

What is rustc?

Welcome to "The rustc book"! `rustc` is the compiler for the Rust programming language, provided by the project itself. Compilers take your source code and produce binary code, either as a library or executable.

Most Rust programmers don't invoke `rustc` directly, but instead do it through [Cargo](#). It's all in service of `rustc` though! If you want to see how Cargo calls `rustc`, you can

```
$ cargo build --verbose
```

And it will print out each `rustc` invocation. This book can help you understand what each of these options does. Additionally, while most Rustaceans use Cargo, not all do: sometimes they integrate `rustc` into other build systems. This book should provide a guide to all of the options you'd need to do so.

Basic usage

Let's say you've got a little hello world program in a file `hello.rs`:

```
fn main() {  
    println!("Hello, world!");  
}
```

To turn this source code into an executable, you can use `rustc`:

```
$ rustc hello.rs  
$ ./hello # on a *NIX  
$ .\hello.exe # on Windows
```

Note that we only ever pass `rustc` the *crate root*, not every file we wish to compile. For example, if we had a `main.rs` that looked like this:

```
mod foo;

fn main() {
    foo::hello();
}
```

And a `foo.rs` that had this:

```
pub fn hello() {
    println!("Hello, world!");
}
```

To compile this, we'd run this command:

```
$ rustc main.rs
```

No need to tell `rustc` about `foo.rs`; the `mod` statements give it everything that it needs. This is different than how you would use a C compiler, where you invoke the compiler on each file, and then link everything together. In other words, the *crate* is a translation unit, not a particular module.

Command-line Arguments

Here's a list of command-line arguments to `rustc` and what they do.

`-h/--help`: get help

This flag will print out help information for `rustc`.

`--cfg`: configure the compilation environment

This flag can turn on or off various `#[cfg]` settings for [conditional compilation](#).

The value can either be a single identifier or two identifiers separated by `=`.

For examples, `--cfg 'verbose'` or `--cfg 'feature="serde"'`. These correspond to `#[cfg(verbose)]` and `#[cfg(feature = "serde")]` respectively.

`--check-cfg`: configure compile-time checking of conditional compilation

This flag enables checking conditional configurations of the crate at compile-time, specifically it helps configure the set of expected `cfg` names and values, in order to check that every *reachable* `#[cfg]` matches the expected config names and values.

This is different from the `--cfg` flag above which activates some config but do not expect them. This is useful to prevent stalled conditions, typos, ...

Refer to the [Checking conditional configurations](#) of this book for further details and explanation.

For examples, `--check-cfg 'cfg(verbose)'` or `--check-cfg 'cfg(feature,`

`values("serde"))'`. These correspond to `#[cfg(verbose)]` and `#[cfg(feature = "serde")]` respectively.

–L: add a directory to the library search path

The `-L` flag adds a path to search for external crates and libraries.

The kind of search path can optionally be specified with the form `-L KIND=PATH` where `KIND` may be one of:

- `dependency` — Only search for transitive dependencies in this directory.
- `crate` — Only search for this crate's direct dependencies in this directory.
- `native` — Only search for native libraries in this directory.
- `framework` — Only search for macOS frameworks in this directory.
- `all` — Search for all library kinds in this directory. This is the default if `KIND` is not specified.

–l: link the generated crate to a native library

Syntax: `-l [KIND[:MODIFIERS]=]NAME[:RENAME]`.

This flag allows you to specify linking to a specific native library when building a crate.

The kind of library can optionally be specified with the form `-l KIND=lib` where `KIND` may be one of:

- `dylib` — A native dynamic library.
- `static` — A native static library (such as a `.a` archive).
- `framework` — A macOS framework.

If the kind is specified, then linking modifiers can be attached to it. Modifiers are specified as a comma-delimited string with each modifier prefixed with either a `+` or `-` to indicate that the modifier is enabled or disabled, respectively. Specifying multiple `modifiers` arguments in a single `link` attribute, or multiple identical modifiers in the same `modifiers` argument

is not currently supported.

Example: `-l static:+whole-archive=mylib`.

The kind of library and the modifiers can also be specified in a `#[link] attribute`. If the kind is not specified in the `link` attribute or on the command-line, it will link a dynamic library by default, except when building a static executable. If the kind is specified on the command-line, it will override the kind specified in a `link` attribute.

The name used in a `link` attribute may be overridden using the form `-l ATTR_NAME:LINK_NAME` where `ATTR_NAME` is the name in the `link` attribute, and `LINK_NAME` is the name of the actual library that will be linked.

Linking modifiers: **whole-archive**

This modifier is only compatible with the `static` linking kind. Using any other kind will result in a compiler error.

`+whole-archive` means that the static library is linked as a whole archive without throwing any object files away.

This modifier translates to `--whole-archive` for `ld`-like linkers, to `/WHOLEARCHIVE` for `link.exe`, and to `-force_load` for `ld64`. The modifier does nothing for linkers that don't support it.

The default for this modifier is `-whole-archive`.

NOTE: The default may currently be different in some cases for backward compatibility, but it is not guaranteed. If you need whole archive semantics use `+whole-archive` explicitly.

Linking modifiers: **bundle**

This modifier is only compatible with the `static` linking kind. Using any other kind will result in a compiler error.

When building a `rlib` or `staticlib` `+bundle` means that the native static library will be packed into the `rlib` or `staticlib` archive, and then retrieved from there during linking of the final binary.

When building a `rlib` `-bundle` means that the native static library is registered as a dependency of that `rlib` "by name", and object files from it are included only during linking of the final binary, the file search by that name is also performed during final linking. When building a `staticlib` `-bundle` means that the native static library is simply not included into the archive and some higher level build system will need to add it later during linking of the final binary.

This modifier has no effect when building other targets like executables or dynamic libraries.

The default for this modifier is `+bundle`.

Linking modifiers: **verbatim**

This modifier is compatible with all linking kinds.

`+verbatim` means that rustc itself won't add any target-specified library prefixes or suffixes (like `lib` or `.a`) to the library name, and will try its best to ask for the same thing from the linker.

For `ld`-like linkers supporting GNU extensions rustc will use the `-l:filename` syntax (note the colon) when passing the library, so the linker won't add any prefixes or suffixes to it. See `-l namespec` in `ld` documentation for more details.

For linkers not supporting any verbatim modifiers (e.g. `link.exe` or `ld64`) the library name will be passed as is. So the most reliable cross-platform use scenarios for this option are when no linker is involved, for example bundling native libraries into `rlibs`.

`-verbatim` means that rustc will either add a target-specific prefix and suffix to the library name before passing it to linker, or won't prevent linker from implicitly adding it. In case of `raw-dylib` kind in particular `.dll` will be added to the library name on Windows.

The default for this modifier is `-verbatim`.

NOTE: Even with `+verbatim` and `-l:filename` syntax `ld`-like linkers do not typically support passing absolute paths to libraries. Usually such paths need to be passed as input files without using any options like `-l`, e.g. `ld /my/absolute/path`.

`-Clink-arg=/my/absolute/path` can be used for doing this from stable rustc.

`--crate-type`: a list of types of crates for the compiler to emit

This instructs `rustc` on which crate type to build. This flag accepts a comma-separated list of values, and may be specified multiple times. The valid crate types are:

- `lib` — Generates a library kind preferred by the compiler, currently defaults to `rlib`.
- `rlib` — A Rust static library.
- `staticlib` — A native static library.
- `dylib` — A Rust dynamic library.
- `cdylib` — A native dynamic library.
- `bin` — A runnable executable program.
- `proc-macro` — Generates a format suitable for a procedural macro library that may be loaded by the compiler.

The crate type may be specified with the `crate_type` attribute. The `--crate-type` command-line value will override the `crate_type` attribute.

More details may be found in the [linkage chapter](#) of the reference.

`--crate-name`: specify the name of the crate being built

This informs `rustc` of the name of your crate.

`--edition`: specify the edition to use

This flag takes a value of `2015`, `2018` or `2021`. The default is `2015`. More information about editions may be found in the [edition guide](#).

`--emit`: specifies the types of output files to generate

This flag controls the types of output files generated by the compiler. It accepts a comma-separated list of values, and may be specified multiple times. The valid emit kinds are:

- `asm` — Generates a file with the crate's assembly code. The default output filename is `CRATE_NAME.s`.
- `dep-info` — Generates a file with Makefile syntax that indicates all the source files that were loaded to generate the crate. The default output filename is `CRATE_NAME.d`.
- `link` — Generates the crates specified by `--crate-type`. The default output filenames depend on the crate type and platform. This is the default if `--emit` is not specified.
- `llvm-bc` — Generates a binary file containing the [LLVM bitcode](#). The default output filename is `CRATE_NAME.bc`.
- `llvm-ir` — Generates a file containing [LLVM IR](#). The default output filename is `CRATE_NAME.ll`.
- `metadata` — Generates a file containing metadata about the crate. The default output filename is `libCRATE_NAME.rmeta`.
- `mir` — Generates a file containing rustc's mid-level intermediate representation. The default output filename is `CRATE_NAME.mir`.
- `obj` — Generates a native object file. The default output filename is `CRATE_NAME.o`.

The output filename can be set with the `-o` flag. A suffix may be added to the filename with the `-C extra-filename` flag.

Output files are written to the current directory unless the `--out-dir` flag is used.

Custom paths for individual emit kinds

Each emit type can optionally be followed by `=` to specify an explicit output path that only applies to the output of that type. For example:

- `--emit=link,dep-info=/path/to/dep-info.d`
 - Emit the crate itself as normal, and also emit dependency info to the specified path.
- `--emit=llvm-ir=-,mir`
 - Emit MIR to the default filename (based on crate name), and emit LLVM IR to stdout.

Emitting to stdout

When using `--emit` or `-o`, output can be sent to stdout by specifying `-` as the path (e.g. `-o -`).

Binary output types can only be written to stdout if it is not a tty. Text output types (`asm`, `dep-info`, `llvm-ir` and `mir`) can be written to stdout regardless of whether it is a tty or not.

Only one type of output can be written to stdout. Attempting to write multiple types to stdout at the same time will result in an error.

`--print`: print compiler information

This flag prints out various information about the compiler. This flag may be specified multiple times, and the information is printed in the order the flags are specified. Specifying a `--print` flag will usually disable the `--emit` step and will only print the requested information. The valid types of print values are:

- `crate-name` — The name of the crate.
- `file-names` — The names of the files created by the `link` emit kind.
- `sysroot` — Path to the sysroot.
- `target-libdir` - Path to the target libdir.
- `cfg` — List of cfg values. See [conditional compilation](#) for more information about cfg values.
- `target-list` — List of known targets. The target may be selected with the `--target` flag.
- `target-cpus` — List of available CPU values for the current target. The target CPU may be selected with the `-C target-cpu=val` flag.
- `target-features` — List of available target features for the current target. Target features may be enabled with the `-C target-feature=val` flag. This flag is unsafe. See [known issues](#) for more details.
- `relocation-models` — List of relocation models. Relocation models may be selected with the `-C relocation-model=val` flag.
- `code-models` — List of code models. Code models may be selected with the `-C code-`

`model=val` flag.

- `tls-models` — List of Thread Local Storage models supported. The model may be selected with the `-Z tls-model=val` flag.
- `native-static-libs` — This may be used when creating a `staticlib` crate type. If this is the only flag, it will perform a full compilation and include a diagnostic note that indicates the linker flags to use when linking the resulting static library. The note starts with the text `native-static-libs:` to make it easier to fetch the output.
- `link-args` — This flag does not disable the `--emit` step. When linking, this flag causes `rustc` to print the full linker invocation in a human-readable form. This can be useful when debugging linker options. The exact format of this debugging output is not a stable guarantee, other than that it will include the linker executable and the text of each command-line argument passed to the linker.
- `deployment-target` - The currently selected [deployment target](#) (or minimum OS version) for the selected Apple platform target. This value can be used or passed along to other components alongside a Rust build that need this information, such as C compilers. This returns rustc's minimum supported deployment target if no `*_DEPLOYMENT_TARGET` variable is present in the environment, or otherwise returns the variable's parsed value.

A filepath may optionally be specified for each requested information kind, in the format `--print KIND=PATH`, just like for `--emit`. When a path is specified, information will be written there instead of to stdout.

–g: include debug information

A synonym for `-C debuginfo=2`.

–O: optimize your code

A synonym for `-C opt-level=2`.

-o: filename of the output

This flag controls the output filename.

--out-dir: directory to write the output in

The outputted crate will be written to this directory. This flag is ignored if the [-o](#) flag is used.

--explain: provide a detailed explanation of an error message

Each error of `rustc` 's comes with an error code; this will print out a longer explanation of a given error.

--test: build a test harness

When compiling this crate, `rustc` will ignore your `main` function and instead produce a test harness. See the [Tests chapter](#) for more information about tests.

--target: select a target triple to build

This controls which [target](#) to produce.

-W: set lint warnings

This flag will set which lints should be set to the [warn level](#).

Note: The order of these lint level arguments is taken into account, see [lint level via compiler flag](#) for more information.

--force-warn: force a lint to warn

This flag sets the given lint to the [forced warn level](#) and the level cannot be overridden, even ignoring the [lint caps](#).

-A: set lint allowed

This flag will set which lints should be set to the [allow level](#).

Note: The order of these lint level arguments is taken into account, see [lint level via compiler flag](#) for more information.

-D: set lint denied

This flag will set which lints should be set to the [deny level](#).

Note: The order of these lint level arguments is taken into account, see [lint level via compiler flag](#) for more information.

-F: set lint forbidden

This flag will set which lints should be set to the [forbid level](#).

Note: The order of these lint level arguments is taken into account, see [lint level via compiler flag](#) for more information.

-Z: set unstable options

This flag will allow you to set unstable options of rustc. In order to set multiple options, the -Z flag can be used multiple times. For example: `rustc -Z verbose-internals -Z time-passes`. Specifying options with -Z is only available on nightly. To view all available options run: `rustc -Z help`, or see [The Unstable Book](#).

--cap-lints: set the most restrictive lint level

This flag lets you 'cap' lints, for more, [see here](#).

-C/--codegen: code generation options

This flag will allow you to set [codegen options](#).

-V/--version: print a version

This flag will print out rustc's version.

-v/--verbose: use verbose output

This flag, when combined with other flags, makes them produce extra output.

--extern: specify where an external library is located

This flag allows you to pass the name and location for an external crate of a direct dependency. Indirect dependencies (dependencies of dependencies) are located using the -

L flag. The given crate name is added to the [extern prelude](#), similar to specifying `extern crate` within the root module. The given crate name does not need to match the name the library was built with.

Specifying `--extern` has one behavior difference from `extern crate`: `--extern` merely makes the crate a *candidate* for being linked; it does not actually link it unless it's actively used. In rare occasions you may wish to ensure a crate is linked even if you don't actively use it from your code: for example, if it changes the global allocator or if it contains `#[no_mangle]` symbols for use by other programming languages. In such cases you'll need to use `extern crate`.

This flag may be specified multiple times. This flag takes an argument with either of the following formats:

- `CRATENAME=PATH` — Indicates the given crate is found at the given path.
- `CRATENAME` — Indicates the given crate may be found in the search path, such as within the sysroot or via the `-L` flag.

The same crate name may be specified multiple times for different crate types. If both an `rlib` and `dylib` are found, an internal algorithm is used to decide which to use for linking. The `-C prefer-dynamic` flag may be used to influence which is used.

If the same crate name is specified with and without a path, the one with the path is used and the pathless flag has no effect.

`--sysroot`: Override the system root

The "sysroot" is where `rustc` looks for the crates that come with the Rust distribution; this flag allows that to be overridden.

`--error-format`: control how errors are produced

This flag lets you control the format of messages. Messages are printed to `stderr`. The valid options are:

- `human` — Human-readable output. This is the default.
- `json` — Structured JSON output. See [the JSON chapter](#) for more detail.
- `short` — Short, one-line messages.

`--color`: configure coloring of output

This flag lets you control color settings of the output. The valid options are:

- `auto` — Use colors if output goes to a tty. This is the default.
- `always` — Always use colors.
- `never` — Never colorize output.

`--diagnostic-width`: specify the terminal width for diagnostics

This flag takes a number that specifies the width of the terminal in characters. Formatting of diagnostics will take the width into consideration to make them better fit on the screen.

`--remap-path-prefix`: remap source names in output

Remap source path prefixes in all output, including compiler diagnostics, debug information, macro expansions, etc. It takes a value of the form `FROM=TO` where a path prefix equal to `FROM` is rewritten to the value `TO`. The `FROM` may itself contain an `=` symbol, but the `TO` value may not. This flag may be specified multiple times.

This is useful for normalizing build products, for example by removing the current directory out of pathnames emitted into the object files. The replacement is purely textual, with no consideration of the current system's pathname syntax. For example `--remap-path-prefix foo=bar` will match `foo/lib.rs` but not `./foo/lib.rs`.

When multiple remappings are given and several of them match, the **last** matching one is

applied.

`--json`: configure json messages printed by the compiler

When the `--error-format=json` option is passed to rustc then all of the compiler's diagnostic output will be emitted in the form of JSON blobs. The `--json` argument can be used in conjunction with `--error-format=json` to configure what the JSON blobs contain as well as which ones are emitted.

With `--error-format=json` the compiler will always emit any compiler errors as a JSON blob, but the following options are also available to the `--json` flag to customize the output:

- `diagnostic-short` - json blobs for diagnostic messages should use the "short" rendering instead of the normal "human" default. This means that the output of `--error-format=short` will be embedded into the JSON diagnostics instead of the default `--error-format=human`.
- `diagnostic-rendered-ansi` - by default JSON blobs in their `rendered` field will contain a plain text rendering of the diagnostic. This option instead indicates that the diagnostic should have embedded ANSI color codes intended to be used to colorize the message in the manner rustc typically already does for terminal outputs. Note that this is usefully combined with crates like `fwdansi` to translate these ANSI codes on Windows to console commands or `strip-ansi-escapes` if you'd like to optionally remove the ansi colors afterwards.
- `artifacts` - this instructs rustc to emit a JSON blob for each artifact that is emitted. An artifact corresponds to a request from the `--emit CLI argument`, and as soon as the artifact is available on the filesystem a notification will be emitted.
- `future-incompat` - includes a JSON message that contains a report if the crate contains any code that may fail to compile in the future.

Note that it is invalid to combine the `--json` argument with the `--color` argument, and it is required to combine `--json` with `--error-format=json`.

See [the JSON chapter](#) for more detail.

@path: load command-line flags from a path

If you specify `@path` on the command-line, then it will open `path` and read command line options from it. These options are one per line; a blank line indicates an empty option. The file can use Unix or Windows style line endings, and must be encoded as UTF-8.

Codegen Options

All of these options are passed to `rustc` via the `-C` flag, short for "codegen." You can see a version of this list for your exact compiler by running `rustc -C help`.

`ar`

This option is deprecated and does nothing.

`code-model`

This option lets you choose which code model to use.

Code models put constraints on address ranges that the program and its symbols may use. With smaller address ranges machine instructions may be able to use more compact addressing modes.

The specific ranges depend on target architectures and addressing modes available to them. For x86 more detailed description of its code models can be found in [System V Application Binary Interface](#) specification.

Supported values for this option are:

- `tiny` - Tiny code model.
- `small` - Small code model. This is the default model for majority of supported targets.
- `kernel` - Kernel code model.
- `medium` - Medium code model.
- `large` - Large code model.

Supported values can also be discovered by running `rustc --print code-models`.

`codegen-units`

This flag controls the maximum number of code generation units the crate is split into. It takes an integer greater than 0.

When a crate is split into multiple codegen units, LLVM is able to process them in parallel. Increasing parallelism may speed up compile times, but may also produce slower code. Setting this to 1 may improve the performance of generated code, but may be slower to compile.

The default value, if not specified, is 16 for non-incremental builds. For incremental builds the default is 256 which allows caching to be more granular.

collapse-macro-debuginfo

This flag controls whether code locations from a macro definition are collapsed into a single location associated with that macro's call site, when generating debuginfo for this crate.

This option, if passed, overrides both default collapsing behavior and `#[collapse_debuginfo]` attributes in code.

- `y`, `yes`, `on`, `true` : collapse code locations in debuginfo.
- `n`, `no`, `off` or `false` : do not collapse code locations in debuginfo.
- `external` : collapse code locations in debuginfo only if the macro comes from a different crate.

control-flow-guard

This flag controls whether LLVM enables the Windows [Control Flow Guard](#) platform security feature. This flag is currently ignored for non-Windows targets. It takes one of the following values:

- `y`, `yes`, `on`, `true`, `checks`, or no value: enable Control Flow Guard.
- `nochecks` : emit Control Flow Guard metadata without runtime enforcement checks (this should only be used for testing purposes as it does not provide security enforcement).

- `n`, `no`, `off`, `false` : do not enable Control Flow Guard (the default).

debug-assertions

This flag lets you turn `cfg(debug_assertions)` [conditional compilation](#) on or off. It takes one of the following values:

- `y`, `yes`, `on`, `true`, or no value: enable debug-assertions.
- `n`, `no`, `off` or `false` : disable debug-assertions.

If not specified, debug assertions are automatically enabled only if the [opt-level](#) is 0.

debuginfo

This flag controls the generation of debug information. It takes one of the following values:

- `0` or `none` : no debug info at all (the default).
- `line-directives-only` : line info directives only. For the `nvptx*` targets this enables [profiling](#). For other use cases, `line-tables-only` is the better, more compatible choice.
- `line-tables-only` : line tables only. Generates the minimal amount of debug info for backtraces with filename/line number info, but not anything else, i.e. no variable or function parameter info.
- `1` or `limited` : debug info without type or variable-level information.
- `2` or `full` : full debug info.

Note: The `-g` [flag](#) is an alias for `-C debuginfo=2`.

default-linker-libraries

This flag controls whether or not the linker includes its default libraries. It takes one of the following values:

- `y`, `yes`, `on`, `true`: include default libraries.
- `n`, `no`, `off` or `false` or no value: exclude default libraries (the default).

For example, for gcc flavor linkers, this issues the `-nodefaultlibs` flag to the linker.

dlltool

On `windows-gnu` targets, this flag controls which `dlltool` `rustc` invokes to generate import libraries when using the `raw-dylib` link kind. It takes a path to [the dlltool executable](#). If this flag is not specified, a `dlltool` executable will be inferred based on the host environment and target.

embed-bitcode

This flag controls whether or not the compiler embeds LLVM bitcode into object files. It takes one of the following values:

- `y`, `yes`, `on`, `true` or no value: put bitcode in rlibs (the default).
- `n`, `no`, `off` or `false`: omit bitcode from rlibs.

LLVM bitcode is required when `rustc` is performing link-time optimization (LTO). It is also required on some targets like iOS ones where vendors look for LLVM bitcode. Embedded bitcode will appear in `rustc`-generated object files inside of a section whose name is defined by the target platform. Most of the time this is `.llvmbc`.

The use of `-C embed-bitcode=no` can significantly improve compile times and reduce generated file sizes if your compilation does not actually need bitcode (e.g. if you're not compiling for iOS or you're not performing LTO). For these reasons, Cargo uses `-C embed-bitcode=no` whenever possible. Likewise, if you are building directly with `rustc` we recommend using `-C embed-bitcode=no` whenever you are not using LTO.

If combined with `-C lto`, `-C embed-bitcode=no` will cause `rustc` to abort at start-up, because the combination is invalid.

Note: if you're building Rust code with LTO then you probably don't even need the `embed-bitcode` option turned on. You'll likely want to use `-Clinker-plugin-lto` instead which skips generating object files entirely and simply replaces object files with LLVM bitcode. The only purpose for `-Cembed-bitcode` is when you're generating an rlib that is both being used with and without LTO. For example Rust's standard library ships with embedded bitcode since users link to it both with and without LTO.

This also may make you wonder why the default is `yes` for this option. The reason for that is that it's how it was for rustc 1.44 and prior. In 1.45 this option was added to turn off what had always been the default.

extra-filename

This option allows you to put extra data in each output filename. It takes a string to add as a suffix to the filename. See the `--emit` flag for more information.

force-frame-pointers

This flag forces the use of frame pointers. It takes one of the following values:

- `y`, `yes`, `on`, `true` or no value: force-enable frame pointers.
- `n`, `no`, `off` or `false`: do not force-enable frame pointers. This does not necessarily mean frame pointers will be removed.

The default behaviour, if frame pointers are not force-enabled, depends on the target.

force-unwind-tables

This flag forces the generation of unwind tables. It takes one of the following values:

- `y`, `yes`, `on`, `true` or no value: Unwind tables are forced to be generated.

- `n`, `no`, `off` or `false`: Unwind tables are not forced to be generated. If unwind tables are required by the target an error will be emitted.

The default if not specified depends on the target.

incremental

This flag allows you to enable incremental compilation, which allows `rustc` to save information after compiling a crate to be reused when recompiling the crate, improving re-compile times. This takes a path to a directory where incremental files will be stored.

inline-threshold

This option is deprecated and does nothing.

Consider using `-Cllvm-args=--inline-threshold=...`.

instrument-coverage

This option enables instrumentation-based code coverage support. See the chapter on [instrumentation-based code coverage](#) for more information.

Note that while the `-C instrument-coverage` option is stable, the profile data format produced by the resulting instrumentation may change, and may not work with coverage tools other than those built and shipped with the compiler.

link-arg

This flag lets you append a single extra argument to the linker invocation.

"Append" is significant; you can pass this flag multiple times to add multiple arguments.

link-args

This flag lets you append multiple extra arguments to the linker invocation. The options should be separated by spaces.

link-dead-code

This flag controls whether the linker will keep dead code. It takes one of the following values:

- `y`, `yes`, `on`, `true` or no value: keep dead code.
- `n`, `no`, `off` or `false`: remove dead code (the default).

An example of when this flag might be useful is when trying to construct code coverage metrics.

link-self-contained

On `windows-gnu`, `linux-musl`, and `wasi` targets, this flag controls whether the linker will use libraries and objects shipped with Rust instead of those in the system. It takes one of the following values:

- no value: rustc will use heuristic to disable self-contained mode if system has necessary tools.
- `y`, `yes`, `on`, `true`: use only libraries/objects shipped with Rust.
- `n`, `no`, `off` or `false`: rely on the user or the linker to provide non-Rust libraries/objects.

This allows overriding cases when detection fails or user wants to use shipped libraries.

linker

This flag controls which linker `rustc` invokes to link your code. It takes a path to the linker executable. If this flag is not specified, the linker will be inferred based on the target. See also the [linker-flavor](#) flag for another way to specify the linker.

linker-flavor

This flag controls the linker flavor used by `rustc`. If a linker is given with the `-C linker flag`, then the linker flavor is inferred from the value provided. If no linker is given then the linker flavor is used to determine the linker to use. Every `rustc` target defaults to some linker flavor. Valid options are:

- `em`: use [Emscripten](#) `emcc`.
- `gcc`: use the `cc` executable, which is typically `gcc` or `clang` on many systems.
- `ld`: use the `ld` executable.
- `msvc`: use the `link.exe` executable from Microsoft Visual Studio MSVC.
- `wasm-ld`: use the [wasm-ld](#) executable, a port of LLVM `lld` for WebAssembly.
- `ld64.lld`: use the LLVM `lld` executable with the `-flavor darwin` flag for Apple's `ld`.
- `ld.lld`: use the LLVM `lld` executable with the `-flavor gnu` flag for GNU binutils' `ld`.
- `lld-link`: use the LLVM `lld` executable with the `-flavor link` flag for Microsoft's `link.exe`.

linker-plugin-lto

This flag defers LTO optimizations to the linker. See [linker-plugin-LTO](#) for more details. It takes one of the following values:

- `y`, `yes`, `on`, `true` or no value: enable linker plugin LTO.
- `n`, `no`, `off` or `false`: disable linker plugin LTO (the default).
- A path to the linker plugin.

More specifically this flag will cause the compiler to replace its typical object file output with LLVM bitcode files. For example an rlib produced with `-Clinker-plugin-lto` will still have `*.o` files in it, but they'll all be LLVM bitcode instead of actual machine code. It is expected that the native platform linker is capable of loading these LLVM bitcode files and generating code at link time (typically after performing optimizations).

Note that rustc can also read its own object files produced with `-Clinker-plugin-lto`. If an rlib is only ever going to get used later with a `-Clto` compilation then you can pass `-Clinker-plugin-lto` to speed up compilation and avoid generating object files that aren't used.

llvm-args

This flag can be used to pass a list of arguments directly to LLVM.

The list must be separated by spaces.

Pass `--help` to see a list of options.

lto

This flag controls whether LLVM uses [link time optimizations](#) to produce better optimized code, using whole-program analysis, at the cost of longer linking time. It takes one of the following values:

- `y`, `yes`, `on`, `true`, `fat`, or no value: perform "fat" LTO which attempts to perform optimizations across all crates within the dependency graph.
- `n`, `no`, `off`, `false`: disables LTO.
- `thin`: perform ["thin" LTO](#). This is similar to "fat", but takes substantially less time to run while still achieving performance gains similar to "fat".

If `-C lto` is not specified, then the compiler will attempt to perform "thin local LTO" which performs "thin" LTO on the local crate only across its [codegen units](#). When `-C lto` is not specified, LTO is disabled if codegen units is 1 or optimizations are disabled (`-C opt-`

`level=0`). That is:

- When `-C lto` is not specified:
 - `codegen-units=1` : disable LTO.
 - `opt-level=0` : disable LTO.
- When `-C lto` is specified:
 - `lto` : 16 codegen units, perform fat LTO across crates.
 - `codegen-units=1 + lto` : 1 codegen unit, fat LTO across crates.

See also [linker-plugin-lto](#) for cross-language LTO.

metadata

This option allows you to control the metadata used for symbol mangling. This takes a space-separated list of strings. Mangled symbols will incorporate a hash of the metadata. This may be used, for example, to differentiate symbols between two different versions of the same crate being linked.

no-prepopulate-passes

This flag tells the pass manager to use an empty list of passes, instead of the usual pre-populated list of passes.

no-redzone

This flag allows you to disable [the red zone](#). It takes one of the following values:

- `y` , `yes` , `on` , `true` or no value: disable the red zone.
- `n` , `no` , `off` or `false` : enable the red zone.

The default behaviour, if the flag is not specified, depends on the target.

no-stack-check

This option is deprecated and does nothing.

no-vectorize-loops

This flag disables [loop vectorization](#).

no-vectorize-slp

This flag disables vectorization using [superword-level parallelism](#).

opt-level

This flag controls the optimization level.

- `0` : no optimizations, also turns on [cfg\(debug_assertions\)](#) (the default).
- `1` : basic optimizations.
- `2` : some optimizations.
- `3` : all optimizations.
- `s` : optimize for binary size.
- `z` : optimize for binary size, but also turn off loop vectorization.

Note: The `-O` [flag](#) is an alias for `-C opt-level=2`.

The default is `0`.

overflow-checks

This flag allows you to control the behavior of [runtime integer overflow](#). When overflow-checks are enabled, a panic will occur on overflow. This flag takes one of the following values:

- `y`, `yes`, `on`, `true` or no value: enable overflow checks.
- `n`, `no`, `off` or `false`: disable overflow checks.

If not specified, overflow checks are enabled if [debug-assertions](#) are enabled, disabled otherwise.

panic

This option lets you control what happens when the code panics.

- `abort`: terminate the process upon panic
- `unwind`: unwind the stack upon panic

If not specified, the default depends on the target.

passes

This flag can be used to add extra [LLVM passes](#) to the compilation.

The list must be separated by spaces.

See also the [no-prepopulate-passes](#) flag.

prefer-dynamic

By default, `rustc` prefers to statically link dependencies. This option will indicate that dynamic linking should be used if possible if both a static and dynamic versions of a library are available.

There is [an internal algorithm](#) for determining whether or not it is possible to statically or dynamically link with a dependency.

This flag takes one of the following values:

- `y`, `yes`, `on`, `true` or no value: prefer dynamic linking.
- `n`, `no`, `off` or `false`: prefer static linking (the default).

profile-generate

This flag allows for creating instrumented binaries that will collect profiling data for use with profile-guided optimization (PGO). The flag takes an optional argument which is the path to a directory into which the instrumented binary will emit the collected data. See the chapter on [profile-guided optimization](#) for more information.

profile-use

This flag specifies the profiling data file to be used for profile-guided optimization (PGO). The flag takes a mandatory argument which is the path to a valid `.profdata` file. See the chapter on [profile-guided optimization](#) for more information.

relocation-model

This option controls generation of [position-independent code \(PIC\)](#).

Supported values for this option are:

Primary relocation models

- `static` - non-relocatable code, machine instructions may use absolute addressing modes.

- `pic` - fully relocatable position independent code, machine instructions need to use relative addressing modes.
Equivalent to the "uppercase" `-fPIC` or `-fPIE` options in other compilers, depending on the produced crate types.
This is the default model for majority of supported targets.
- `pie` - position independent executable, relocatable code but without support for symbol interpositioning (replacing symbols by name using `LD_PRELOAD` and similar).
Equivalent to the "uppercase" `-fPIE` option in other compilers. `pie` code cannot be linked into shared libraries (you'll get a linking error on attempt to do this).

Special relocation models

- `dynamic-no-pic` - relocatable external references, non-relocatable code.
Only makes sense on Darwin and is rarely used.
If StackOverflow tells you to use this as an opt-out of PIC or PIE, don't believe it, use `-C relocation-model=static` instead.
- `ropi`, `rwpi` and `ropi-rwpi` - relocatable code and read-only data, relocatable read-write data, and combination of both, respectively.
Only makes sense for certain embedded ARM targets.
- `default` - relocation model default to the current target.
Only makes sense as an override for some other explicitly specified relocation model previously set on the command line.

Supported values can also be discovered by running `rustc --print relocation-models`.

Linking effects

In addition to codegen effects, `relocation-model` has effects during linking.

If the relocation model is `pic` and the current target supports position-independent executables (PIE), the linker will be instructed (`-pie`) to produce one.

If the target doesn't support both position-independent and statically linked executables, then `-C target-feature=+crt-static` "wins" over `-C relocation-model=pic`, and the linker is instructed (`-static`) to produce a statically linked but not position-independent executable.

relro-level

This flag controls what level of RELRO (Relocation Read-Only) is enabled. RELRO is an exploit mitigation which makes the Global Offset Table (GOT) read-only.

Supported values for this option are:

- `off` : Dynamically linked functions are resolved lazily and the GOT is writable.
- `partial` : Dynamically linked functions are resolved lazily and written into the Procedure Linking Table (PLT) part of the GOT (`.got.plt`). The non-PLT part of the GOT (`.got`) is made read-only and both are moved to prevent writing from buffer overflows.
- `full` : Dynamically linked functions are resolved at the start of program execution and the Global Offset Table (`.got` / `.got.plt`) is populated eagerly and then made read-only. The GOT is also moved to prevent writing from buffer overflows. Full RELRO uses more memory and increases process startup time.

This flag is ignored on platforms where RELRO is not supported (targets which do not use the ELF binary format), such as Windows or macOS. Each rustc target has its own default for RELRO. rustc enables Full RELRO by default on platforms where it is supported.

remark

This flag lets you print remarks for optimization passes.

The list of passes should be separated by spaces.

`all` will remark on every pass.

rpath

This flag controls whether `rpath` is enabled. It takes one of the following values:

- `y` , `yes` , `on` , `true` or no value: enable rpath.

- `n`, `no`, `off` or `false` : disable rpath (the default).

save-temps

This flag controls whether temporary files generated during compilation are deleted once compilation finishes. It takes one of the following values:

- `y`, `yes`, `on`, `true` or no value: save temporary files.
- `n`, `no`, `off` or `false` : delete temporary files (the default).

soft-float

This option controls whether `rustc` generates code that emulates floating point instructions in software. It takes one of the following values:

- `y`, `yes`, `on`, `true` or no value: use soft floats.
- `n`, `no`, `off` or `false` : use hardware floats (the default).

split-debuginfo

This option controls the emission of "split debuginfo" for debug information that `rustc` generates. The default behavior of this option is platform-specific, and not all possible values for this option work on all platforms. Possible values are:

- `off` - This is the default for platforms with ELF binaries and windows-gnu (not Windows MSVC and not macOS). This typically means that DWARF debug information can be found in the final artifact in sections of the executable. This option is not supported on Windows MSVC. On macOS this options prevents the final execution of `dsymutil` to generate debuginfo.
- `packed` - This is the default for Windows MSVC and macOS. The term "packed" here means that all the debug information is packed into a separate file from the main

executable. On Windows MSVC this is a `*.pdb` file, on macOS this is a `*.dsym` folder, and on other platforms this is a `*.dwp` file.

- `unpacked` - This means that debug information will be found in separate files for each compilation unit (object file). This is not supported on Windows MSVC. On macOS this means the original object files will contain debug information. On other Unix platforms this means that `*.dwo` files will contain debug information.

Note that all three options are supported on Linux and Apple platforms, `packed` is supported on Windows-MSVC, and all other platforms support `off`. Attempting to use an unsupported option requires using the nightly channel with the `-Z unstable-options` flag.

strip

The option `-C strip=val` controls stripping of debuginfo and similar auxiliary data from binaries during linking.

Supported values for this option are:

- `none` - debuginfo and symbols are not modified.
- `debuginfo` - debuginfo sections and debuginfo symbols from the symbol table section are stripped at link time and are not copied to the produced binary. This should leave backtraces mostly-intact but may make using a debugger like `gdb` or `lldb` ineffectual. Prior to 1.79, this unintentionally disabled the generation of `*.pdb` files on MSVC, resulting in the absence of symbols.
- `symbols` - same as `debuginfo`, but the rest of the symbol table section is stripped as well, depending on platform support. On platforms which depend on this symbol table for backtraces, profiling, and similar, this can affect them so negatively as to make the trace incomprehensible. Programs which may be combined with others, such as CLI pipelines and developer tooling, or even anything which wants crash-reporting, should usually avoid `-Cstrip=symbols`.

Note that, at any level, removing debuginfo only necessarily impacts "friendly" introspection. `-Cstrip` cannot be relied on as a meaningful security or obfuscation measure, as disassemblers and decompilers can extract considerable information even in the absence of symbols.

symbol-mangling-version

This option controls the [name mangling](#) format for encoding Rust item names for the purpose of generating object code and linking.

Supported values for this option are:

- `v0` — The "v0" mangling scheme.

The default, if not specified, will use a compiler-chosen default which may change in the future.

See the [Symbol Mangling](#) chapter for details on symbol mangling and the mangling format.

target-cpu

This instructs `rustc` to generate code specifically for a particular processor.

You can run `rustc --print target-cpus` to see the valid options to pass and the default target CPU for the current build target. Each target has a default base CPU. Special values include:

- `native` can be passed to use the processor of the host machine.
- `generic` refers to an LLVM target with minimal features but modern tuning.

target-feature

Individual targets will support different features; this flag lets you control enabling or disabling a feature. Each feature should be prefixed with a `+` to enable it or `-` to disable it.

Features from multiple `-C target-feature` options are combined.

Multiple features can be specified in a single option by separating them with commas - `-C target-feature=+x,-y`.

If some feature is specified more than once with both `+` and `-`, then values passed later

override values passed earlier.

For example, `-C target-feature=+x,-y,+z` `-Ctarget-feature=-x,+y` is equivalent to `-C target-feature=-x,+y,+z`.

To see the valid options and an example of use, run `rustc --print target-features`.

Using this flag is unsafe and might result in [undefined runtime behavior](#).

See also the [target_feature attribute](#) for controlling features per-function.

This also supports the feature `+crt-static` and `-crt-static` to control [static C runtime linkage](#).

Each target and [target-cpu](#) has a default set of enabled features.

tune-cpu

This instructs `rustc` to schedule code specifically for a particular processor. This does not affect the compatibility (instruction sets or ABI), but should make your code slightly more efficient on the selected CPU.

The valid options are the same as those for [target-cpu](#). The default is `None`, which LLVM translates as the `target-cpu`.

This is an unstable option. Use `-Z tune-cpu=machine` to specify a value.

Due to limitations in LLVM (12.0.0-git9218f92), this option is currently effective only for x86 targets.

Jobserver

Internally, `rustc` may take advantage of parallelism. `rustc` will coordinate with the build system calling it if a [GNU Make jobserver](#) is passed in the `MAKEFLAGS` environment variable. Other flags may have an effect as well, such as [CARGO_MAKEFLAGS](#). If a jobserver is not passed, then `rustc` will choose the number of jobs to use.

Starting with Rust 1.76.0, `rustc` will warn if a jobserver appears to be available but is not accessible, e.g.:

```
$ echo 'fn main() {}' | MAKEFLAGS=--jobserver-auth=3,4 rustc -
warning: failed to connect to jobserver from environment variable `MAKEFLAGS="--
-jobserver-auth=3,4"`: cannot open file descriptor 3 from the jobserver
environment variable value: Bad file descriptor (os error 9)
|
= note: the build environment is likely misconfigured
```

Integration with build systems

The following subsections contain recommendations on how to integrate `rustc` with build systems so that the jobserver is handled appropriately.

GNU Make

When calling `rustc` from GNU Make, it is recommended that all `rustc` invocations are marked as recursive in the `Makefile` (by prefixing the command line with the `+` indicator), so that GNU Make enables the jobserver for them. For instance:

```
x:
    +@echo 'fn main() {}' | rustc -
```

In particular, GNU Make 4.3 (a widely used version as of 2024) passes a simple pipe jobserver in `MAKEFLAGS` even when it was not made available for the child process, which in turn means `rustc` will warn about it. For instance, if the `+` indicator is removed from the

example above and GNU Make is called with e.g. `make -j2`, then the aforementioned warning will trigger.

For calls to `rustc` inside `$(shell ...)` inside a recursive Make, one can disable the jobserver manually by clearing the `MAKEFLAGS` variable, e.g.:

```
S := $(shell MAKEFLAGS= rustc --print sysroot)
```

```
x:
    @$(MAKE) y
```

```
y:
    @echo $(S)
```

CMake

CMake 3.28 supports the `JOB_SERVER_AWARE` option in its `add_custom_target` command, e.g.:

```
cmake_minimum_required(VERSION 3.28)
project(x)
add_custom_target(x
    JOB_SERVER_AWARE TRUE
    COMMAND echo 'fn main() {}' | rustc -
)
```

For earlier versions, when using CMake with the Makefile generator, one workaround is to have `$(MAKE)` somewhere in the command so that GNU Make treats it as a recursive Make call, e.g.:

```
cmake_minimum_required(VERSION 3.22)
project(x)
add_custom_target(x
    COMMAND DUMMY_VARIABLE=$(MAKE) echo 'fn main() {}' | rustc -
)
```

Lints

In software, a "lint" is a tool used to help improve your source code. The Rust compiler contains a number of lints, and when it compiles your code, it will also run the lints. These lints may produce a warning, an error, or nothing at all, depending on how you've configured things.

Here's a small example:

```
$ cat main.rs
fn main() {
    let x = 5;
}
$ rustc main.rs
warning: unused variable: `x`
--> main.rs:2:9
   |
2  |     let x = 5;
   |         ^
   = note: `#[warn(unused_variables)]` on by default
   = note: to avoid this warning, consider using `_x` instead
```

This is the `unused_variables` lint, and it tells you that you've introduced a variable that you don't use in your code. That's not *wrong*, so it's not an error, but it might be a bug, so you get a warning.

Future-incompatible lints

Sometimes the compiler needs to be changed to fix an issue that can cause existing code to stop compiling. "Future-incompatible" lints are issued in these cases to give users of Rust a smooth transition to the new behavior. Initially, the compiler will continue to accept the problematic code and issue a warning. The warning has a description of the problem, a notice that this will become an error in the future, and a link to a tracking issue that provides detailed information and an opportunity for feedback. This gives users some time to fix the code to accommodate the change. After some time, the warning may become an error.

The following is an example of what a future-incompatible looks like:

```
warning: borrow of packed field is unsafe and requires unsafe function or block
(error E0133)
--> lint_example.rs:11:13
   |
11 |     let y = &x.data.0;
   |               ^^^^^^^^^
   |
= note: `[warn(safe_packed_borrows)]` on by default
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #46043 <https://github.com/rust-
lang/rust/issues/46043>
= note: fields of packed structs might be misaligned: dereferencing a
misaligned pointer or even just creating a misaligned reference is undefined
behavior
```

For more information about the process and policy of future-incompatible changes, see [RFC 1589](https://rust-lang.org/rfcs/1589).

Lint Levels

In `rustc`, lints are divided into six *levels*:

1. allow
2. expect
3. warn
4. force-warn
5. deny
6. forbid

Each lint has a default level (explained in the lint listing later in this chapter), and the compiler has a default warning level. First, let's explain what these levels mean, and then we'll talk about configuration.

allow

These lints exist, but by default, do nothing. For example, consider this source:

```
pub fn foo() {}
```

Compiling this file produces no warnings:

```
$ rustc lib.rs --crate-type=lib
$
```

But this code violates the `missing_docs` lint.

These lints exist mostly to be manually turned on via configuration, as we'll talk about later in this section.

expect

Sometimes, it can be helpful to suppress lints, but at the same time ensure that the code in question still emits them. The 'expect' level does exactly this. If the lint in question is not emitted, the `unfulfilled_lint_expectation` lint triggers on the `expect` attribute, notifying you that the expectation is no longer fulfilled.

```
fn main() {
    #[expect(unused_variables)]
    let unused = "Everyone ignores me";

    #[expect(unused_variables)] // `unused_variables` lint is not emitted
    let used = "I'm useful";    // the expectation is therefore unfulfilled
    println!("The `used` value is equal to: {:?}", used);
}
```

This will produce the following warning:

```
warning: this lint expectation is unfulfilled
--> src/main.rs:7:14
   |
7  |     #[expect(unused_variables)]
   |               ^^^^^^^^^^^^^^^^^
   |
= note: `#[warn(unfulfilled_lint_expectations)]` on by default
```

This level can only be defined via the `#[expect]` attribute, there is no equivalent flag. Lints with the special 'force-warn' level will still be emitted as usual.

warn

The 'warn' lint level will produce a warning if you violate the lint. For example, this code runs afoul of the `unused_variables` lint:

```
pub fn foo() {
    let x = 5;
}
```

This will produce this warning:

```
$ rustc lib.rs --crate-type=lib
warning: unused variable: `x`
--> lib.rs:2:9
  |
2 |     let x = 5;
  |         ^
= note: `#[warn(unused_variables)]` on by default
= note: to avoid this warning, consider using `_x` instead
```

force-warn

'force-warn' is a special lint level. It's the same as 'warn' in that a lint at this level will produce a warning, but unlike the 'warn' level, the 'force-warn' level cannot be overridden. If a lint is set to 'force-warn', it is guaranteed to warn: no more, no less. This is true even if the overall lint level is capped via cap-lints.

deny

A 'deny' lint produces an error if you violate it. For example, this code runs into the `exceeding_bitshifts` lint.

```
fn main() {
    100u8 << 10;
}

$ rustc main.rs
error: bitshift exceeds the type's number of bits
--> main.rs:2:13
  |
2 |     100u8 << 10;
  |     ^^^^^^^^^^^
= note: `#[deny(exceeding_bitshifts)]` on by default
```

What's the difference between an error from a lint and a regular old error? Lints are

configurable via levels, so in a similar way to 'allow' lints, warnings that are 'deny' by default let you allow them. Similarly, you may wish to set up a lint that is `warn` by default to produce an error instead. This lint level gives you that.

forbid

'forbid' is a special lint level that fills the same role for 'deny' that 'force-warn' does for 'warn'. It's the same as 'deny' in that a lint at this level will produce an error, but unlike the 'deny' level, the 'forbid' level can not be overridden to be anything lower than an error. However, lint levels may still be capped with `--cap-lints` (see below) so `rustc --cap-lints warn` will make lints set to 'forbid' just warn.

Configuring warning levels

Remember our `missing_docs` example from the 'allow' lint level?

```
$ cat lib.rs
pub fn foo() {}
$ rustc lib.rs --crate-type=lib
$
```

We can configure this lint to operate at a higher level, both with compiler flags, as well as with an attribute in the source code.

You can also "cap" lints so that the compiler can choose to ignore certain lint levels. We'll talk about that last.

Via compiler flag

The `-A`, `-W`, `--force-warn`, `-D`, and `-F` flags let you turn one or more lints into allowed, warning, force-warn, deny, or forbid levels, like this:

```
$ rustc lib.rs --crate-type=lib -W missing-docs
warning: missing documentation for crate
--> lib.rs:1:1
|
1 | pub fn foo() {}
|   ^^^^^^^^^^^^^^^
|
= note: requested on the command line with `-W missing-docs`
```

```
warning: missing documentation for a function
--> lib.rs:1:1
|
1 | pub fn foo() {}
|   ^^^^^^^^^^^^^^^
```

```
$ rustc lib.rs --crate-type=lib -D missing-docs
error: missing documentation for crate
--> lib.rs:1:1
|
1 | pub fn foo() {}
|   ^^^^^^^^^^^^^^^
|
= note: requested on the command line with `-D missing-docs`
```

```
error: missing documentation for a function
--> lib.rs:1:1
|
1 | pub fn foo() {}
|   ^^^^^^^^^^^^^^^
```

```
error: aborting due to 2 previous errors
```

You can also pass each flag more than once for changing multiple lints:

```
$ rustc lib.rs --crate-type=lib -D missing-docs -D unused-variables
```

And of course, you can mix these five flags together:

```
$ rustc lib.rs --crate-type=lib -D missing-docs -A unused-variables
```

The order of these command line arguments is taken into account. The following allows the `unused-variables` lint, because it is the last argument for that lint:

```
$ rustc lib.rs --crate-type=lib -D unused-variables -A unused-variables
```

You can make use of this behavior by overriding the level of one specific lint out of a group of lints. The following example denies all the lints in the `unused` group, but explicitly allows the `unused-variables` lint in that group (forbid still trumps everything regardless of ordering):

```
$ rustc lib.rs --crate-type=lib -D unused -A unused-variables
```

Since `force-warn` and `forbid` cannot be overridden, setting one of them will prevent any later level for the same lint from taking effect.

Via an attribute

You can also modify the lint level with a crate-wide attribute:

```
$ cat lib.rs
#![warn(missing_docs)]

pub fn foo() {}

$ rustc lib.rs --crate-type=lib
warning: missing documentation for crate
--> lib.rs:1:1
   |
1 | / #[warn(missing_docs)]
2 | |
3 | | pub fn foo() {}
   | | _____^
   | |
note: lint level defined here
--> lib.rs:1:9
   |
1 | #[warn(missing_docs)]
   |          ^^^^^^^^^^^^^
warning: missing documentation for a function
--> lib.rs:3:1
   |
3 | pub fn foo() {}
   |          ^^^^^^^^^^^^^
```

`warn`, `allow`, `deny`, and `forbid` all work this way. There is no way to set a lint to `force-warn` using an attribute.

You can also pass in multiple lints per attribute:

```
#![warn(missing_docs, unused_variables)]

pub fn foo() {}
```

And use multiple attributes together:

```
#![warn(missing_docs)]
#![deny(unused_variables)]

pub fn foo() {}
```

All lint attributes support an additional `reason` parameter, to give context why a certain attribute was added. This reason will be displayed as part of the lint message, if the lint is emitted at the defined level.

```
use std::path::PathBuf;
pub fn get_path() -> PathBuf {
    #[allow(unused_mut, reason = "this is only modified on some platforms")]
    let mut file_name = PathBuf::from("git");
    #[cfg(target_os = "windows")]
    file_name.set_extension("exe");
    file_name
}
```

Capping lints

`rustc` supports a flag, `--cap-lints LEVEL` that sets the "lint cap level." This is the maximum level for all lints. So for example, if we take our code sample from the "deny" lint level above:

```
fn main() {
    100u8 << 10;
}
```

And we compile it, capping lints to warn:

```
$ rustc lib.rs --cap-lints warn
warning: bitshift exceeds the type's number of bits
--> lib.rs:2:5
   |
2  |     100u8 << 10;
   |     ^^^^^^^^^^^
   = note: `[warn(exceeding_bitshifts)]` on by default

warning: this expression will panic at run-time
--> lib.rs:2:5
   |
2  |     100u8 << 10;
   |     ^^^^^^^^^^^ attempt to shift left with overflow
```

It now only warns, rather than errors. We can go further and allow all lints:

```
$ rustc lib.rs --cap-lints allow
$
```

This feature is used heavily by Cargo; it will pass `--cap-lints allow` when compiling your dependencies, so that if they have any warnings, they do not pollute the output of your build.

Lint Groups

rustc has the concept of a "lint group", where you can toggle several warnings through one name.

For example, the `nonstandard-style` lint sets `non-camel-case-types`, `non-snake-case`, and `non-upper-case-globals` all at once. So these are equivalent:

```
$ rustc -D nonstandard-style
$ rustc -D non-camel-case-types -D non-snake-case -D non-upper-case-globals
```

Here's a list of each lint group, and the lints that they are made up of:

Group	Description	Lints
warnings	All lints that are set to issue warnings	See warn-by-default for the default set of warnings
future-incompatible	Lints that detect code that has future-compatibility problems	ambiguous-associated-items , ambiguous-glob-imports , byte-slice-in-packed-struct-with-derive , cenum-impl-drop-cast , coherence-leak-check , conflicting-repr-hints , const-eval-mutable-ptr-in-final-value , const-evaluatable-unchecked , dependency-on-unit-never-type-fallback , deprecated-cfg-attr-crate-type-name , deref-into-dyn-supertrait , elided-lifetimes-in-associated-constant , forbidden-lint-groups , ill-formed-attribute-input , invalid-type-param-default , late-bound-lifetime-arguments , legacy-derive-helpers , macro-expanded-macro-exports-accessed-by-absolute-paths , missing-fragment-specifier , never-type-fallback-flowing-into-unsafe , order-dependent-trait-objects , out-of-scope-macro-calls , patterns-in-fns-without-body , proc-macro-derive-resolution-fallback , ptr-cast-add-auto-to-object , pub-use-of-private-extern-crate , repr-

		transparent-external-private-fields, self- constructor-from-outer-item, semicolon-in- expressions-from-macros, soft-unstable, uncovered-param-in-projection, uninhabited- static, unstable-name-collisions, unstable- syntax-pre-expansion, unsupported-calling- conventions, wasm-c-abi, writes-through- immutable-pointer
keyword- idents	Lints that detect identifiers which will be come keywords in later editions	keyword-idents-2018, keyword-idents-2024
let- underscore	Lints that detect wildcard let bindings that are likely to be invalid	let-underscore-drop, let-underscore-lock
nonstandard- style	Violation of standard naming conventions	non-camel-case-types, non-snake-case, non- upper-case-globals
refining-impl- trait	Detects refinement of <code>impl Trait</code> return types by trait implementations	refining-impl-trait-internal, refining-impl- trait-reachable
rust-2018- compatibility	Lints used to transition code from the 2015 edition to 2018	absolute-paths-not-starting-with-crate, anonymous-parameters, keyword-idents- 2018, tyvar-behind-raw-pointer
rust-2018- idioms	Lints to nudge you toward idiomatic features of Rust 2018	bare-trait-objects, elided-lifetimes-in-paths, ellipsis-inclusive-range-patterns, explicit- outlives-requirements, unused-extern-crates

rust-2021-compatibility	Lints used to transition code from the 2018 edition to 2021	array-into-iter, bare-trait-objects, ellipsis-inclusive-range-patterns, non-fmt-panics, rust-2021-incompatible-closure-captures, rust-2021-incompatible-or-patterns, rust-2021-prefixes-incompatible-syntax, rust-2021-prelude-collisions
rust-2024-compatibility	Lints used to transition code from the 2021 edition to 2024	boxed-slice-into-iter, deprecated-safe, edition-2024-expr-fragment-specifier, keyword-idents-2024, missing-unsafe-on-extern, static-mut-refs, unsafe-attr-outside-unsafe, unsafe-op-in-unsafe-fn
unused	Lints that detect things being declared but not used, or excess syntax	dead-code, map-unit-fn, path-statements, redundant-semicolon, unreachable-code, unreachable-patterns, unused-allocation, unused-assignments, unused-attributes, unused-braces, unused-doc-comments, unused-extern-crates, unused-features, unused-imports, unused-labels, unused-macro-rules, unused-macros, unused-must-use, unused-mut, unused-parens, unused-unsafe, unused-variables

Additionally, there's a `bad-style` lint group that's a deprecated alias for `nonstandard-style`.

Finally, you can also see the table above by invoking `rustc -W help`. This will give you the exact values for the specific compiler you have installed.

Lint Listing

This section lists out all of the lints, grouped by their default lint levels.

You can also see this list by running `rustc -W help`.

Allowed-by-default Lints

These lints are all set to the 'allow' level by default. As such, they won't show up unless you set them to a higher lint level with a flag or attribute.

- `absolute_paths_not_starting_with_crate`
- `async-idents`
- `closure_returning_async_block`
- `deprecated_safe`
- `disjoint-capture-migration`
- `edition_2024_expr_fragment_specifier`
- `elided-lifetime-in-path`
- `elided_lifetimes_in_paths`
- `explicit_outlives_requirements`
- `ffi_unwind_calls`
- `fuzzy_provenance_casts`
- `impl_trait_overcaptures`
- `keyword-idents`
- `keyword_idents_2018`
- `keyword_idents_2024`
- `let_underscore_drop`
- `lossy_provenance_casts`
- `macro_use_extern_crate`
- `meta_variable_misuse`
- `missing_abi`
- `missing_copy_implementations`
- `missing_debug_implementations`
- `missing_docs`
- `missing_unsafe_on_extern`
- `multiple_supertrait_upcastable`
- `must_not_suspend`
- `non_ascii_idents`
- `non_exhaustive_omitted_patterns`
- `non_local_definitions`
- `or-patterns-back-compat`

- `redundant_lifetimes`
- `rust_2021_incompatible_closure_captures`
- `rust_2021_incompatible_or_patterns`
- `rust_2021_prefixes_incompatible_syntax`
- `rust_2021_prelude_collisions`
- `rust_2024_incompatible_pat`
- `single-use-lifetime`
- `single_use_lifetimes`
- `trivial_casts`
- `trivial_numeric_casts`
- `unit_bindings`
- `unnameable_types`
- `unreachable_pub`
- `unsafe_attr_outside_unsafe`
- `unsafe_code`
- `unsafe_op_in_unsafe_fn`
- `unstable_features`
- `unused_crate_dependencies`
- `unused_extern_crates`
- `unused_import_braces`
- `unused_lifetimes`
- `unused_macro_rules`
- `unused_qualifications`
- `unused_results`
- `variant_size_differences`

absolute-paths-not-starting-with-crate

The `absolute_paths_not_starting_with_crate` lint detects fully qualified paths that start with a module name instead of `crate`, `self`, or an extern crate name

Example

```
#![deny(absolute_paths_not_starting_with_crate)]

mod foo {
    pub fn bar() {}
}

fn main() {
    ::foo::bar();
}
```

This will produce:

```
error: absolute paths must start with `self`, `super`, `crate`, or an external
crate name in the 2018 edition
--> lint_example.rs:8:5
   |
8  |     ::foo::bar();
   |     ^^^^^^^^^^^^^ help: use `crate`: `crate::foo::bar`
   |
= warning: this is accepted in the current edition (Rust 2015) but is a hard
error in Rust 2018!
= note: for more information, see issue #53130 <https://github.com/rust-
lang/rust/issues/53130>
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(absolute_paths_not_starting_with_crate)]
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

Rust [editions](#) allow the language to evolve without breaking backwards compatibility. This lint catches code that uses absolute paths in the style of the 2015 edition. In the 2015 edition, absolute paths (those starting with `::`) refer to either the crate root or an external crate. In the 2018 edition it was changed so that they only refer to external crates. The path prefix `crate::` should be used instead to reference items from the crate root.

If you switch the compiler from the 2015 to 2018 edition without updating the code, then it will fail to compile if the old style paths are used. You can manually change the paths to use the `crate::` prefix to transition to the 2018 edition.

This lint solves the problem automatically. It is "allow" by default because the code is perfectly valid in the 2015 edition. The `cargo fix` tool with the `--edition` flag will switch this lint to "warn" and automatically apply the suggested fix from the compiler. This provides a completely automated way to update old code to the 2018 edition.

async-idents

The lint `async-idents` has been renamed to `keyword-idents`.

closure-returning-async-block

The `closure_returning_async_block` lint detects cases where users write a closure that returns an async block.

Example

```
#![warn(closure_returning_async_block)]  
let c = |x: &str| async {};
```

This will produce:


```
warning: unknown lint: `closure_returning_async_block`
--> lint_example.rs:1:1
   |
1  | #![warn(closure_returning_async_block)]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
= note: the `closure_returning_async_block` lint is unstable
= note: see issue #62290 <https://github.com/rust-lang/rust/issues/62290> for
more information
= help: add `#![feature(async_closure)]` to the crate attributes to enable
= note: this compiler was built on 2024-07-25; consider upgrading it if it is
out of date
= note: `#![warn(unknown_lints)]` on by default
```

Explanation

Using an async closure is preferable over a closure that returns an async block, since async closures are less restrictive in how its captures are allowed to be used.

For example, this code does not work with a closure returning an async block:

```
async fn callback(x: &str) {}

let captured_str = String::new();
let c = move || async {
    callback(&captured_str).await;
};
```

But it does work with async closures:

```
#![feature(async_closure)]

async fn callback(x: &str) {}

let captured_str = String::new();
let c = async move || {
    callback(&captured_str).await;
};
```

The `deprecated_safe` lint detects unsafe functions being used as safe functions.

Example

```
#![deny(deprecated_safe)]
// edition 2021
use std::env;
fn enable_backtrace() {
    env::set_var("RUST_BACKTRACE", "1");
}
```

This will produce:

```
error: call to deprecated safe function `std::env::set_var` is unsafe and requires unsafe block  
--> lint_example.rs:6:5  
|  
6 |     env::set_var("RUST_BACKTRACE", "1");  
|       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ call to unsafe function  
  
= warning: this is accepted in the current edition (Rust 2021) but is a hard error in Rust 2024!  
= note: for more information, see issue #27970 <https://github.com/rust-lang/rust/issues/27970>  
note: the lint level is defined here  
--> lint_example.rs:1:9  
|  
1 | #[deny(deprecated_safe)]  
|   ^^^^^^^^^^^^^^^^^^^^^  
  
help: you can wrap the call in an `unsafe` block if you can guarantee the code is only ever called from single-threaded code  
|  
6 + // TODO: Audit that the environment access only happens in single-  
threaded code.  
7 ~ unsafe { env::set_var("RUST_BACKTRACE", "1") };  
|
```

Explanation

Rust [editions](#) allow the language to evolve without breaking backward compatibility. This lint catches code that uses `unsafe` functions that were declared as `safe` (`non-unsafe`) in earlier editions. If you switch the compiler to a new edition without updating the code, then it will fail to compile if you are using a function previously marked as `safe`.

You can audit the code to see if it suffices the preconditions of the `unsafe` code, and if it does, you can wrap it in an `unsafe` block. If you can't fulfill the preconditions, you probably need to switch to a different way of doing what you want to achieve.

This lint can automatically wrap the calls in `unsafe` blocks, but this obviously cannot verify that the preconditions of the `unsafe` functions are fulfilled, so that is still up to the user.

The lint is currently "allow" by default, but that might change in the future.

disjoint-capture-migration

The lint `disjoint-capture-migration` has been renamed to [rust-2021-incompatible-closure-captures](#).

edition-2024-expr-fragment-specifier

The `edition_2024_expr_fragment_specifier` lint detects the use of `expr` fragments in macros during migration to the 2024 edition.

The `expr` fragment specifier will accept more expressions in the 2024 edition. To maintain the behavior from the 2021 edition and earlier, use the `expr_2021` fragment specifier.

Example

```

#![deny(edition_2024_expr_fragment_specifier)]
macro_rules! m {
    ($e:expr) => {
        $e
    }
}

fn main() {
    m!(1);
}

```

This will produce:

```

error: the `expr` fragment specifier will accept more expressions in the 2024
edition
--> lint_example.rs:3:7
   |
3  |     ($e:expr) => {
   |           ^^^^
   |
= warning: this changes meaning in Rust 2024
= note: for more information, see Migration Guide <https://doc.rust-
lang.org/nightly/edition-guide/rust-2024/macro-fragment-specifiers.html>
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(edition_2024_expr_fragment_specifier)]
   |           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: to keep the existing behavior, use the `expr_2021` fragment specifier
   |
3  |     ($e:expr_2021) => {
   |           ~~~~~

```

Explanation

Rust [editions](#) allow the language to evolve without breaking backwards compatibility. This lint catches code that uses [macro matcher fragment specifiers](#) that have changed meaning in the 2024 edition. If you switch to the new edition without updating the code, your macros may behave differently.

In the 2024 edition, the `expr` fragment specifier `expr` will also match `const { ... }` blocks. This means if a macro had a pattern that matched `$e:expr` and another that matches `const { $e: expr }`, for example, that under the 2024 edition the first pattern would match while in the 2021 and earlier editions the second pattern would match. To keep the old behavior, use the `expr_2021` fragment specifier.

This lint detects macros whose behavior might change due to the changing meaning of the `expr` fragment specifier. It is "allow" by default because the code is perfectly valid in older editions. The `cargo fix` tool with the `--edition` flag will switch this lint to "warn" and automatically apply the suggested fix from the compiler. This provides a completely automated way to update old code for a new edition.

Using `cargo fix --edition` with this lint will ensure that your code retains the same behavior. This may not be the desired, as macro authors often will want their macros to use the latest grammar for matching expressions. Be sure to carefully review changes introduced by this lint to ensure the macros implement the desired behavior.

elided-lifetime-in-path

The lint `elided-lifetime-in-path` has been renamed to `elided-lifetimes-in-paths`.

elided-lifetimes-in-paths

The `elided_lifetimes_in_paths` lint detects the use of hidden lifetime parameters.

Example

```

#![deny(elided_lifetimes_in_paths)]
#![deny(warnings)]
struct Foo<'a> {
    x: &'a u32
}

fn foo(x: &Foo) {
}

```

This will produce:

```

error: hidden lifetime parameters in types are deprecated
--> lint_example.rs:8:12
   |
8  | fn foo(x: &Foo) {
   |             ^^^ expected lifetime parameter
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(elided_lifetimes_in_paths)]
   |             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: indicate the anonymous lifetime
   |
8  | fn foo(x: &Foo<'_>) {
   |             ++++

```

Explanation

Elided lifetime parameters can make it difficult to see at a glance that borrowing is occurring. This lint ensures that lifetime parameters are always explicitly stated, even if it is the `'_placeholder lifetime`.

This lint is "allow" by default because it has some known issues, and may require a significant transition for old code.

explicit-outlives-requirements

The `explicit_outlives_requirements` lint detects unnecessary lifetime bounds that can be inferred.

Example

```
#![deny(explicit_outlives_requirements)]
#![deny(warnings)]

struct SharedRef<'a, T>
where
    T: 'a,
{
    data: &'a T,
}
```

This will produce:

```
error: outlives requirements can be inferred
--> lint_example.rs:6:24
   |
 6 |     struct SharedRef<'a, T>
   |     ^
 7 | | where
 8 | |     T: 'a,
   | |-----^ help: remove this bound
   |
note: the lint level is defined here
--> lint_example.rs:2:9
   |
 2 | #![deny(explicit_outlives_requirements)]
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

If a `struct` contains a reference, such as `&'a T`, the compiler requires that `T` outlives the lifetime `'a`. This historically required writing an explicit lifetime bound to indicate this requirement. However, this can be overly explicit, causing clutter and unnecessary complexity. The language was changed to automatically infer the bound if it is not specified.

Specifically, if the struct contains a reference, directly or indirectly, to τ with lifetime $'x$, then it will infer that $\tau: 'x$ is a requirement.

This lint is "allow" by default because it can be noisy for existing code that already had these requirements. This is a stylistic choice, as it is still valid to explicitly state the bound. It also has some false positives that can cause confusion.

See [RFC 2093](#) for more details.

ffi-unwind-calls

The `ffi_unwind_calls` lint detects calls to foreign functions or function pointers with `c-unwind` or other FFI-unwind ABIs.

Example

```
#![warn(ffi_unwind_calls)]

extern "C-unwind" {
    fn foo();
}

fn bar() {
    unsafe { foo(); }
    let ptr: unsafe extern "C-unwind" fn() = foo;
    unsafe { ptr(); }
}
```

This will produce:


```

warning: call to foreign function with FFI-unwind ABI
--> lint_example.rs:9:14
  |
9 |     unsafe { foo(); }
  |               ^^^^^ call to foreign function with FFI-unwind ABI
note: the lint level is defined here
--> lint_example.rs:1:9
  |
1 | #![warn(ffi_unwind_calls)]
  |               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```



```

warning: call to function pointer with FFI-unwind ABI
--> lint_example.rs:11:14
  |
11 |     unsafe { ptr(); }
  |               ^^^^^ call to function pointer with FFI-unwind ABI

```

Explanation

For crates containing such calls, if they are compiled with `-C panic=unwind` then the produced library cannot be linked with crates compiled with `-C panic=abort`. For crates that desire this ability it is therefore necessary to avoid such calls.

fuzzy-provenance-casts

The `fuzzy_provenance_casts` lint detects an `as` cast between an integer and a pointer.

Example

```
#![feature(strict_provenance)]
#![warn(fuzzy_provenance_casts)]

fn main() {
    let _dangling = 16_usize as *const u8;
}
```

This will produce:

```
warning: strict provenance disallows casting integer `usize` to pointer `*const u8`
--> lint_example.rs:5:21
   |
5 |     let _dangling = 16_usize as *const u8;
   |                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   = help: if you can't comply with strict provenance and don't have a pointer
with the correct provenance you can use `std::ptr::with_exposed_provenance()`
instead
note: the lint level is defined here
--> lint_example.rs:2:9
   |
2 |     #[warn(fuzzy_provenance_casts)]
   |               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: use `.with_addr()` to adjust a valid pointer in the same allocation, to
this address
5 |     let _dangling = (...).with_addr(16_usize);
   |                               +++++~
```

Explanation

This lint is part of the strict provenance effort, see [issue #95228](#). Casting an integer to a pointer is considered bad style, as a pointer contains, besides the *address* also a *provenance*, indicating what memory the pointer is allowed to read/write. Casting an integer, which doesn't have provenance, to a pointer requires the compiler to assign (guess) provenance. The compiler assigns "all exposed valid" (see the docs of [ptr::with_exposed_provenance](#) for more information about this "exposing"). This penalizes the optimiser and is not well suited for dynamic analysis/dynamic program verification (e.g. Miri or CHERI platforms).

It is much better to use `ptr::with_addr` instead to specify the provenance you want. If using this function is not possible because the code relies on exposed provenance then there is as an escape hatch `ptr::with_exposed_provenance`.

impl-trait-overcaptures

The `impl_trait_overcaptures` lint warns against cases where lifetime capture behavior will differ in edition 2024.

In the 2024 edition, `impl Trait`s will capture all lifetimes in scope, rather than just the lifetimes that are mentioned in the bounds of the type. Often these sets are equal, but if not, it means that the `impl Trait` may cause erroneous borrow-checker errors.

Example

```
let mut x = vec![];
x.push(1);

fn test(x: &Vec<i32>) -> impl Display {
    x[0]
}

let element = test(&x);
x.push(2);
println!("{element}");
```

This will produce:

```

error: `impl std::fmt::Display` will capture more lifetimes than possibly
intended in edition 2024
--> lint_example.rs:9:26
   |
 9 | fn test(x: &Vec<i32>) -> impl Display {
   |                               ^^^^^^^^^^^^^^^^^^^
note: specifically, this lifetime is in scope but not mentioned in the type's
bounds
--> lint_example.rs:9:12
   |
 9 | fn test(x: &Vec<i32>) -> impl Display {
   |               ^
   = note: all lifetimes in scope will be captured by `impl Trait`s in edition
2024
note: the lint level is defined here
--> lint_example.rs:3:9
   |
 3 | #![deny(impl_trait_overcaptures)]
   |               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: use the precise capturing `use<...>` syntax to make the captures explicit
   |
 9 | fn test(x: &Vec<i32>) -> impl Display + use<> {
   |                               ++++++++

```

Explanation

In edition < 2024, the returned `impl Display` doesn't capture the lifetime from the `&Vec<i32>`, so the vector can be mutably borrowed while the `impl Display` is live.

To fix this, we can explicitly state that the `impl Display` doesn't capture any lifetimes, using `impl Display + use<>`.

keyword-idents

The lint `keyword-idents` has been renamed to [keyword-idents-2018](#).

keyword-idents-2018

The `keyword_idents_2018` lint detects edition keywords being used as an identifier.

Example

```
#![deny(keyword_idents_2018)]
// edition 2015
fn dyn() {}
```

This will produce:

```
error: `dyn` is a keyword in the 2018 edition
--> lint_example.rs:4:4
   |
4 | fn dyn() {}
   |    ^^^ help: you can use a raw identifier to stay compatible: `r#dyn`
   |
= warning: this is accepted in the current edition (Rust 2015) but is a hard
error in Rust 2018!
= note: for more information, see issue #49716 <https://github.com/rust-
lang/rust/issues/49716>
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1 | #![deny(keyword_idents_2018)]
   |    ^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

Rust [editions](#) allow the language to evolve without breaking backwards compatibility. This lint catches code that uses new keywords that are added to the language that are used as identifiers (such as a variable name, function name, etc.). If you switch the compiler to a new edition without updating the code, then it will fail to compile if you are using a new keyword as an identifier.

You can manually change the identifiers to a non-keyword, or use a [raw identifier](#), for

example `r#dyn`, to transition to a new edition.

This lint solves the problem automatically. It is "allow" by default because the code is perfectly valid in older editions. The `cargo fix` tool with the `--edition` flag will switch this lint to "warn" and automatically apply the suggested fix from the compiler (which is to use a raw identifier). This provides a completely automated way to update old code for a new edition.

keyword-idents-2024

The `keyword_idents_2024` lint detects edition keywords being used as an identifier.

Example

```
#![deny(keyword_idents_2024)]
// edition 2015
fn gen() {}
```

This will produce:

```
error: `gen` is a keyword in the 2024 edition
--> lint_example.rs:4:4
   |
4 | fn gen() {}
   |    ^^^ help: you can use a raw identifier to stay compatible: `r#gen`
   = warning: this is accepted in the current edition (Rust 2015) but is a hard
error in Rust 2024!
   = note: for more information, see issue #49716 <https://github.com/rust-
lang/rust/issues/49716>
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1 | #![deny(keyword_idents_2024)]
   |    ^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

Rust [editions](#) allow the language to evolve without breaking backwards compatibility. This lint catches code that uses new keywords that are added to the language that are used as identifiers (such as a variable name, function name, etc.). If you switch the compiler to a new edition without updating the code, then it will fail to compile if you are using a new keyword as an identifier.

You can manually change the identifiers to a non-keyword, or use a [raw identifier](#), for example `r#gen`, to transition to a new edition.

This lint solves the problem automatically. It is "allow" by default because the code is perfectly valid in older editions. The [cargo fix](#) tool with the `--edition` flag will switch this lint to "warn" and automatically apply the suggested fix from the compiler (which is to use a raw identifier). This provides a completely automated way to update old code for a new edition.

let-underscore-drop

The `let_underscore_drop` lint checks for statements which don't bind an expression which has a non-trivial `Drop` implementation to anything, causing the expression to be dropped immediately instead of at end of scope.

Example

```

struct SomeStruct;
impl Drop for SomeStruct {
    fn drop(&mut self) {
        println!("Dropping SomeStruct");
    }
}

fn main() {
    #[warn(let_underscore_drop)]
    // SomeStruct is dropped immediately instead of at end of scope,
    // so "Dropping SomeStruct" is printed before "end of main".
    // The order of prints would be reversed if SomeStruct was bound to
    // a name (such as "_foo").
    let _ = SomeStruct;
    println!("end of main");
}

```

This will produce:

```

warning: non-binding let on a type that implements `Drop`
--> lint_example.rs:14:5
   |
14 |     let _ = SomeStruct;
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
note: the lint level is defined here
--> lint_example.rs:9:11
   |
 9 |     #[warn(let_underscore_drop)]
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: consider binding to an unused variable to avoid immediately dropping the
value
   |
14 |     let _unused = SomeStruct;
   |     ~~~~~
help: consider immediately dropping the value
   |
14 |     drop(SomeStruct);
   |     ~~~~~+

```

Explanation

Statements which assign an expression to an underscore causes the expression to immediately drop instead of extending the expression's lifetime to the end of the scope. This is usually unintended, especially for types like `MutexGuard`, which are typically used to lock a mutex for the duration of an entire scope.

If you want to extend the expression's lifetime to the end of the scope, assign an underscore-prefixed name (such as `_foo`) to the expression. If you do actually want to drop the expression immediately, then calling `std::mem::drop` on the expression is clearer and helps convey intent.

lossy-provenance-casts

The `lossy_provenance_casts` lint detects an `as` cast between a pointer and an integer.

Example

```
#![feature(strict_provenance)]
#![warn(lossy_provenance_casts)]

fn main() {
    let x: u8 = 37;
    let _addr: usize = &x as *const u8 as usize;
}
```

This will produce:

```
warning: under strict provenance it is considered bad style to cast pointer
`*const u8` to integer `usize`
--> lint_example.rs:6:24
6 |         let _addr: usize = &x as *const u8 as usize;
  |                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
= help: if you can't comply with strict provenance and need to expose the
pointer provenance you can use `.expose_provenance()` instead
note: the lint level is defined here
--> lint_example.rs:2:9
2 |     #![warn(lossy_provenance_casts)]
  |           ^^^^^^^^^^^^^^^^^^^^^^^^^
help: use `.addr()` to obtain the address of a pointer
6 |         let _addr: usize = (&x as *const u8).addr();
  |                               +             ~~~~~
```

Explanation

This lint is part of the strict provenance effort, see [issue #95228](#). Casting a pointer to an integer is a lossy operation, because beyond just an *address* a pointer may be associated with a particular *provenance*. This information is used by the optimiser and for dynamic analysis/dynamic program verification (e.g. Miri or CHERI platforms).

Since this cast is lossy, it is considered good style to use the `ptr::addr` method instead, which has a similar effect, but doesn't "expose" the pointer provenance. This improves optimisation potential. See the docs of `ptr::addr` and `ptr::expose_provenance` for more information about exposing pointer provenance.

If your code can't comply with strict provenance and needs to expose the provenance, then there is `ptr::expose_provenance` as an escape hatch, which preserves the behaviour of `as usize` casts while being explicit about the semantics.

macro-use-extern-crate

The `macro_use_extern_crate` lint detects the use of the [macro_use attribute](#).

Example

```
#![deny(macro_use_extern_crate)]

#[macro_use]
extern crate serde_json;

fn main() {
    let _ = json!{{}};
}
```

This will produce:

```
error: applying the `#[macro_use]` attribute to an `extern crate` item is
deprecated
--> src/main.rs:3:1
   |
3  | #[macro_use]
   | ^^^^^^^^^^^^^^^^^
   |
   = help: remove it and import macros at use sites with a `use` item instead
note: the lint level is defined here
--> src/main.rs:1:9
   |
1  | #![deny(macro_use_extern_crate)]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

The [macro_use attribute](#) on an [extern crate](#) item causes macros in that external crate to be brought into the prelude of the crate, making the macros in scope everywhere. As part of the efforts to simplify handling of dependencies in the [2018 edition](#), the use of `extern crate` is being phased out. To bring macros from extern crates into scope, it is recommended to use a [use import](#).

This lint is "allow" by default because this is a stylistic choice that has not been settled, see

[issue #52043](#) for more information.

meta-variable-misuse

The `meta_variable_misuse` lint detects possible meta-variable misuse in macro definitions.

Example

```
#![deny(meta_variable_misuse)]

macro_rules! foo {
    () => {};
    $($i:ident = $($j:ident),+ );* => { $( $( $i = $k; )+ )* };
}

fn main() {
    foo!();
}
```

This will produce:

```
error: unknown macro variable `k`
--> lint_example.rs:5:55
   |
5  |     $($i:ident = $($j:ident),+ );* => { $( $( $i = $k; )+ )* };
   |                                     ^^
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(meta_variable_misuse)]
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

There are quite a few different ways a `macro_rules` macro can be improperly defined. Many

of these errors were previously only detected when the macro was expanded or not at all. This lint is an attempt to catch some of these problems when the macro is *defined*.

This lint is "allow" by default because it may have false positives and other issues. See [issue #61053](#) for more details.

missing-abi

The `missing_abi` lint detects cases where the ABI is omitted from extern declarations.

Example

```
#![deny(missing_abi)]

extern fn foo() {}
```

This will produce:

```
error: extern declarations without an explicit ABI are deprecated
--> lint_example.rs:4:1
   |
4 | extern fn foo() {}
   | ^^^^^^^^^^^^^^^^^^ ABI should be specified here
   |
   = help: the default ABI is C
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1 | #![deny(missing_abi)]
   | ^^^^^^^^^^^^^^^^^^
```

Explanation

Historically, Rust implicitly selected C as the ABI for extern declarations. We expect to add new ABIs, like `C-unwind`, in the future, though this has not yet happened, and especially

with their addition seeing the ABI easily will make code review easier.

missing-copy-implementations

The `missing_copy_implementations` lint detects potentially-forgotten implementations of `Copy` for public types.

Example

```
#![deny(missing_copy_implementations)]
pub struct Foo {
    pub field: i32
}
```

This will produce:

```
error: type could implement `Copy`; consider adding `impl Copy`
--> lint_example.rs:2:1
  |
2 | / pub struct Foo {
3 | |     pub field: i32
4 | | }
  | | ^
  | |_-
  |
note: the lint level is defined here
--> lint_example.rs:1:9
1 | #[deny(missing_copy_implementations)]
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

Historically (before 1.0), types were automatically marked as `Copy` if possible. This was changed so that it required an explicit opt-in by implementing the `Copy` trait. As part of this change, a lint was added to alert if a copyable type was not marked `Copy`.

This lint is "allow" by default because this code isn't bad; it is common to write newtypes like this specifically so that a `Copy` type is no longer `Copy`. `Copy` types can result in unintended copies of large data which can impact performance.

missing-debug-implementations

The `missing_debug_implementations` lint detects missing implementations of `fmt::Debug` for public types.

Example

```
#![deny(missing_debug_implementations)]
pub struct Foo;
```

This will produce:

```
error: type does not implement `Debug`; consider adding `#[derive(Debug)]` or a
manual implementation
--> lint_example.rs:2:1
  |
2 | pub struct Foo;
  | ^^^^^^^^^^^^^^^^^
  |
note: the lint level is defined here
--> lint_example.rs:1:9
1 | #[deny(missing_debug_implementations)]
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

Having a `Debug` implementation on all types can assist with debugging, as it provides a convenient way to format and display a value. Using the `#[derive(Debug)]` attribute will automatically generate a typical implementation, or a custom implementation can be added

by manually implementing the `Debug` trait.

This lint is "allow" by default because adding `Debug` to all types can have a negative impact on compile time and code size. It also requires boilerplate to be added to every type, which can be an impediment.

missing-docs

The `missing_docs` lint detects missing documentation for public items.

Example

```
#![deny(missing_docs)]
pub fn foo() {}
```

This will produce:

```
error: missing documentation for the crate
--> lint_example.rs:1:1
   |
1  | / #![deny(missing_docs)]
2  | | fn main() {
3  | | pub fn foo() {}
4  | | }
   | | ^
   | |
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(missing_docs)]
   |          ^^^^^^^^^^^^^
```

Explanation

This lint is intended to ensure that a library is well-documented. Items without

documentation can be difficult for users to understand how to use properly.

This lint is "allow" by default because it can be noisy, and not all projects may want to enforce everything to be documented.

missing-unsafe-on-extern

The `missing_unsafe_on_extern` lint detects missing `unsafe` keyword on extern declarations.

Example

```
#![feature(unsafe_extern_blocks)]
#![warn(missing_unsafe_on_extern)]
#![allow(dead_code)]

extern "C" {
    fn foo(_: i32);
}

fn main() {}
```

This will produce:

```

warning: extern blocks should be unsafe
--> lint_example.rs:5:1
   |
5  |     extern "C" {
   |     ^
   |     |
   |     _help: needs `unsafe` before the extern keyword: `unsafe`
   |
6  |         fn foo(_: i32);
7  |     }
   |     ^
   |
= warning: this is accepted in the current edition (Rust 2021) but is a hard
error in Rust 2024!
= note: for more information, see issue #123743 <https://github.com/rust-
lang/rust/issues/123743>
note: the lint level is defined here
--> lint_example.rs:2:9
   |
2  | #![warn(missing_unsafe_on_extern)]
   |          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

Explanation

Declaring extern items, even without ever using them, can cause Undefined Behavior. We should consider all sources of Undefined Behavior to be unsafe.

This is a [future-incompatible](#) lint to transition this to a hard error in the future.

multiple-supertrait-upcastable

The `multiple_supertrait_upcastable` lint detects when an object-safe trait has multiple supertraits.

Example

```
#![feature(multiple_supertrait_upcastable)]
trait A {}
trait B {}

#[warn(multiple_supertrait_upcastable)]
trait C: A + B {}
```

This will produce:

```
warning: `C` is object-safe and has multiple supertraits
--> lint_example.rs:7:1
  |
7 | trait C: A + B {}
  | ^^^^^^^^^^^^^^^^^
  |
note: the lint level is defined here
--> lint_example.rs:6:8
  |
6 | #[warn(multiple_supertrait_upcastable)]
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

To support upcasting with multiple supertraits, we need to store multiple vtables and this can result in extra space overhead, even if no code actually uses upcasting. This lint allows users to identify when such scenarios occur and to decide whether the additional overhead is justified.

must-not-suspend

The `must_not_suspend` lint guards against values that shouldn't be held across suspend points (`.await`)

Example

```

#![feature(must_not_suspend)]
#![warn(must_not_suspend)]

#[must_not_suspend]
struct SyncThing {}

async fn yield_now() {}

pub async fn uhoh() {
    let guard = SyncThing {};
    yield_now().await;
    let _guard = guard;
}

```

This will produce:

```

warning: `SyncThing` held across a suspend point, but should not be
--> lint_example.rs:11:9
   |
11 |     let guard = SyncThing {};
   |           ^^^^^^
12 |     yield_now().await;
   |                   ----- the value is held across this suspend point
help: consider using a block (`{ ... }`) to shrink the value's scope, ending
before the suspend point
--> lint_example.rs:11:9
   |
11 |     let guard = SyncThing {};
   |           ^^^^^^
note: the lint level is defined here
--> lint_example.rs:2:9
   |
 2 | #![warn(must_not_suspend)]
   |           ^^^^^^^^^^^^^^^^^^^^^

```

Explanation

The `must_not_suspend` lint detects values that are marked with the `#[must_not_suspend]` attribute being held across suspend points. A "suspend" point is usually a `.await` in an `async` function.

This attribute can be used to mark values that are semantically incorrect across suspends (like certain types of timers), values that have async alternatives, and values that regularly cause problems with the `Send`-ness of async fn's returned futures (like `MutexGuard`'s)

non-ascii-idents

The `non_ascii_idents` lint detects non-ASCII identifiers.

Example

```
#![deny(non_ascii_idents)]
fn main() {
    let föö = 1;
}
```

This will produce:

```
error: identifier contains non-ASCII characters
--> lint_example.rs:4:9
   |
4  |     let föö = 1;
   |           ^^^
note: the lint level is defined here
--> lint_example.rs:2:9
   |
2  | #![deny(non_ascii_idents)]
   |           ^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

This lint allows projects that wish to retain the limit of only using ASCII characters to switch this lint to "forbid" (for example to ease collaboration or for security reasons). See [RFC 2457](https://rust-lang.org/rfcs/2457-non-ascii-idents) for more details.

non-exhaustive-omitted-patterns

The `non_exhaustive_omitted_patterns` lint aims to help consumers of a `#[non_exhaustive]` struct or enum who want to match all of its fields/variants explicitly.

The `#[non_exhaustive]` annotation forces matches to use wildcards, so exhaustiveness checking cannot be used to ensure that all fields/variants are matched explicitly. To remedy this, this allow-by-default lint warns the user when a match mentions some but not all of the fields/variants of a `#[non_exhaustive]` struct or enum.

Example

```
// crate A
#[non_exhaustive]
pub enum Bar {
    A,
    B, // added variant in non breaking change
}

// in crate B
#![feature(non_exhaustive_omitted_patterns_lint)]
#[warn(non_exhaustive_omitted_patterns)]
match Bar::A {
    Bar::A => {},
    _ => {},
}
```

This will produce:

```

warning: some variants are not matched explicitly
--> $DIR/reachable-patterns.rs:70:9
|
LL |         match Bar::A {
|           ^ pattern `Bar::B` not covered
|
note: the lint level is defined here
--> $DIR/reachable-patterns.rs:69:16
|
LL |         #[warn(non_exhaustive_omitted_patterns)]
|           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
= help: ensure that all variants are matched explicitly by adding the
suggested match arms
= note: the matched value is of type `Bar` and the
`non_exhaustive_omitted_patterns` attribute was found

```

Warning: setting this to `deny` will make upstream non-breaking changes (adding fields or variants to a `#[non_exhaustive]` struct or enum) break your crate. This goes against expected semver behavior.

Explanation

Structs and enums tagged with `#[non_exhaustive]` force the user to add a (potentially redundant) wildcard when pattern-matching, to allow for future addition of fields or variants. The `non_exhaustive_omitted_patterns` lint detects when such a wildcard happens to actually catch some fields/variants. In other words, when the match without the wildcard would not be exhaustive. This lets the user be informed if new fields/variants were added.

non-local-definitions

The `non_local_definitions` lint checks for `impl` blocks and `#[macro_export]` macro inside bodies (functions, enum discriminant, ...).

Example

```

#![warn(non_local_definitions)]
trait MyTrait {}
struct MyStruct;

fn foo() {
    impl MyTrait for MyStruct {}
}

```

This will produce:

```

warning: non-local `impl` definition, `impl` blocks should be written at the
same level as their item
--> lint_example.rs:7:5
  |
6 | fn foo() {
  | ----- move the `impl` block outside of this function `foo` and up 2
bodies
7 |     impl MyTrait for MyStruct {}
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^
  |           |           |
  |           |           | `MyStruct` is not local
  |           |           | `MyTrait` is not local
  |
= note: `impl` may be usable in bounds, etc. from outside the expression,
which might e.g. make something constructible that previously wasn't, because
it's still on a publicly-visible type
= note: an `impl` is never scoped, even when it is nested inside an item, as
it may impact type checking outside of that item, which can be the case if
neither the trait or the self type are at the same nesting level as the `impl`
= note: this lint may become deny-by-default in the edition 2024 and higher,
see the tracking issue <https://github.com/rust-lang/rust/issues/120363>
note: the lint level is defined here
--> lint_example.rs:1:9
  |
1 | #![warn(non_local_definitions)]
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

Explanation

Creating non-local definitions go against expectation and can create discrepancies in tooling. It should be avoided. It may become deny-by-default in edition 2024 and higher, see the

tracking issue <https://github.com/rust-lang/rust/issues/120363>.

An `impl` definition is non-local if it is nested inside an item and neither the type nor the trait are at the same nesting level as the `impl` block.

All nested bodies (functions, enum discriminant, array length, consts) (except for `const _: Ty = { ... }` in top-level module, which is still undecided) are checked.

or-patterns-back-compat

The lint `or-patterns-back-compat` has been renamed to [rust-2021-incompatible-or-patterns](#).

redundant-lifetimes

The `redundant_lifetimes` lint detects lifetime parameters that are redundant because they are equal to another named lifetime.

Example

```
#[deny(redundant_lifetimes)]

// `a` = 'static', so all usages of `a` can be replaced with `static`
pub fn bar<a: 'static>() {}

// `a` = 'b', so all usages of `b` can be replaced with `a`
pub fn bar<a: 'b, 'b: 'a>() {}
```

This will produce:

```

error: unnecessary lifetime parameter ``a`
--> lint_example.rs:5:12
   |
5  | pub fn bar<'a: 'static>() {}
   |           ^^
   |
   = note: you can use the ``static` lifetime directly, in place of ``a`
note: the lint level is defined here
--> lint_example.rs:2:8
   |
2  | #[deny(redundant_lifetimes)]
   |           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

Explanation

Unused lifetime parameters may signal a mistake or unfinished code. Consider removing the parameter.

rust-2021-incompatible-closure-captures

The `rust_2021_incompatible_closure_captures` lint detects variables that aren't completely captured in Rust 2021, such that the `Drop` order of their fields may differ between Rust 2018 and 2021.

It can also detect when a variable implements a trait like `Send`, but one of its fields does not, and the field is captured by a closure and used with the assumption that said field implements the same trait as the root variable.

Example of drop reorder

```
#![deny(rust_2021_incompatible_closure_captures)]

struct FancyInteger(i32);

impl Drop for FancyInteger {
    fn drop(&mut self) {
        println!("Just dropped {}", self.0);
    }
}

struct Point { x: FancyInteger, y: FancyInteger }

fn main() {
    let p = Point { x: FancyInteger(10), y: FancyInteger(20) };

    let c = || {
        let x = p.x;
    };

    c();

    // ... More code ...
}
```

This will produce:

```

error: changes to closure capture in Rust 2021 will affect drop order
--> lint_example.rs:17:11
   |
17 |     let c = || {
   |             ^^
18 |         let x = p.x;
   |               --- in Rust 2018, this closure captures all of `p`, but in
   |               Rust 2021, it will only capture `p.x`
...
24 | }
   | - in Rust 2018, `p` is dropped here, but in Rust 2021, only `p.x` will be
   |   dropped here as part of the closure
   |
   | = note: for more information, see <https://doc.rust-lang.org/nightly/edition-guide/rust-2021/disjoint-capture-in-closures.html>
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(rust_2021_incompatible_closure_captures)]
   |          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: add a dummy let to cause `p` to be fully captured
   |
17 ~     let c = || {
18 +         let _ = &p;
   |

```

Explanation

In the above example, `p.y` will be dropped at the end of `f` instead of with `c` in Rust 2021.

Example of auto-trait

```

#![deny(rust_2021_incompatible_closure_captures)]
use std::thread;

struct Pointer(*mut i32);
unsafe impl Send for Pointer {}

fn main() {
    let mut f = 10;
    let fptr = Pointer(&mut f as *mut i32);
    thread::spawn(move || unsafe {
        *fptr.0 = 20;
    });
}

```

This will produce:

error: changes to closure capture in Rust 2021 will affect which traits the closure implements

```

--> lint_example.rs:10:19
|
10 |     thread::spawn(move || unsafe {
|                               ^^^^^^^ in Rust 2018, this closure implements `Send` as
`fptr` implements `Send`, but in Rust 2021, this closure will no longer
implement `Send` because `fptr` is not fully captured and `fptr.0` does not
implement `Send`

```

```

11 |         *fptr.0 = 20;
|         ----- in Rust 2018, this closure captures all of `fptr`, but in
Rust 2021, it will only capture `fptr.0`

```

= note: for more information, see <<https://doc.rust-lang.org/nightly/edition-guide/rust-2021/disjoint-capture-in-closures.html>>

note: the lint level is defined here

```

--> lint_example.rs:1:9
|
1 | #![deny(rust_2021_incompatible_closure_captures)]
|   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

help: add a dummy let to cause `fptr` to be fully captured

```

10 ~     thread::spawn(move || { let _ = &fptr; unsafe {
11 |         *fptr.0 = 20;
12 ~     } });
|

```

Explanation

In the above example, only `fptr.0` is captured in Rust 2021. The field is of type `*mut i32`, which doesn't implement `Send`, making the code invalid as the field cannot be sent between threads safely.

rust-2021-incompatible-or-patterns

The `rust_2021_incompatible_or_patterns` lint detects usage of old versions of or-patterns.

Example

```
#![deny(rust_2021_incompatible_or_patterns)]

macro_rules! match_any {
    ( $expr:expr , $( $( $pat:pat )|+ => $expr_arm:expr ),+ ) => {
        match $expr {
            $(
                $(
                    $( $pat => $expr_arm, )+
                )+
            )
        };
    }
}

fn main() {
    let result: Result<i64, i32> = Err(42);
    let int: i64 = match_any!(result, Ok(i) | Err(i) => i.into());
    assert_eq!(int, 42);
}
```

This will produce:

```

error: the meaning of the `pat` fragment specifier is changing in Rust 2021,
which may affect this macro
--> lint_example.rs:4:26
  |
4 |      ( $expr:expr , $( $( $pat:pat )|+ => $expr_arm:expr ),+ ) => {
  |                                ^^^^^^^^^ help: use pat_param to preserve
semantics: ` $pat:pat_param `
  |
  = warning: this is accepted in the current edition (Rust 2018) but is a hard
error in Rust 2021!
  = note: for more information, see <https://doc.rust-lang.org/nightly/edition-guide/rust-2021/or-patterns-macro-rules.html>
note: the lint level is defined here
--> lint_example.rs:1:9
  |
1 |  #[deny(rust_2021_incompatible_or_patterns)]
  |  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

Explanation

In Rust 2021, the `pat` matcher will match additional patterns, which include the `|` character.

rust-2021-prefixes-incompatible-syntax

The `rust_2021_prefixes_incompatible_syntax` lint detects identifiers that will be parsed as a prefix instead in Rust 2021.

Example

```
#![deny(rust_2021_prefixes_incompatible_syntax)]

macro_rules! m {
    (z $x:expr) => ();
}

m!(z"hey");
```

This will produce:

```
error: prefix `z` is unknown
--> lint_example.rs:8:4
  |
8 | m!(z"hey");
  |   ^ unknown prefix
  |
  = warning: this is accepted in the current edition (Rust 2018) but is a hard
error in Rust 2021!
  = note: for more information, see <https://doc.rust-lang.org/nightly/edition-guide/rust-2021/reserving-syntax.html>
note: the lint level is defined here
--> lint_example.rs:1:9
  |
1 | #![deny(rust_2021_prefixes_incompatible_syntax)]
  |           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: insert whitespace here to avoid this being parsed as a prefix in Rust
2021
  |
8 | m!(z "hey");
  |      +
```

Explanation

In Rust 2015 and 2018, `z"hey"` is two tokens: the identifier `z` followed by the string literal `"hey"`. In Rust 2021, the `z` is considered a prefix for `"hey"`.

This lint suggests to add whitespace between the `z` and `"hey"` tokens to keep them separated in Rust 2021.

rust-2021-prelude-collisions

The `rust_2021_prelude_collisions` lint detects the usage of trait methods which are ambiguous with traits added to the prelude in future editions.

Example

```
#![deny(rust_2021_prelude_collisions)]

trait Foo {
    fn try_into(self) -> Result<String, !>;
}

impl Foo for &str {
    fn try_into(self) -> Result<String, !> {
        Ok(String::from(self))
    }
}

fn main() {
    let x: String = "3".try_into().unwrap();
    //           ^^^^^^^^^^^
    // This call to try_into matches both Foo::try_into and TryInto::try_into
    as
    // `TryInto` has been added to the Rust prelude in 2021 edition.
    println!("{}", x);
}
```

This will produce:

```

error: trait method `try_into` will become ambiguous in Rust 2021
--> lint_example.rs:14:21
   |
14 |         let x: String = "3".try_into().unwrap();
   |                                ^^^^^^^^^^^^^^^^^ help: disambiguate the associated
function: `Foo::try_into(&*3)`
   |
   = warning: this is accepted in the current edition (Rust 2018) but is a hard
error in Rust 2021!
   = note: for more information, see <https://doc.rust-
lang.org/nightly/edition-guide/rust-2021/prelude.html>
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(rust_2021_prelude_collisions)]
   |                                ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

Explanation

In Rust 2021, one of the important introductions is the [prelude changes](#), which add `TryFrom`, `TryInto`, and `FromIterator` into the standard library's prelude. Since this results in an ambiguity as to which method/function to call when an existing `try_into` method is called via dot-call syntax or a `try_from` / `from_iter` associated function is called directly on a type.

rust-2024-incompatible-pat

The `rust_2024_incompatible_pat` lint detects patterns whose meaning will change in the Rust 2024 edition.

Example

```
#![feature(ref_pat_eat_one_layer_2024)]
#![warn(rust_2024_incompatible_pat)]

if let Some(&a) = &Some(&0u8) {
    let _: u8 = a;
}
if let Some(mut _a) = &mut Some(0u8) {
    _a = 7u8;
}
```

This will produce:

```
warning: the semantics of this pattern will change in edition 2024
--> lint_example.rs:5:8
```

```
|
5 | if let Some(&a) = &Some(&0u8) {
|           _^^^^^^^^
|           |
|           help: desugar the match ergonomics: `&`
|
```

```
note: the lint level is defined here
```

```
--> lint_example.rs:2:9
|
2 | #![warn(rust_2024_incompatible_pat)]
|           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
warning: the semantics of this pattern will change in edition 2024
--> lint_example.rs:8:8
```

```
|
8 | if let Some(mut _a) = &mut Some(0u8) {
|           _^^^^^^^^^^^^^^^^
|           |
|           help: desugar the match ergonomics: `&mut`
```

Explanation

In Rust 2024 and above, the `mut` keyword does not reset the pattern binding mode, and nor do `&` or `&mut` patterns. The lint will suggest code that has the same meaning in all editions.

single-use-lifetime

The lint `single-use-lifetime` has been renamed to `single-use-lifetimes`.

single-use-lifetimes

The `single_use_lifetimes` lint detects lifetimes that are only used once.

Example

```
#![deny(single_use_lifetimes)]

fn foo<'a>(x: &'a u32) {}
```

This will produce:

```
error: lifetime parameter `'a` only used once
--> lint_example.rs:4:8
   |
4 | fn foo<'a>(x: &'a u32) {}
   |           ^^      -- ...is used only here
   |           |
   |           this lifetime...
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1 | #![deny(single_use_lifetimes)]
   |           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: elide the single-use lifetime
   |
4 - fn foo<'a>(x: &'a u32) {}
4 + fn foo(x: &u32) {}
   |
```

Explanation

Specifying an explicit lifetime like `'a` in a function or `impl` should only be used to link together two things. Otherwise, you should just use `'_` to indicate that the lifetime is not linked to anything, or elide the lifetime altogether if possible.

This lint is "allow" by default because it was introduced at a time when `'_` and elided lifetimes were first being introduced, and this lint would be too noisy. Also, there are some known false positives that it produces. See [RFC 2115](#) for historical context, and [issue #44752](#) for more details.

trivial-casts

The `trivial_casts` lint detects trivial casts which could be replaced with coercion, which may require a temporary variable.

Example

```
#![deny(trivial_casts)]  
let x: &u32 = &42;  
let y = x as *const u32;
```

This will produce:

```

error: trivial cast: `&u32` as `*const u32`
--> lint_example.rs:4:9
   |
4  | let y = x as *const u32;
   |             ^^^^^^^^^^^^^^^^^
   |
= help: cast can be replaced by coercion; this might require a temporary
variable
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(trivial_casts)]
   |             ^^^^^^^^^^^^^^^^^

```

Explanation

A trivial cast is a cast `e as T` where `e` has type `U` and `U` is a subtype of `T`. This type of cast is usually unnecessary, as it can be usually be inferred.

This lint is "allow" by default because there are situations, such as with FFI interfaces or complex type aliases, where it triggers incorrectly, or in situations where it will be more difficult to clearly express the intent. It may be possible that this will become a warning in the future, possibly with an explicit syntax for coercions providing a convenient way to work around the current issues. See [RFC 401 \(coercions\)](#), [RFC 803 \(type ascription\)](#) and [RFC 3307 \(remove type ascription\)](#) for historical context.

trivial-numeric-casts

The `trivial_numeric_casts` lint detects trivial numeric casts of types which could be removed.

Example

```
#![deny(trivial_numeric_casts)]
let x = 42_i32 as i32;
```

This will produce:

```
error: trivial numeric cast: `i32` as `i32`
--> lint_example.rs:3:9
   |
3 | let x = 42_i32 as i32;
   |             ^^^^^^^^^^^^^^^^^
   |
   = help: cast can be replaced by coercion; this might require a temporary variable
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1 | #![deny(trivial_numeric_casts)]
   |             ^^^^^^^^^^^^^^^^^
```

Explanation

A trivial numeric cast is a cast of a numeric type to the same numeric type. This type of cast is usually unnecessary.

This lint is "allow" by default because there are situations, such as with FFI interfaces or complex type aliases, where it triggers incorrectly, or in situations where it will be more difficult to clearly express the intent. It may be possible that this will become a warning in the future, possibly with an explicit syntax for coercions providing a convenient way to work around the current issues. See [RFC 401 \(coercions\)](#), [RFC 803 \(type ascription\)](#) and [RFC 3307 \(remove type ascription\)](#) for historical context.

unit-bindings

The `unit_bindings` lint detects cases where bindings are useless because they have the unit type `()` as their inferred type. The lint is suppressed if the user explicitly annotates the

let binding with the unit type `()`, or if the let binding uses an underscore wildcard pattern, i.e. `let _ = expr`, or if the binding is produced from macro expansions.

Example

```
#![deny(unit_bindings)]

fn foo() {
    println!("do work");
}

pub fn main() {
    let x = foo(); // useless binding
}
```

This will produce:

```
error: binding has unit type `()`
--> lint_example.rs:8:5
  |
8 |     let x = foo(); // useless binding
  |     ^^^^^^^^^^^^^^^
  |
  |     this pattern is inferred to be the unit type `()`
  |
note: the lint level is defined here
--> lint_example.rs:1:9
1 | #![deny(unit_bindings)]
  |     ^^^^^^^^^^^^^^^^^
```

Explanation

Creating a local binding with the unit type `()` does not do much and can be a sign of a user error, such as in this example:


```
fn main() {
    let mut x = [1, 2, 3];
    x[0] = 5;
    let y = x.sort(); // useless binding as `sort` returns `()` and not the
sorted array.
    println!("{:?}", y); // prints "()"
}
```

unnameable-types

The `unnameable_types` lint detects types for which you can get objects of that type, but cannot name the type itself.

Example

```
#![deny(unnameable_types)]
mod m {
    pub struct S;
}

pub fn get_unnameable() -> m::S { m::S }
```

This will produce:

```
error: struct `S` is reachable but cannot be named
--> lint_example.rs:4:5
  |
4 |     pub struct S;
  |     ^^^^^^^^^^^^^ reachable at visibility `pub`, but can only be named at
visibility `pub(crate)`
  |
note: the lint level is defined here
--> lint_example.rs:2:9
  |
2 | #![deny(unnameable_types)]
  |     ^^^^^^^^^^^^^^^^^^^^^
```

Explanation

It is often expected that if you can obtain an object of type τ , then you can name the type τ as well, this lint attempts to enforce this rule. The recommended action is to either reexport the type properly to make it nameable, or document that users are not supposed to be able to name it for one reason or another.

Besides types, this lint applies to traits because traits can also leak through signatures, and you may obtain objects of their `dyn Trait` or `impl Trait` types.

unreachable-pub

The `unreachable_pub` lint triggers for `pub` items not reachable from other crates - that means neither directly accessible, nor reexported, nor leaked through things like return types.

Example

```
#![deny(unreachable_pub)]
mod foo {
    pub mod bar {

    }
}
```

This will produce:

```

error: unreachable `pub` item
--> lint_example.rs:4:5
  |
4 |     pub mod bar {
  |     ----^^^^^^^^
  |     |
  |     help: consider restricting its visibility: `pub(crate)`
  |
    = help: or consider exporting it for use by other crates
note: the lint level is defined here
--> lint_example.rs:1:9
  |
1 | #![deny(unreachable_pub)]
  |           ^^^^^^^^^^^^^^^^^^^^^

```

Explanation

The `pub` keyword both expresses an intent for an item to be publicly available, and also signals to the compiler to make the item publicly accessible. The intent can only be satisfied, however, if all items which contain this item are *also* publicly accessible. Thus, this lint serves to identify situations where the intent does not match the reality.

If you wish the item to be accessible elsewhere within the crate, but not outside it, the `pub(crate)` visibility is recommended to be used instead. This more clearly expresses the intent that the item is only visible within its own crate.

This lint is "allow" by default because it will trigger for a large amount existing Rust code, and has some false-positives. Eventually it is desired for this to become warn-by-default.

unsafe-attr-outside-unsafe

The `unsafe_attr_outside_unsafe` lint detects a missing `unsafe` keyword on attributes considered unsafe.

Example

```
#![feature(unsafe_attributes)]
#![warn(unsafe_attr_outside_unsafe)]

#[no_mangle]
extern "C" fn foo() {}

fn main() {}
```

This will produce:

```
warning: unsafe attribute used without unsafe
--> lint_example.rs:4:3
  |
4 | #[no_mangle]
  |      ^^^^^^^^^ usage of unsafe attribute
  |
  = warning: this is accepted in the current edition (Rust 2021) but is a hard
error in Rust 2024!
  = note: for more information, see issue #123757 <https://github.com/rust-
lang/rust/issues/123757>
note: the lint level is defined here
--> lint_example.rs:2:9
  |
2 | #![warn(unsafe_attr_outside_unsafe)]
  |      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: wrap the attribute in `unsafe(...)`
  |
4 | #[unsafe(no_mangle)]
  |      ++++++++      +
```

Explanation

Some attributes (e.g. `no_mangle`, `export_name`, `link_section` -- see [issue #82499](#) for a more complete list) are considered "unsafe" attributes. An unsafe attribute must only be used inside `unsafe(...)`.

This lint can automatically wrap the attributes in `unsafe(...)`, but this obviously cannot verify that the preconditions of the `unsafe` attributes are fulfilled, so that is still up to the user.

The lint is currently "allow" by default, but that might change in the future.

unsafe-code

The `unsafe_code` lint catches usage of `unsafe` code and other potentially unsound constructs like `no_mangle`, `export_name`, and `link_section`.

Example

```
#![deny(unsafe_code)]
fn main() {
    unsafe {

    }
}

#[no_mangle]
fn func_0() { }

#[export_name = "exported_symbol_name"]
pub fn name_in_rust() { }

#[no_mangle]
#[link_section = ".example_section"]
pub static VAR1: u32 = 1;
```

This will produce:

```
error: usage of an `unsafe` block
--> lint_example.rs:3:5
  |
3 | /     unsafe {
4 | |
5 | |     }
  | |_____^
  |
note: the lint level is defined here
--> lint_example.rs:1:9
  |
```

```
1 | #![deny(unsafe_code)]
  |           ^^^^^^^^^^^^^
```

error: declaration of a `no\_mangle` function

--> lint_example.rs:8:1

```
|
8 | #[no_mangle]
  | ^^^^^^^^^^^^^
```

= note: the linker's behavior with multiple libraries exporting duplicate symbol names is undefined and Rust cannot provide guarantees when you manually override them

error: declaration of a function with `export\_name`

--> lint_example.rs:11:1

```
|
11 | #[export_name = "exported_symbol_name"]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

= note: the linker's behavior with multiple libraries exporting duplicate symbol names is undefined and Rust cannot provide guarantees when you manually override them

error: declaration of a `no\_mangle` static

--> lint_example.rs:14:1

```
|
14 | #[no_mangle]
   | ^^^^^^^^^^^^^
```

= note: the linker's behavior with multiple libraries exporting duplicate symbol names is undefined and Rust cannot provide guarantees when you manually override them

error: declaration of a static with `link\_section`

--> lint_example.rs:15:1

```
|
15 | #[link_section = ".example_section"]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

= note: the program's behavior with overridden link sections on items is unpredictable and Rust cannot provide guarantees when you manually override them

Explanation

This lint is intended to restrict the usage of `unsafe` blocks and other constructs (including, but not limited to `no_mangle`, `link_section` and `export_name` attributes) wrong usage of which causes undefined behavior.

unsafe-op-in-unsafe-fn

The `unsafe_op_in_unsafe_fn` lint detects unsafe operations in unsafe functions without an explicit `unsafe` block.

Example

```
#![deny(unsafe_op_in_unsafe_fn)]

unsafe fn foo() {}

unsafe fn bar() {
    foo();
}

fn main() {}
```

This will produce:

```

error[E0133]: call to unsafe function `foo` is unsafe and requires unsafe block
--> lint_example.rs:6:5
   |
6  |     foo();
   |     ^^^^^ call to unsafe function
   |
   = note: for more information, see issue #71668 <https://github.com/rust-
lang/rust/issues/71668>
   = note: consult the function's documentation for information on how to avoid
undefined behavior
note: an unsafe function restricts its caller, but its body is safe by default
--> lint_example.rs:5:1
   |
5  | unsafe fn bar() {
   | ^^^^^^^^^^^^^^^^^
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(unsafe_op_in_unsafe_fn)]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

Explanation

Currently, an `unsafe fn` allows any `unsafe` operation within its body. However, this can increase the surface area of code that needs to be scrutinized for proper behavior. The `unsafe block` provides a convenient way to make it clear exactly which parts of the code are performing unsafe operations. In the future, it is desired to change it so that unsafe operations cannot be performed in an `unsafe fn` without an `unsafe block`.

The fix to this is to wrap the unsafe code in an `unsafe block`.

This lint is "allow" by default on editions up to 2021, from 2024 it is "warn" by default; the plan for increasing severity further is still being considered. See [RFC #2585](#) and [issue #71668](#) for more details.

unstable-features

The `unstable_features` lint detects uses of `#![feature]`.

Example

```
#![deny(unstable_features)]
#![feature(test)]
```

This will produce:

```
error: use of an unstable feature
--> lint_example.rs:2:12
   |
2  | #![feature(test)]
   |                ^^^^
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(unstable_features)]
   |                ^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

In larger nightly-based projects which

- consist of a multitude of crates where a subset of crates has to compile on stable either unconditionally or depending on a `cfg` flag to for example allow stable users to depend on them,
- don't use nightly for experimental features but for, e.g., unstable options only,

this lint may come in handy to enforce policies of these kinds.

unused-crate-dependencies

The `unused_crate_dependencies` lint detects crate dependencies that are never used.

Example

```
#![deny(unused_crate_dependencies)]
```

This will produce:

```
error: extern crate `regex` is unused in crate `lint_example`
  |
  = help: remove the dependency or add `use regex as _;` to the crate root
note: the lint level is defined here
--> src/lib.rs:1:9
  |
1 | #![deny(unused_crate_dependencies)]
  |          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

After removing the code that uses a dependency, this usually also requires removing the dependency from the build configuration. However, sometimes that step can be missed, which leads to time wasted building dependencies that are no longer used. This lint can be enabled to detect dependencies that are never used (more specifically, any dependency passed with the `--extern` command-line flag that is never referenced via `use`, `extern crate`, or in any `path`).

This lint is "allow" by default because it can provide false positives depending on how the build system is configured. For example, when using Cargo, a "package" consists of multiple crates (such as a library and a binary), but the dependencies are defined for the package as a whole. If there is a dependency that is only used in the binary, but not the library, then the lint will be incorrectly issued in the library.

unused-extern-crates

The `unused_extern_crates` lint guards against `extern crate` items that are never used.

Example

```
#![deny(unused_extern_crates)]
#![deny(warnings)]
extern crate proc_macro;
```

This will produce:

```
error: unused extern crate
--> lint_example.rs:4:1
   |
 4 | extern crate proc_macro;
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ help: remove it
   |
note: the lint level is defined here
--> lint_example.rs:1:9
   |
 1 | #![deny(unused_extern_crates)]
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

`extern crate` items that are unused have no effect and should be removed. Note that there are some cases where specifying an `extern crate` is desired for the side effect of ensuring the given crate is linked, even though it is not otherwise directly referenced. The lint can be silenced by aliasing the crate to an underscore, such as `extern crate foo as _`. Also note that it is no longer idiomatic to use `extern crate` in the [2018 edition](#), as extern crates are now automatically added in scope.

This lint is "allow" by default because it can be noisy, and produce false-positives. If a dependency is being removed from a project, it is recommended to remove it from the build configuration (such as `Cargo.toml`) to ensure stale build entries aren't left behind.

unused-import-braces

The `unused_import_braces` lint catches unnecessary braces around an imported item.

Example

```
#![deny(unused_import_braces)]
use test::{A};

pub mod test {
    pub struct A;
}
```

This will produce:

```
error: braces around A is unnecessary
--> lint_example.rs:2:1
   |
2  | use test::{A};
   | ^^^^^^^^^^^^^^^^^
   |
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(unused_import_braces)]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

If there is only a single item, then remove the braces (`use test::A;` for example).

This lint is "allow" by default because it is only enforcing a stylistic choice.

unused-lifetimes

The `unused_lifetimes` lint detects lifetime parameters that are never used.

Example

```
#[deny(unused_lifetimes)]
```

```
pub fn foo<'a>() {}
```

This will produce:

```
error: lifetime parameter ``a` never used
--> lint_example.rs:4:12
   |
4 | pub fn foo<'a>() {}
   |             ^^^ help: elide the unused lifetime
note: the lint level is defined here
--> lint_example.rs:2:8
   |
2 | #[deny(unused_lifetimes)]
   |          ^^^^^^^^^^^^^^^^^
```

Explanation

Unused lifetime parameters may signal a mistake or unfinished code. Consider removing the parameter.

unused-macro-rules

The `unused_macro_rules` lint detects macro rules that were not used.

Note that the lint is distinct from the `unused_macros` lint, which fires if the entire macro is never called, while this lint fires for single unused rules of the macro that is otherwise used. `unused_macro_rules` fires only if `unused_macros` wouldn't fire.

Example

```
#[warn(unused_macro_rules)]
macro_rules! unused_empty {
    (hello) => { println!("Hello, world!") }; // This rule is unused
    () => { println!("empty") }; // This rule is used
}

fn main() {
    unused_empty!(hello);
}
```

This will produce:

```
warning: rule #2 of macro `unused_empty` is never used
--> lint_example.rs:4:5
 4 |      () => { println!("empty") }; // This rule is used
   |      ^^
   |
note: the lint level is defined here
--> lint_example.rs:1:8
 1 |  #[warn(unused_macro_rules)]
   |      ^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

Unused macro rules may signal a mistake or unfinished code. Furthermore, they slow down compilation. Right now, silencing the warning is not supported on a single rule level, so you have to add an allow to the entire macro definition.

If you intended to export the macro to make it available outside of the crate, use the [macro_export](#) attribute.

unused-qualifications

The `unused_qualifications` lint detects unnecessarily qualified names.

Example

```
#![deny(unused_qualifications)]
mod foo {
    pub fn bar() {}
}

fn main() {
    use foo::bar;
    foo::bar();
    bar();
}
```

This will produce:

```
error: unnecessary qualification
--> lint_example.rs:8:5
   |
8  |     foo::bar();
   |     ^^^^^^^^^
   |
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(unused_qualifications)]
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: remove the unnecessary path segments
   |
8  -     foo::bar();
8  +     bar();
   |
```

Explanation

If an item from another module is already brought into scope, then there is no need to qualify it in this case. You can call `bar()` directly, without the `foo::`.

This lint is "allow" by default because it is somewhat pedantic, and doesn't indicate an actual problem, but rather a stylistic choice, and can be noisy when refactoring or moving around code.

unused-results

The `unused_results` lint checks for the unused result of an expression in a statement.

Example

```
#![deny(unused_results)]
fn foo<T>() -> T { panic!() }

fn main() {
    foo::usize();
}
```

This will produce:

```
error: unused result of type `usize`
--> lint_example.rs:5:5
   |
5  |     foo::usize();
   |     ^^^^^^^^^^^^^^^^^
note: the lint level is defined here
--> lint_example.rs:1:9
1  | #![deny(unused_results)]
   |     ^^^^^^^^^^^^^^^^^
```

Explanation

Ignoring the return value of a function may indicate a mistake. In cases where it is almost certain that the result should be used, it is recommended to annotate the function with the

`must_use` attribute. Failure to use such a return value will trigger the `unused_must_use` lint which is warn-by-default. The `unused_results` lint is essentially the same, but triggers for *all* return values.

This lint is "allow" by default because it can be noisy, and may not be an actual problem. For example, calling the `remove` method of a `Vec` or `HashMap` returns the previous value, which you may not care about. Using this lint would require explicitly ignoring or discarding such values.

variant-size-differences

The `variant_size_differences` lint detects enums with widely varying variant sizes.

Example

```
#![deny(variant_size_differences)]
enum En {
    V0(u8),
    VBig([u8; 1024]),
}
```

This will produce:

```
error: enum variant is more than three times larger (1024 bytes) than the next
largest
--> lint_example.rs:5:5
   |
5 |         VBig([u8; 1024]),
   |         ^^^^^^^^^^^^^^^^^
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1 | #![deny(variant_size_differences)]
   |         ^^^^^^^^^^^^^^^^^
```

Explanation

It can be a mistake to add a variant to an enum that is much larger than the other variants, bloating the overall size required for all variants. This can impact performance and memory usage. This is triggered if one variant is more than 3 times larger than the second-largest variant.

Consider placing the large variant's contents on the heap (for example via [Box](#)) to keep the overall size of the enum itself down.

This lint is "allow" by default because it can be noisy, and may not be an actual problem. Decisions about this should be guided with profiling and benchmarking.

Warn-by-default Lints

These lints are all set to the 'warn' level by default.

- `ambiguous_glob_imports`
- `ambiguous_glob_reexports`
- `ambiguous_wide_pointer_comparisons`
- `anonymous_parameters`
- `array_into_iter`
- `asm_sub_register`
- `async_fn_in_trait`
- `bad_asm_style`
- `bare-trait-object`
- `bare_trait_objects`
- `boxed_slice_into_iter`
- `break_with_label_and_loop`
- `byte_slice_in_packed_struct_with_derive`
- `clashing_extern_declarations`
- `coherence_leak_check`
- `confusable_idents`
- `const_eval_mutable_ptr_in_final_value`
- `const_evaluatable_unchecked`
- `const_item_mutation`
- `dead_code`
- `dependency_on_unit_never_type_fallback`
- `deprecated`
- `deprecated_where_clause_location`
- `deref_into_dyn_supertrait`
- `deref_nullptr`
- `drop_bounds`
- `dropping_copy_types`
- `dropping_references`
- `duplicate_macro_attributes`
- `dyn_drop`
- `ellipsis_inclusive_range_patterns`

- `exported_private_dependencies`
- `for_loops_over_fallibles`
- `forbidden_lint_groups`
- `forgetting_copy_types`
- `forgetting_references`
- `function_item_references`
- `hidden_glob_reexports`
- `impl_trait_redundant_captures`
- `improper_ctypes`
- `improper_ctypes_definitions`
- `incomplete_features`
- `inline_no_sanitize`
- `internal_features`
- `invalid_from_utf8`
- `invalid_macro_export_arguments`
- `invalid_nan_comparisons`
- `invalid_value`
- `irrefutable_let_patterns`
- `large_assignments`
- `late_bound_lifetime_arguments`
- `legacy_derive_helpers`
- `map_unit_fn`
- `mixed_script_confusables`
- `named_arguments_used_positionally`
- `never_type_fallback_flow_into_unsafe`
- `no_mangle_generic_items`
- `non-fmt-panic`
- `non_camel_case_types`
- `non_contiguous_range_endpoints`
- `non_fmt_panics`
- `non_shorthand_field_patterns`
- `non_snake_case`
- `non_upper_case_globals`
- `noop_method_call`
- `opaque_hidden_inferred_bound`

- `out_of_scope_macro_calls`
- `overlapping-patterns`
- `overlapping_range_endpoints`
- `path_statements`
- `private_bounds`
- `private_interfaces`
- `private_macro_use`
- `ptr_cast_add_auto_to_object`
- `redundant-semicolon`
- `redundant_semicolons`
- `refining_impl_trait_internal`
- `refining_impl_trait_reachable`
- `renamed_and_removed_lints`
- `repr_transparent_external_private_fields`
- `self_constructor_from_outer_item`
- `semicolon_in_expressions_from_macros`
- `special_module_name`
- `stable_features`
- `static-mut-ref`
- `static_mut_refs`
- `suspicious_double_ref_op`
- `temporary_cstring_as_ptr`
- `trivial_bounds`
- `type_alias_bounds`
- `tyvar_behind_raw_pointer`
- `uncommon_codepoints`
- `unconditional_recursion`
- `uncovered_param_in_projection`
- `undefined_naked_function_abi`
- `unexpected_cfgs`
- `unfulfilled_lint_expectations`
- `ungated_async_fn_track_caller`
- `uninhabited_static`
- `unknown_lints`
- `unknown_or_malformed_diagnostic_attributes`

- `unnameable_test_items`
- `unreachable_code`
- `unreachable_patterns`
- `unstable-name-collision`
- `unstable_name_collisions`
- `unstable_syntax_pre_expansion`
- `unsupported_calling_conventions`
- `unused-doc-comment`
- `unused-tuple-struct-fields`
- `unused_allocation`
- `unused_assignments`
- `unused_associated_type_bounds`
- `unused_attributes`
- `unused_braces`
- `unused_comparisons`
- `unused_doc_comments`
- `unused_features`
- `unused_imports`
- `unused_labels`
- `unused_macros`
- `unused_must_use`
- `unused_mut`
- `unused_parens`
- `unused_unsafe`
- `unused_variables`
- `useless_ptr_null_checks`
- `warnings`
- `wasm_c_abi`
- `while_true`
- `writes_through_immutable_pointer`

ambiguous-glob-imports

The `ambiguous_glob_imports` lint detects glob imports that should report ambiguity errors,

but previously didn't do that due to rustc bugs.

Example

```
#![deny(ambiguous_glob_imports)]
pub fn foo() -> u32 {
    use sub::*;
    C
}

mod sub {
    mod mod1 { pub const C: u32 = 1; }
    mod mod2 { pub const C: u32 = 2; }

    pub use mod1::*;
    pub use mod2::*;
}
```

This will produce:

```

error: `C` is ambiguous
--> lint_example.rs:5:5
   |
5  |     C
   |     ^ ambiguous name
   |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #114095 <https://github.com/rust-
lang/rust/issues/114095>
= note: ambiguous because of multiple glob imports of a name in the same
module
note: `C` could refer to the constant imported here
--> lint_example.rs:12:13
   |
12 |     pub use mod1::*;
   |           ^^^^^^^
   |
= help: consider adding an explicit import of `C` to disambiguate
note: `C` could also refer to the constant imported here
--> lint_example.rs:13:13
   |
13 |     pub use mod2::*;
   |           ^^^^^^^
   |
= help: consider adding an explicit import of `C` to disambiguate
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(ambiguous_glob_imports)]
   |           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

Explanation

Previous versions of Rust compile it successfully because it had lost the ambiguity error when resolve `use sub::mod2::*`.

This is a [future-incompatible](#) lint to transition this to a hard error in the future.

ambiguous-glob-reexports

The `ambiguous_glob_reexports` lint detects cases where names re-exported via globs collide. Downstream users trying to use the same name re-exported from multiple globs will receive a warning pointing out redefinition of the same name.

Example

```
#![deny(ambiguous_glob_reexports)]
pub mod foo {
    pub type X = u8;
}

pub mod bar {
    pub type Y = u8;
    pub type X = u8;
}

pub use foo::*;
pub use bar::*;

pub fn main() {}
```

This will produce:

```
error: ambiguous glob re-exports
  --> lint_example.rs:11:9
   |
11 | pub use foo::*;
   |           ^^^^^^ the name `X` in the type namespace is first re-exported
here
12 | pub use bar::*;
   |           ----- but the name `X` in the type namespace is also re-exported
here
note: the lint level is defined here
  --> lint_example.rs:1:9
   |
1  | #![deny(ambiguous_glob_reexports)]
   |           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

This was previously accepted but it could silently break a crate's downstream users code. For example, if `foo::x` and `bar::x` were re-exported before `bar::x` was added to the re-exports, down stream users could use `this_crate::x` without problems. However, adding `bar::x` would cause compilation errors in downstream crates because `x` is defined multiple times in the same namespace of `this_crate`.

ambiguous-wide-pointer-comparisons

The `ambiguous_wide_pointer_comparisons` lint checks comparison of `*const/*mut ?Sized` as the operands.

Example

```
let ab = (A, B);
let a = &ab.0 as *const dyn T;
let b = &ab.1 as *const dyn T;

let _ = a == b;
```

This will produce:

warning: ambiguous wide pointer comparison, the comparison includes metadata which may not be expected

```
--> lint_example.rs:13:9
|
13 | let _ = a == b;
|           ^^^^^^
|
= note: `[warn(ambiguous_wide_pointer_comparisons)]` on by default
help: use `std::ptr::addr_eq` or untyped pointers to only compare their addresses
|
13 | let _ = std::ptr::addr_eq(a, b);
|           ++++++ ~ +
```

Explanation

The comparison includes metadata which may not be expected.

anonymous-parameters

The `anonymous_parameters` lint detects anonymous parameters in trait definitions.

Example

```
#![deny(anonymous_parameters)]
// edition 2015
pub trait Foo {
    fn foo(usize);
}
fn main() {}
```

This will produce:

```

error: anonymous parameters are deprecated and will be removed in the next
edition
--> lint_example.rs:4:12
   |
4  |     fn foo(usize);
   |               ^^^^^ help: try naming the parameter or explicitly ignoring it:
   | `_: usize`
   |
   = warning: this is accepted in the current edition (Rust 2015) but is a hard
error in Rust 2018!
   = note: for more information, see issue #41686 <https://github.com/rust-
lang/rust/issues/41686>
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(anonymous_parameters)]
   |               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

Explanation

This syntax is mostly a historical accident, and can be worked around quite easily by adding an `_` pattern or a descriptive identifier:

```

trait Foo {
    fn foo(_: usize);
}

```

This syntax is now a hard error in the 2018 edition. In the 2015 edition, this lint is "warn" by default. This lint enables the `cargo fix` tool with the `--edition` flag to automatically transition old code from the 2015 edition to 2018. The tool will run this lint and automatically apply the suggested fix from the compiler (which is to add `_` to each parameter). This provides a completely automated way to update old code for a new edition. See [issue #41686](https://github.com/rust-lang/rust/issues/41686) for more details.

array-into-iter

The `array_into_iter` lint detects calling `into_iter` on arrays.

Example

```
[1, 2, 3].into_iter().for_each(|n| { *n; });
```

This will produce:

```
warning: this method call resolves to `<[T; N] as IntoIterator>::into_iter`
(due to backwards compatibility), but will resolve to `<[T; N] as
IntoIterator>::into_iter` in Rust 2021
--> lint_example.rs:3:11
   |
3 | [1, 2, 3].into_iter().for_each(|n| { *n; });
   |           ^^^^^^^^^^^
   |
   = warning: this changes meaning in Rust 2021
   = note: for more information, see <https://doc.rust-lang.org/nightly/edition-guide/rust-2021/IntoIterator-for-arrays.html>
   = note: `#[warn(array_into_iter)]` on by default
help: use `.iter()` instead of `.into_iter()` to avoid ambiguity
   |
3 | [1, 2, 3].iter().for_each(|n| { *n; });
   |           ~~~~
help: or use `IntoIterator::into_iter(..)` instead of `.into_iter()` to
explicitly iterate by value
   |
3 | IntoIterator::into_iter([1, 2, 3]).for_each(|n| { *n; });
   | +++++~
```

Explanation

Since Rust 1.53, arrays implement `IntoIterator`. However, to avoid breakage, `array.into_iter()` in Rust 2015 and 2018 code will still behave as `(&array).into_iter()`, returning an iterator over references, just like in Rust 1.52 and earlier. This only applies to the method call syntax `array.into_iter()`, not to any other syntax such as `for _ in array` or `IntoIterator::into_iter(array)`.

asm-sub-register

The `asm_sub_register` lint detects using only a subset of a register for inline asm inputs.

Example

```
#[cfg(target_arch="x86_64")]
use std::arch::asm;

fn main() {
    #[cfg(target_arch="x86_64")]
    unsafe {
        asm!("mov {0}, {0}", in(reg) 0i16);
    }
}
```

This will produce:

```
warning: formatting may not be suitable for sub-register argument
--> src/main.rs:7:19
7 |         asm!("mov {0}, {0}", in(reg) 0i16);
  |                   ^^^  ^^^          ---- for this argument
= note: `#[warn(asm_sub_register)]` on by default
= help: use the `x` modifier to have the register formatted as `ax`
= help: or use the `r` modifier to keep the default formatting of `rax`
```

Explanation

Registers on some architectures can use different names to refer to a subset of the register. By default, the compiler will use the name for the full register size. To explicitly use a subset of the register, you can override the default by using a modifier on the template string operand to specify when subregister to use. This lint is issued if you pass in a value with a smaller data type than the default register size, to alert you of possibly using the incorrect width. To fix this, add the suggested modifier to the template, or cast the value to the correct size.

See [register template modifiers](#) in the reference for more details.

async-fn-in-trait

The `async_fn_in_trait` lint detects use of `async fn` in the definition of a publicly-reachable trait.

Example

```
pub trait Trait {
    async fn method(&self);
}
```

This will produce:

```
warning: use of `async fn` in public traits is discouraged as auto trait bounds
cannot be specified
--> lint_example.rs:2:5
   |
2 |     async fn method(&self);
   |     ^^^^^^
   |
= note: you can suppress this lint if you plan to use the trait only in your
own code, or do not care about auto traits like `Send` on the `Future`
= note: `#[warn(async_fn_in_trait)]` on by default
help: you can alternatively desugar to a normal `fn` that returns `impl Future`
and add any desired bounds such as `Send`, but these cannot be relaxed without
a breaking API change
   |
2 -     async fn method(&self);
2 +     fn method(&self) -> impl std::future::Future<Output = ()> + Send;
   |
```

Explanation

When `async fn` is used in a trait definition, the trait does not promise that the opaque `Future` returned by the associated function or method will implement any `auto traits` such as `Send`. This may be surprising and may make the associated functions or methods on the trait less useful than intended. On traits exposed publicly from a crate, this may affect downstream crates whose authors cannot alter the trait definition.

For example, this code is invalid:

```
pub trait Trait {
    async fn method(&self) {}
}

fn test<T: Trait>(x: T) {
    fn spawn<T: Send>(_: T) {}
    spawn(x.method()); // Not OK.
}
```

This lint exists to warn authors of publicly-reachable traits that they may want to consider desugaring the `async fn` to a normal `fn` that returns an opaque `impl Future<..> + Send` type.

For example, instead of:

```
pub trait Trait {
    async fn method(&self) {}
}
```

The author of the trait may want to write:

```
use core::future::Future;
pub trait Trait {
    fn method(&self) -> impl Future<Output = ()> + Send { async {} }
}
```

This still allows the use of `async fn` within impls of the trait. However, it also means that the trait will never be compatible with impls where the returned `Future` of the method does not implement `Send`.

Conversely, if the trait is used only locally, if it is never used in generic functions, or if it is only used in single-threaded contexts that do not care whether the returned `Future`

implements `Send`, then the lint may be suppressed.

bad-asm-style

The `bad_asm_style` lint detects the use of the `.intel_syntax` and `.att_syntax` directives.

Example

```
#[cfg(target_arch="x86_64")]
use std::arch::asm;

fn main() {
    #[cfg(target_arch="x86_64")]
    unsafe {
        asm!(
            ".att_syntax",
            "movq %{0}, %{0}", in(reg) 0usize
        );
    }
}
```

This will produce:

```
warning: avoid using `.att_syntax`, prefer using `options(att_syntax)` instead
--> src/main.rs:8:14
 8 |         ".att_syntax",
   |         ^^^^^^^^^^^^^
= note: `#[warn(bad_asm_style)]` on by default
```

Explanation

On x86, `asm!` uses the intel assembly syntax by default. While this can be switched using assembler directives like `.att_syntax`, using the `att_syntax` option is recommended instead because it will also properly prefix register placeholders with `%` as required by AT&T

syntax.

bare-trait-object

The lint `bare-trait-object` has been renamed to `bare-trait-objects`.

bare-trait-objects

The `bare_trait_objects` lint suggests using `dyn Trait` for trait objects.

Example

```
trait Trait { }

fn takes_trait_object(_: Box<Trait>) {
}
```

This will produce:

```
warning: trait objects without an explicit `dyn` are deprecated
--> lint_example.rs:4:30
  |
4 | fn takes_trait_object(_: Box<Trait>) {
  |                               ^^^^^^
  |
= warning: this is accepted in the current edition (Rust 2018) but is a hard
error in Rust 2021!
= note: for more information, see <https://doc.rust-lang.org/nightly/edition-
guide/rust-2021/warnings-promoted-to-error.html>
= note: `#[warn(bare_trait_objects)]` on by default
help: if this is an object-safe trait, use `dyn`
  |
4 | fn takes_trait_object(_: Box<dyn Trait>) {
  |                               +++
```

Explanation

Without the `dyn` indicator, it can be ambiguous or confusing when reading code as to whether or not you are looking at a trait object. The `dyn` keyword makes it explicit, and adds a symmetry to contrast with `impl Trait`.

boxed-slice-into-iter

The `boxed_slice_into_iter` lint detects calling `into_iter` on boxed slices.

Example

```
vec![1, 2, 3].into_boxed_slice().into_iter().for_each(|n| { *n; });
```

This will produce:

```
warning: this method call resolves to `&Box<T> as IntoIterator>::into_iter`
(due to backwards compatibility), but will resolve to `<Box<T> as
IntoIterator>::into_iter` in Rust 2024
--> lint_example.rs:3:34
  |
3 | vec![1, 2, 3].into_boxed_slice().into_iter().for_each(|n| { *n; });
  |                                     ^^^^^^^^^^^^^
  |
= warning: this changes meaning in Rust 2024
= note: `#[warn(boxed_slice_into_iter)]` on by default
help: use `.iter()` instead of `.into_iter()` to avoid ambiguity
  |
3 | vec![1, 2, 3].into_boxed_slice().iter().for_each(|n| { *n; });
  |                                     ~~~~~
help: or use `IntoIterator::into_iter(..)` instead of `.into_iter()` to
explicitly iterate by value
  |
3 | IntoIterator::into_iter(vec![1, 2, 3].into_boxed_slice()).for_each(|n| {
  | *n; });
  | +++++~
```

Explanation

Since Rust 1.80.0, boxed slices implement `IntoIterator`. However, to avoid breakage, `boxed_slice.into_iter()` in Rust 2015, 2018, and 2021 code will still behave as `(&boxed_slice).into_iter()`, returning an iterator over references, just like in Rust 1.79.0 and earlier. This only applies to the method call syntax `boxed_slice.into_iter()`, not to any other syntax such as `for _ in boxed_slice` or `IntoIterator::into_iter(boxed_slice)`.

break-with-label-and-loop

The `break_with_label_and_loop` lint detects labeled `break` expressions with an unlabeled loop as their value expression.

Example

```
'label: loop {
    break 'label loop { break 42; };
};
```

This will produce:

```
warning: this labeled break expression is easy to confuse with an unlabeled
break with a labeled value expression
```

```
--> lint_example.rs:3:5
```

```
3 | |      break 'label loop { break 42; };
  | |      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
= note: `#[warn(break_with_label_and_loop)]` on by default
help: wrap this expression in parentheses
```

```
3 | |      break 'label (loop { break 42; });
  | |                  +               +
```

Explanation

In Rust, loops can have a label, and `break` expressions can refer to that label to break out of specific loops (and not necessarily the innermost one). `break` expressions can also carry a value expression, which can be another loop. A labeled `break` with an unlabeled loop as its value expression is easy to confuse with an unlabeled `break` with a labeled loop and is thus discouraged (but allowed for compatibility); use parentheses around the loop expression to silence this warning. Unlabeled `break` expressions with labeled loops yield a hard error, which can also be silenced by wrapping the expression in parentheses.

byte-slice-in-packed-struct-with-derive

The `byte_slice_in_packed_struct_with_derive` lint detects cases where a byte slice field (`[u8]`) or string slice field (`str`) is used in a `packed` struct that derives one or more built-in traits.

Example

```
#[repr(packed)]
#[derive(Hash)]
struct FlexZeroSlice {
    width: u8,
    data: [u8],
}
```

This will produce:

```
warning: byte slice in a packed struct that derives a built-in trait
--> lint_example.rs:6:5
  |
3 | #[derive(Hash)]
  |          ---- in this derive macro expansion
...
6 |     data: [u8],
  |           ^^^^^^^^^^^^^
  |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #107457 <https://github.com/rust-lang/rust/issues/107457>
= help: consider implementing the trait by hand, or remove the `packed`
attribute
= note: `#[warn(byte_slice_in_packed_struct_with_derive)]` on by default
= note: this warning originates in the derive macro `Hash` (in Nightly
builds, run with -Z macro-backtrace for more info)
```

Explanation

This was previously accepted but is being phased out, because fields in packed structs are now required to implement `Copy` for `derive` to work. Byte slices and string slices are a temporary exception because certain crates depended on them.

clashing-extern-declarations

The `clashing_extern_declarations` lint detects when an `extern fn` has been declared with the same name but different types.

Example

```

mod m {
    extern "C" {
        fn foo();
    }
}

extern "C" {
    fn foo(_: u32);
}

```

This will produce:

```

warning: `foo` redeclared with a different signature
--> lint_example.rs:9:5
   |
4  |         fn foo();
   |         ----- `foo` previously declared here
...
9  |         fn foo(_: u32);
   |         ^^^^^^^^^^^^^^^^^ this signature doesn't match the previous declaration
   |
= note: expected `unsafe extern "C" fn()`
         found `unsafe extern "C" fn(u32)`
= note: `#[warn(clashing_extern_declarations)]` on by default

```

Explanation

Because two symbols of the same name cannot be resolved to two different functions at link time, and one function cannot possibly have two types, a clashing extern declaration is almost certainly a mistake. Check to make sure that the `extern` definitions are correct and equivalent, and possibly consider unifying them in one location.

This lint does not run between crates because a project may have dependencies which both rely on the same extern function, but declare it in a different (but valid) way. For example, they may both declare an opaque type for one or more of the arguments (which would end up distinct types), or use types that are valid conversions in the language the `extern fn` is defined in. In these cases, the compiler can't say that the clashing declaration is incorrect.

coherence-leak-check

The `coherence_leak_check` lint detects conflicting implementations of a trait that are only distinguished by the old leak-check code.

Example

```
trait SomeTrait { }
impl SomeTrait for for<'a> fn(&'a u8) { }
impl<'a> SomeTrait for fn(&'a u8) { }
```

This will produce:

```
warning: conflicting implementations of trait `SomeTrait` for type `for<'a>
fn(&'a u8)`
--> lint_example.rs:4:1
   |
3 | impl SomeTrait for for<'a> fn(&'a u8) { }
   | ----- first implementation here
4 | impl<'a> SomeTrait for fn(&'a u8) { }
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ conflicting implementation for `for<'a>
fn(&'a u8)`
   |
   = warning: the behavior may change in a future release
   = note: for more information, see issue #56105 <https://github.com/rust-
lang/rust/issues/56105>
   = note: this behavior recently changed as a result of a bug fix; see rust-
lang/rust#56105 for details
   = note: `#[warn(coherence_leak_check)]` on by default
```

Explanation

In the past, the compiler would accept trait implementations for identical functions that differed only in where the lifetime binder appeared. Due to a change in the borrow checker implementation to fix several bugs, this is no longer allowed. However, since this affects existing code, this is a [future-incompatible](#) lint to transition this to a hard error in the future.

Code relying on this pattern should introduce "newtypes", like `struct Foo(for<'a> fn(&'a u8))` .

See [issue #56105](#) for more details.

confusable-idents

The `confusable_idents` lint detects visually confusable pairs between identifiers.

Example

```
// Latin Capital Letter E With Caron
pub const Ě: i32 = 1;
// Latin Capital Letter E With Breve
pub const Ę: i32 = 2;
```

This will produce:

```
warning: found both `Ě` and `Ė` as identifiers, which look alike
--> lint_example.rs:5:11
   |
3 | pub const Ě: i32 = 1;
   |             - other identifier used here
4 | // Latin Capital Letter E With Breve
5 | pub const Ė: i32 = 2;
   |             ^ this identifier can be confused with `Ě`
   = note: `#[warn(confusable_idents)]` on by default
```

Explanation

This lint warns when different identifiers may appear visually similar, which can cause confusion.

The confusable detection algorithm is based on [Unicode® Technical Standard #39 Unicode](#)

[Security Mechanisms Section 4 Confusable Detection](#). For every distinct identifier X execute the function `skeleton(X)`. If there exist two distinct identifiers X and Y in the same crate where `skeleton(X) = skeleton(Y)` report it. The compiler uses the same mechanism to check if an identifier is too similar to a keyword.

Note that the set of confusable characters may change over time. Beware that if you "forbid" this lint that existing code may fail in the future.

const-eval-mutable-ptr-in-final-value

The `const_eval_mutable_ptr_in_final_value` lint detects if a mutable pointer has leaked into the final value of a `const` expression.

Example

```
pub enum JsValue {
    Undefined,
    Object(std::cell::Cell<bool>),
}

impl ::std::ops::Drop for JsValue {
    fn drop(&mut self) {}
}

const UNDEFINED: &JsValue = &JsValue::Undefined;

fn main() {
}
```

This will produce:

```
warning: encountered mutable pointer in final value of constant
--> lint_example.rs:10:1
   |
10 | const UNDEFINED: &JsValue = &JsValue::Undefined;
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #122153 <https://github.com/rust-
lang/rust/issues/122153>
= note: `[warn(const_eval_mutable_ptr_in_final_value)]` on by default
```

Explanation

In the 1.77 release, the const evaluation machinery adopted some stricter rules to reject expressions with values that could end up holding mutable references to state stored in static memory (which is inherently immutable).

This is a [future-incompatible](#) lint to ease the transition to an error. See [issue #122153](#) for more details.

const-evaluatable-unchecked

The `const_evaluatable_unchecked` lint detects a generic constant used in a type.

Example

```
const fn foo<T>() -> usize {
    if std::mem::size_of::<*mut T>() < 8 { // size of *mut T does not depend on
T
        4
    } else {
        8
    }
}

fn test<T>() {
    let _ = [0; foo::<T>()];
}
```

This will produce:

```
warning: cannot use constants which depend on generic parameters in types
--> lint_example.rs:11:17
   |
11 |     let _ = [0; foo::<T>()];
   |                      ^^^^^^^^^^^^^
   |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #76200 <https://github.com/rust-
lang/rust/issues/76200>
= note: `#[warn(const_evaluable_unchecked)]` on by default
```

Explanation

In the 1.43 release, some uses of generic parameters in array repeat expressions were accidentally allowed. This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #76200](#) for a more detailed description and possible fixes.

const-item-mutation

The `const_item_mutation` lint detects attempts to mutate a `const` item.

Example

```
const F00: [i32; 1] = [0];

fn main() {
    F00[0] = 1;
    // This will print "[0]".
    println!("{:?}", F00);
}
```

This will produce:

```
warning: attempting to modify a `const` item
--> lint_example.rs:4:5
 4 |         F00[0] = 1;
   |         ^^^^^^^^^^^
   = note: each usage of a `const` item creates a new temporary; the original
`const` item will not be modified
note: `const` item defined here
--> lint_example.rs:1:1
 1 | const F00: [i32; 1] = [0];
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   = note: `#[warn(const_item_mutation)]` on by default
```

Explanation

Trying to directly mutate a `const` item is almost always a mistake. What is happening in the example above is that a temporary copy of the `const` is mutated, but the original `const` is not. Each time you refer to the `const` by name (such as `F00` in the example above), a separate copy of the value is inlined at that location.

This lint checks for writing directly to a field (`F00.field = some_value`) or array entry (`F00[0] = val`), or taking a mutable reference to the const item (`&mut F00`), including through an autoderef (`F00.some_mut_self_method()`).

There are various alternatives depending on what you are trying to accomplish:

- First, always reconsider using mutable globals, as they can be difficult to use correctly, and can make the code more difficult to use or understand.
- If you are trying to perform a one-time initialization of a global:
 - If the value can be computed at compile-time, consider using const-compatible values (see [Constant Evaluation](#)).
 - For more complex single-initialization cases, consider using `std::sync::LazyLock`.
- If you truly need a mutable global, consider using a `static`, which has a variety of options:
 - Simple data types can be directly defined and mutated with an `atomic` type.
 - More complex types can be placed in a synchronization primitive like a `Mutex`, which can be initialized with one of the options listed above.
 - A `mutable static` is a low-level primitive, requiring `unsafe`. Typically This should be avoided in preference of something higher-level like one of the above.

dead-code

The `dead_code` lint detects unused, unexported items.

Example

```
fn foo() {}
```

This will produce:

```
warning: function `foo` is never used
--> lint_example.rs:2:4
   |
2  | fn foo() {}
   |     ^^^
   |
= note: `#[warn(dead_code)]` on by default
```

Explanation

Dead code may signal a mistake or unfinished code. To silence the warning for individual items, prefix the name with an underscore such as `_foo`. If it was intended to expose the item outside of the crate, consider adding a visibility modifier like `pub`.

To preserve the numbering of tuple structs with unused fields, change the unused fields to have unit type or use `PhantomData`.

Otherwise consider removing the unused code.

Limitations

Removing fields that are only used for side-effects and never read will result in behavioral changes. Examples of this include:

- If a field's value performs an action when it is dropped.
- If a field's type does not implement an auto trait (e.g. `Send`, `Sync`, `Unpin`).

For side-effects from dropping field values, this lint should be allowed on those fields. For side-effects from containing field types, `PhantomData` should be used.

dependency-on-unit-never-type-fallback

The `dependency_on_unit_never_type_fallback` lint detects cases where code compiles with `never type fallback` being `()`, but will stop compiling with fallback being `!`.

Example

```

#![deny(dependency_on_unit_never_type_fallback)]
fn main() {
    if true {
        // return has type `!` which, in some cases, causes never type fallback
        return
    } else {
        // the type produced by this call is not specified explicitly,
        // so it will be inferred from the previous branch
        Default::default()
    };
    // depending on the fallback, this may compile (because `()` implements
    // `Default`),
    // or it may not (because `!` does not implement `Default`)
}

```

This will produce:

```

error: this function depends on never type fallback being `()`
--> lint_example.rs:2:1
|
2 | fn main() {
|   ^^^^^^^^^^^
|
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #123748 <https://github.com/rust-
lang/rust/issues/123748>
= help: specify the types explicitly
note: in edition 2024, the requirement `!: Default` will fail
--> lint_example.rs:9:9
|
9 |         Default::default()
|         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
note: the lint level is defined here
--> lint_example.rs:1:9
1 | #![deny(dependency_on_unit_never_type_fallback)]
|   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

Explanation

Due to historic reasons never type fallback was `()`, meaning that `!` got spontaneously coerced to `()`. There are plans to change that, but they may make the code such as above not compile. Instead of depending on the fallback, you should specify the type explicitly:

```
if true {
    return
} else {
    // type is explicitly specified, fallback can't hurt us no more
    <() as Default>::default()
};
```

See [Tracking Issue for making `!` fall back to `!`](#).

deprecated

The `deprecated` lint detects use of deprecated items.

Example

```
#[deprecated]
fn foo() {}

fn bar() {
    foo();
}
```

This will produce:

```
warning: use of deprecated function `main::foo`
--> lint_example.rs:6:5
   |
6  |     foo();
   |     ^^^
   |
= note: `#[warn(deprecated)]` on by default
```

Explanation

Items may be marked "deprecated" with the `deprecated` attribute to indicate that they should no longer be used. Usually the attribute should include a note on what to use instead, or check the documentation.

deprecated-where-clause-location

The `deprecated_where_clause_location` lint detects when a where clause in front of the equals in an associated type.

Example

```
trait Trait {
    type Assoc<'a> where Self: 'a;
}

impl Trait for () {
    type Assoc<'a> where Self: 'a = ();
}
```

This will produce:

```
warning: where clause not allowed here
--> lint_example.rs:7:18
   |
7  |     type Assoc<'a> where Self: 'a = ();
   |                               ^^^^^^^^^^^^^^^^^
   |
   = note: see issue #89122 <https://github.com/rust-lang/rust/issues/89122> for
more information
   = note: `#[warn(deprecated_where_clause_location)]` on by default
help: move it to the end of the type declaration
   |
7 -     type Assoc<'a> where Self: 'a = ();
7 +     type Assoc<'a> = () where Self: 'a;
```

Explanation

The preferred location for where clauses on associated types is after the type. However, for most of generic associated types development, it was only accepted before the equals. To provide a transition period and further evaluate this change, both are currently accepted. At some point in the future, this may be disallowed at an edition boundary; but, that is undecided currently.

deref-into-dyn-supertrait

The `deref_into_dyn_supertrait` lint is output whenever there is a use of the `Deref` implementation with a `dyn SuperTrait` type as `Output`.

These implementations will become shadowed when the `trait_upcasting` feature is stabilized. The `deref` functions will no longer be called implicitly, so there might be behavior change.

Example

```
#![deny(deref_into_dyn_supertrait)]
#![allow(dead_code)]

use core::ops::Deref;

trait A {}
trait B: A {}

impl<'a> Deref for dyn 'a + B {
    type Target = dyn A;
    fn deref(&self) -> &Self::Target {
        todo!()
    }
}

fn take_a(_: &dyn A) { }

fn take_b(b: &dyn B) {
    take_a(b);
}
```

This will produce:

```
error: this `Deref` implementation is covered by an implicit supertrait coercion
--> lint_example.rs:9:1
|
9 | impl<'a> Deref for dyn 'a + B {
|   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ `dyn B` implements `Deref<Target = dyn A>`,
which conflicts with supertrait `A`
10 |     type Target = dyn A;
    |     ----- target type is a supertrait of `dyn B`
    |
= warning: this will change its meaning in a future release!
= note: for more information, see issue #89460 <https://github.com/rust-  
lang/rust/issues/89460>
note: the lint level is defined here
--> lint_example.rs:1:9
|
1 | #[deny(deref_into_dyn_supertrait)]
  |   ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

The dyn upcasting coercion feature adds new coercion rules, taking priority over certain other coercion rules, which will cause some behavior change.

deref-nullptr

The `deref_nullptr` lint detects when an null pointer is dereferenced, which causes [undefined behavior](#).

Example

```
use std::ptr;
unsafe {
    let x = &*ptr::null::<i32>();
    let x = ptr::addr_of!(*ptr::null::<i32>());
    let x = *(0 as *const i32);
}
```

This will produce:

```
warning: dereferencing a null pointer
--> lint_example.rs:5:14
   |
5  |     let x = &*ptr::null::<i32>();
   |               ^^^^^^^^^^^^^^^^^^^^^ this code causes undefined behavior when
executed
   |
   = note: `[warn(deref_nullptr)]` on by default
```

```
warning: dereferencing a null pointer
--> lint_example.rs:6:27
   |
6  |     let x = ptr::addr_of!(*ptr::null::<i32>());
   |                           ^^^^^^^^^^^^^^^^^^^^^ this code causes undefined
behavior when executed
```

```
warning: dereferencing a null pointer
--> lint_example.rs:7:13
   |
7  |     let x = *(0 as *const i32);
   |               ^^^^^^^^^^^^^^^^^ this code causes undefined behavior when
executed
```

Explanation

Dereferencing a null pointer causes [undefined behavior](#) even as a place expression, like `&*(0 as *const i32)` Or `addr_of!(*(0 as *const i32))`.

drop-bounds

The `drop_bounds` lint checks for generics with `std::ops::Drop` as bounds.

Example

```
fn foo<T: Drop>() {}
```

This will produce:

```
warning: bounds on `T: Drop` are most likely incorrect, consider instead using
`std::mem::needs_drop` to detect whether a type can be trivially dropped
--> lint_example.rs:2:11
   |
2  | fn foo<T: Drop>() {}
   |           ^^^^
   |
= note: `[warn(drop_bounds)]` on by default
```

Explanation

A generic trait bound of the form `T: Drop` is most likely misleading and not what the programmer intended (they probably should have used `std::mem::needs_drop` instead).

`Drop` bounds do not actually indicate whether a type can be trivially dropped or not, because a composite type containing `Drop` types does not necessarily implement `Drop` itself. Naïvely, one might be tempted to write an implementation that assumes that a type can be trivially dropped while also supplying a specialization for `T: Drop` that actually calls the destructor. However, this breaks down e.g. when `T` is `String`, which does not implement `Drop` itself but contains a `Vec`, which does implement `Drop`, so assuming `T` can be trivially dropped would lead to a memory leak here.

Furthermore, the `Drop` trait only contains one method, `Drop::drop`, which may not be called explicitly in user code (E0040), so there is really no use case for using `Drop` in trait bounds, save perhaps for some obscure corner cases, which can use `#[allow(drop_bounds)]`.

dropping-copy-types

The `dropping_copy_types` lint checks for calls to `std::mem::drop` with a value that derives

the Copy trait.

Example

```
let x: i32 = 42; // i32 implements Copy
std::mem::drop(x); // A copy of x is passed to the function, leaving the
                  // original unaffected
```

This will produce:

```
warning: calls to `std::mem::drop` with a value that implements `Copy` does
nothing
--> lint_example.rs:3:1
   |
3  | std::mem::drop(x); // A copy of x is passed to the function, leaving the
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |                                     |
   |                                     argument has type `i32`
   |
   = note: `#[warn(dropping_copy_types)]` on by default
help: use `let _ = ...` to ignore the expression or result
   |
3  - std::mem::drop(x); // A copy of x is passed to the function, leaving the
3  + let _ = x; // A copy of x is passed to the function, leaving the
   |
```

Explanation

Calling `std::mem::drop` [does nothing for types that implement Copy](#), since the value will be copied and moved into the function on invocation.

dropping-references

The `dropping_references` lint checks for calls to `std::mem::drop` with a reference instead of an owned value.

Example

```
let mut lock_guard = mutex.lock();
std::mem::drop(&lock_guard); // Should have been drop(lock_guard), mutex
// still locked
operation_that_requires_mutex_to_be_unlocked();
```

This will produce:

```
warning: calls to `std::mem::drop` with a reference instead of an owned value
does nothing
--> lint_example.rs:5:1
5 | std::mem::drop(&lock_guard); // Should have been drop(lock_guard), mutex
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
  | argument has type `&Result<MutexGuard<'_, i32>,
PoisonError<MutexGuard<'_, i32>>>`
  |
  = note: `#[warn(dropping_references)]` on by default
help: use `let _ = ...` to ignore the expression or result
5 - std::mem::drop(&lock_guard); // Should have been drop(lock_guard), mutex
5 + let _ = &lock_guard; // Should have been drop(lock_guard), mutex
```

Explanation

Calling `drop` on a reference will only drop the reference itself, which is a no-op. It will not call the `drop` method (from the `Drop` trait implementation) on the underlying referenced value, which is likely what was intended.

duplicate-macro-attributes

The `duplicate_macro_attributes` lint detects when a `#[test]`-like built-in macro attribute is duplicated on an item. This lint may trigger on `bench`, `cfg_eval`, `test` and `test_case`.

Example

```
#[test]
#[test]
fn foo() {}
```

This will produce:

```
warning: duplicated attribute
--> src/lib.rs:2:1
   |
2  | #[test]
   | ^^^^^^^
   |
   = note: `#[warn(duplicate_macro_attributes)]` on by default
```

Explanation

A duplicated attribute may erroneously originate from a copy-paste and the effect of it being duplicated may not be obvious or desirable.

For instance, doubling the `#[test]` attributes registers the test to be run twice with no change to its environment.

dyn-drop

The `dyn_drop` lint checks for trait objects with `std::ops::Drop`.

Example

```
fn foo(_x: Box<dyn Drop>) {}
```

This will produce:

warning: types that do not implement `Drop` can still have drop glue, consider instead using `std::mem::needs\_drop` to detect whether a type is trivially dropped

```
--> lint_example.rs:2:20
  |
2 | fn foo(_x: Box<dyn Drop>) {}
  |                ^^^^
  |
= note: `#[warn(dyn_drop)]` on by default
```

Explanation

A trait object bound of the form `dyn Drop` is most likely misleading and not what the programmer intended.

`Drop` bounds do not actually indicate whether a type can be trivially dropped or not, because a composite type containing `Drop` types does not necessarily implement `Drop` itself. Naïvely, one might be tempted to write a deferred drop system, to pull cleaning up memory out of a latency-sensitive code path, using `dyn Drop` trait objects. However, this breaks down e.g. when `T` is `String`, which does not implement `Drop`, but should probably be accepted.

To write a trait object bound that accepts anything, use a placeholder trait with a blanket implementation.

```
trait Placeholder {}
impl<T> Placeholder for T {}
fn foo(_x: Box<dyn Placeholder>) {}
```

ellipsis-inclusive-range-patterns

The `ellipsis_inclusive_range_patterns` lint detects the `... range pattern`, which is deprecated.

Example

```
let x = 123;
match x {
    0...100 => {}
    _ => {}
}
```

This will produce:

```
warning: `...` range patterns are deprecated
--> lint_example.rs:4:6
   |
4  |     0...100 => {}
   |     ^^^ help: use `..=` for an inclusive range
   |
= warning: this is accepted in the current edition (Rust 2018) but is a hard
error in Rust 2021!
= note: for more information, see <https://doc.rust-lang.org/nightly/edition-
guide/rust-2021/warnings-promoted-to-error.html>
= note: `[warn(ellipsis_inclusive_range_patterns)]` on by default
```

Explanation

The `...` range pattern syntax was changed to `..=` to avoid potential confusion with the [..`..` range expression](#). Use the new form instead.

exported-private-dependencies

The `exported_private_dependencies` lint detects private dependencies that are exposed in a public interface.

Example

```
pub fn foo() -> Option<some_private_dependency::Thing> {
    None
}
```

This will produce:

```
warning: type `bar::Thing` from private dependency 'bar' in public interface
--> src/lib.rs:3:1
   |
3  | pub fn foo() -> Option<bar::Thing> {
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
= note: `#[warn(exported_private_dependencies)]` on by default
```

Explanation

Dependencies can be marked as "private" to indicate that they are not exposed in the public interface of a crate. This can be used by Cargo to independently resolve those dependencies because it can assume it does not need to unify them with other packages using that same dependency. This lint is an indication of a violation of that contract.

To fix this, avoid exposing the dependency in your public interface. Or, switch the dependency to a public dependency.

Note that support for this is only available on the nightly channel. See [RFC 1977](#) for more details, as well as the [Cargo documentation](#).

for-loops-over-fallibles

The `for_loops_over_fallibles` lint checks for `for` loops over `Option` or `Result` values.

Example

```
let opt = Some(1);
for x in opt { /* ... */}
```

This will produce:

```
warning: for loop over an `Option`. This is more readably written as an `if
let` statement
--> lint_example.rs:3:10
  |
3 | for x in opt { /* ... */}
  |             ^^^
  |
  = note: `[warn(for_loops_over_fallibles)]` on by default
help: to check pattern in a loop use `while let`
  |
3 | while let Some(x) = opt { /* ... */}
  | ~~~~~~
help: consider using `if let` to clear intent
  |
3 | if let Some(x) = opt { /* ... */}
  | ~~~~~~
```

Explanation

Both `Option` and `Result` implement `IntoIterator` trait, which allows using them in a `for` loop. `for` loop over `Option` or `Result` will iterate either 0 (if the value is `None` / `Err(_)`) or 1 time (if the value is `Some(_)` / `Ok(_)`). This is not very useful and is more clearly expressed via `if let`.

`for` loop can also be accidentally written with the intention to call a function multiple times, while the function returns `Some(_)`, in these cases `while let` loop should be used instead.

The "intended" use of `IntoIterator` implementations for `Option` and `Result` is passing them to generic code that expects something implementing `IntoIterator`. For example using `.chain(option)` to optionally add a value to an iterator.

forbidden-lint-groups

The `forbidden_lint_groups` lint detects violations of `forbid` applied to a lint group. Due to a bug in the compiler, these used to be overlooked entirely. They now generate a warning.

Example

```
#![forbid(warnings)]
#![deny(bad_style)]

fn main() {}
```

This will produce:

```
warning: deny(bad_style) incompatible with previous forbid
--> lint_example.rs:2:9
  |
1 | #![forbid(warnings)]
  |           ----- `forbid` level set here
2 | #![deny(bad_style)]
  |           ^^^^^^^^^^ overruled by previous forbid
  |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #81670 <https://github.com/rust-lang/rust/issues/81670>
= note: `#[warn(forbidden_lint_groups)]` on by default
```

Recommended fix

If your crate is using `#![forbid(warnings)]`, we recommend that you change to `#![deny(warnings)]`.

Explanation

Due to a compiler bug, applying `forbid` to lint groups previously had no effect. The bug is now fixed but instead of enforcing `forbid` we issue this future-compatibility warning to avoid breaking existing crates.

forgetting-copy-types

The `forgetting_copy_types` lint checks for calls to `std::mem::forget` with a value that derives the `Copy` trait.

Example

```
let x: i32 = 42; // i32 implements Copy
std::mem::forget(x); // A copy of x is passed to the function, leaving the
                      // original unaffected
```

This will produce:

```
warning: calls to `std::mem::forget` with a value that implements `Copy` does
nothing
--> lint_example.rs:3:1
   |
3  | std::mem::forget(x); // A copy of x is passed to the function, leaving the
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |                                     |
   |                                     argument has type `i32`
   |
   = note: `[warn(forgetting_copy_types)]` on by default
help: use `let _ = ...` to ignore the expression or result
   |
3  - std::mem::forget(x); // A copy of x is passed to the function, leaving the
3  + let _ = x; // A copy of x is passed to the function, leaving the
   |
```

Explanation

An alternative, but also valid, explanation is that Copy types do not implement the Drop trait, which means they have no destructors. Without a destructor, there is nothing for `std::mem::forget` to ignore.

The `forgetting_references` lint checks for calls to `std::mem::forget` with a reference instead of an owned value.

```
let x = Box::new(1);
std::mem::forget(&x); // Should have been forget(x), x will still be dropped
```

```
warning: calls to `std::mem::forget` with a reference instead of an owned value
does nothing
--> lint_example.rs:3:1
|
3 | std::mem::forget(&x); // Should have been forget(x), x will still be
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^--^
  |                               |
  |                               argument has type `&Box<i32>`
= note: `#[warn(forgetting_references)]` on by default
help: use `let _ = ...` to ignore the expression or result
3 - std::mem::forget(&x); // Should have been forget(x), x will still be
  |
3 + let _ = &x; // Should have been forget(x), x will still be dropped
  |
```

Explanation

Calling `forget` on a reference will only forget the reference itself, which is a no-op. It will not forget the underlying referenced value, which is likely what was intended.

function-item-references

The `function_item_references` lint detects function references that are formatted with `fmt::Pointer` or transmuted.

Example

```
fn foo() { }
```

```
fn main() {
    println!("{:p}", &foo);
}
```

This will produce:

```
warning: taking a reference to a function item does not give a function pointer
--> lint_example.rs:4:22
   |
4  |     println!("{:p}", &foo);
   |                      ^^^^^ help: cast `foo` to obtain a function pointer:
   | `foo as fn()`
   |
   = note: `#[warn(function_item_references)]` on by default
```

Explanation

Taking a reference to a function may be mistaken as a way to obtain a pointer to that function. This can give unexpected results when formatting the reference as a pointer or transmuting it. This lint is issued when function references are formatted as pointers,

passed as arguments bound by `fmt::Pointer` or transmuted.

hidden-glob-reexports

The `hidden_glob_reexports` lint detects cases where glob re-export items are shadowed by private items.

Example

```
#![deny(hidden_glob_reexports)]

pub mod upstream {
    mod inner { pub struct Foo {}; pub struct Bar {}; }
    pub use self::inner::*;
    struct Foo {} // private item shadows `inner::Foo`
}

// mod downstream {
//     fn test() {
//         let _ = crate::upstream::Foo; // inaccessible
//     }
// }

pub fn main() {}
```

This will produce:

```

error: private item shadows public glob re-export
--> lint_example.rs:6:5
  |
6 |     struct Foo {} // private item shadows `inner::Foo`
  |     ^^^^^^^^^^^^^^^^^
note: the name `Foo` in the type namespace is supposed to be publicly re-exported here
--> lint_example.rs:5:13
  |
5 |     pub use self::inner::*;
  |           ^^^^^^^^^^^^^^^
note: but the private item here shadows it
--> lint_example.rs:6:5
  |
6 |     struct Foo {} // private item shadows `inner::Foo`
  |     ^^^^^^^^^^^^^^^^^
note: the lint level is defined here
--> lint_example.rs:1:9
  |
1 | #![deny(hidden_glob_reexports)]
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^

```

Explanation

This was previously accepted without any errors or warnings but it could silently break a crate's downstream user code. If the `struct Foo` was added, `dep::inner::Foo` would silently become inaccessible and trigger a "struct Foo is private" visibility error at the downstream use site.

impl-trait-redundant-captures

The `impl_trait_redundant_captures` lint warns against cases where use of the precise capturing `use<...>` syntax is not needed.

In the 2024 edition, `impl Trait`s will capture all lifetimes in scope. If precise-capturing `use<...>` syntax is used, and the set of parameters that are captured are *equal* to the set of

parameters in scope, then the syntax is redundant, and can be removed.

Example

```
fn test<'a>(x: &'a i32) -> impl Sized + use<'a> { x }
```

This will produce:

```
error: all possible in-scope parameters are already captured, so `use<...>`
syntax is redundant
--> lint_example.rs:5:28
   |
5  | fn test<'a>(x: &'a i32) -> impl Sized + use<'a> { x }
   |                                     ^^^^^^^^^^^^^^^^^^-----
   |                                     |
   |                                     help: remove the `use<...>` syntax

note: the lint level is defined here
--> lint_example.rs:3:9
   |
3  | #[deny(impl_trait_redundant_captures)]
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

To fix this, remove the `use<'a>` , since the lifetime is already captured since it is in scope.

improper-ctypes

The `improper_ctypes` lint detects incorrect use of types in foreign modules.

Example

```
extern "C" {
    static STATIC: String;
}
```

This will produce:

```
warning: `extern` block uses type `String`, which is not FFI-safe
--> lint_example.rs:3:20
  |
3 |     static STATIC: String;
  |                      ^^^^^^^ not FFI-safe
  |
= help: consider adding a `#[repr(C)]` or `#[repr(transparent)]` attribute to
this struct
= note: this struct has unspecified layout
= note: `#[warn(improper_ctypes)]` on by default
```

Explanation

The compiler has several checks to verify that types used in `extern` blocks are safe and follow certain rules to ensure proper compatibility with the foreign interfaces. This lint is issued when it detects a probable mistake in a definition. The lint usually should provide a description of the issue, along with possibly a hint on how to resolve it.

improper-ctypes-definitions

The `improper_ctypes_definitions` lint detects incorrect use of `extern function` definitions.

Example

```
pub extern "C" fn str_type(p: &str) { }
```

This will produce:

```

warning: `extern` fn uses type `str`, which is not FFI-safe
--> lint_example.rs:3:31
   |
3  | pub extern "C" fn str_type(p: &str) { }
   |                               ^^^^ not FFI-safe
   |
= help: consider using `*const u8` and a length instead
= note: string slices have no C equivalent
= note: `#[warn(improper_ctypes_definitions)]` on by default

```

Explanation

There are many parameter and return types that may be specified in an `extern` function that are not compatible with the given ABI. This lint is an alert that these types should not be used. The lint usually should provide a description of the issue, along with possibly a hint on how to resolve it.

incomplete-features

The `incomplete_features` lint detects unstable features enabled with the [feature attribute](#) that may function improperly in some or all cases.

Example

```
#![feature(generic_const_exprs)]
```

This will produce:

```
warning: the feature `generic_const_exprs` is incomplete and may not be safe to
use and/or cause compiler crashes
--> lint_example.rs:1:12
   |
1 | #![feature(generic_const_exprs)]
   |           ^^^^^^^^^^^^^^^^^^^^^
   |
= note: see issue #76560 <https://github.com/rust-lang/rust/issues/76560> for
more information
= note: `#[warn(incomplete_features)]` on by default
```

Explanation

Although it is encouraged for people to experiment with unstable features, some of them are known to be incomplete or faulty. This lint is a signal that the feature has not yet been finished, and you may experience problems with it.

inline-no-sanitize

The `inline_no_sanitize` lint detects incompatible use of `#[inline(always)]` and `#[no_sanitize(...)]`.

Example

```
#![feature(no_sanitize)]

#[inline(always)]
#[no_sanitize(address)]
fn x() {}

fn main() {
    x()
}
```

This will produce:


```
warning: `no_sanitizе` will have no effect after inlining
--> lint_example.rs:4:1
|
4 | #[no_sanitizе(address)]
|  ^^^^^^^^^^^^^^^^^^^^^
|
note: inlining requested here
--> lint_example.rs:3:1
|
3 | #[inline(always)]
|  ^^^^^^^^^^^^^^^
= note: `#[warn(inline_no_sanitizе)]` on by default
```

Explanation

The use of the `#[inline(always)]` attribute prevents the the `#[no_sanitize(...)]` attribute from working. Consider temporarily removing `inline` attribute.

internal-features

The `internal_features` lint detects unstable features enabled with the `feature` attribute that are internal to the compiler or standard library.

Example

```
#![feature(rustc_attrs)]
```

This will produce:

```
warning: the feature `rustc_attrs` is internal to the compiler or standard
library
--> lint_example.rs:1:12
   |
1  | #![feature(rustc_attrs)]
   |              ^^^^^^^^^^^^^^^
   |
= note: using it is strongly discouraged
= note: `#[warn(internal_features)]` on by default
```

Explanation

These features are an implementation detail of the compiler and standard library and are not supposed to be used in user code.

invalid-from-utf8

The `invalid_from_utf8` lint checks for calls to `std::str::from_utf8` and `std::str::from_utf8_mut` with a known invalid UTF-8 value.

Example

```
std::str::from_utf8(b"Ru\x82st");
```

This will produce:

```

warning: calls to `std::str::from_utf8` with a invalid literal always return an
error
--> lint_example.rs:3:1
   |
3 | std::str::from_utf8(b"Ru\x82st");
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   |                                     |
   |                                     the literal was valid UTF-8 up to the 2 bytes
   |
= note: `#[warn(invalid_from_utf8)]` on by default

```

Explanation

Trying to create such a `str` would always return an error as per documentation for `std::str::from_utf8` and `std::str::from_utf8_mut`.

invalid-macro-export-arguments

The `invalid_macro_export_arguments` lint detects cases where `#[macro_export]` is being used with invalid arguments.

Example

```

#![deny(invalid_macro_export_arguments)]

#[macro_export(invalid_parameter)]
macro_rules! myMacro {
    () => {
        // [...]
    }
}

#[macro_export(too, many, items)]

```

This will produce:

```

error: `invalid_parameter` isn't a valid `#[macro_export]` argument
--> lint_example.rs:4:16
   |
4  | #[macro_export(invalid_parameter)]
   |                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(invalid_macro_export_arguments)]
   |          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

Explanation

The only valid argument is `#[macro_export(local_inner_macros)]` or no argument (`#[macro_export]`). You can't have multiple arguments in a `#[macro_export(..)]`, or mention arguments other than `local_inner_macros`.

invalid-nan-comparisons

The `invalid_nan_comparisons` lint checks comparison with `f32::NAN` or `f64::NAN` as one of the operand.

Example

```

let a = 2.3f32;
if a == f32::NAN {}

```

This will produce:

```

warning: incorrect NaN comparison, NaN cannot be directly compared to itself
--> lint_example.rs:3:4
   |
3  | if a == f32::NAN {}
   |          ^^^^^^^
   |
   = note: `[warn(invalid_nan_comparisons)]` on by default
help: use `f32::is_nan()` or `f64::is_nan()` instead
   |
3  - if a == f32::NAN {}
3  + if a.is_nan() {}
   |

```

Explanation

NaN does not compare meaningfully to anything – not even itself – so those comparisons are always false.

invalid-value

The `invalid_value` lint detects creating a value that is not valid, such as a null reference.

Example

```

unsafe {
    let x: &'static i32 = std::mem::zeroed();
}

```

This will produce:

```

warning: the type `i32` does not permit zero-initialization
--> lint_example.rs:4:27
  |
4 |     let x: &'static i32 = std::mem::zeroed();
  |                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |                               |
  |                               this code causes undefined behavior when executed
  |                               help: use `MaybeUninit<T>` instead, and only call
`assume_init` after initialization is done
  |
= note: references must be non-null
= note: `#[warn(invalid_value)]` on by default

```

Explanation

In some situations the compiler can detect that the code is creating an invalid value, which should be avoided.

In particular, this lint will check for improper use of `mem::zeroed`, `mem::uninitialized`, `mem::transmute`, and `MaybeUninit::assume_init` that can cause [undefined behavior](#). The lint should provide extra information to indicate what the problem is and a possible solution.

irrefutable-let-patterns

The `irrefutable_let_patterns` lint detects [irrefutable patterns](#) in `if let s`, `while let s`, and `if let guards`.

Example

```

if let _ = 123 {
    println!("always runs!");
}

```

This will produce:

```
warning: irrefutable `if let` pattern
--> lint_example.rs:2:4
  |
2 | if let _ = 123 {
  |      ^^^^^^^^^^^
  |
= note: this pattern will always match, so the `if let` is useless
= help: consider replacing the `if let` with a `let`
= note: `#[warn(irrefutable_let_patterns)]` on by default
```

Explanation

There usually isn't a reason to have an irrefutable pattern in an `if let` or `while let` statement, because the pattern will always match successfully. A `let` or `loop` statement will suffice. However, when generating code with a macro, forbidding irrefutable patterns would require awkward workarounds in situations where the macro doesn't know if the pattern is refutable or not. This lint allows macros to accept this form, while alerting for a possibly incorrect use in normal code.

See [RFC 2086](#) for more details.

large-assignments

The `large_assignments` lint detects when objects of large types are being moved around.

Example

```
let x = [0; 50000];
let y = x;
```

produces:

```
warning: moving a large value
--> $DIR/move-large.rs:1:3
let y = x;
    - Copied large value here
```

Explanation

When using a large type in a plain assignment or in a function argument, idiomatic code can be inefficient. Ideally appropriate optimizations would resolve this, but such optimizations are only done in a best-effort manner. This lint will trigger on all sites of large moves and thus allow the user to resolve them in code.

late-bound-lifetime-arguments

The `late_bound_lifetime_arguments` lint detects generic lifetime arguments in path segments with late bound lifetime parameters.

Example

```
struct S;

impl S {
    fn late(self, _: &u8, _: &u8) {}
}

fn main() {
    S.late::<'static>(&0, &0);
}
```

This will produce:


```
warning: cannot specify lifetime arguments explicitly if late bound lifetime
parameters are present
--> lint_example.rs:8:14
   |
4  |     fn late(self, _: &u8, _: &u8) {}
   |                               - the late bound lifetime parameter is introduced here
...
8  |     S.late::<'static>(&0, &0);
   |               ^^^^^^^
   |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #42868 <https://github.com/rust-
lang/rust/issues/42868>
= note: `#[warn(late_bound_lifetime_arguments)]` on by default
```

Explanation

It is not clear how to provide arguments for early-bound lifetime parameters if they are intermixed with late-bound parameters in the same list. For now, providing any explicit arguments will trigger this lint if late-bound parameters are present, so in the future a solution can be adopted without hitting backward compatibility issues. This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #42868](#) for more details, along with a description of the difference between early and late-bound parameters.

legacy-derive-helpers

The `legacy_derive_helpers` lint detects derive helper attributes that are used before they are introduced.

Example

```
#[serde(rename_all = "camelCase")]
#[derive(Deserialize)]
struct S { /* fields */ }
```

produces:

```
warning: derive helper attribute is used before it is introduced
--> $DIR/legacy-derive-helpers.rs:1:3
   |
1  | #[serde(rename_all = "camelCase")]
   |      ^^^^^^
...
2  | #[derive(Deserialize)]
   |           ----- the attribute is introduced here
```

Explanation

Attributes like this work for historical reasons, but attribute expansion works in left-to-right order in general, so, to resolve `#[serde]`, compiler has to try to "look into the future" at not yet expanded part of the item, but such attempts are not always reliable.

To fix the warning place the helper attribute after its corresponding derive.

```
#[derive(Deserialize)]
#[serde(rename_all = "camelCase")]
struct S { /* fields */ }
```

map-unit-fn

The `map_unit_fn` lint checks for `Iterator::map` receive a callable that returns `()`.

Example

```
fn foo(items: &mut Vec<u8>) {
    items.sort();
}

fn main() {
    let mut x: Vec<Vec<u8>> = vec![
        vec![0, 2, 1],
        vec![5, 4, 3],
    ];
    x.iter_mut().map(foo);
}
```

This will produce:

```
warning: `Iterator::map` call that discard the iterator's values
--> lint_example.rs:10:18
   |
10 | fn foo(items: &mut Vec<u8>) {
   | ----- this function returns `()`, which is likely
   | not what you wanted
...
10 |     x.iter_mut().map(foo);
   |                      ^^^^---^
   |                      |   |
   |                      |   called `Iterator::map` with callable that returns
   | `()`
   | after this call to map, the resulting iterator is `impl
   | Iterator<Item = ()>`, which means the only information carried by the iterator
   | is the number of items
   |
   | = note: `Iterator::map`, like many of the methods on `Iterator`, gets
   | executed lazily, meaning that its effects won't be visible until it is iterated
   | = note: `#[warn(map_unit_fn)]` on by default
   | help: you might have meant to use `Iterator::for_each`
10 |     x.iter_mut().for_each(foo);
   |                      ~~~~~~
```

Explanation

Mapping to `()` is almost always a mistake.

mixed-script-confusables

The `mixed_script_confusables` lint detects visually confusable characters in identifiers between different [scripts](#).

Example

```
// The Japanese katakana character ェ can be confused with the Han character 丂.  
const 丂: &'static str = "アイウ";
```

This will produce:

```
warning: the usage of Script Group `Japanese, Katakana` in this crate consists  
solely of mixed script confusables  
--> lint_example.rs:3:7  
   |  
3 | const 丂: &'static str = "アイウ";  
   |          ^^  
   |  
= note: the usage includes '丂' (U+30A8)  
= note: please recheck to make sure their usages are indeed what you want  
= note: `#[warn(mixed_script_confusables)]` on by default
```

Explanation

This lint warns when characters between different scripts may appear visually similar, which can cause confusion.

If the crate contains other identifiers in the same script that have non-confusable characters, then this lint will *not* be issued. For example, if the example given above has another identifier with katakana characters (such as `let カタカナ = 123;`), then this indicates that you are intentionally using katakana, and it will not warn about it.

Note that the set of confusable characters may change over time. Beware that if you "forbid" this lint that existing code may fail in the future.

named-arguments-used-positionally

The `named_arguments_used_positionally` lint detects cases where named arguments are only used positionally in format strings. This usage is valid but potentially very confusing.

Example

```
#![deny(named_arguments_used_positionally)]
fn main() {
    let _x = 5;
    println!("{}", _x = 1); // Prints 1, will trigger lint

    println!("{}", _x); // Prints 5, no lint emitted
    println!("{}", _x); // Prints 5, no lint emitted
}
```

This will produce:

```
error: named argument `_x` is not used by name
--> lint_example.rs:4:20
  |
4 |     println!("{}", _x = 1); // Prints 1, will trigger lint
  |                  --  ^^ this named argument is referred to by position in
formatting string
  |                  |
  |                  this formatting argument uses named argument `_x` by position
note: the lint level is defined here
--> lint_example.rs:1:9
  |
1 | #![deny(named_arguments_used_positionally)]
  |          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: use the named argument by name to avoid ambiguity
  |
4 |     println!("{}", _x); // Prints 1, will trigger lint
  |                  ++
```

Explanation

Rust formatting strings can refer to named arguments by their position, but this usage is potentially confusing. In particular, readers can incorrectly assume that the declaration of named arguments is an assignment (which would produce the unit type). For backwards compatibility, this is not a hard error.

never-type-fallback-flowing-into-unsafe

The `never_type_fallback_flowng_into_unsafe` lint detects cases where `never` type fallback affects `unsafe` function calls.

Never type fallback

When the compiler sees a value of type `!` it implicitly inserts a coercion (if possible), to allow type check to infer any type:

```
// this
let x: u8 = panic!();

// is (essentially) turned by the compiler into
let x: u8 = absurd(panic!());

// where absurd is a function with the following signature
// (it's sound, because `!` always marks unreachable code):
fn absurd<T>(never: !) -> T { ... }
```

While it's convenient to be able to use non-diverging code in one of the branches (like `if a { b } else { return }`) this could lead to compilation errors:

```
// this
{ panic!() };

// gets turned into this
{ absurd(panic!()) }; // error: can't infer the type of `absurd`
```

To prevent such errors, compiler remembers where it inserted `absurd` calls, and if it can't infer their type, it sets the type to fallback. `{ absurd::<Fallback>(panic!()) };`. This is

what is known as "never type fallback".

Example

```
#![deny(never_type_fallback_flowng_into_unsafe)]
fn main() {
    if true {
        // return has type `!` which, in some cases, causes never type fallback
        return
    } else {
        // `zeroed` is an unsafe function, which returns an unbounded type
        unsafe { std::mem::zeroed() }
    };
    // depending on the fallback, `zeroed` may create `()` (which is completely
    // sound),
    // or `!` (which is instant undefined behavior)
}
```

This will produce:

```
error: never type fallback affects this call to an `unsafe` function
--> lint_example.rs:8:18
   |
 8 |         unsafe { std::mem::zeroed() }
   |                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
= warning: this will change its meaning in a future release!
= note: for more information, see issue #123748 <https://github.com/rust-
lang/rust/issues/123748>
= help: specify the type explicitly
note: the lint level is defined here
--> lint_example.rs:1:9
   |
 1 | #![deny(never_type_fallback_flowng_into_unsafe)]
   |                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

Due to historic reasons never type fallback was `()`, meaning that `!` got spontaneously

coerced to `()`. There are plans to change that, but they may make the code such as above unsound. Instead of depending on the fallback, you should specify the type explicitly:

```
if true {
    return
} else {
    // type is explicitly specified, fallback can't hurt us no more
    unsafe { std::mem::zeroed::<()>() }
};
```

See [Tracking Issue](#) for making `! fall back to !`.

no-mangle-generic-items

The `no_mangle_generic_items` lint detects generic items that must be mangled.

Example

```
#[no_mangle]
fn foo<T>(t: T) {

}
```

This will produce:

```
warning: functions generic over types or consts must be mangled
--> lint_example.rs:3:1
   |
2  |     #[no_mangle]
   |     ----- help: remove this attribute
3  | / fn foo<T>(t: T) {
4  | |
5  | | }
   | | ^
   |
= note: `#[warn(no_mangle_generic_items)]` on by default
```


Explanation

A function with generics must have its symbol mangled to accommodate the generic parameter. The `no_mangle` attribute has no effect in this situation, and should be removed.

non-fmt-panic

The lint `non-fmt-panic` has been renamed to `non-fmt-panics`.

non-camel-case-types

The `non_camel_case_types` lint detects types, variants, traits and type parameters that don't have camel case names.

Example

```
struct my_struct;
```

This will produce:

```
warning: type `my_struct` should have an upper camel case name
--> lint_example.rs:2:8
   |
 2 | struct my_struct;
   |          ^^^^^^^^^ help: convert the identifier to upper camel case:
`MyStruct`
   |
   = note: `#[warn(non_camel_case_types)]` on by default
```

Explanation

The preferred style for these identifiers is to use "camel case", such as `MyStruct`, where the first letter should not be lowercase, and should not use underscores between letters. Underscores are allowed at the beginning and end of the identifier, as well as between non-letters (such as `x86_64`).

non-contiguous-range-endpoints

The `non_contiguous_range_endpoints` lint detects likely off-by-one errors when using exclusive [range patterns](#).

Example

```
let x = 123u32;
match x {
    0..100 => { println!("small"); }
    101..1000 => { println!("large"); }
    _ => { println!("larger"); }
}
```

This will produce:

```
warning: multiple ranges are one apart
--> lint_example.rs:4:5
 4 |         0..100 => { println!("small"); }
   |         ^^^^^^^
   |         |
   |         this range doesn't match `100_u32` because `..` is an exclusive range
   |         help: use an inclusive range instead: `0_u32..=100_u32`
 5 |         101..1000 => { println!("large"); }
   |         ----- this could appear to continue range `0_u32..100_u32`, but
`100_u32` isn't matched by either of them
   |
   = note: `#[warn(non_contiguous_range_endpoints)]` on by default
```

Explanation

It is likely a mistake to have range patterns in a match expression that miss out a single number. Check that the beginning and end values are what you expect, and keep in mind that with `..=` the right bound is inclusive, and with `..` it is exclusive.

non-fmt-panics

The `non_fmt_panics` lint detects `panic!(..)` invocations where the first argument is not a formatting string.

Example

```
panic!("{}",);  
panic!(123);
```

This will produce:

```
warning: panic message contains an unused formatting placeholder
--> lint_example.rs:2:9
  |
2 | panic!("{}",);
  |           ^^
  |
= note: this message is not used as a format string when given without
arguments, but will be in Rust 2021
= note: `#[warn(non_fmt_panics)]` on by default
help: add the missing argument
  |
2 | panic!("{}", ...);
  |           +++++
help: or add a "{}" format string to use the message literally
  |
2 | panic!("{}", "{}");
  |           +++++
```

```
warning: panic message is not a string literal
--> lint_example.rs:3:8
  |
3 | panic!(123);
  |       ^^^
  |
= note: this usage of `panic!()` is deprecated; it will be a hard error in
Rust 2021
= note: for more information, see <https://doc.rust-lang.org/nightly/edition-guide/rust-2021/panic-macro-consistency.html>
help: add a "{}" format string to `Display` the message
  |
3 | panic!("{}", 123);
  |       +++++
help: or use std::panic::panic_any instead
  |
3 | std::panic::panic_any(123);
  | ~~~~~~~~~~~~~~~~~~~~~~
```

Explanation

In Rust 2018 and earlier, `panic!(x)` directly uses `x` as the message. That means that `panic!("{}",)` panics with the message `"{}"` instead of using it as a formatting string, and

`panic!(123)` will panic with an `i32` as message.

Rust 2021 always interprets the first argument as format string.

non-shorthand-field-patterns

The `non_shorthand_field_patterns` lint detects using `Struct { x: x }` instead of `Struct { x }` in a pattern.

Example

```
struct Point {  
    x: i32,  
    y: i32,  
}  
  
fn main() {  
    let p = Point {  
        x: 5,  
        y: 5,  
    };  
  
    match p {  
        Point { x: x, y: y } => (),  
    }  
}
```

This will produce:

```
warning: the `x:` in this pattern is redundant
--> lint_example.rs:14:17
   |
14 |         Point { x: x, y: y } => (),
   |                   ^^^^ help: use shorthand field pattern: `x`
   |
= note: `#[warn(non_shorthand_field_patterns)]` on by default
```

```
warning: the `y:` in this pattern is redundant
--> lint_example.rs:14:23
   |
14 |         Point { x: x, y: y } => (),
   |                           ^^^^ help: use shorthand field pattern: `y`
```

Explanation

The preferred style is to avoid the repetition of specifying both the field name and the binding name if both identifiers are the same.

non-snake-case

The `non_snake_case` lint detects variables, methods, functions, lifetime parameters and modules that don't have snake case names.

Example

```
let MY_VALUE = 5;
```

This will produce:

```
warning: variable `MY_VALUE` should have a snake case name
--> lint_example.rs:2:5
  |
2 | let MY_VALUE = 5;
  |         ^^^^^^^^^ help: convert the identifier to snake case: `my_value`
  |
= note: `[warn(non_snake_case)]` on by default
```

Explanation

The preferred style for these identifiers is to use "snake case", where all the characters are in lowercase, with words separated with a single underscore, such as `my_value`.

non-upper-case-globals

The `non_upper_case_globals` lint detects static items that don't have uppercase identifiers.

Example

```
static max_points: i32 = 5;
```

This will produce:

```
warning: static variable `max_points` should have an upper case name
--> lint_example.rs:2:8
  |
2 | static max_points: i32 = 5;
  |         ^^^^^^^^^ help: convert the identifier to upper case: `MAX_POINTS`
  |
= note: `[warn(non_upper_case_globals)]` on by default
```

Explanation

The preferred style is for static item names to use all uppercase letters such as `MAX_POINTS` .

noop-method-call

The `noop_method_call` lint detects specific calls to noop methods such as a calling `<&T as Clone>::clone` where `T: !Clone` .

Example

```
struct Foo;
let foo = &Foo;
let clone: &Foo = foo.clone();
```

This will produce:

```
warning: call to `.clone()` on a reference in this situation does nothing
--> lint_example.rs:5:22
   |
5  | let clone: &Foo = foo.clone();
   |                               ^^^^^^^^^^
   |
= note: the type `Foo` does not implement `Clone`, so calling `clone` on
`&Foo` copies the reference, which does not do anything and can be removed
= note: `#[warn(noop_method_call)]` on by default
help: remove this redundant call
   |
5 - let clone: &Foo = foo.clone();
5 + let clone: &Foo = foo;
   |
help: if you meant to clone `Foo`, implement `Clone` for it
   |
3 + #[derive(Clone)]
4 | struct Foo;
```


Explanation

Some method calls are noops meaning that they do nothing. Usually such methods are the result of blanket implementations that happen to create some method invocations that end up not doing anything. For instance, `clone` is implemented on all `&T`, but calling `clone` on a `&T` where `T` does not implement `clone`, actually doesn't do anything as references are copy. This lint detects these calls and warns the user about them.

opaque-hidden-inferred-bound

The `opaque_hidden_inferred_bound` lint detects cases in which nested `impl Trait` in associated type bounds are not written generally enough to satisfy the bounds of the associated type.

Explanation

This functionality was removed in #97346, but then rolled back in #99860 because it caused regressions.

We plan on reintroducing this as a hard error, but in the meantime, this lint serves to warn and suggest fixes for any use-cases which rely on this behavior.

Example

```

#![feature(type_alias_impl_trait)]

trait Duh {}

impl Duh for i32 {}

trait Trait {
    type Assoc: Duh;
}

impl<F: Duh> Trait for F {
    type Assoc = F;
}

type Tait = impl Sized;

fn test() -> impl Trait<Assoc = Tait> {
    42
}

fn main() {}

```

This will produce:

```

warning: opaque type `impl Trait<Assoc = Tait>` does not satisfy its associated
type bounds
--> lint_example.rs:17:25
   |
 8 |         type Assoc: Duh;
   |                      --- this associated type bound is unsatisfied for `Tait`
...
17 | fn test() -> impl Trait<Assoc = Tait> {
   |                               ^^^^^^^^^^^^^^^^^
   |
   = note: `#[warn(opaque_hidden_inferred_bound)]` on by default

```

In this example, `test` declares that the associated type `Assoc` for `impl Trait` is `impl sized`, which does not satisfy the bound `Duh` on the associated type.

Although the hidden type, `i32` does satisfy this bound, we do not consider the return type to be well-formed with this lint. It can be fixed by changing `Tait = impl Sized` into `Tait = impl Sized + Duh`.

out-of-scope-macro-calls

The `out_of_scope_macro_calls` lint detects `macro_rules` called when they are not in scope, above their definition, which may happen in key-value attributes.

Example

```
#![doc = in_root!()]

macro_rules! in_root { () => { "" } }

fn main() {}
```

This will produce:

```
warning: cannot find macro `in_root` in this scope
--> lint_example.rs:1:10
   |
1  | #![doc = in_root!()]
   |           ^^^^^^^^
   |
   = warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
   = note: for more information, see issue #124535 <https://github.com/rust-
lang/rust/issues/124535>
   = help: import `macro_rules` with `use` to make it callable above its
definition
   = note: `#[warn(out_of_scope_macro_calls)]` on by default
```

Explanation

The scope in which a `macro_rules` item is visible starts at that item and continues below it. This is more similar to `let` than to other items, which are in scope both above and below their definition. Due to a bug `macro_rules` were accidentally in scope inside some key-value attributes above their definition. The lint catches such cases. To address the issue turn the `macro_rules` into a regularly scoped item by importing it with `use`.

This is a [future-incompatible](#) lint to transition this to a hard error in the future.

overlapping-patterns

The lint `overlapping-patterns` has been renamed to [overlapping-range-endpoints](#).

overlapping-range-endpoints

The `overlapping_range_endpoints` lint detects `match` arms that have [range patterns](#) that overlap on their endpoints.

Example

```
let x = 123u8;
match x {
    0..=100 => { println!("small"); }
    100..=255 => { println!("large"); }
}
```

This will produce:

```
warning: multiple patterns overlap on their endpoints
--> lint_example.rs:5:5
 4 |         0..=100 => { println!("small"); }
   |         ----- this range overlaps on `100_u8`...
 5 |         100..=255 => { println!("large"); }
   |         ^^^^^^^^^^^ ... with this range
   |
= note: you likely meant to write mutually exclusive ranges
= note: `#[warn(overlapping_range_endpoints)]` on by default
```

Explanation

It is likely a mistake to have range patterns in a match expression that overlap in this way. Check that the beginning and end values are what you expect, and keep in mind that with `..=` the left and right bounds are inclusive.

path-statements

The `path_statements` lint detects path statements with no effect.

Example

```
let x = 42;

x;
```

This will produce:

```
warning: path statement with no effect
--> lint_example.rs:4:1
   |
4  | x;
   | ^^
   |
= note: `[warn(path_statements)]` on by default
```

Explanation

It is usually a mistake to have a statement that has no effect.

private-bounds

The `private_bounds` lint detects types in a secondary interface of an item, that are more private than the item itself. Secondary interface of an item consists of bounds on generic parameters and where clauses, including supertraits for trait items.

Example

```
#![deny(private_bounds)]
```

```
struct PrivTy;
pub struct S
    where PrivTy:
{}

```

This will produce:

```
error: type `PrivTy` is more private than the item `S`
--> lint_example.rs:5:1
|
5 | pub struct S
|   ^^^^^^^^^^^^^^^^^ struct `S` is reachable at visibility `pub`
|
note: but type `PrivTy` is only usable at visibility `pub(crate)`
--> lint_example.rs:4:1
|
4 | struct PrivTy;
|   ^^^^^^^^^^^^^^^^^
note: the lint level is defined here
--> lint_example.rs:2:9
|
2 | #![deny(private_bounds)]
|   ^^^^^^^^^^^^^^^^^^^^^

```

Explanation

Having private types or traits in item bounds makes it less clear what interface the item actually provides.

The `private_interfaces` lint detects types in a primary interface of an item, that are more private than the item itself. Primary interface of an item is all its interface except for bounds on generic parameters and where clauses.

```
#![deny(private_interfaces)]
struct SemiPriv;

mod m1 {
    struct Priv;
    impl crate::SemiPriv {
        pub fn f(_: Priv) {}
    }
}
```

```
error: type `Priv` is more private than the item `m1::<impl SemiPriv>::f`
--> lint_example.rs:8:9
|
8 |         pub fn f(_: Priv) {}
|         ^^^^^^^^^^^^^^^^^^^^^ associated function `m1::<impl SemiPriv>::f` is
reachable at visibility `pub(crate)`
|
note: but type `Priv` is only usable at visibility `pub(self)`
--> lint_example.rs:6:5
|
6 |     struct Priv;
|     ^^^^^^^^^^^
note: the lint level is defined here
--> lint_example.rs:2:9
|
2 |     #![deny(private_interfaces)]
|     ^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

Having something private in primary interface guarantees that the item will be unusable from outer modules due to type privacy.

private-macro-use

The `private_macro_use` lint detects private macros that are imported with `#[macro_use]`.

Example

```
// extern_macro.rs
macro_rules! foo_ { () => {}; }
use foo_ as foo;

// code.rs

#![deny(private_macro_use)]

#[macro_use]
extern crate extern_macro;

fn main() {
    foo!();
}
```

This will produce:

```
error: cannot find macro `foo` in this scope
```

Explanation

This lint arises from overlooking visibility checks for macros in an external crate.

This is a [future-incompatible](#) lint to transition this to a hard error in the future.

ptr-cast-add-auto-to-object

The `ptr_cast_add_auto_to_object` lint detects casts of raw pointers to trait objects, which add auto traits.

Example

```
let ptr: *const dyn core::any::Any = &();
_ = ptr as *const dyn core::any::Any + Send;
```

This will produce:

```
warning: adding an auto trait `Send` to a trait object in a pointer cast may
cause UB later on
--> lint_example.rs:3:5
   |
3 | _ = ptr as *const dyn core::any::Any + Send;
   |           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #127323 <https://github.com/rust-
lang/rust/issues/127323>
= note: `#[warn(ptr_cast_add_auto_to_object)]` on by default
```

Explanation

Adding an auto trait can make the vtable invalid, potentially causing UB in safe code afterwards. For example:

```
#![feature(arbitrary_self_types)]

trait Trait {
    fn f(self: *const Self)
    where
        Self: Send;
}

impl Trait for *const () {
    fn f(self: *const Self) {
        unreachable!()
    }
}

fn main() {
    let unsend: *const () = &();
    let unsend: *const dyn Trait = &unsend;
    let send_bad: *const (dyn Trait + Send) = unsend as _;
    send_bad.f(); // this crashes, since vtable for `*const ()` does not have
an entry for `f`
}
```

Generally you must ensure that vtable is right for the pointer's type, before passing the pointer to safe code.

redundant-semicolon

The lint `redundant-semicolon` has been renamed to `redundant-semicolons`.

redundant-semicolons

The `redundant-semicolons` lint detects unnecessary trailing semicolons.

Example

```
let _ = 123;;
```

This will produce:

```
warning: unnecessary trailing semicolon
--> lint_example.rs:2:13
   |
2  | let _ = 123;;
   |             ^ help: remove this semicolon
   = note: `#[warn(redundant_semicolons)]` on by default
```

Explanation

Extra semicolons are not needed, and may be removed to avoid confusion and visual clutter.

refining-impl-trait-internal

The `refining_impl_trait_internal` lint detects `impl Trait` return types in method signatures that are refined by a trait implementation, meaning the implementation adds information about the return type that is not present in the trait.

Example

```
#![deny(refining_impl_trait)]

use std::fmt::Display;

trait AsDisplay {
    fn as_display(&self) -> impl Display;
}

impl<'s> AsDisplay for &'s str {
    fn as_display(&self) -> Self {
        *self
    }
}

fn main() {
    // users can observe that the return type of
    // `<&str as AsDisplay>::as_display()` is `&str`.
    let _x: &str = "".as_display();
}
```

This will produce:

```

error: impl trait in impl method signature does not match trait method
signature
--> lint_example.rs:10:29
   |
6  |         fn as_display(&self) -> impl Display;
   |                                ----- return type from trait method
defined here
...
10 |         fn as_display(&self) -> Self {
   |                                ^^^^
   |
   = note: add `#[allow(refining_impl_trait)]` if it is intended for this to be
part of the public API of this crate
   = note: we are soliciting feedback, see issue #121718
<https://github.com/rust-lang/rust/issues/121718> for more information
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  |     #[deny(refining_impl_trait)]
   |           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   = note: `#[deny(refining_impl_trait_internal)]` implied by `#[deny(refining_impl_trait)]`
help: replace the return type so that it matches the trait
   |
10 |         fn as_display(&self) -> impl std::fmt::Display {
   |                                ~~~~~

```

Explanation

Callers of methods for types where the implementation is known are able to observe the types written in the impl signature. This may be intended behavior, but may also lead to implementation details being revealed unintentionally. In particular, it may pose a semver hazard for authors of libraries who do not wish to make stronger guarantees about the types than what is written in the trait signature.

`refining_impl_trait` is a lint group composed of two lints:

- `refining_impl_trait_reachable`, for refinements that are publically reachable outside a crate, and
- `refining_impl_trait_internal`, for refinements that are only visible within a crate.

We are seeking feedback on each of these lints; see issue [#121718](#) for more information.

refining-impl-trait-reachable

The `refining_impl_trait_reachable` lint detects `impl Trait` return types in method signatures that are refined by a publically reachable trait implementation, meaning the implementation adds information about the return type that is not present in the trait.

Example

```
#![deny(refining_impl_trait)]

use std::fmt::Display;

pub trait AsDisplay {
    fn as_display(&self) -> impl Display;
}

impl<'s> AsDisplay for &'s str {
    fn as_display(&self) -> Self {
        *self
    }
}

fn main() {
    // users can observe that the return type of
    // `<&str as AsDisplay>::as_display()` is `<&str`'.
    let _x: &str = "".as_display();
}
```

This will produce:

```

error: impl trait in impl method signature does not match trait method
signature
--> lint_example.rs:10:29
6 |         fn as_display(&self) -> impl Display;
    |                                ----- return type from trait method
defined here
...
10 |         fn as_display(&self) -> Self {
    |                                ^^^^
    = note: add `#[allow(refining_impl_trait)]` if it is intended for this to be
part of the public API of this crate
    = note: we are soliciting feedback, see issue #121718
<https://github.com/rust-lang/rust/issues/121718> for more information
note: the lint level is defined here
--> lint_example.rs:1:9
1 |     #![deny(refining_impl_trait)]
    |             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    = note: `#[deny(refining_impl_trait_reachable)]` implied by `#[deny(refining_impl_trait)]`
help: replace the return type so that it matches the trait
10 |         fn as_display(&self) -> impl std::fmt::Display {
    |                                ~~~~~

```

Explanation

Callers of methods for types where the implementation is known are able to observe the types written in the impl signature. This may be intended behavior, but may also lead to implementation details being revealed unintentionally. In particular, it may pose a semver hazard for authors of libraries who do not wish to make stronger guarantees about the types than what is written in the trait signature.

`refining_impl_trait` is a lint group composed of two lints:

- `refining_impl_trait_reachable`, for refinements that are publically reachable outside a crate, and
- `refining_impl_trait_internal`, for refinements that are only visible within a crate.

We are seeking feedback on each of these lints; see issue [#121718](#) for more information.

renamed-and-removed-lints

The `renamed_and_removed_lints` lint detects lints that have been renamed or removed.

Example

```
#![deny(raw_pointer_derive)]
```

This will produce:

```
warning: lint `raw_pointer_derive` has been removed: using derive with raw
pointers is ok
--> lint_example.rs:1:9
  |
1 | #![deny(raw_pointer_derive)]
  |           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
= note: `#[warn(renamed_and_removed_lints)]` on by default
```

Explanation

To fix this, either remove the lint or use the new name. This can help avoid confusion about lints that are no longer valid, and help maintain consistency for renamed lints.

repr-transparent-external-private-fields

The `repr_transparent_external_private_fields` lint detects types marked `# [repr(transparent)]` that (transitively) contain an external ZST type marked `# [non_exhaustive]` or containing private fields


```
#![deny(repr_transparent_external_private_fields)]
use foo::NonExhaustiveZst;

#[repr(transparent)]
struct Bar(u32, ([u32; 0], NonExhaustiveZst));
```

```

error: zero-sized fields in repr(transparent) cannot contain external non-
exhaustive types
--> src/main.rs:5:28
|
5 | struct Bar(u32, ([u32; 0], NonExhaustiveZst));
|                               ^^^^^^^^^^^^^^^^^
|
note: the lint level is defined here
--> src/main.rs:1:9
|
1 | #![deny(repr_transparent_external_private_fields)]
|                               ^^^^^^^^^^^^^^^^^
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #78586 <https://github.com/rust-
lang/rust/issues/78586>
= note: this struct contains `NonExhaustiveZst`, which is marked with `#
[non_exhaustive]`, and makes it not a breaking change to become non-zero-sized
in the future.

```

Previous, Rust accepted fields that contain external private zero-sized types, even though it should not be a breaking change to add a non-zero-sized field to that private type.

This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #78586](#) for more details.

self-constructor-from-outer-item

The `self_constructor_from_outer_item` lint detects cases where the `self` constructor was silently allowed due to a bug in the resolver, and which may produce surprising and unintended behavior.

Using a `self` type alias from an outer item was never intended, but was silently allowed. This is deprecated -- and is a hard error when the `self` type alias references generics that are not in scope.

Example

```
#![deny(self_constructor_from_outer_item)]

struct S0(usize);

impl S0 {
    fn foo() {
        const C: S0 = Self(0);
        fn bar() -> S0 {
            Self(0)
        }
    }
}
```

This will produce:

```

error: can't reference `Self` constructor from outer item
--> lint_example.rs:8:23
|
6 | impl S0 {
| ----- the inner item doesn't inherit generics from this impl, so `Self`
is invalid to reference
7 |     fn foo() {
8 |         const C: S0 = Self(0);
|                        ^^^^^ help: replace `Self` with the actual type: `S0`
|
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #124186 <https://github.com/rust-
lang/rust/issues/124186>
note: the lint level is defined here
--> lint_example.rs:1:9
|
1 | #![deny(self_constructor_from_outer_item)]
|           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

error: can't reference `Self` constructor from outer item
--> lint_example.rs:10:13
|
6 | impl S0 {
| ----- the inner item doesn't inherit generics from this impl, so `Self`
is invalid to reference
...
10 |         Self(0)
|         ^^^^^ help: replace `Self` with the actual type: `S0`
|
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #124186 <https://github.com/rust-
lang/rust/issues/124186>

```

Explanation

The `self` type alias should not be reachable because nested items are not associated with the scope of the parameters from the parent item.

semicolon-in-expressions-from-macros

The `semicolon_in_expressions_from_macros` lint detects trailing semicolons in macro bodies when the macro is invoked in expression position. This was previously accepted, but is being phased out.

Example

```
#![deny(semicolon_in_expressions_from_macros)]
macro_rules! foo {
    () => { true; }
}

fn main() {
    let val = match true {
        true => false,
        _ => foo!()
    };
}
```

This will produce:

```

error: trailing semicolon in macro used in expression position
--> lint_example.rs:3:17
   |
3  |     () => { true; }
   |               ^
...
9  |     _ => foo!()
   |         ----- in this macro invocation
   |
   = warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
   = note: for more information, see issue #79813 <https://github.com/rust-
lang/rust/issues/79813>
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(semicolons_in_expressions_from_macros)]
   |               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   = note: this error originates in the macro `foo` (in Nightly builds, run with
-Z macro-backtrace for more info)

```

Explanation

Previous, Rust ignored trailing semicolon in a macro body when a macro was invoked in expression position. However, this makes the treatment of semicolons in the language inconsistent, and could lead to unexpected runtime behavior in some circumstances (e.g. if the macro author expects a value to be dropped).

This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #79813](#) for more details.

special-module-name

The `special_module_name` lint detects module declarations for files that have a special meaning.

Example

```
mod lib;

fn main() {
    lib::run();
}
```

This will produce:

```
warning: found module declaration for lib.rs
--> lint_example.rs:1:1
|
1 | mod lib;
  | ^^^^^^^^^
|
= note: lib.rs is the root of this crate's library target
= help: to refer to it from other targets, use the library's name as the path
= note: `[warn(special_module_name)]` on by default
```

Explanation

Cargo recognizes `lib.rs` and `main.rs` as the root of a library or binary crate, so declaring them as modules will lead to miscompilation of the crate unless configured explicitly.

To access a library from a binary target within the same crate, use `your_crate_name::` as the path instead of `lib::`:

```
// bar/src/lib.rs
fn run() {
    // ...
}

// bar/src/main.rs
fn main() {
    bar::run();
}
```

Binary targets cannot be used as libraries and so declaring one as a module is not allowed.

stable-features

The `stable-features` lint detects a `feature attribute` that has since been made stable.

Example

```
#![feature(test_accepted_feature)]
fn main() {}
```

This will produce:

```
warning: the feature `test_accepted_feature` has been stable since 1.0.0 and no
longer requires an attribute to enable
--> lint_example.rs:1:12
   |
1  | #![feature(test_accepted_feature)]
   |             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   = note: `#[warn(stable-features)]` on by default
```

Explanation

When a feature is stabilized, it is no longer necessary to include a `#![feature]` attribute for it. To fix, simply remove the `#![feature]` attribute.

static-mut-ref

The lint `static-mut-ref` has been renamed to `static-mut-refs`.

static-mut-refs

The `static_mut_refs` lint checks for shared or mutable references of mutable static inside `unsafe` blocks and `unsafe` functions.

Example

```
fn main() {
    static mut X: i32 = 23;
    static mut Y: i32 = 24;

    unsafe {
        let y = &X;
        let ref x = X;
        let (x, y) = (&X, &Y);
        foo(&X);
    }
}

unsafe fn _foo() {
    static mut X: i32 = 23;
    static mut Y: i32 = 24;

    let y = &X;
    let ref x = X;
    let (x, y) = (&X, &Y);
    foo(&X);
}

fn foo<'a>(_x: &'a i32) {}
```

This will produce:

```
warning: creating a shared reference to mutable static is discouraged
--> lint_example.rs:6:17
   |
6  |         let y = &X;
   |                   ^^ shared reference to mutable static
   |
= note: for more information, see issue #114447 <https://github.com/rust-lang/rust/issues/114447>
= note: this will be a hard error in the 2024 edition
= note: this shared reference has lifetime `static`, but if the static ever gets mutated, or a mutable reference is created, then any further use of this shared reference is Undefined Behavior
```



```

= note: `#[warn(static_mut_refs)]` on by default
help: use `addr_of!` instead to create a raw pointer

```

```

6 |         let y = addr_of!(X);
   |                   ~~~~~ +

```

```

warning: creating a shared reference to mutable static is discouraged
--> lint_example.rs:7:21

```

```

7 |         let ref x = X;
   |                   ^ shared reference to mutable static

```

```

= note: for more information, see issue #114447 <https://github.com/rust-
lang/rust/issues/114447>

```

```

= note: this will be a hard error in the 2024 edition
= note: this shared reference has lifetime `'static`, but if the static ever
gets mutated, or a mutable reference is created, then any further use of this
shared reference is Undefined Behavior
help: use `addr_of!` instead to create a raw pointer

```

```

7 |         let ref x = addr_of!(X);
   |                   ++++++ +

```

```

warning: creating a shared reference to mutable static is discouraged
--> lint_example.rs:8:23

```

```

8 |         let (x, y) = (&X, &Y);
   |                   ^^ shared reference to mutable static

```

```

= note: for more information, see issue #114447 <https://github.com/rust-
lang/rust/issues/114447>

```

```

= note: this will be a hard error in the 2024 edition
= note: this shared reference has lifetime `'static`, but if the static ever
gets mutated, or a mutable reference is created, then any further use of this
shared reference is Undefined Behavior
help: use `addr_of!` instead to create a raw pointer

```

```

8 |         let (x, y) = (addr_of!(X), &Y);
   |                   ~~~~~ +

```

```

warning: creating a shared reference to mutable static is discouraged
--> lint_example.rs:8:27

```

```

8 |         let (x, y) = (&X, &Y);

```

```

|                                     ^^ shared reference to mutable static
|
= note: for more information, see issue #114447 <https://github.com/rust-lang/rust/issues/114447>
= note: this will be a hard error in the 2024 edition
= note: this shared reference has lifetime `'static`, but if the static ever
gets mutated, or a mutable reference is created, then any further use of this
shared reference is Undefined Behavior
help: use `addr_of!` instead to create a raw pointer
|
8 |         let (x, y) = (&X, addr_of!(Y));
|                               ~~~~~ +

```

warning: creating a shared reference to mutable static is discouraged
--> lint_example.rs:9:13

```

|
9 |         foo(&X);
|             ^^ shared reference to mutable static
|
= note: for more information, see issue #114447 <https://github.com/rust-lang/rust/issues/114447>
= note: this will be a hard error in the 2024 edition
= note: this shared reference has lifetime `'static`, but if the static ever
gets mutated, or a mutable reference is created, then any further use of this
shared reference is Undefined Behavior
help: use `addr_of!` instead to create a raw pointer
|
9 |         foo(addr_of!(X));
|             ~~~~~ +

```

warning: creating a shared reference to mutable static is discouraged
--> lint_example.rs:17:13

```

|
17 |         let y = &X;
|             ^^ shared reference to mutable static
|
= note: for more information, see issue #114447 <https://github.com/rust-lang/rust/issues/114447>
= note: this will be a hard error in the 2024 edition
= note: this shared reference has lifetime `'static`, but if the static ever
gets mutated, or a mutable reference is created, then any further use of this
shared reference is Undefined Behavior
help: use `addr_of!` instead to create a raw pointer
|
17 |         let y = addr_of!(X);

```

```

|               ~~~~~~ +

warning: creating a shared reference to mutable static is discouraged
--> lint_example.rs:18:17
|
18 |         let ref x = X;
|               ^ shared reference to mutable static
|
= note: for more information, see issue #114447 <https://github.com/rust-lang/rust/issues/114447>
= note: this will be a hard error in the 2024 edition
= note: this shared reference has lifetime `'static`, but if the static ever
gets mutated, or a mutable reference is created, then any further use of this
shared reference is Undefined Behavior
help: use `addr_of!` instead to create a raw pointer

```

```

|
18 |         let ref x = addr_of!(X);
|               ++++++++ +

warning: creating a shared reference to mutable static is discouraged
--> lint_example.rs:19:19
|
19 |         let (x, y) = (&X, &Y);
|               ^^ shared reference to mutable static
|
= note: for more information, see issue #114447 <https://github.com/rust-lang/rust/issues/114447>
= note: this will be a hard error in the 2024 edition
= note: this shared reference has lifetime `'static`, but if the static ever
gets mutated, or a mutable reference is created, then any further use of this
shared reference is Undefined Behavior
help: use `addr_of!` instead to create a raw pointer

```

```

|
19 |         let (x, y) = (addr_of!(X), &Y);
|               ~~~~~~ +

warning: creating a shared reference to mutable static is discouraged
--> lint_example.rs:19:23
|
19 |         let (x, y) = (&X, &Y);
|               ^^ shared reference to mutable static
|
= note: for more information, see issue #114447 <https://github.com/rust-lang/rust/issues/114447>

```

```

= note: this will be a hard error in the 2024 edition
= note: this shared reference has lifetime `'static`, but if the static ever
gets mutated, or a mutable reference is created, then any further use of this
shared reference is Undefined Behavior
help: use `addr_of!` instead to create a raw pointer

```

```

19 |         let (x, y) = (&X, addr_of!(Y));
    |                        ~~~~~ +

```

```

warning: creating a shared reference to mutable static is discouraged

```

```

--> lint_example.rs:20:9

```

```

20 |         foo(&X);
    |             ^^ shared reference to mutable static

```

```

= note: for more information, see issue #114447 <https://github.com/rust-
lang/rust/issues/114447>

```

```

= note: this will be a hard error in the 2024 edition
= note: this shared reference has lifetime `'static`, but if the static ever
gets mutated, or a mutable reference is created, then any further use of this
shared reference is Undefined Behavior
help: use `addr_of!` instead to create a raw pointer

```

```

20 |         foo(addr_of!(X));
    |             ~~~~~ +

```

Explanation

Shared or mutable references of mutable static are almost always a mistake and can lead to undefined behavior and various other problems in your code.

This lint is "warn" by default on editions up to 2021, in 2024 there is a hard error instead.

suspicious-double-ref-op

The `suspicious_double_ref_op` lint checks for usage of `.clone()` / `.borrow()` / `.deref()` on an `&&T` when `T: !Deref/Borrow/Clone`, which means the call will return the inner `&T`, instead of performing the operation on the underlying `T` and can be confusing.

Example

```
struct Foo;
let foo = &&Foo;
let clone: &Foo = foo.clone();
```

This will produce:

```
warning: using `.clone()` on a double reference, which returns `&Foo` instead
of cloning the inner type
--> lint_example.rs:5:22
   |
5  | let clone: &Foo = foo.clone();
   |                      ^^^^^^^^^
   |
= note: `#[warn(suspicious_double_ref_op)]` on by default
```

Explanation

Since `Foo` doesn't implement `Clone`, running `.clone()` only dereferences the double reference, instead of cloning the inner type which should be what was intended.

temporary-cstring-as-ptr

The `temporary_cstring_as_ptr` lint detects getting the inner pointer of a temporary `CString`.

Example

```
let c_str = CString::new("foo").unwrap().as_ptr();
```

This will produce:

```
warning: getting the inner pointer of a temporary `CString`
--> lint_example.rs:4:42
  |
4 | let c_str = CString::new("foo").unwrap().as_ptr();
  |                                     ^^^^^^^ this pointer will be
invalid
  |
  | this `CString` is deallocated at the end of the statement, bind
it to a variable to extend its lifetime
  |
= note: pointers do not have a lifetime; when calling `as_ptr` the `CString`
will be deallocated at the end of the statement because nothing is referencing
it as far as the type system is concerned
= help: for more information, see https://doc.rust-
lang.org/reference/destructors.html
= note: `#[warn(temporary_cstring_as_ptr)]` on by default
```

Explanation

The inner pointer of a `cstring` lives only as long as the `cstring` it points to. Getting the inner pointer of a *temporary* `cstring` allows the `cstring` to be dropped at the end of the statement, as it is not being referenced as far as the typesystem is concerned. This means outside of the statement the pointer will point to freed memory, which causes undefined behavior if the pointer is later dereferenced.

trivial-bounds

The `trivial_bounds` lint detects trait bounds that don't depend on any type parameters.

Example

```
#![feature(trivial_bounds)]
pub struct A where i32: Copy;
```

This will produce:

```

warning: trait bound i32: Copy does not depend on any type or lifetime
parameters
--> lint_example.rs:3:25
   |
3 | pub struct A where i32: Copy;
   |                      ^^^^
   = note: `[warn(trivial_bounds)]` on by default

```

Explanation

Usually you would not write a trait bound that you know is always true, or never true. However, when using macros, the macro may not know whether or not the constraint would hold or not at the time when generating the code. Currently, the compiler does not alert you if the constraint is always true, and generates an error if it is never true. The

`trivial_bounds` feature changes this to be a warning in both cases, giving macros more freedom and flexibility to generate code, while still providing a signal when writing non-macro code that something is amiss.

See [RFC 2056](#) for more details. This feature is currently only available on the nightly channel, see [tracking issue #48214](#).

type-alias-bounds

The `type_alias_bounds` lint detects bounds in type aliases.

Example

```
type SendVec<T: Send> = Vec<T>;
```

This will produce:

```
warning: bounds on generic parameters are not enforced in type aliases
--> lint_example.rs:2:17
|
2 | type SendVec<T: Send> = Vec<T>;
|                   ^^^^
|
= note: `#[warn(type_alias_bounds)]` on by default
help: the bound will not be checked when the type alias is used, and should be
removed
|
2 - type SendVec<T: Send> = Vec<T>;
2 + type SendVec<T> = Vec<T>;
|
```

Explanation

The trait bounds in a type alias are currently ignored, and should not be included to avoid confusion. This was previously allowed unintentionally; this may become a hard error in the future.

tyvar-behind-raw-pointer

The `tyvar_behind_raw_pointer` lint detects raw pointer to an inference variable.

Example

```
// edition 2015
let data = std::ptr::null();
let _ = &data as *const *const ();

if data.is_null() {}
```

This will produce:


```
warning: type annotations needed
--> lint_example.rs:6:9
   |
6  | if data.is_null() {}
   |           ^^^^^^^
   |
= warning: this is accepted in the current edition (Rust 2015) but is a hard
error in Rust 2018!
= note: for more information, see issue #46906 <https://github.com/rust-
lang/rust/issues/46906>
= note: `#[warn(tyvar_behind_raw_pointer)]` on by default
```

Explanation

This kind of inference was previously allowed, but with the future arrival of [arbitrary self types](#), this can introduce ambiguity. To resolve this, use an explicit type instead of relying on type inference.

This is a [future-incompatible](#) lint to transition this to a hard error in the 2018 edition. See [issue #46906](#) for more details. This is currently a hard-error on the 2018 edition, and is "warn" by default in the 2015 edition.

uncommon-codepoints

The `uncommon_codepoints` lint detects uncommon Unicode codepoints in identifiers.

Example

```
const μ: f64 = 0.000001;
```

This will produce:

```
warning: identifier contains a non normalized (NFKC) character: 'μ'
--> lint_example.rs:3:7
   |
3  | const μ: f64 = 0.000001;
   |      ^
   |
= note: this character is included in the Not_NFKC Unicode general security
profile
= note: `[warn(uncommon_codepoints)]` on by default
```

Explanation

This lint warns about using characters which are not commonly used, and may cause visual confusion.

This lint is triggered by identifiers that contain a codepoint that is not part of the set of "Allowed" codepoints as described by [Unicode® Technical Standard #39 Unicode Security Mechanisms Section 3.1 General Security Profile for Identifiers](#).

Note that the set of uncommon codepoints may change over time. Beware that if you "forbid" this lint that existing code may fail in the future.

unconditional-recursion

The `unconditional_recursion` lint detects functions that cannot return without calling themselves.

Example

```
fn foo() {
    foo();
}
```

This will produce:

```

warning: function cannot return without recursing
--> lint_example.rs:2:1
   |
2  | fn foo() {
   | ^^^^^^^^^ cannot return without recursing
3  |     foo();
   |     ----- recursive call site
   |
= help: a `loop` may express intention better if this is on purpose
= note: `#[warn(unconditional_recursion)]` on by default

```

Explanation

It is usually a mistake to have a recursive call that does not have some condition to cause it to terminate. If you really intend to have an infinite loop, using a `loop` expression is recommended.

uncovered-param-in-projection

The `uncovered_param_in_projection` lint detects a violation of one of Rust's orphan rules for foreign trait implementations that concerns the use of type parameters inside trait associated type paths ("projections") whose output may not be a local type that is mistakenly considered to "cover" said parameters which is **unsound** and which may be rejected by a future version of the compiler.

Originally reported in [#99554](#).

Example

```

// dependency.rs
#![crate_type = "lib"]

pub trait Trait<T, U> {}

```

```
// dependent.rs
trait Identity {
    type Output;
}

impl<T> Identity for T {
    type Output = T;
}

struct Local;

impl<T> dependency::Trait<Local, T> for <T as Identity>::Output {}

fn main() {}
```

This will produce:

```
warning[E0210]: type parameter `T` must be covered by another type when it
appears before the first local type (`Local`)
  --> dependent.rs:11:6
    |
11 | impl<T> dependency::Trait<Local, T> for <T as Identity>::Output {}
    |      ^ type parameter `T` must be covered by another type when it appears
before the first local type (`Local`)
    |
    = warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
    = note: for more information, see issue #124559 <https://github.com/rust-
lang/rust/issues/124559>
    = note: implementing a foreign trait is only possible if at least one of the
types for which it is implemented is local, and no uncovered type parameters
appear before that first local type
    = note: in this case, 'before' refers to the following order: `impl<..>
ForeignTrait<T1, ..., Tn> for T0`, where `T0` is the first and `Tn` is the last
    = note: `#[warn(uncovered_param_in_projection)]` on by default
```

Explanation

FIXME(fmease): Write explainer.

undefined-naked-function-abi

The `undefined_naked_function_abi` lint detects naked function definitions that either do not specify an ABI or specify the Rust ABI.

Example

```
#![feature(asm_experimental_arch, naked_functions)]
```

```
use std::arch::asm;
```

```
#[naked]
pub fn default_abi() -> u32 {
    unsafe { asm!("", options(noreturn)); }
}
```

```
#[naked]
pub extern "Rust" fn rust_abi() -> u32 {
    unsafe { asm!("", options(noreturn)); }
}
```

This will produce:

```
warning: Rust ABI is unsupported in naked functions
--> lint_example.rs:7:1
  |
7 | pub fn default_abi() -> u32 {
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
= note: `#[warn(unknown_abi)]` on by default
```

```
warning: Rust ABI is unsupported in naked functions
--> lint_example.rs:12:1
  |
12 | pub extern "Rust" fn rust_abi() -> u32 {
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Explanation

The Rust ABI is currently undefined. Therefore, naked functions should specify a non-Rust ABI.

unexpected-cfgs

The `unexpected_cfgs` lint detects unexpected conditional compilation conditions.

Example

```
rustc --check-cfg 'cfg()'
```

```
#[cfg(widnows)]
fn foo() {}
```

This will produce:

```
warning: unexpected `cfg` condition name: `widnows`
--> lint_example.rs:1:7
   |
1  | #[cfg(widnows)]
   |          ^^^^^^^
   |
   = note: `#[warn(unexpected_cfgs)]` on by default
```

Explanation

This lint is only active when `--check-cfg` arguments are being passed to the compiler and triggers whenever an unexpected condition name or value is used.

See the [Checking Conditional Configurations](#) section for more details.

See the [Cargo Specifics](#) section for configuring this lint in `Cargo.toml`.

unfulfilled-lint-expectations

The `unfulfilled_lint_expectations` lint detects when a lint expectation is unfulfilled.

Example

```
#![cfg_attr(bootstrap, feature(lint_reasons))]

#[expect(unused_variables)]
let x = 10;
println!("{}", x);
```

This will produce:

```
warning: this lint expectation is unfulfilled
--> lint_example.rs:4:10
   |
4  | #[expect(unused_variables)]
   |           ^^^^^^^^^^^^^^^^^
   |
   = note: `#[warn(unfulfilled_lint_expectations)]` on by default
```

Explanation

The `#[expect]` attribute can be used to create a lint expectation. The expectation is fulfilled, if a `#[warn]` attribute at the same location would result in a lint emission. If the expectation is unfulfilled, because no lint was emitted, this lint will be emitted on the attribute.

ungated-async-fn-track-caller

The `ungated_async_fn_track_caller` lint warns when the `#[track_caller]` attribute is used on an async function without enabling the corresponding unstable feature flag.

Example

```
#[track_caller]
async fn foo() {}
```

This will produce:

```
warning: `#[track_caller]` on async functions is a no-op
--> lint_example.rs:2:1
   |
 2 | #[track_caller]
   | ^^^^^^^^^^^^^^^^^
 3 | async fn foo() {}
   | ----- this function will not propagate the caller location
   |
   = note: see issue #110011 <https://github.com/rust-lang/rust/issues/110011>
   for more information
   = help: add `#![feature(async_fn_track_caller)]` to the crate attributes to
   enable
   = note: this compiler was built on 2024-07-25; consider upgrading it if it is
   out of date
   = note: `#[warn(ungated_async_fn_track_caller)]` on by default
```

Explanation

The attribute must be used in conjunction with the [async_fn_track_caller](#) feature flag. Otherwise, the `#[track_caller]` annotation will function as a no-op.

uninhabited-static

The `uninhabited_static` lint detects uninhabited statics.

Example


```
enum Void {}
extern {
    static EXTERN: Void;
}
```

This will produce:

```
warning: static of uninhabited type
--> lint_example.rs:4:5
   |
4  |     static EXTERN: Void;
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #74840 <https://github.com/rust-lang/rust/issues/74840>
= note: uninhabited statics cannot be initialized, and any access would be an
immediate error
= note: `#[warn(uninhabited_static)]` on by default
```

Explanation

Statics with an uninhabited type can never be initialized, so they are impossible to define. However, this can be side-stepped with an `extern static`, leading to problems later in the compiler which assumes that there are no initialized uninhabited places (such as locals or statics). This was accidentally allowed, but is being phased out.

unknown-lints

The `unknown_lints` lint detects unrecognized lint attributes.

Example

```
#![allow(not_a_real_lint)]
```

This will produce:

```
warning: unknown lint: `not_a_real_lint`
--> lint_example.rs:1:10
   |
1  | #![allow(not_a_real_lint)]
   |           ^^^^^^^^^^^^^^^^^
   |
   = note: `#[warn(unknown_lints)]` on by default
```

Explanation

It is usually a mistake to specify a lint that does not exist. Check the spelling, and check the lint listing for the correct name. Also consider if you are using an old version of the compiler, and the lint is only available in a newer version.

unknown-or-malformed-diagnostic-attributes

The `unknown_or_malformed_diagnostic_attributes` lint detects unrecognized or otherwise malformed diagnostic attributes.

Example

```
#![feature(diagnostic_namespace)]
#[diagnostic::does_not_exist]
struct Foo;
```

This will produce:

```
warning: unknown diagnostic attribute
--> lint_example.rs:3:15
|
3 | #[diagnostic::does_not_exist]
|               ^^^^^^^^^^^^^^^^^
= note: `#[warn(unknown_or_malformed_diagnostic_attributes)]` on by default
```

Explanation

It is usually a mistake to specify a diagnostic attribute that does not exist. Check the spelling, and check the diagnostic attribute listing for the correct name. Also consider if you are using an old version of the compiler, and the attribute is only available in a newer version.

unnameable-test-items

The `unnameable_test_items` lint detects `#[test]` functions that are not able to be run by the test harness because they are in a position where they are not nameable.

Example

```
fn main() {
    #[test]
    fn foo() {
        // This test will not fail because it does not run.
        assert_eq!(1, 2);
    }
}
```

This will produce:

```
warning: cannot test inner items
--> lint_example.rs:2:5
  |
2 |     #[test]
  |     ^^^^^^^
= note: `#[warn(unnameable_test_items)]` on by default
= note: this warning originates in the attribute macro `test` (in Nightly builds, run with -Z macro-backtrace for more info)
```

Explanation

In order for the test harness to run a test, the test function must be located in a position where it can be accessed from the crate root. This generally means it must be defined in a module, and not anywhere else such as inside another function. The compiler previously allowed this without an error, so a lint was added as an alert that a test is not being used. Whether or not this should be allowed has not yet been decided, see [RFC 2471](#) and [issue #36629](#).

unreachable-code

The `unreachable_code` lint detects unreachable code paths.

Example

```
panic!("we never go past here!");

let x = 5;
```

This will produce:

```

warning: unreachable statement
--> lint_example.rs:4:1
   |
2  | panic!("we never go past here!");
   | ----- any code following this expression is
unreachable
3  |
4  | let x = 5;
   | ^^^^^^^^^^^^^ unreachable statement
   |
   = note: `#[warn(unreachable_code)]` on by default

```

Explanation

Unreachable code may signal a mistake or unfinished code. If the code is no longer in use, consider removing it.

unreachable-patterns

The `unreachable_patterns` lint detects unreachable patterns.

Example

```

let x = 5;
match x {
    y => (),
    5 => (),
}

```

This will produce:

```
warning: unreachable pattern
--> lint_example.rs:5:5
   |
4  |     y => (),
   |     - matches any value
5  |     5 => (),
   |     ^ unreachable pattern
   |
= note: `[warn(unreachable_patterns)]` on by default
```

Explanation

This usually indicates a mistake in how the patterns are specified or ordered. In this example, the `y` pattern will always match, so the five is impossible to reach. Remember, match arms match in order, you probably wanted to put the `5` case above the `y` case.

unstable-name-collision

The lint `unstable-name-collision` has been renamed to `unstable-name-collisions`.

unstable-name-collisions

The `unstable_name_collisions` lint detects that you have used a name that the standard library plans to add in the future.

Example

```

trait MyIterator : Iterator {
    // is_sorted is an unstable method that already exists on the Iterator
    trait
    fn is_sorted(self) -> bool where Self: Sized {true}
}

impl<T: ?Sized> MyIterator for T where T: Iterator { }

let x = vec![1, 2, 3];
let _ = x.iter().is_sorted();

```

This will produce:

```

warning: a method with this name may be added to the standard library in the
future
--> lint_example.rs:10:18
   |
10 | let _ = x.iter().is_sorted();
   |                      ^^^^^^^^^^^
   = warning: once this associated item is added to the standard library, the
ambiguity may cause an error or change in behavior!
   = note: for more information, see issue #48919 <https://github.com/rust-
lang/rust/issues/48919>
   = help: call with fully qualified syntax `MyIterator::is_sorted(...)` to
keep using the current method
   = note: `#[warn(unstable_name_collisions)]` on by default
help: add `#![feature(is_sorted)]` to the crate attributes to enable
`is_sorted`
1  + #![feature(is_sorted)]
   |

```

Explanation

When new methods are added to traits in the standard library, they are usually added in an "unstable" form which is only available on the [nightly channel](#) with a [feature attribute](#). If there is any preexisting code which extends a trait to have a method with the same name, then the names will collide. In the future, when the method is stabilized, this will cause an error due to the ambiguity. This lint is an early-warning to let you know that there may be a

collision in the future. This can be avoided by adding type annotations to disambiguate which trait method you intend to call, such as `MyIterator::is_sorted(my_iter)` or renaming or removing the method.

unstable-syntax-pre-expansion

The `unstable_syntax_pre_expansion` lint detects the use of unstable syntax that is discarded during attribute expansion.

Example

```
#[cfg(FALSE)]
macro foo() {}
```

This will produce:

```
warning: `macro` is experimental
--> lint_example.rs:3:1
   |
3  | macro foo() {}
   | ^^^^^^^^^^^^^^^^^
   |
= note: see issue #39412 <https://github.com/rust-lang/rust/issues/39412> for
more information
= help: add `#![feature(decl_macro)]` to the crate attributes to enable
= note: this compiler was built on 2024-07-25; consider upgrading it if it is
out of date
= warning: unstable syntax can change at any point in the future, causing a
hard error!
= note: for more information, see issue #65860 <https://github.com/rust-lang/rust/issues/65860>
```

Explanation

The input to active attributes such as `#[cfg]` or procedural macro attributes is required to

be valid syntax. Previously, the compiler only gated the use of unstable syntax features after resolving `#[cfg]` gates and expanding procedural macros.

To avoid relying on unstable syntax, move the use of unstable syntax into a position where the compiler does not parse the syntax, such as a functionlike macro.

```
macro_rules! identity {  
    ( $($tokens:tt)* ) => { $($tokens)* }  
}  
  
#[cfg(FALSE)]  
identity! {  
    macro foo() {}  
}
```

This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #65860](#) for more details.

unsupported-calling-conventions

The `unsupported_calling_conventions` lint is output whenever there is a use of the `stdcall`, `fastcall`, `thiscall`, `vectorcall` calling conventions (or their unwind variants) on targets that cannot meaningfully be supported for the requested target.

For example `stdcall` does not make much sense for a `x86_64` or, more apparently, `powerpc` code, because this calling convention was never specified for those targets.

Historically MSVC toolchains have fallen back to the regular C calling convention for targets other than `x86`, but Rust doesn't really see a similar need to introduce a similar hack across many more targets.

Example

```
extern "stdcall" fn stdcall() {}
```

This will produce:

```
warning: use of calling convention not supported on this target
--> $DIR/unsupported.rs:39:1
  |
LL | extern "stdcall" fn stdcall() {}
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
= note: `[warn(unsupported_calling_conventions)]` on by default
= warning: this was previously accepted by the compiler but is being phased
out;
           it will become a hard error in a future release!
= note: for more information, see issue ...
```

Explanation

On most of the targets the behaviour of `stdcall` and similar calling conventions is not defined at all, but was previously accepted due to a bug in the implementation of the compiler.

unused-doc-comment

The lint `unused-doc-comment` has been renamed to [unused-doc-comments](#).

unused-tuple-struct-fields

The lint `unused-tuple-struct-fields` has been renamed to [dead-code](#).

unused-allocation

The `unused_allocation` lint detects unnecessary allocations that can be eliminated.

Example

```
fn main() {
    let a = Box::new([1, 2, 3]).len();
}
```

This will produce:

```
warning: unnecessary allocation, use `&` instead
--> lint_example.rs:2:13
   |
2  |     let a = Box::new([1, 2, 3]).len();
   |                      ^^^^^^^^^^^^^^^^^
   |
= note: `#[warn(unused_allocation)]` on by default
```

Explanation

When a `box` expression is immediately coerced to a reference, then the allocation is unnecessary, and a reference (using `&` or `&mut`) should be used instead to avoid the allocation.

unused-assignments

The `unused_assignments` lint detects assignments that will never be read.

Example

```
let mut x = 5;
x = 6;
```

This will produce:

```
warning: value assigned to `x` is never read
--> lint_example.rs:3:1
   |
3  | x = 6;
   | ^
   |
= help: maybe it is overwritten before being read?
= note: `[warn(unused_assignments)]` on by default
```

Explanation

Unused assignments may signal a mistake or unfinished code. If the variable is never used after being assigned, then the assignment can be removed. Variables with an underscore prefix such as `_x` will not trigger this lint.

unused-associated-type-bounds

The `unused_associated_type_bounds` lint is emitted when an associated type bound is added to a trait object, but the associated type has a `where Self: Sized` bound, and is thus unavailable on the trait object anyway.

Example

```
trait Foo {
    type Bar where Self: Sized;
}
type Mop = dyn Foo<Bar = ()>;
```

This will produce:

```

warning: unnecessary associated type bound for not object safe associated type
--> lint_example.rs:5:20
|
5 | type Mop = dyn Foo<Bar = ()>;
|               ^^^^^^^^^ help: remove this bound
|
= note: this associated type has a `where Self: Sized` bound, and while the
associated type can be specified, it cannot be used because trait objects are
never `Sized`
= note: `#[warn(unused_associated_type_bounds)]` on by default

```

Explanation

Just like methods with `self: Sized` bounds are unavailable on trait objects, associated types can be removed from the trait object.

unused-attributes

The `unused_attributes` lint detects attributes that were not used by the compiler.

Example

```
#![ignore]
```

This will produce:

```

warning: `#[ignore]` only has an effect on functions
--> lint_example.rs:1:1
|
1 | #![ignore]
|   ^^^^^^^^^^^^^
|
= note: `#[warn(unused_attributes)]` on by default

```

Explanation

Unused [attributes](#) may indicate the attribute is placed in the wrong position. Consider removing it, or placing it in the correct position. Also consider if you intended to use an *inner attribute* (with a `!` such as `#![allow(unused)]`) which applies to the item the attribute is within, or an *outer attribute* (without a `!` such as `#[allow(unused)]`) which applies to the item *following* the attribute.

unused-braces

The `unused_braces` lint detects unnecessary braces around an expression.

Example

```
if { true } {
    // ...
}
```

This will produce:

```
warning: unnecessary braces around `if` condition
--> lint_example.rs:2:4
  |
2 | if { true } {
  |   ^^      ^^
  |
  = note: `#[warn(unused_braces)]` on by default
help: remove these braces
  |
2 - if { true } {
2 + if true {
  |
```

Explanation

The braces are not needed, and should be removed. This is the preferred style for writing these expressions.

unused-comparisons

The `unused_comparisons` lint detects comparisons made useless by limits of the types involved.

Example

```
fn foo(x: u8) {
    x >= 0;
}
```

This will produce:

```
warning: comparison is useless due to type limits
--> lint_example.rs:3:5
   |
3  |     x >= 0;
   |     ^^^^^^
   |
= note: `[warn(unused_comparisons)]` on by default
```

Explanation

A useless comparison may indicate a mistake, and should be fixed or removed.

unused-doc-comments

The `unused_doc_comments` lint detects doc comments that aren't used by `rustdoc`.

Example

```
/// docs for x
let x = 12;
```

This will produce:

```
warning: unused doc comment
--> lint_example.rs:2:1
   |
 2 | /// docs for x
   | ^^^^^^^^^^^^^^^^^
 3 | let x = 12;
   | ----- rustdoc does not generate documentation for statements
   |
   = help: use `//` for a plain comment
   = note: `[warn(unused_doc_comments)]` on by default
```

Explanation

rustdoc does not use doc comments in all positions, and so the doc comment will be ignored. Try changing it to a normal comment with `//` to avoid the warning.

unused-features

The `unused_features` lint detects unused or unknown features found in crate-level [feature attributes](#).

Note: This lint is currently not functional, see [issue #44232](#) for more details.

unused-imports

The `unused_imports` lint detects imports that are never used.

Example

```
use std::collections::HashMap;
```

This will produce:

```
warning: unused import: `std::collections::HashMap`
--> lint_example.rs:2:5
  |
2 | use std::collections::HashMap;
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
= note: `#[warn(unused_imports)]` on by default
```

Explanation

Unused imports may signal a mistake or unfinished code, and clutter the code, and should be removed. If you intended to re-export the item to make it available outside of the module, add a visibility modifier like `pub`.

unused-labels

The `unused_labels` lint detects [labels](#) that are never used.

Example

```
'unused_label: loop {}
```

This will produce:

```
warning: unused label
--> lint_example.rs:2:1
   |
2  | 'unused_label: loop {}
   | ^^^^^^^^^^^^^^^^^^^
   |
= note: `#[warn(unused_labels)]` on by default
```

Explanation

Unused labels may signal a mistake or unfinished code. To silence the warning for the individual label, prefix it with an underscore such as `'_my_label: .`

unused-macros

The `unused_macros` lint detects macros that were not used.

Note that this lint is distinct from the `unused_macro_rules` lint, which checks for single rules that never match of an otherwise used macro, and thus never expand.

Example

```
macro_rules! unused {
    () => {};
}

fn main() {
}
```

This will produce:

```
warning: unused macro definition: `unused`
--> lint_example.rs:1:14
   |
1  | macro_rules! unused {
   |               ^^^^^
   = note: `#[warn(unused_macros)]` on by default
```

Explanation

Unused macros may signal a mistake or unfinished code. To silence the warning for the individual macro, prefix the name with an underscore such as `_my_macro`. If you intended to export the macro to make it available outside of the crate, use the [macro_export attribute](#).

unused-must-use

The `unused_must_use` lint detects unused result of a type flagged as `#[must_use]`.

Example

```
fn returns_result() -> Result<(), ()> {
    Ok(())
}

fn main() {
    returns_result();
}
```

This will produce:

```
warning: unused `Result` that must be used
--> lint_example.rs:6:5
   |
6  |     returns_result();
   |     ^^^^^^^^^^^^^^^^^
   |
= note: this `Result` may be an `Err` variant, which should be handled
= note: `[warn(unused_must_use)]` on by default
help: use `let _ = ...` to ignore the resulting value
   |
6  |     let _ = returns_result();
   |     ++++++
```

Explanation

The `#[must_use]` attribute is an indicator that it is a mistake to ignore the value. See [the reference](#) for more details.

unused-mut

The `unused_mut` lint detects `mut` variables which don't need to be mutable.

Example

```
let mut x = 5;
```

This will produce:

```
warning: variable does not need to be mutable
--> lint_example.rs:2:5
  |
2 | let mut x = 5;
  |      ----^
  |      |
  |      help: remove this `mut`
= note: `[warn(unused_mut)]` on by default
```

Explanation

The preferred style is to only mark variables as `mut` if it is required.

unused-parens

The `unused_parens` lint detects `if`, `match`, `while` and `return` with parentheses; they do not need them.

Examples

```
if(true) {}
```

This will produce:

```

warning: unnecessary parentheses around `if` condition
--> lint_example.rs:2:3
  |
2 | if(true) {}
  |   ^    ^
  |
  = note: `[warn(unused_parens)]` on by default
help: remove these parentheses
  |
2 - if(true) {}
2 + if true {}
  |

```

Explanation

The parentheses are not needed, and should be removed. This is the preferred style for writing these expressions.

unused-unsafe

The `unused_unsafe` lint detects unnecessary use of an `unsafe` block.

Example

```
unsafe {}
```

This will produce:

```
warning: unnecessary `unsafe` block
--> lint_example.rs:2:1
  |
2 | unsafe {}
  | ^^^^^^ unnecessary `unsafe` block
  |
= note: `#[warn(unused_unsafe)]` on by default
```

Explanation

If nothing within the block requires `unsafe`, then remove the `unsafe` marker because it is not required and may cause confusion.

unused-variables

The `unused_variables` lint detects variables which are not used in any way.

Example

```
let x = 5;
```

This will produce:

```
warning: unused variable: `x`
--> lint_example.rs:2:5
  |
2 | let x = 5;
  |     ^ help: if this is intentional, prefix it with an underscore: `_x`
  |
= note: `#[warn(unused_variables)]` on by default
```

Explanation

Unused variables may signal a mistake or unfinished code. To silence the warning for the individual variable, prefix it with an underscore such as `_x`.

useless_ptr_null_checks

The `useless_ptr_null_checks` lint checks for useless null checks against pointers obtained from non-null types.

Example

```
let fn_ptr: fn() = /* somehow obtained nullable function pointer */

if (fn_ptr as *const()).is_null() { /* ... */ }
```

This will produce:

```
warning: function pointers are not nullable, so checking them for null will
always return false
--> lint_example.rs:6:4
  |
6 | if (fn_ptr as *const()).is_null() { /* ... */ }
  |   ^-----^
  |   |
  |   expression has type `fn()`
  |
= help: wrap the function pointer inside an `Option` and use
`Option::is_none` to check for null pointer value
= note: `#[warn(useless_ptr_null_checks)]` on by default
```

Explanation

Function pointers and references are assumed to be non-null, checking them for null will

always return false.

warnings

The `warnings` lint allows you to change the level of other lints which produce warnings.

Example

```
#![deny(warnings)]
fn foo() {}
```

This will produce:

```
error: function `foo` is never used
--> lint_example.rs:3:4
   |
3  | fn foo() {}
   |     ^^^
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(warnings)]
   |     ^^^^^^^^^^^
   = note: `#[deny(dead_code)]` implied by `#[deny(warnings)]`
```

Explanation

The `warnings` lint is a bit special; by changing its level, you change every other warning that would produce a warning to whatever value you'd like. As such, you won't ever trigger this lint in your code directly.

wasm-c-abi

The `wasm_c_abi` lint detects crate dependencies that are incompatible with future versions of Rust that will emit spec-compliant C ABI.

Example

```
#![deny(wasm_c_abi)]
```

This will produce:

```
error: the following packages contain code that will be rejected by a future
version of Rust: wasm-bindgen v0.2.87
  |
note: the lint level is defined here
--> src/lib.rs:1:9
  |
1 | #![deny(wasm_c_abi)]
  |           ^^^^^^^^^^^
```

Explanation

Rust has historically emitted non-spec-compliant C ABI. This has caused incompatibilities between other compilers and Wasm targets. In a future version of Rust this will be fixed and therefore dependencies relying on the non-spec-compliant C ABI will stop functioning.

while-true

The `while_true` lint detects `while true { }`.

Example

```
while true {

}
```

This will produce:

```
warning: denote infinite loops with `loop { ... }`
--> lint_example.rs:2:1
  |
2 | while true {
  | ^^^^^^^^^^^^^ help: use `loop`
  |
= note: `#[warn(while_true)]` on by default
```

Explanation

`while true` should be replaced with `loop`. A `loop` expression is the preferred way to write an infinite loop because it more directly expresses the intent of the loop.

writes-through-immutable-pointer

The `writes_through_immutable_pointer` lint detects writes through pointers derived from shared references.

Example

```
#![feature(const_mut_refs)]
const WRITE_AFTER_CAST: () = unsafe {
    let mut x = 0;
    let ptr = &x as *const i32 as *mut i32;
    *ptr = 0;
};
```

This will produce:

```
warning: writing through a pointer that was derived from a shared (immutable)
reference
--> lint_example.rs:6:5
   |
6  |     *ptr = 0;
   |     ^^^^^^^
   |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #X <https://github.com/rust-
lang/rust/issues/X>
= note: `#[warn(writes_through_immutable_pointer)]` on by default
```

Explanation

Shared references are immutable (when there is no `UnsafeCell` involved), and writing through them or through pointers derived from them is Undefined Behavior. The compiler recently learned to detect such Undefined Behavior during compile-time evaluation, and in the future this will raise a hard error.

Deny-by-default Lints

These lints are all set to the 'deny' level by default.

- `ambiguous_associated_items`
- `arithmetic_overflow`
- `binary_asm_labels`
- `bindings_with_variant_name`
- `cenum_impl_drop_cast`
- `conflicting_repr_hints`
- `deprecated_cfg_attr_crate_type_name`
- `elided_lifetimes_in_associated_constant`
- `enum_intrinsics_non_enums`
- `exceeding_bitshifts`
- `ill_formed_attribute_input`
- `incomplete_include`
- `ineffective_unstable_trait_impl`
- `invalid_atomic_ordering`
- `invalid_doc_attributes`
- `invalid_from_utf8_unchecked`
- `invalid_reference_casting`
- `invalid_type_param_default`
- `let_underscore_lock`
- `long_running_const_eval`
- `macro_expanded_macro_exports_accessed_by_absolute_paths`
- `missing_fragment_specifier`
- `mutable_transmutes`
- `named_asm_labels`
- `no_mangle_const_items`
- `order_dependent_trait_objects`
- `overflowing_literals`
- `patterns_in_fns_without_body`
- `proc_macro_derive_resolution_fallback`
- `pub_use_of_private_extern_crate`
- `soft_unstable`

- [test_unstable_lint](#)
- [text_direction_codepoint_in_comment](#)
- [text_direction_codepoint_in_literal](#)
- [unconditional_panic](#)
- [undropped_manually_drops](#)
- [unknown_crate_types](#)
- [useless_deprecated](#)

ambiguous-associated-items

The `ambiguous_associated_items` lint detects ambiguity between [associated items](#) and [enum variants](#).

Example

```
enum E {  
    V  
}  
  
trait Tr {  
    type V;  
    fn foo() -> Self::V;  
}  
  
impl Tr for E {  
    type V = u8;  
    // `Self::V` is ambiguous because it may refer to the associated type or  
    // the enum variant.  
    fn foo() -> Self::V { 0 }  
}
```

This will produce:

```

error: ambiguous associated item
--> lint_example.rs:15:17
   |
15 |         fn foo() -> Self::V { 0 }
   |                        ^^^^^^^ help: use fully-qualified syntax: `<E as Tr>::V`
   |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #57644 <https://github.com/rust-
lang/rust/issues/57644>
note: `V` could refer to the variant defined here
--> lint_example.rs:3:5
   |
3  |         V
   |         ^
note: `V` could also refer to the associated type defined here
--> lint_example.rs:7:5
   |
7  |         type V;
   |         ^^^^^^^
   |
= note: `#[deny(ambiguous_associated_items)]` on by default

```

Explanation

Previous versions of Rust did not allow accessing enum variants through [type aliases](#). When this ability was added (see [RFC 2338](#)), this introduced some situations where it can be ambiguous what a type was referring to.

To fix this ambiguity, you should use a [qualified path](#) to explicitly state which type to use. For example, in the above example the function can be written as `fn f() -> <Self as Tr>::V { 0 }` to specifically refer to the associated type.

This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #57644](#) for more details.

arithmetic-overflow

The `arithmetic_overflow` lint detects that an arithmetic operation will [overflow](#).

Example

```
1_i32 << 32;
```

This will produce:

```
error: this arithmetic operation will overflow
--> lint_example.rs:2:1
   |
2  | 1_i32 << 32;
   | ^^^^^^^^^^^^^ attempt to shift left by `32_i32`, which would overflow
   |
   = note: `[deny(arithmetic_overflow)]` on by default
```

Explanation

It is very likely a mistake to perform an arithmetic operation that overflows its value. If the compiler is able to detect these kinds of overflows at compile-time, it will trigger this lint. Consider adjusting the expression to avoid overflow, or use a data type that will not overflow.

binary-asm-labels

The `binary_asm_labels` lint detects the use of numeric labels containing only binary digits in the inline `asm!` macro.

Example


```

#![cfg(target_arch = "x86_64")]

use std::arch::asm;

fn main() {
    unsafe {
        asm!("0: jmp 0b");
    }
}

```

This will produce:

```

error: avoid using labels containing only the digits `0` and `1` in inline
assembly
--> <source>:7:15
   |
7  |         asm!("0: jmp 0b");
   |               ^ use a different label that doesn't start with `0` or `1`
   |
   = help: start numbering with `2` instead
   = note: an LLVM bug makes these labels ambiguous with a binary literal number
on x86
   = note: see <https://github.com/llvm/llvm-project/issues/99547> for more
information
   = note: `[deny(binary_asm_labels)]` on by default

```

Explanation

An [LLVM bug](#) causes this code to fail to compile because it interprets the `0b` as a binary literal instead of a reference to the previous local label `0`. To work around this bug, don't use labels that could be confused with a binary literal.

This behavior is platform-specific to x86 and x86-64.

See the explanation in [Rust By Example](#) for more details.

bindings-with-variant-name

The `bindings_with_variant_name` lint detects pattern bindings with the same name as one of the matched variants.

Example

```
pub enum Enum {
    Foo,
    Bar,
}

pub fn foo(x: Enum) {
    match x {
        Foo => {}
        Bar => {}
    }
}
```

This will produce:

```
error[E0170]: pattern binding `Foo` is named the same as one of the variants of
the type `main::Enum`
--> lint_example.rs:9:9
  |
9 |         Foo => {}
  |         ^^^ help: to match on the variant, qualify the path:
`main::Enum::Foo`
  |
  = note: `[deny(bindings_with_variant_name)]` on by default
```

Explanation

It is usually a mistake to specify an enum variant name as an [identifier pattern](#). In the example above, the `match` arms are specifying a variable name to bind the value of `x` to. The second arm is ignored because the first one matches *all* values. The likely intent is that the arm was intended to match on the enum variant.

Two possible solutions are:

- Specify the enum variant using a [path pattern](#), such as `Enum::Foo`.
- Bring the enum variants into local scope, such as adding `use Enum::*;` to the beginning of the `foo` function in the example above.

cenum-impl-drop-cast

The `cenum_impl_drop_cast` lint detects an `as` cast of a field-less `enum` that implements [Drop](#).

Example

```
enum E {  
    A,  
}  
  
impl Drop for E {  
    fn drop(&mut self) {  
        println!("Drop");  
    }  
}  
  
fn main() {  
    let e = E::A;  
    let i = e as u32;  
}
```

This will produce:

```

error: cannot cast enum `E` into integer `u32` because it implements `Drop`
--> lint_example.rs:14:13
   |
14 |         let i = e as u32;
   |                   ^^^^^^^
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #73333 <https://github.com/rust-
lang/rust/issues/73333>
= note: `#[deny(cenum_impl_drop_cast)]` on by default

```

Explanation

Casting a field-less `enum` that does not implement `Copy` to an integer moves the value without calling `drop`. This can result in surprising behavior if it was expected that `drop` should be called. Calling `drop` automatically would be inconsistent with other move operations. Since neither behavior is clear or consistent, it was decided that a cast of this nature will no longer be allowed.

This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #73333](#) for more details.

conflicting-repr-hints

The `conflicting_repr_hints` lint detects [repr attributes](#) with conflicting hints.

Example

```

#[repr(u32, u64)]
enum Foo {
    Variant1,
}

```

This will produce:

```
error[E0566]: conflicting representation hints
--> lint_example.rs:2:8
  |
2 | #[repr(u32, u64)]
  |           ^^^  ^^^
  |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #68585 <https://github.com/rust-
lang/rust/issues/68585>
= note: `#[deny(conflicting_repr_hints)]` on by default
```

Explanation

The compiler incorrectly accepted these conflicting representations in the past. This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #68585](#) for more details.

To correct the issue, remove one of the conflicting hints.

deprecated-cfg-attr-crate-type-name

The `deprecated_cfg_attr_crate_type_name` lint detects uses of the `#![cfg_attr(..., crate_type = "...")]` and `#![cfg_attr(..., crate_name = "...")]` attributes to conditionally specify the crate type and name in the source code.

Example

```
#![cfg_attr(debug_assertions, crate_type = "lib")]
```

This will produce:

```

error: `crate_type` within an `#![cfg_attr]` attribute is deprecated
--> lint_example.rs:1:31
   |
1 | #![cfg_attr(debug_assertions, crate_type = "lib")]
   |                                     ^^^^^^^^^^^^^^^^^^^^^
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #91632 <https://github.com/rust-
lang/rust/issues/91632>
= note: `#[deny(deprecated_cfg_attr_crate_type_name)]` on by default

```

Explanation

The `#![crate_type]` and `#![crate_name]` attributes require a hack in the compiler to be able to change the used crate type and crate name after macros have been expanded. Neither attribute works in combination with Cargo as it explicitly passes `--crate-type` and `--crate-name` on the commandline. These values must match the value used in the source code to prevent an error.

To fix the warning use `--crate-type` on the commandline when running rustc instead of `#![cfg_attr(..., crate_type = "...")]` and `--crate-name` instead of `#![cfg_attr(..., crate_name = "...")]`.

elided-lifetimes-in-associated-constant

The `elided_lifetimes_in_associated_constant` lint detects elided lifetimes in associated constants when there are other lifetimes in scope. This was accidentally supported, and this lint was later relaxed to allow eliding lifetimes to `'static` when there are no lifetimes in scope.

Example

```
#![deny(elided_lifetimes_in_associated_constant)]

struct Foo<'a>(&'a ());

impl<'a> Foo<'a> {
    const STR: &str = "hello, world";
}
```

This will produce:

```
error: `` without an explicit lifetime name cannot be used here
--> lint_example.rs:7:16
   |
7  |     const STR: &str = "hello, world";
   |                  ^
   |
   = warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
   = note: for more information, see issue #115010 <https://github.com/rust-
lang/rust/issues/115010>
note: cannot automatically infer ``static` because of other lifetimes in scope
--> lint_example.rs:6:6
   |
6  | impl<'a> Foo<'a> {
   |      ^^
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1  | #![deny(elided_lifetimes_in_associated_constant)]
   |          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
help: use the ``static` lifetime
   |
7  |     const STR: &'static str = "hello, world";
   |                  +++++++
```

Explanation

Previous version of Rust

Implicit static-in-const behavior was decided [against](#) for associated constants because of ambiguity. This, however, regressed and the compiler erroneously treats elided lifetimes in

associated constants as lifetime parameters on the impl.

This is a [future-incompatible](#) lint to transition this to a hard error in the future.

enum-intrinsics-non-enums

The `enum_intrinsics_non_enums` lint detects calls to intrinsic functions that require an `enum` (`core::mem::discriminant`, `core::mem::variant_count`), but are called with a non-enum type.

Example

```
#![deny(enum_intrinsics_non_enums)]
core::mem::discriminant::<i32>(&123);
```

This will produce:

```
error: the return value of `mem::discriminant` is unspecified when called with
a non-enum type
--> lint_example.rs:3:1
   |
3 | core::mem::discriminant::<i32>(&123);
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
note: the argument to `discriminant` should be a reference to an enum, but it
was passed a reference to a `i32`, which is not an enum
--> lint_example.rs:3:32
   |
3 | core::mem::discriminant::<i32>(&123);
   |                                ^^^^^
note: the lint level is defined here
--> lint_example.rs:1:9
   |
1 | #![deny(enum_intrinsics_non_enums)]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```


Explanation

In order to accept any enum, the `mem::discriminant` and `mem::variant_count` functions are generic over a type `T`. This makes it technically possible for `T` to be a non-enum, in which case the return value is unspecified.

This lint prevents such incorrect usage of these functions.

exceeding-bitshifts

The lint `exceeding-bitshifts` has been renamed to `arithmetic-overflow`.

ill-formed-attribute-input

The `ill_formed_attribute_input` lint detects ill-formed attribute inputs that were previously accepted and used in practice.

Example

```
#[inline = "this is not valid"]  
fn foo() {}
```

This will produce:

```

error: valid forms for the attribute are `#[inline]` and `#[inline(always|never)]`
--> lint_example.rs:2:1
   |
2 | #[inline = "this is not valid"]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #57571 <https://github.com/rust-
lang/rust/issues/57571>
= note: `#[deny(ill_formed_attribute_input)]` on by default

```

Explanation

Previously, inputs for many built-in attributes weren't validated and nonsensical attribute inputs were accepted. After validation was added, it was determined that some existing projects made use of these invalid forms. This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #57571](#) for more details.

Check the [attribute reference](#) for details on the valid inputs for attributes.

incomplete-include

The `incomplete_include` lint detects the use of the `include!` macro with a file that contains more than one expression.

Example

```

fn main() {
    include!("foo.txt");
}

```

where the file `foo.txt` contains:

```
println!("hi!");
```

produces:

```
error: include macro expected single expression in source
--> foo.txt:1:14
   |
1  | println!("1");
   |               ^
   |
   = note: `[deny(incomplete_include)]` on by default
```

Explanation

The `include!` macro is currently only intended to be used to include a single [expression](#) or multiple [items](#). Historically it would ignore any contents after the first expression, but that can be confusing. In the example above, the `println!` expression ends just before the semicolon, making the semicolon "extra" information that is ignored. Perhaps even more surprising, if the included file had multiple print statements, the subsequent ones would be ignored!

One workaround is to place the contents in braces to create a [block expression](#). Also consider alternatives, like using functions to encapsulate the expressions, or use [proc-macros](#).

This is a lint instead of a hard error because existing projects were found to hit this error. To be cautious, it is a lint for now. The future semantics of the `include!` macro are also uncertain, see [issue #35560](#).

ineffective-unstable-trait-impl

The `ineffective_unstable_trait_impl` lint detects `#[unstable]` attributes which are not used.

Example

```
#![feature(staged_api)]

#[derive(Clone)]
#[stable(feature = "x", since = "1")]
struct S {}

#[unstable(feature = "y", issue = "none")]
impl Copy for S {}
```

This will produce:

```
error: an `#[unstable]` annotation here has no effect
--> lint_example.rs:8:1
   |
 8 | #[unstable(feature = "y", issue = "none")]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   = note: see issue #55436 <https://github.com/rust-lang/rust/issues/55436> for
more information
   = note: `#[deny(ineffective_unstable_trait_impl)]` on by default
```

Explanation

`staged_api` does not currently support using a stability attribute on `impl` blocks. `impl` s are always stable if both the type and trait are stable, and always unstable otherwise.

invalid-atomic-ordering

The `invalid_atomic_ordering` lint detects passing an `Ordering` to an atomic operation that does not support that ordering.

Example

```
let atom = AtomicU8::new(0);
let value = atom.load(Ordering::Release);
```

This will produce:

```
error: atomic loads cannot have `Release` or `AcqRel` ordering
--> lint_example.rs:4:23
   |
4 | let value = atom.load(Ordering::Release);
   |                        ^^^^^^^^^^^^^^^^^
= help: consider using ordering modes `Acquire`, `SeqCst` or `Relaxed`
= note: `#[deny(invalid_atomic_ordering)]` on by default
```

Explanation

Some atomic operations are only supported for a subset of the `atomic::Ordering` variants. Passing an unsupported variant will cause an unconditional panic at runtime, which is detected by this lint.

This lint will trigger in the following cases: (where `AtomicType` is an atomic type from `core::sync::atomic`, such as `AtomicBool`, `AtomicPtr`, `AtomicUsize`, or any of the other integer atomics).

- Passing `Ordering::Acquire` or `Ordering::AcqRel` to `AtomicType::store`.
- Passing `Ordering::Release` or `Ordering::AcqRel` to `AtomicType::load`.
- Passing `Ordering::Relaxed` to `core::sync::atomic::fence` or `core::sync::atomic::compiler_fence`.
- Passing `Ordering::Release` or `Ordering::AcqRel` as the failure ordering for any of `AtomicType::compare_exchange`, `AtomicType::compare_exchange_weak`, or `AtomicType::fetch_update`.

invalid-doc-attributes

The `invalid_doc_attributes` lint detects when the `#[doc(...)]` is misused.

Example

```
#![deny(warnings)]

pub mod submodule {
    #![doc(test(no_crate_inject))]
}
```

This will produce:

```
error: this attribute can only be applied at the crate level
--> lint_example.rs:5:12
   |
5  |     #![doc(test(no_crate_inject))]
   |           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
= note: read <https://doc.rust-lang.org/nightly/rustdoc/the-doc-attribute.html#at-the-crate-level> for more information
= note: `#![deny(invalid_doc_attributes)]` on by default
```

Explanation

Previously, incorrect usage of the `#[doc(...)]` attribute was not being validated. Usually these should be rejected as a hard error, but this lint was introduced to avoid breaking any existing crates which included them.

invalid-from-utf8-unchecked

The `invalid_from_utf8_unchecked` lint checks for calls to `std::str::from_utf8_unchecked` and `std::str::from_utf8_unchecked_mut` with a known invalid UTF-8 value.

Example

```
unsafe {
    std::str::from_utf8_unchecked(b"Ru\x82st");
}
```

This will produce:

```
error: calls to `std::str::from_utf8_unchecked` with a invalid literal are
undefined behavior
--> lint_example.rs:4:5
   |
4  |     std::str::from_utf8_unchecked(b"Ru\x82st");
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |                                                                           |
   |                                                                           the literal was valid UTF-8 up to the 2
bytes
   |
   = note: `#[deny(invalid_from_utf8_unchecked)]` on by default
```

Explanation

Creating such a `str` would result in undefined behavior as per documentation for `std::str::from_utf8_unchecked` and `std::str::from_utf8_unchecked_mut`.

invalid-reference-casting

The `invalid_reference_casting` lint checks for casts of `&T` to `&mut T` without using interior mutability.

Example

```
fn x(r: &i32) {
    unsafe {
        *(r as *const i32 as *mut i32) += 1;
    }
}
```

This will produce:

```
error: assigning to `&T` is undefined behavior, consider using an `UnsafeCell`
--> lint_example.rs:4:9
   |
4  |         *(r as *const i32 as *mut i32) += 1;
   |         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
= note: for more information, visit <https://doc.rust-lang.org/book/ch15-05-interior-mutability.html>
= note: `#[deny(invalid_reference_casting)]` on by default
```

Explanation

Casting `&T` to `&mut T` without using interior mutability is undefined behavior, as it's a violation of Rust reference aliasing requirements.

`UnsafeCell` is the only way to obtain aliasable data that is considered mutable.

invalid-type-param-default

The `invalid_type_param_default` lint detects type parameter defaults erroneously allowed in an invalid location.

Example

```
fn foo<T=i32>(t: T) {}
```


This will produce:

```
error: defaults for type parameters are only allowed in `struct`, `enum`,
`type`, or `trait` definitions
--> lint_example.rs:2:8
  |
2 | fn foo<T=i32>(t: T) {}
  |           ^^^^^
  |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #36887 <https://github.com/rust-
lang/rust/issues/36887>
= note: `#[deny(invalid_type_param_default)]` on by default
```

Explanation

Default type parameters were only intended to be allowed in certain situations, but historically the compiler allowed them everywhere. This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #36887](#) for more details.

let-underscore-lock

The `let_underscore_lock` lint checks for statements which don't bind a mutex to anything, causing the lock to be released immediately instead of at end of scope, which is typically incorrect.

Example

```

use std::sync::{Arc, Mutex};
use std::thread;
let data = Arc::new(Mutex::new(0));

thread::spawn(move || {
    // The lock is immediately released instead of at the end of the
    // scope, which is probably not intended.
    let _ = data.lock().unwrap();
    println!("doing some work");
    let mut lock = data.lock().unwrap();
    *lock += 1;
});

```

This will produce:

```

error: non-binding let on a synchronization lock
--> lint_example.rs:9:9
   |
9  |     let _ = data.lock().unwrap();
   |         ^ this lock is not assigned to a binding and is immediately dropped
   |
   = note: `[deny(let_underscore_lock)]` on by default
help: consider binding to an unused variable to avoid immediately dropping the
value
   |
9  |     let _unused = data.lock().unwrap();
   |         ~~~~~~
help: consider immediately dropping the value
   |
9  |     drop(data.lock().unwrap());
   |     ~~~~~~

```

Explanation

Statements which assign an expression to an underscore causes the expression to immediately drop instead of extending the expression's lifetime to the end of the scope. This is usually unintended, especially for types like `MutexGuard`, which are typically used to lock a mutex for the duration of an entire scope.

If you want to extend the expression's lifetime to the end of the scope, assign an

underscore-prefixed name (such as `_foo`) to the expression. If you do actually want to drop the expression immediately, then calling `std::mem::drop` on the expression is clearer and helps convey intent.

long-running-const-eval

The `long_running_const_eval` lint is emitted when const eval is running for a long time to ensure rustc terminates even if you accidentally wrote an infinite loop.

Example

```
const F00: () = loop {};
```

This will produce:

```
error: constant evaluation is taking a long time
--> lint_example.rs:2:17
  |
2 | const F00: () = loop {};
  |                  ^^^^^^^
= note: this lint makes sure the compiler doesn't get stuck due to infinite
loops in const eval.
      If your compilation actually takes a long time, you can safely allow
the lint.
help: the constant being evaluated
--> lint_example.rs:2:1
  |
2 | const F00: () = loop {};
  | ^^^^^^^^^^^^^^^^^^^
= note: `#[deny(long_running_const_eval)]` on by default
```

Explanation

Loops allow const evaluation to compute arbitrary code, but may also cause infinite loops or

just very long running computations. Users can enable long running computations by allowing the lint on individual constants or for entire crates.

Unconditional warnings

Note that regardless of whether the lint is allowed or set to warn, the compiler will issue warnings if constant evaluation runs significantly longer than this lint's limit. These warnings are also shown to downstream users from crates.io or similar registries. If you are above the lint's limit, both you and downstream users might be exposed to these warnings. They might also appear on compiler updates, as the compiler makes minor changes about how complexity is measured: staying below the limit ensures that there is enough room, and given that the lint is disabled for people who use your dependency it means you will be the only one to get the warning and can put out an update in your own time.

macro-expanded-macro-exports-accessed-by-absolute-paths

The `macro_expanded_macro_exports_accessed_by_absolute_paths` lint detects macro-expanded `macro_export` macros from the current crate that cannot be referred to by absolute paths.

Example

```
macro_rules! define_exported {
    () => {
        #[macro_export]
        macro_rules! exported {
            () => {};
        }
    };
}

define_exported!();

fn main() {
    crate::exported!();
}
```

This will produce:

```
error: macro-expanded `macro_export` macros from the current crate cannot be
referred to by absolute paths
--> lint_example.rs:13:5
   |
13 |     crate::exported!();
   |     ^^^^^^^^^^^^^^^^^^^
   |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #52234 <https://github.com/rust-
lang/rust/issues/52234>
note: the macro is defined here
--> lint_example.rs:4:9
   |
 4 | /         macro_rules! exported {
 5 | |             () => {};
 6 | |         }
   | |_____| ^
   |
...
10 |     define_exported!();
   |     ----- in this macro invocation
= note: `#[deny(macro_expanded_macro_exports_accessed_by_absolute_paths)]`
on by default
= note: this error originates in the macro `define_exported` (in Nightly
builds, run with -Z macro-backtrace for more info)
```

Explanation

The intent is that all macros marked with the `#[macro_export]` attribute are made available in the root of the crate. However, when a `macro_rules!` definition is generated by another macro, the macro expansion is unable to uphold this rule. This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #53495](#) for more details.

missing-fragment-specifier

The `missing_fragment_specifier` lint is issued when an unused pattern in a `macro_rules!` macro definition has a meta-variable (e.g. `$e`) that is not followed by a fragment specifier (e.g. `:expr`).

This warning can always be fixed by removing the unused pattern in the `macro_rules!` macro definition.

Example

```
macro_rules! foo {  
    () => {};  
    ($name) => { };  
}  
  
fn main() {  
    foo!();  
}
```

This will produce:

```

error: missing fragment specifier
--> lint_example.rs:3:5
   |
3  |     ($name) => { };
   |     ^^^^^^
   |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #40107 <https://github.com/rust-lang/rust/issues/40107>
= note: `#[deny(missing_fragment_specifier)]` on by default

```

Explanation

To fix this, remove the unused pattern from the `macro_rules!` macro definition:

```

macro_rules! foo {
    () => {};
}
fn main() {
    foo!();
}

```

mutable-transmutes

The `mutable_transmutes` lint catches transmuting from `&T` to `&mut T` because it is [undefined behavior](#).

Example

```

unsafe {
    let y = std::mem::transmute:::<&i32, &mut i32>(&5);
}

```

This will produce:

```
error: transmuting &T to &mut T is undefined behavior, even if the reference is
unused, consider instead using an UnsafeCell
--> lint_example.rs:3:13
   |
3  |     let y = std::mem::transmute:::<&i32, &mut i32>(&5);
   |               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
= note: `#[deny(mutable_transmutes)]` on by default
```

Explanation

Certain assumptions are made about aliasing of data, and this transmute violates those assumptions. Consider using [UnsafeCell](#) instead.

named-asm-labels

The `named_asm_labels` lint detects the use of named labels in the inline `asm!` macro.

Example

```
use std::arch::asm;

fn main() {
    unsafe {
        asm!("foo: bar");
    }
}
```

This will produce:


```

error: avoid using named labels in inline assembly
--> lint_example.rs:6:15
   |
6  |         asm!("foo: bar");
   |               ^^^
= help: only local labels of the form `<number>:` should be used in inline
asm
= note: see the asm section of Rust By Example <https://doc.rust-
lang.org/nightly/rust-by-example/unsafe/asm.html#labels> for more information
= note: `#[deny(named_asm_labels)]` on by default

```

Explanation

LLVM is allowed to duplicate inline assembly blocks for any reason, for example when it is in a function that gets inlined. Because of this, GNU assembler [local labels](#) *must* be used instead of labels with a name. Using named labels might cause assembler or linker errors.

See the explanation in [Rust By Example](#) for more details.

no-mangle-const-items

The `no_mangle_const_items` lint detects any `const` items with the [no_mangle](#) attribute.

Example

```

#[no_mangle]
const F00: i32 = 5;

```

This will produce:

```

error: const items should never be `#[no_mangle]`
--> lint_example.rs:3:1
   |
3  | const F00: i32 = 5;
   | -----^^^^^^^^^^^^^^^^^^^^
   | |
   | help: try a static value: `pub static`
   |
= note: `#[deny(no_mangle_const_items)]` on by default

```

Explanation

Constants do not have their symbols exported, and therefore, this probably means you meant to use a `static`, not a `const`.

order-dependent-trait-objects

The `order_dependent_trait_objects` lint detects a trait coherency violation that would allow creating two trait impls for the same dynamic trait object involving marker traits.

Example

```

pub trait Trait {}

impl Trait for dyn Send + Sync { }
impl Trait for dyn Sync + Send { }

```

This will produce:

```

error: conflicting implementations of trait `Trait` for type `(dyn Send + Sync
+ 'static)': (E0119)
--> lint_example.rs:5:1
   |
4 | impl Trait for dyn Send + Sync { }
   | ----- first implementation here
5 | impl Trait for dyn Sync + Send { }
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ conflicting implementation for `(dyn Send +
Sync + 'static)`
   |
   = warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
   = note: for more information, see issue #56484 <https://github.com/rust-
lang/rust/issues/56484>
   = note: `#[deny(order_dependent_trait_objects)]` on by default

```

Explanation

A previous bug caused the compiler to interpret traits with different orders (such as `Send + Sync` and `Sync + Send`) as distinct types when they were intended to be treated the same. This allowed code to define separate trait implementations when there should be a coherence error. This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #56484](#) for more details.

overflowing-literals

The `overflowing_literals` lint detects literal out of range for its type.

Example

```
let x: u8 = 1000;
```

This will produce:

```

error: literal out of range for `u8`
--> lint_example.rs:2:13
   |
2  | let x: u8 = 1000;
   |             ^^^^
   = note: the literal `1000` does not fit into the type `u8` whose range is
`0..=255`
   = note: `[deny(overflowing_literals)]` on by default

```

Explanation

It is usually a mistake to use a literal that overflows the type where it is used. Either use a literal that is within range, or change the type to be within the range of the literal.

patterns-in-fns-without-body

The `patterns_in_fns_without_body` lint detects `mut` identifier patterns as a parameter in functions without a body.

Example

```

trait Trait {
    fn foo(mut arg: u8);
}

```

This will produce:

```

error: patterns aren't allowed in functions without bodies
--> lint_example.rs:3:12
   |
3  |     fn foo(mut arg: u8);
   |           ^^^^^^^ help: remove `mut` from the parameter: `arg`
   |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #35203 <https://github.com/rust-
lang/rust/issues/35203>
= note: `#[deny(patterns_in_fns_without_body)]` on by default

```

Explanation

To fix this, remove `mut` from the parameter in the trait definition; it can be used in the implementation. That is, the following is OK:

```

trait Trait {
    fn foo(arg: u8); // Removed `mut` here
}

impl Trait for i32 {
    fn foo(mut arg: u8) { // `mut` here is OK
    }
}

```

Trait definitions can define functions without a body to specify a function that implementors must define. The parameter names in the body-less functions are only allowed to be `_` or an [identifier](#) for documentation purposes (only the type is relevant). Previous versions of the compiler erroneously allowed [identifier patterns](#) with the `mut` keyword, but this was not intended to be allowed. This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #35203](#) for more details.

proc-macro-derive-resolution-fallback

The `proc_macro_derive_resolution_fallback` lint detects proc macro derives using inaccessible names from parent modules.

Example

```
// foo.rs
#![crate_type = "proc-macro"]

extern crate proc_macro;

use proc_macro::*;

#[proc_macro_derive(Foo)]
pub fn foo1(a: TokenStream) -> TokenStream {
    drop(a);
    "mod __bar { static mut BAR: Option<Something> = None; }".parse().unwrap()
}

// bar.rs
#[macro_use]
extern crate foo;

struct Something;

#[derive(Foo)]
struct Another;

fn main() {}
```

This will produce:

```
warning: cannot find type `Something` in this scope
--> src/main.rs:8:10
  |
8 | #[derive(Foo)]
  |           ^^^ names from parent modules are not accessible without an
explicit import
  |
  = note: `[warn(proc_macro_derive_resolution_fallback)]` on by default
  = warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
  = note: for more information, see issue #50504 <https://github.com/rust-
lang/rust/issues/50504>
```

Explanation

If a proc-macro generates a module, the compiler unintentionally allowed items in that module to refer to items in the crate root without importing them. This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #50504](#) for more details.

pub-use-of-private-extern-crate

The `pub_use_of_private_extern_crate` lint detects a specific situation of re-exporting a private extern crate.

Example

```
extern crate core;
pub use core as reexported_core;
```

This will produce:

```

error[E0365]: extern crate `core` is private and cannot be re-exported
--> lint_example.rs:3:9
   |
3  | pub use core as reexported_core;
   |           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
= warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
= note: for more information, see issue #127909 <https://github.com/rust-
lang/rust/issues/127909>
= note: `#[deny(pub_use_of_private_extern_crate)]` on by default
help: consider making the `extern crate` item publicly accessible
   |
2  | pub extern crate core;
   | +++

```

Explanation

A public `use` declaration should not be used to publicly re-export a private `extern crate`. `pub extern crate` should be used instead.

This was historically allowed, but is not the intended behavior according to the visibility rules. This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #127909](#) for more details.

soft-unstable

The `soft_unstable` lint detects unstable features that were unintentionally allowed on stable.

Example


```
#[cfg(test)]
extern crate test;

#[bench]
fn name(b: &mut test::Bencher) {
    b.iter(|| 123)
}
```

This will produce:

```
error: use of unstable library feature 'test': `bench` is a part of custom test
frameworks which are unstable
--> lint_example.rs:5:3
   |
5  | #[bench]
   |      ^^^^^
   |
   = warning: this was previously accepted by the compiler but is being phased
out; it will become a hard error in a future release!
   = note: for more information, see issue #64266 <https://github.com/rust-
lang/rust/issues/64266>
   = note: `#[deny(soft_unstable)]` on by default
```

Explanation

The `bench` attribute was accidentally allowed to be specified on the [stable release channel](#). Turning this to a hard error would have broken some projects. This lint allows those projects to continue to build correctly when `--cap-lints` is used, but otherwise signal an error that `#[bench]` should not be used on the stable channel. This is a [future-incompatible](#) lint to transition this to a hard error in the future. See [issue #64266](#) for more details.

test-unstable-lint

The `test_unstable_lint` lint tests unstable lints and is perma-unstable.

Example

```
#![allow(test_unstable_lint)]
```

This will produce:

```
warning: unknown lint: `test_unstable_lint`
--> lint_example.rs:1:1
   |
1  | #![allow(test_unstable_lint)]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   = note: the `test_unstable_lint` lint is unstable
   = help: add `#![feature(test_unstable_lint)]` to the crate attributes to
enable
   = note: this compiler was built on 2024-07-25; consider upgrading it if it is
out of date
   = note: `#[warn(unknown_lints)]` on by default
```

Explanation

In order to test the behavior of unstable lints, a permanently-unstable lint is required. This lint can be used to trigger warnings and errors from the compiler related to unstable lints.

text-direction-codepoint-in-comment

The `text_direction_codepoint_in_comment` lint detects Unicode codepoints in comments that change the visual representation of text on screen in a way that does not correspond to their on memory representation.

Example

```
#![deny(text_direction_codepoint_in_comment)]
fn main() {
    println!("{:?}",); // ';(
}
```

This will produce:

```
error: unicode codepoint changing visible direction of text present in comment
--> lint_example.rs:3:23
  |
3 |     println!("{:?}",); // ');
  |                        ^^^^^-^^
  |                        |   |
  |                        |   '\u{202e}'
  |                        this comment contains an invisible unicode text flow
control codepoint
  |
  = note: these kind of unicode codepoints change the way text flows on
applications that support them, but can cause confusion because they change the
order of characters on the screen
note: the lint level is defined here
--> lint_example.rs:1:9
  |
1 | #![deny(text_direction_codepoint_in_comment)]
  |          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
  = help: if their presence wasn't intentional, you can remove them
```

Explanation

Unicode allows changing the visual flow of text on screen in order to support scripts that are written right-to-left, but a specially crafted comment can make code that will be compiled appear to be part of a comment, depending on the software used to read the code. To avoid potential problems or confusion, such as in CVE-2021-42574, by default we deny their use.

text-direction-codepoint-in-literal

The `text_direction_codepoint_in_literal` lint detects Unicode codepoints that change

the visual representation of text on screen in a way that does not correspond to their on memory representation.

Explanation

The unicode characters `\u{202A}`, `\u{202B}`, `\u{202D}`, `\u{202E}`, `\u{2066}`, `\u{2067}`, `\u{2068}`, `\u{202C}` and `\u{2069}` make the flow of text on screen change its direction on software that supports these codepoints. This makes the text "abc" display as "cba" on screen. By leveraging software that supports these, people can write specially crafted literals that make the surrounding code seem like it's performing one action, when in reality it is performing another. Because of this, we proactively lint against their presence to avoid surprises.

Example

```
#![deny(text_direction_codepoint_in_literal)]
fn main() {
    println!("{:?}", ';( '
}
```

This will produce:

```

error: unicode codepoint changing visible direction of text present in literal
--> lint_example.rs:3:22
  |
3 |     println!("{:?}", ' ');
  |                      ^-
  |                      ||
  |                      |'\u{202e}'
  |                      this literal contains an invisible unicode text flow
control codepoint
  |
  = note: these kind of unicode codepoints change the way text flows on
applications that support them, but can cause confusion because they change the
order of characters on the screen
note: the lint level is defined here
--> lint_example.rs:1:9
  |
1 | #![deny(text_direction_codepoint_in_literal)]
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  = help: if their presence wasn't intentional, you can remove them
help: if you want to keep them but make them visible in your source code, you
can escape them
  |
3 |     println!("{:?}", '\u{202e}');
  |                      ~~~~~~

```

unconditional-panic

The `unconditional_panic` lint detects an operation that will cause a panic at runtime.

Example

```
let x = 1 / 0;
```

This will produce:

```
error: this operation will panic at runtime
--> lint_example.rs:3:9
   |
3  | let x = 1 / 0;
   |           ^^^^^ attempt to divide `1_i32` by zero
   |
= note: `[deny(unconditional_panic)]` on by default
```

Explanation

This lint detects code that is very likely incorrect because it will always panic, such as division by zero and out-of-bounds array accesses. Consider adjusting your code if this is a bug, or using the `panic!` or `unreachable!` macro instead in case the panic is intended.

undropped-manually-drops

The `undropped_manually_drops` lint check for calls to `std::mem::drop` with a value of `std::mem::ManuallyDrop` which doesn't drop.

Example

```
struct S;
drop(std::mem::ManuallyDrop::new(S));
```

This will produce:

```

error: calls to `std::mem::drop` with `std::mem::ManuallyDrop` instead of the
inner value does nothing
--> lint_example.rs:3:1
   |
3  | drop(std::mem::ManuallyDrop::new(S));
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   | argument has type `ManuallyDrop<S>`
   |
   = note: `[deny(undropped_manually_drops)]` on by default
help: use `std::mem::ManuallyDrop::into_inner` to get the inner value
   |
3  | drop(std::mem::ManuallyDrop::into_inner(std::mem::ManuallyDrop::new(S)));
   | ++++++

```

Explanation

`ManuallyDrop` does not drop it's inner value so calling `std::mem::drop` will not drop the inner value of the `ManuallyDrop` either.

unknown-crate-types

The `unknown_crate_types` lint detects an unknown crate type found in a `crate_type` attribute.

Example

```

#![crate_type="lol"]
fn main() {}

```

This will produce:

```

error: invalid `crate_type` value
--> lint_example.rs:1:15
   |
1  | #![crate_type="lol"]
   |                ^^^^^
   |
= note: `#[deny(unknown_crate_types)]` on by default

```

Explanation

An unknown value give to the `crate_type` attribute is almost certainly a mistake.

useless-deprecated

The `useless_deprecated` lint detects deprecation attributes with no effect.

Example

```

struct X;

#[deprecated = "message"]
impl Default for X {
    fn default() -> Self {
        X
    }
}

```

This will produce:


```
error: this `#[deprecated]` annotation has no effect
--> lint_example.rs:4:1
   |
4  | #[deprecated = "message"]
   | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ help: remove the unnecessary deprecation
attribute
   |
   = note: `#[deny(useless_deprecated)]` on by default
```

Explanation

Deprecation attributes have no effect on trait implementations.

JSON Output

This chapter documents the JSON structures emitted by `rustc`. JSON may be enabled with the `--error-format=json` flag. Additional options may be specified with the `--json` flag which can change which messages are generated, and the format of the messages.

JSON messages are emitted one per line to `stderr`.

If parsing the output with Rust, the `cargo_metadata` crate provides some support for parsing the messages.

Each type of message has a `$message_type` field which can be used to distinguish the different formats. When parsing, care should be taken to be forwards-compatible with future changes to the format. Optional values may be `null`. New fields may be added. Enumerated fields like "level" or "suggestion_applicability" may add new values.

Diagnostics

Diagnostic messages provide errors or possible concerns generated during compilation. `rustc` provides detailed information about where the diagnostic originates, along with hints and suggestions.

Diagnostics are arranged in a parent/child relationship where the parent diagnostic value is the core of the diagnostic, and the attached children provide additional context, help, and information.

Diagnostics have the following format:

```
{  
  /* Type of this message */  
  "$message_type": "diagnostic",  
  /* The primary message. */  
  "message": "unused variable: `x`",  
  /* The diagnostic code.  
     Some messages may set this value to null.  
  */  
  "code": {
```

```

    /* A unique string identifying which diagnostic triggered. */
    "code": "unused_variables",
    /* An optional string explaining more detail about the diagnostic code.
*/
    "explanation": null
},
/* The severity of the diagnostic.
Values may be:
- "error": A fatal error that prevents compilation.
- "warning": A possible error or concern.
- "note": Additional information or context about the diagnostic.
- "help": A suggestion on how to resolve the diagnostic.
- "failure-note": A note attached to the message for further
information.
- "error: internal compiler error": Indicates a bug within the compiler.
*/
"level": "warning",
/* An array of source code locations to point out specific details about
where the diagnostic originates from. This may be empty, for example
for some global messages, or child messages attached to a parent.

Character offsets are offsets of Unicode Scalar Values.
*/
"spans": [
{
    /* The file where the span is located.
Note that this path may not exist. For example, if the path
points to the standard library, and the rust src is not
available in the sysroot, then it may point to a nonexistent
file. Beware that this may also point to the source of an
external crate.
*/
    "file_name": "lib.rs",
    /* The byte offset where the span starts (0-based, inclusive). */
    "byte_start": 21,
    /* The byte offset where the span ends (0-based, exclusive). */
    "byte_end": 22,
    /* The first line number of the span (1-based, inclusive). */
    "line_start": 2,
    /* The last line number of the span (1-based, inclusive). */
    "line_end": 2,
    /* The first character offset of the line_start (1-based,
inclusive). */
    "column_start": 9,
    /* The last character offset of the line_end (1-based, exclusive).
*/
    "column_end": 10,

```

```
/* Whether or not this is the "primary" span.
```

This indicates that this span is the focal point of the diagnostic.

There are rare cases where multiple spans may be marked as primary. For example, "immutable borrow occurs here" and "mutable borrow ends here" can be two separate primary spans.

The top (parent) message should always have at least one primary span, unless it has zero spans. Child messages may have zero or more primary spans.

```
*/
"is_primary": true,
/* An array of objects showing the original source code for this
   span. This shows the entire lines of text where the span is
   located. A span across multiple lines will have a separate
   value for each line.
*/
"text": [
  {
    /* The entire line of the original source code. */
    "text": "    let x = 123;",
    /* The first character offset of the line of
       where the span covers this line (1-based, inclusive). */
    "highlight_start": 9,
    /* The last character offset of the line of
       where the span covers this line (1-based, exclusive). */
    "highlight_end": 10
  }
],
/* An optional message to display at this span location.
   This is typically null for primary spans.
*/
"label": null,
/* An optional string of a suggested replacement for this span to
   solve the issue. Tools may try to replace the contents of the
   span with this text.
*/
"suggested_replacement": null,
/* An optional string that indicates the confidence of the
   "suggested_replacement". Tools may use this value to determine
   whether or not suggestions should be automatically applied.
```

Possible values may be:

- "MachineApplicable": The suggestion is definitely what the user intended. This suggestion should be automatically

```

        applied.
    - "MaybeIncorrect": The suggestion may be what the user
      intended, but it is uncertain. The suggestion should result
      in valid Rust code if it is applied.
    - "HasPlaceholders": The suggestion contains placeholders like
      `(...)`. The suggestion cannot be applied automatically
      because it will not result in valid Rust code. The user will
      need to fill in the placeholders.
    - "Unspecified": The applicability of the suggestion is unknown.
  */
  "suggestion_applicability": null,
  /* An optional object indicating the expansion of a macro within
     this span.

     If a message occurs within a macro invocation, this object will
     provide details of where within the macro expansion the message
     is located.
  */
  "expansion": {
    /* The span of the macro invocation.
       Uses the same span definition as the "spans" array.
    */
    "span": { /*...*/ }
    /* Name of the macro, such as "foo!" or "#[derive(Eq)]". */
    "macro_decl_name": "some_macro!",
    /* Optional span where the relevant part of the macro is
       defined. */
    "def_site_span": { /*...*/ },
  }
},
/* Array of attached diagnostic messages.
   This is an array of objects using the same format as the parent
   message. Children are not nested (children do not themselves
   contain "children" definitions).
*/
"children": [
  {
    "message": "`#[warn(unused_variables)]` on by default",
    "code": null,
    "level": "note",
    "spans": [],
    "children": [],
    "rendered": null
  },
  {
    "message": "if this is intentional, prefix it with an underscore",

```

```

"code": null,
"level": "help",
"spans": [
  {
    "file_name": "lib.rs",
    "byte_start": 21,
    "byte_end": 22,
    "line_start": 2,
    "line_end": 2,
    "column_start": 9,
    "column_end": 10,
    "is_primary": true,
    "text": [
      {
        "text": "    let x = 123;",
        "highlight_start": 9,
        "highlight_end": 10
      }
    ],
    "label": null,
    "suggested_replacement": "_x",
    "suggestion_applicability": "MachineApplicable",
    "expansion": null
  }
],
"children": [],
"rendered": null
},
],
/* Optional string of the rendered version of the diagnostic as displayed
   by rustc. Note that this may be influenced by the `--json` flag.
*/
"rendered": "warning: unused variable: `x`\n --> lib.rs:2:9\n  |\n2 | \nlet x = 123;\n  |           ^ help: if this is intentional, prefix it with an\nunderscore: `_x`\n  |\n   = note: `[warn(unused_variables)]` on by default\n\n"
}

```

Artifact notifications

Artifact notifications are emitted when the `--json=artifacts` flag is used. They indicate that a file artifact has been saved to disk. More information about emit kinds may be found in the `--emit` flag documentation. Notifications can contain more than one file for each

type, for example when using multiple codegen units.

```
{
  /* Type of this message */
  "$message_type": "artifact",
  /* The filename that was generated. */
  "artifact": "libfoo.rlib",
  /* The kind of artifact that was generated. Possible values:
   - "link": The generated crate as specified by the crate-type.
   - "dep-info": The `.d` file with dependency information in a Makefile-
like syntax.
   - "metadata": The Rust `.rmeta` file containing metadata about the
crate.
   - "asm": The `.s` file with generated assembly
   - "llvm-ir": The `.ll` file with generated textual LLVM IR
   - "llvm-bc": The `.bc` file with generated LLVM bytecode
   - "mir": The `.mir` file with rustc's mid-level intermediate
representation.
   - "obj": The `.o` file with generated native object code
  */
  "emit": "link"
}
```

Future-incompatible reports

If the `--json=future-incompat` flag is used, then a separate JSON structure will be emitted if the crate may stop compiling in the future. This contains diagnostic information about the particular warnings that may be turned into a hard error in the future. This will include the diagnostic information, even if the diagnostics have been suppressed (such as with an `# [allow]` attribute or the `--cap-lints` option).

```

{
  /* Type of this message */
  "$message_type": "future_incompat",
  /* An array of objects describing a warning that will become a hard error
     in the future.
  */
  "future_incompat_report":
  [
    {
      /* A diagnostic structure as defined in
         https://doc.rust-lang.org/rustc/json.html#diagnostics
      */
      "diagnostic": {...},
    }
  ]
}

```

Unused Dependency Notifications

The options `--json=unused-externs` and `--json=unused-externs-silent` in conjunction with the `unused-crate-dependencies` lint will emit JSON structures reporting any crate dependencies (specified with `--extern`) which never had any symbols referenced. These are intended to be consumed by the build system which can then emit diagnostics telling the user to remove the unused dependencies from `Cargo.toml` (or whatever build-system file defines dependencies).

The JSON structure is:

```

{
  "lint_level": "deny", /* Level of the warning */
  "unused_names": [
    "foo" /* Names of unused crates, as specified with --extern
foo=libfoo.rlib */
  ],
}

```

The warn/deny/forbid lint level (as defined either on the command line or in the source) dictates the `lint_level` in the JSON. With `unused-externs`, a `deny` or `forbid` level diagnostic will also cause `rustc` to exit with a failure exit code.

`unused-externs-silent` will report the diagnostic the same way, but will not cause `rustc` to exit with failure - it's up to the consumer to flag failure appropriately. (This is needed by Cargo which shares the same dependencies across multiple build targets, so it should only report an unused dependency if its not used by any of the targets.)

Tests

`rustc` has a built-in facility for building and running tests for a crate. More information about writing and running tests may be found in the [Testing Chapter](#) of the Rust Programming Language book.

Tests are written as free functions with the `#[test]` attribute. For example:

```
#[test]
fn it_works() {
    assert_eq!(2 + 2, 4);
}
```

Tests "pass" if they return without an error. They "fail" if they [panic](#), or return a type such as `Result` that implements the [Termination](#) trait with a non-zero value.

By passing the `--test` option to `rustc`, the compiler will build the crate in a special mode to construct an executable that will run the tests in the crate. The `--test` flag will make the following changes:

- The crate will be built as a `bin` [crate type](#), forcing it to be an executable.
- Links the executable with `libtest`, the test harness that is part of the standard library, which handles running the tests.
- Synthesizes a `main` function which will process command-line arguments and run the tests. This new `main` function will replace any existing `main` function as the entry point of the executable, though the existing `main` will still be compiled.
- Enables the `test` [cfg option](#), which allows your code to use conditional compilation to detect if it is being built as a test.
- Enables building of functions annotated with the `test` and `bench` attributes, which will be run by the test harness.

After the executable is created, you can run it to execute the tests and receive a report on what passes and fails. If you are using [Cargo](#) to manage your project, it has a built-in `cargo test` command which handles all of this automatically. An example of the output looks like this:

```
running 4 tests
test it_works ... ok
test check_valid_args ... ok
test invalid_characters ... ok
test walks_the_dog ... ok
```

```
test result: ok. 4 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

Note: Tests must be built with the [unwind panic strategy](#). This is because all tests run in the same process, and they are intended to catch panics, which is not possible with the `abort` strategy. See the unstable [-Z panic=abort-tests](#) option for experimental support of the `abort` strategy by spawning tests in separate processes.

Test attributes

Tests are indicated using attributes on free functions. The following attributes are used for testing, see the linked documentation for more details:

- `#[test]` — Indicates a function is a test to be run.
- `#[bench]` — Indicates a function is a benchmark to be run. Benchmarks are currently unstable and only available in the nightly channel, see the [unstable docs](#) for more details.
- `#[should_panic]` — Indicates that the test function will only pass if the function panics.
- `#[ignore]` — Indicates that the test function will be compiled, but not run by default. See the `--ignored` and `--include-ignored` options to run these tests.

CLI arguments

The libtest harness has several command-line arguments to control its behavior.

Note: When running with `cargo test`, the libtest CLI arguments must be passed after the `--` argument to differentiate between flags for Cargo and those for the harness. For example: `cargo test -- --nocapture`

Filters

Positional arguments (those without a `-` prefix) are treated as filters which will only run tests whose name matches one of those strings. The filter will match any substring found in the full path of the test function. For example, if the test function `it_works` is located in the module `utils::paths::tests`, then any of the filters `works`, `path`, `utils::`, or `utils::paths::tests::it_works` will match that test.

See [Selection options](#) for more options to control which tests are run.

Action options

The following options perform different actions other than running tests.

`--list`

Prints a list of all tests and benchmarks. Does not run any of the tests. [Filters](#) can be used to list only matching tests.

`-h, --help`

Displays usage information and command-line options.

Selection options

The following options change how tests are selected.

`--test`

This is the default mode where all tests will be run as well as running all benchmarks with only a single iteration (to ensure the benchmark works, without taking the time to actually perform benchmarking). This can be combined with the `--bench` flag to run both tests and perform full benchmarking.

`--bench`

This runs in a mode where tests are ignored, and only runs benchmarks. This can be combined with `--test` to run both benchmarks and tests.

`--exact`

This forces [filters](#) to match the full path of the test exactly. For example, if the test `it_works` is in the module `utils::paths::tests`, then only the string `utils::paths::tests::it_works` will match that test.

`--skip FILTER`

Skips any tests whose name contains the given *FILTER* string. This flag may be passed multiple times.

`--ignored`

Runs only tests that are marked with the [ignore](#) attribute.

`--include-ignored`

Runs both [ignored](#) and non-ignored tests.

`--exclude-should-panic`

Excludes tests marked with the [should_panic](#) attribute.

 This option is [unstable](#), and requires the `-Z unstable-options` flag. See [tracking issue #82348](#) for more information.

Execution options

The following options affect how tests are executed.

`--test-threads` *NUM_THREADS*

Sets the number of threads to use for running tests in parallel. By default, uses the amount of concurrency available on the hardware as indicated by [available_parallelism](#).

This can also be specified with the `RUST_TEST_THREADS` environment variable.

`--force-run-in-process`

Forces the tests to run in a single process when using the [abort panic strategy](#).

 This only works with the unstable `-Z panic-abort-tests` option, and requires the `-Z unstable-options` flag. See [tracking issue #67650](#) for more information.

`--ensure-time`

 This option is [unstable](#), and requires the `-Z unstable-options` flag. See [tracking issue #64888](#) and the [unstable docs](#) for more information.

`--shuffle`

Runs the tests in random order, as opposed to the default alphabetical order.

This may also be specified by setting the `RUST_TEST_SHUFFLE` environment variable to anything but `0`.

The random number generator seed that is output can be passed to `--shuffle-seed` to run the tests in the same order again.

Note that `--shuffle` does not affect whether the tests are run in parallel. To run the tests in random order sequentially, use `--shuffle --test-threads 1`.

 This option is [unstable](#), and requires the `-Z unstable-options` flag. See [tracking](#)

[issue #89583](#) for more information.

`--shuffle-seed SEED`

Like `--shuffle`, but seeds the random number generator with *SEED*. Thus, calling the test harness with `--shuffle-seed SEED` twice runs the tests in the same order both times.

SEED is any 64-bit unsigned integer, for example, one produced by `--shuffle`.

This can also be specified with the `RUST_TEST_SHUFFLE_SEED` environment variable.

 This option is [unstable](#), and requires the `-Z unstable-options` flag. See [tracking issue #89583](#) for more information.

Output options

The following options affect the output behavior.

`-q, --quiet`

Displays one character per test instead of one line per test. This is an alias for `--format=terse`.

`--nocapture`

Does not capture the stdout and stderr of the test, and allows tests to print to the console. Usually the output is captured, and only displayed if the test fails.

This may also be specified by setting the `RUST_TEST_NOCAPTURE` environment variable to anything but `0`.

`--show-output`

Displays the stdout and stderr of successful tests after all tests have run.

Contrast this with `--nocapture` which allows tests to print *while they are running*, which can

cause interleaved output if there are multiple tests running in parallel, `--show-output` ensures the output is contiguous, but requires waiting for all tests to finish.

`--color` *COLOR*

Control when colored terminal output is used. Valid options:

- `auto` : Colorize if stdout is a tty and `--nocapture` is not used. This is the default.
- `always` : Always colorize the output.
- `never` : Never colorize the output.

`--format` *FORMAT*

Controls the format of the output. Valid options:

- `pretty` : This is the default format, with one line per test.
- `terse` : Displays only a single character per test. `--quiet` is an alias for this option.
- `json` : Emits JSON objects, one per line.   This option is [unstable](#), and requires the `-Z unstable-options` flag. See [tracking issue #49359](#) for more information.

`--logfile` *PATH*

Writes the results of the tests to the given file.

`--report-time`

  This option is [unstable](#), and requires the `-Z unstable-options` flag. See [tracking issue #64888](#) and the [unstable docs](#) for more information.

Unstable options

Some CLI options are added in an "unstable" state, where they are intended for experimentation and testing to determine if the option works correctly, has the right design, and is useful. The option may not work correctly, break, or change at any time. To signal that you acknowledge that you are using an unstable option, they require passing the `-Z`

`unstable-options` command-line flag.

Benchmarks

The libtest harness supports running benchmarks for functions annotated with the `#[bench]` attribute. Benchmarks are currently unstable, and only available on the [nightly channel](#). More information may be found in the [unstable book](#).

Custom test frameworks

Experimental support for using custom test harnesses is available on the [nightly channel](#). See [tracking issue #50297](#) and the [custom_test_frameworks](#) documentation for more information.

Platform Support

Support for different platforms ("targets") are organized into three tiers, each with a different set of guarantees. For more information on the policies for targets at each tier, see the [Target Tier Policy](#).

Targets are identified by their "target triple" which is the string to inform the compiler what kind of output should be produced.

Component availability is tracked [here](#).

Tier 1 with Host Tools

Tier 1 targets can be thought of as "guaranteed to work". The Rust project builds official binary releases for each tier 1 target, and automated testing ensures that each tier 1 target builds and passes tests after each change.

Tier 1 targets with host tools additionally support running tools like `rustc` and `cargo` natively on the target, and automated testing ensures that tests pass for the host tools as well. This allows the target to be used as a development platform, not just a compilation target. For the full requirements, see [Tier 1 with Host Tools](#) in the Target Tier Policy.

All tier 1 targets with host tools support the full standard library.

target	notes
aarch64-unknown-linux-gnu	ARM64 Linux (kernel 4.1, glibc 2.17+)
i686-pc-windows-gnu	32-bit MinGW (Windows 10+, Windows Server 2016+) ¹
i686-pc-windows-msvc	32-bit MSVC (Windows 10+, Windows Server 2016+) ¹
i686-unknown-linux-gnu	32-bit Linux (kernel 3.2+, glibc 2.17+) ¹
x86_64-apple-darwin	64-bit macOS (10.12+, Sierra+)
x86_64-pc-windows-gnu	64-bit MinGW (Windows 10+, Windows Server 2016+)

<code>x86_64-pc-windows-msvc</code>	64-bit MSVC (Windows 10+, Windows Server 2016+)
<code>x86_64-unknown-linux-gnu</code>	64-bit Linux (kernel 3.2+, glibc 2.17+)

¹ Due to limitations of the C ABI, floating-point support on `i686` targets is non-compliant: floating-point return values are passed via an x87 register, so NaN payload bits can be lost. Functions with the default Rust ABI are not affected. See [issue #115567](#).

Tier 1

Tier 1 targets can be thought of as "guaranteed to work". The Rust project builds official binary releases for each tier 1 target, and automated testing ensures that each tier 1 target builds and passes tests after each change. For the full requirements, see [Tier 1 target policy](#) in the Target Tier Policy.

At this time, all Tier 1 targets are [Tier 1 with Host Tools](#).

Tier 2 with Host Tools

Tier 2 targets can be thought of as "guaranteed to build". The Rust project builds official binary releases of the standard library (or, in some cases, only the `core` library) for each tier 2 target, and automated builds ensure that each tier 2 target can be used as build target after each change. Automated tests are not always run so it's not guaranteed to produce a working build, but tier 2 targets often work to quite a good degree and patches are always welcome!

Tier 2 target-specific code is not closely scrutinized by Rust team(s) when modifications are made. Bugs are possible in all code, but the level of quality control for these targets is likely to be lower. See [library team policy](#) for details on the review practices for standard library code.

Tier 2 targets with host tools additionally support running tools like `rustc` and `cargo` natively on the target, and automated builds ensure that the host tools build as well. This allows the target to be used as a development platform, not just a compilation target. For the full requirements, see [Tier 2 with Host Tools](#) in the Target Tier Policy.

All tier 2 targets with host tools support the full standard library.

NOTE: The `rust-docs` component is not usually built for tier 2 targets, so Rustup may install the documentation for a similar tier 1 target instead.

target	notes
aarch64-apple-darwin	ARM64 macOS (11.0+, Big Sur+)
<code>aarch64-pc-windows-msvc</code>	ARM64 Windows MSVC
<code>aarch64-unknown-linux-musl</code>	ARM64 Linux with musl 1.2.3
<code>arm-unknown-linux-gnueabi</code>	Armv6 Linux (kernel 3.2, glibc 2.17)
<code>arm-unknown-linux-gnueabihf</code>	Armv6 Linux, hardfloat (kernel 3.2, glibc 2.17)
<code>armv7-unknown-linux-gnueabihf</code>	Armv7-A Linux, hardfloat (kernel 3.2, glibc 2.17)
loongarch64-unknown-linux-gnu	LoongArch64 Linux, LP64D ABI (kernel 5.19, glibc 2.36)
loongarch64-unknown-linux-musl	LoongArch64 Linux, LP64D ABI (kernel 5.19, musl 1.2.5)
<code>powerpc-unknown-linux-gnu</code>	PowerPC Linux (kernel 3.2, glibc 2.17)
<code>powerpc64-unknown-linux-gnu</code>	PPC64 Linux (kernel 3.2, glibc 2.17)
<code>powerpc64le-unknown-linux-gnu</code>	PPC64LE Linux (kernel 3.10, glibc 2.17)
<code>riscv64gc-unknown-linux-gnu</code>	RISC-V Linux (kernel 4.20, glibc 2.29)
<code>s390x-unknown-linux-gnu</code>	S390x Linux (kernel 3.2, glibc 2.17)
<code>x86_64-unknown-freebsd</code>	64-bit FreeBSD
<code>x86_64-unknown-illumos</code>	illumos
<code>x86_64-unknown-linux-musl</code>	64-bit Linux with musl 1.2.3
x86_64-unknown-netbsd	NetBSD/amd64

Tier 2 without Host Tools

Tier 2 targets can be thought of as "guaranteed to build". The Rust project builds official binary releases of the standard library (or, in some cases, only the `core` library) for each tier

2 target, and automated builds ensure that each tier 2 target can be used as build target after each change. Automated tests are not always run so it's not guaranteed to produce a working build, but tier 2 targets often work to quite a good degree and patches are always welcome! For the full requirements, see [Tier 2 target policy](#) in the Target Tier Policy.

The `std` column in the table below has the following meanings:

- ✓ indicates the full standard library is available.
- * indicates the target only supports `no_std` development.
- ? indicates the standard library support is a work-in-progress.

Tier 2 target-specific code is not closely scrutinized by Rust team(s) when modifications are made. Bugs are possible in all code, but the level of quality control for these targets is likely to be lower. See [library team policy](#) for details on the review practices for standard library code.

NOTE: The `rust-docs` component is not usually built for tier 2 targets, so Rustup may install the documentation for a similar tier 1 target instead.

target	std	notes
aarch64-apple-ios	✓	ARM64 iOS
aarch64-apple-ios-sim	✓	Apple iOS Simulator on ARM64
aarch64-fuchsia	✓	Alias for aarch64-unknown-fuchsia
aarch64-unknown-fuchsia	✓	ARM64 Fuchsia
aarch64-linux-android	✓	ARM64 Android
aarch64-pc-windows-gnullvm	✓	ARM64 MinGW (Windows 10+), LLVM ABI
aarch64-unknown-linux-ohos	✓	ARM64 OpenHarmony
aarch64-unknown-none-softfloat	*	Bare ARM64, softfloat
aarch64-unknown-none	*	Bare ARM64, hardfloat
aarch64-unknown-uefi	?	ARM64 UEFI
arm-linux-androideabi	✓	Armv6 Android
arm-unknown-linux-musleabi	✓	Armv6 Linux with musl 1.2.3
		Armv6 Linux with musl 1.2.3,

<code>arm-unknown-linux-musleabihf</code>	✓	hardfloat
<code>arm64ec-pc-windows-msvc</code>	✓	Arm64EC Windows MSVC
<code>armebv7r-none-eabi</code>	*	Bare Armv7-R, Big Endian
<code>armebv7r-none-eabihf</code>	*	Bare Armv7-R, Big Endian, hardfloat
<code>armv5te-unknown-linux-gnueabi</code>	✓	Armv5TE Linux (kernel 4.4, glibc 2.23)
<code>armv5te-unknown-linux-musleabi</code>	✓	Armv5TE Linux with musl 1.2.3
<code>armv7-linux-androideabi</code>	✓	Armv7-A Android
<code>armv7-unknown-linux-gnueabi</code>	✓	Armv7-A Linux (kernel 4.15, glibc 2.27)
<code>armv7-unknown-linux-musleabi</code>	✓	Armv7-A Linux with musl 1.2.3
<code>armv7-unknown-linux-musleabihf</code>	✓	Armv7-A Linux with musl 1.2.3, hardfloat
<code>armv7-unknown-linux-ohos</code>	✓	Armv7-A OpenHarmony
<code>armv7a-none-eabi</code>	*	Bare Armv7-A
<code>armv7r-none-eabi</code>	*	Bare Armv7-R
<code>armv7r-none-eabihf</code>	*	Bare Armv7-R, hardfloat
<code>i586-pc-windows-msvc</code>	*	32-bit Windows w/o SSE ²
<code>i586-unknown-linux-gnu</code>	✓	32-bit Linux w/o SSE (kernel 3.2, glibc 2.17) ²
<code>i586-unknown-linux-musl</code>	✓	32-bit Linux w/o SSE, musl 1.2.3 ²
<code>i686-linux-android</code>	✓	32-bit x86 Android ¹
<code>i686-pc-windows-gnullvm</code>	✓	32-bit x86 MinGW (Windows 10+), LLVM ABI ¹
<code>i686-unknown-freebsd</code>	✓	32-bit FreeBSD ¹
<code>i686-unknown-linux-musl</code>	✓	32-bit Linux with musl 1.2.3 ¹
<code>i686-unknown-uefi</code>	?	32-bit UEFI
		LoongArch64 Bare-metal

loongarch64-unknown-none	*	(LP64D ABI)
loongarch64-unknown-none-softfloat	*	LoongArch64 Bare-metal (LP64S ABI)
nvptx64-nvidia-cuda	*	--emit=asm generates PTX code that runs on NVIDIA GPUs
riscv32imac-unknown-none-elf	*	Bare RISC-V (RV32IMAC ISA)
riscv32i-unknown-none-elf	*	Bare RISC-V (RV32I ISA)
riscv32im-unknown-none-elf	*	Bare RISC-V (RV32IM ISA)
riscv32imc-unknown-none-elf	*	Bare RISC-V (RV32IMC ISA)
riscv32imafc-unknown-none-elf	*	Bare RISC-V (RV32IMAFIC ISA)
riscv64gc-unknown-none-elf	*	Bare RISC-V (RV64IMAFDC ISA)
riscv64imac-unknown-none-elf	*	Bare RISC-V (RV64IMAC ISA)
sparc64-unknown-linux-gnu	✓	SPARC Linux (kernel 4.4, glibc 2.23)
sparcv9-sun-solaris	✓	SPARC Solaris 11, illumos
thumbv6m-none-eabi	*	Bare Armv6-M
thumbv7em-none-eabi	*	Bare Armv7E-M
thumbv7em-none-eabihf	*	Bare Armv7E-M, hardfloat
thumbv7m-none-eabi	*	Bare Armv7-M
thumbv7neon-linux-androideabi	✓	Thumb2-mode Armv7-A Android with NEON
thumbv7neon-unknown-linux-gnueabihf	✓	Thumb2-mode Armv7-A Linux with NEON (kernel 4.4, glibc 2.23)
thumbv8m.base-none-eabi	*	Bare Armv8-M Baseline
thumbv8m.main-none-eabi	*	Bare Armv8-M Mainline
thumbv8m.main-none-eabihf	*	Bare Armv8-M Mainline, hardfloat
wasm32-unknown-emscrip	✓	WebAssembly via Emscripten
wasm32-unknown-unknown	✓	WebAssembly

wasm32-wasi	✓	WebAssembly with WASI (undergoing a rename to wasm32-wasip1)
wasm32-wasip1	✓	WebAssembly with WASI
wasm32-wasip1-threads	✓	WebAssembly with WASI Preview 1 and threads
x86_64-apple-ios	✓	64-bit x86 iOS
x86_64-fortanix-unknown-sgx	✓	Fortanix ABI for 64-bit Intel SGX
x86_64-fuchsia	✓	Alias for x86_64-unknown-fuchsia
x86_64-unknown-fuchsia	✓	64-bit x86 Fuchsia
x86_64-linux-android	✓	64-bit x86 Android
x86_64-pc-solaris	✓	64-bit Solaris 11, illumos
x86_64-pc-windows-gnullvm	✓	64-bit x86 MinGW (Windows 10+), LLVM ABI
x86_64-unknown-linux-gnux32	✓	64-bit Linux (x32 ABI) (kernel 4.15, glibc 2.27)
x86_64-unknown-linux-ohos	✓	x86_64 OpenHarmony
x86_64-unknown-none	*	Freestanding/bare-metal x86_64, softfloat
x86_64-unknown-redox	✓	Redox OS
x86_64-unknown-uefi	?	64-bit UEFI

² Floating-point support on `i586` targets is non-compliant: the `x87` registers and instructions used for these targets do not provide IEEE-754-compliant behavior, in particular when it comes to rounding and NaN payload bits. See [issue #114479](#).

Tier 3

Tier 3 targets are those which the Rust codebase has support for, but which the Rust project does not build or test automatically, so they may or may not work. Official builds are not available. For the full requirements, see [Tier 3 target policy](#) in the Target Tier Policy.

The `std` column in the table below has the following meanings:

- ✓ indicates the full standard library is available.
- * indicates the target only supports `no_std` development.
- ? indicates the standard library support is unknown or a work-in-progress.

Tier 3 target-specific code is not closely scrutinized by Rust team(s) when modifications are made. Bugs are possible in all code, but the level of quality control for these targets is likely to be lower. See [library team policy](#) for details on the review practices for standard library code.

The `host` column indicates whether the codebase includes support for building host tools.

target	std	host	notes
<code>arm64e-apple-ios</code>	✓		ARM64e Apple iOS
<code>arm64e-apple-darwin</code>	✓	✓	ARM64e Apple Darwin
<code>aarch64-apple-ios-macabi</code>	✓		Apple Catalyst on ARM64
<code>aarch64-apple-tvos</code>	✓		ARM64 tvOS
<code>aarch64-apple-tvos-sim</code>	✓		ARM64 tvOS Simulator
<code>aarch64-apple-watchos</code>	✓		ARM64 Apple WatchOS
<code>aarch64-apple-watchos-sim</code>	✓		ARM64 Apple WatchOS Simulator
<code>aarch64-apple-visionos</code>	✓		ARM64 Apple visionOS
<code>aarch64-apple-visionos-sim</code>	✓		ARM64 Apple visionOS Simulator
<code>aarch64-kmc-solid_asp3</code>	✓		ARM64 SOLID with TOPPERS/ASP3
<code>aarch64-nintendo-switch-freestanding</code>	*		ARM64 Nintendo Switch, Horizon
<code>aarch64-unknown-teeos</code>	?		ARM64 TEEOS

aarch64-unknown-nto-qnx710	✓		ARM64 QNX Neutrino 7.1 RTOS
aarch64-unknown-freebsd	✓	✓	ARM64 FreeBSD
aarch64-unknown-hermit	✓		ARM64 Hermit
aarch64-unknown-illumos	✓	✓	ARM64 illumos
aarch64-unknown-linux-gnu_ilp32	✓	✓	ARM64 Linux (ILP32 ABI)
aarch64-unknown-netbsd	✓	✓	ARM64 NetBSD
aarch64-unknown-openbsd	✓	✓	ARM64 OpenBSD
aarch64-unknown-redox	✓		ARM64 Redox OS
aarch64-uwp-windows-msvc	✓		
aarch64-wrs-vxworks	?		
aarch64_be-unknown-linux-gnu_ilp32	✓	✓	ARM64 Linux (big-endian, ILP32 ABI)
aarch64_be-unknown-linux-gnu	✓	✓	ARM64 Linux (big-endian)
aarch64_be-unknown-netbsd	✓	✓	ARM64 NetBSD (big-endian)
arm64_32-apple-watchos	✓		Arm Apple WatchOS 64-bit with 32-bit pointers
armeb-unknown-linux-gnueabi	✓	?	Arm BE8 the default Arm big-endian architecture since Armv6 .
armv4t-none-eabi	*		Bare Armv4T
armv4t-unknown-linux-gnueabi	?		Armv4T Linux
armv5te-none-eabi	*		Bare Armv5TE
armv5te-unknown-linux-uclibceabi	?		Armv5TE Linux with uClibc
armv6-unknown-freebsd	✓	✓	Armv6 FreeBSD
			Armv6 NetBSD

<code>armv6-unknown-netbsd-eabihf</code>	✓	✓	w/hard-float
<code>armv6k-nintendo-3ds</code>	?		Armv6k Nintendo 3DS, Horizon (Requires devkitARM toolchain)
<code>armv7-sony-vita-newlibeabihf</code>	✓		Armv7-A Cortex-A9 Sony PlayStation Vita (requires VITASDK toolchain)
<code>armv7-unknown-linux-uclibceabi</code>	✓	✓	Armv7-A Linux with uClibc, softfloat
<code>armv7-unknown-linux-uclibceabihf</code>	✓	?	Armv7-A Linux with uClibc, hardfloat
<code>armv7-unknown-freebsd</code>	✓	✓	Armv7-A FreeBSD
<code>armv7-unknown-netbsd-eabihf</code>	✓	✓	Armv7-A NetBSD w/hard-float
<code>armv7-wrs-vxworks-eabihf</code>	?		Armv7-A for VxWorks
<code>armv7a-kmc-solid_asp3-eabi</code>	✓		ARM SOLID with TOPPERS/ASP3
<code>armv7a-kmc-solid_asp3-eabihf</code>	✓		ARM SOLID with TOPPERS/ASP3, hardfloat
<code>armv7a-none-eabihf</code>	*		Bare Armv7-A, hardfloat
<code>armv7k-apple-watchos</code>	✓		Armv7-A Apple WatchOS
<code>armv7s-apple-ios</code>	✓		Armv7-A Apple-A6 Apple iOS
<code>armv8r-none-eabihf</code>	*		Bare Armv8-R, hardfloat
<code>avr-unknown-gnu-atmega328</code>	*		AVR. Requires -Z build-std=core
<code>bpfeb-unknown-none</code>	*		BPF (big endian)

<code>bpfel-unknown-none</code>	*		BPF (little endian)
<code>csky-unknown-linux-gnuabiv2</code>	✓		C-SKY abiv2 Linux (little endian)
<code>csky-unknown-linux-gnuabiv2hf</code>	✓		C-SKY abiv2 Linux, hardfloat (little endian)
<code>hexagon-unknown-none-elf</code>	*		Bare Hexagon (v60+, HVX)
<code>hexagon-unknown-linux-musl</code>	✓		Hexagon Linux with musl 1.2.3
<code>i386-apple-ios</code>	✓		32-bit x86 iOS ¹
<code>i586-pc-nto-qnx700</code>	*		32-bit x86 QNX Neutrino 7.0 RTOS ¹
<code>i586-unknown-netbsd</code>	✓		32-bit x86, restricted to Pentium
<code>i686-apple-darwin</code>	✓	✓	32-bit macOS (10.12+, Sierra+) ¹
<code>i686-unknown-haiku</code>	✓	✓	32-bit Haiku ¹
<code>i686-unknown-hurd-gnu</code>	✓	✓	32-bit GNU/Hurd ¹
<code>i686-unknown-netbsd</code>	✓	✓	NetBSD/i386 with SSE2 ¹
<code>i686-unknown-openbsd</code>	✓	✓	32-bit OpenBSD ¹
<code>i686-unknown-redox</code>	✓		i686 Redox OS
<code>i686-uwp-windows-gnu</code>	✓		¹
<code>i686-uwp-windows-msvc</code>	✓		¹
<code>i686-win7-windows-msvc</code>	✓		32-bit Windows 7 support ¹
<code>i686-wrs-vxworks</code>	?		¹
<code>m68k-unknown-linux-gnu</code>	?		Motorola 680x0 Linux

<code>mips-unknown-linux-gnu</code>	✓	✓	MIPS Linux (kernel 4.4, glibc 2.23)
<code>mips-unknown-linux-musl</code>	✓		MIPS Linux with musl 1.2.3
<code>mips-unknown-linux-uclibc</code>	✓		MIPS Linux with uClibc
<code>mips64-openwrt-linux-musl</code>	?		MIPS64 for OpenWrt Linux musl 1.2.3
<code>mips64-unknown-linux-gnuabi64</code>	✓	✓	MIPS64 Linux, N64 ABI (kernel 4.4, glibc 2.23)
<code>mips64-unknown-linux-muslabi64</code>	✓		MIPS64 Linux, N64 ABI, musl 1.2.3
<code>mips64el-unknown-linux-gnuabi64</code>	✓	✓	MIPS64 (little endian) Linux, N64 ABI (kernel 4.4, glibc 2.23)
<code>mips64el-unknown-linux-muslabi64</code>	✓		MIPS64 (little endian) Linux, N64 ABI, musl 1.2.3
<code>mipsel-unknown-linux-gnu</code>	✓	✓	MIPS (little endian) Linux (kernel 4.4, glibc 2.23)
<code>mipsel-unknown-linux-musl</code>	✓		MIPS (little endian) Linux with musl 1.2.3
<code>mipsel-unknown-netbsd</code>	✓	✓	32-bit MIPS (LE), requires mips32 cpu support
<code>mipsel-sony-psp</code>	*		MIPS (LE) Sony PlayStation Portable (PSP)
<code>mipsel-sony-psx</code>	*		MIPS (LE) Sony PlayStation 1 (PSX)
<code>mipsel-unknown-linux-uclibc</code>	✓		MIPS (LE) Linux with

			uClibc
mipsel-unknown-none	*		Bare MIPS (LE) softfloat
mipsisa32r6-unknown-linux-gnu	?		32-bit MIPS Release 6 Big Endian
mipsisa32r6el-unknown-linux-gnu	?		32-bit MIPS Release 6 Little Endian
mipsisa64r6-unknown-linux-gnuabi64	?		64-bit MIPS Release 6 Big Endian
mipsisa64r6el-unknown-linux-gnuabi64	✓	✓	64-bit MIPS Release 6 Little Endian
msp430-none-elf	*		16-bit MSP430 microcontrollers
powerpc-unknown-linux-gnuspe	✓		PowerPC SPE Linux
powerpc-unknown-linux-musl	?		PowerPC Linux with musl 1.2.3
powerpc-unknown-netbsd	✓	✓	NetBSD 32-bit powerpc systems
powerpc-unknown-openbsd	*		
powerpc-wrs-vxworks-spe	?		
powerpc-wrs-vxworks	?		
powerpc64-unknown-freebsd	✓	✓	PPC64 FreeBSD (ELFv1 and ELFv2)
powerpc64le-unknown-freebsd			PPC64LE FreeBSD
powerpc-unknown-freebsd			PowerPC FreeBSD
powerpc64-unknown-linux-musl	?		64-bit PowerPC Linux with musl 1.2.3
powerpc64-wrs-vxworks	?		
powerpc64le-unknown-linux-musl	?		64-bit PowerPC Linux with musl 1.2.3, Little Endian
powerpc64-unknown-openbsd	✓	✓	OpenBSD/powerpc64

<code>powerpc64-ibm-aix</code>	?		64-bit AIX (7.2 and newer)
<code>riscv32gc-unknown-linux-gnu</code>			RISC-V Linux (kernel 5.4, glibc 2.33)
<code>riscv32gc-unknown-linux-musl</code>			RISC-V Linux (kernel 5.4, musl 1.2.3 + RISC-V32 support patches)
<code>riscv32im-risc0-zkvm-elf</code>	?		RISC Zero's zero-knowledge Virtual Machine (RV32IM ISA)
<code>riscv32ima-unknown-none-elf</code>	*		Bare RISC-V (RV32IMA ISA)
<code>riscv32imac-unknown-xous-elf</code>	?		RISC-V Xous (RV32IMAC ISA)
<code>riscv32imc-esp-espidf</code>	✓		RISC-V ESP-IDF
<code>riscv32imac-esp-espidf</code>	✓		RISC-V ESP-IDF
<code>riscv32imafc-esp-espidf</code>	✓		RISC-V ESP-IDF
<code>riscv64gc-unknown-hermit</code>	✓		RISC-V Hermit
<code>riscv64gc-unknown-freebsd</code>			RISC-V FreeBSD
<code>riscv64gc-unknown-fuchsia</code>			RISC-V Fuchsia
<code>riscv64gc-unknown-linux-musl</code>			RISC-V Linux (kernel 4.20, musl 1.2.3)
<code>riscv64gc-unknown-netbsd</code>	✓	✓	RISC-V NetBSD
<code>riscv64gc-unknown-openbsd</code>	✓	✓	OpenBSD/riscv64
<code>riscv64-linux-android</code>			RISC-V 64-bit Android
<code>s390x-unknown-linux-musl</code>			S390x Linux (kernel 3.2, musl 1.2.3)
<code>sparc-unknown-linux-gnu</code>	✓		32-bit SPARC Linux
<code>sparc-unknown-none-elf</code>	*		Bare 32-bit SPARC V7+

sparc64-unknown-netbsd	✓	✓	NetBSD/sparc64
sparc64-unknown-openbsd	✓	✓	OpenBSD/sparc64
thumbv4t-none-eabi	*		Thumb-mode Bare Armv4T
thumbv5te-none-eabi	*		Thumb-mode Bare Armv5TE
thumbv7a-pc-windows-msvc	✓		
thumbv7a-uwp-windows-msvc	✓		
thumbv7neon-unknown-linux-musleabihf	?		Thumb2-mode Armv7-A Linux with NEON, musl 1.2.3
wasm32-wasip2	✓		WebAssembly
wasm64-unknown-unknown	?		WebAssembly
x86_64-apple-ios-macabi	✓		Apple Catalyst on x86_64
x86_64-apple-tvos	✓		x86 64-bit tvOS
x86_64-apple-watchos-sim	✓		x86 64-bit Apple WatchOS simulator
x86_64-pc-nto-qnx710	✓		x86 64-bit QNX Neutrino 7.1 RTOS
x86_64-unikraft-linux-musl	✓		64-bit Unikraft with musl 1.2.3
x86_64-unknown-dragonfly	✓	✓	64-bit DragonFlyBSD
x86_64-unknown-haiku	✓	✓	64-bit Haiku
x86_64-unknown-hermit	✓		x86_64 Hermit
x86_64-unknown-l4re-uclibc	?		
x86_64-unknown-openbsd	✓	✓	64-bit OpenBSD
x86_64-uwp-windows-gnu	✓		
x86_64-uwp-windows-msvc	✓		
x86_64-win7-windows-msvc	✓		64-bit Windows 7 support

x86_64-wrs-vxworks	?		
x86_64h-apple-darwin	✓	✓	macOS with late-gen Intel (at least Haswell)
x86_64-unknown-linux-none	*		64-bit Linux with no libc
xtensa-esp32-none-elf	*		Xtensa ESP32
xtensa-esp32-espidf	✓		Xtensa ESP32
xtensa-esp32s2-none-elf	*		Xtensa ESP32-S2
xtensa-esp32s2-espidf	✓		Xtensa ESP32-S2
xtensa-esp32s3-none-elf	*		Xtensa ESP32-S3
xtensa-esp32s3-espidf	✓		Xtensa ESP32-S3

Target Tier Policy

Table of Contents

- General
- Adding a new target
- Tier 3 target policy
- Tier 2 target policy
 - Tier 2 with host tools
- Tier 1 target policy
 - Tier 1 with host tools

General

Rust provides three tiers of target support:

- Rust provides no guarantees about tier 3 targets; they exist in the codebase, but may or may not build.
- Rust's continuous integration checks that tier 2 targets will always build, but they may or may not pass tests.
- Rust's continuous integration checks that tier 1 targets will always build and pass tests.

Adding a new tier 3 target imposes minimal requirements; we focus primarily on avoiding disruption to other ongoing Rust development.

Tier 2 and tier 1 targets place work on Rust project developers as a whole, to avoid breaking the target. The broader Rust community may also feel more inclined to support higher-tier targets in their crates (though they are not obligated to do so). Thus, these tiers require commensurate and ongoing efforts from the maintainers of the target, to demonstrate value and to minimize any disruptions to ongoing Rust development.

This policy defines the requirements for accepting a proposed target at a given level of

support.

Each tier builds on all the requirements from the previous tier, unless overridden by a stronger requirement. Targets at tier 2 and tier 1 may also provide *host tools* (such as `rustc` and `cargo`); each of those tiers includes a set of supplementary requirements that must be met if supplying host tools for the target. A target at tier 2 or tier 1 is not required to supply host tools, but if it does, it must meet the corresponding additional requirements for host tools.

The policy for each tier also documents the Rust governance teams that must approve the addition of any target at that tier. Those teams are responsible for reviewing and evaluating the target, based on these requirements and their own judgment. Those teams may apply additional requirements, including subjective requirements, such as to deal with issues not foreseen by this policy. (Such requirements may subsequently motivate additions to this policy.)

While these criteria attempt to document the policy, that policy still involves human judgment. Targets must fulfill the spirit of the requirements as well, as determined by the judgment of the approving teams. Reviewers and team members evaluating targets and target-specific patches should always use their own best judgment regarding the quality of work, and the suitability of a target for the Rust project. Neither this policy nor any decisions made regarding targets shall create any binding agreement or estoppel by any party.

Before filing an issue or pull request (PR) to introduce or promote a target, the target should already meet the corresponding tier requirements. This does not preclude an existing target's maintainers using issues (on the Rust repository or otherwise) to track requirements that have not yet been met, as appropriate; however, before officially proposing the introduction or promotion of a target, it should meet all of the necessary requirements. A target proposal must quote the corresponding requirements verbatim and respond to them as part of explaining how the target meets those requirements. (For the requirements that simply state that the target or the target developers must not do something, it suffices to acknowledge the requirement.)

For a list of all supported targets and their corresponding tiers ("tier 3", "tier 2", "tier 2 with host tools", "tier 1", or "tier 1 with host tools"), see [platform support](#).

Several parts of this policy require providing target-specific documentation. Such documentation should typically appear in a subdirectory of the platform-support section of this rustc manual, with a link from the target's entry in [platform support](#). Use `TEMPLATE.md`

as a base, and see other documentation in that directory for examples.

Note that a target must have already received approval for the next lower tier, and spent a reasonable amount of time at that tier, before making a proposal for promotion to the next higher tier; this is true even if a target meets the requirements for several tiers at once. This policy leaves the precise interpretation of "reasonable amount of time" up to the approving teams; those teams may scale the amount of time required based on their confidence in the target and its demonstrated track record at its current tier. At a minimum, multiple stable releases of Rust should typically occur between promotions of a target.

The availability or tier of a target in stable Rust is not a hard stability guarantee about the future availability or tier of that target. Higher-level target tiers are an increasing commitment to the support of a target, and we will take that commitment and potential disruptions into account when evaluating the potential demotion or removal of a target that has been part of a stable release. The promotion or demotion of a target will not generally affect existing stable releases, only current development and future releases.

In this policy, the words "must" and "must not" specify absolute requirements that a target must meet to qualify for a tier. The words "should" and "should not" specify requirements that apply in almost all cases, but for which the approving teams may grant an exception for good reason. The word "may" indicates something entirely optional, and does not indicate guidance or recommendations. This language is based on [IETF RFC 2119](#).

Adding a new target

New targets typically start as Tier 3 and then can be promoted later. To propose addition of a new target, open a pull request on [rust-lang/rust](#) :

- Copy the [Tier 3 target policy](#) to the description and fill it out, see [example](#).
- Add a new description for the target in `src/doc/rustc/src/platform-support` using the [template](#).
- Add the target to the [SUMMARY.md](#) (allows wildcards) and [platform-support.md](#) (must name all targets verbatim). Link to the created description page.
- Ensure the pull request is assigned to a member of the [Rust compiler team](#) by commenting:

r? compiler-team

Tier 3 target policy

At this tier, the Rust project provides no official support for a target, so we place minimal requirements on the introduction of targets.

A proposed new tier 3 target must be reviewed and approved by a member of the compiler team based on these requirements. The reviewer may choose to gauge broader compiler team consensus via a [Major Change Proposal \(MCP\)](#).

A proposed target or target-specific patch that substantially changes code shared with other targets (not just target-specific code) must be reviewed and approved by the appropriate team for that shared code before acceptance.

- A tier 3 target must have a designated developer or developers (the "target maintainers") on record to be CCed when issues arise regarding the target. (The mechanism to track and CC such developers may evolve over time.)
- Targets must use naming consistent with any existing targets; for instance, a target for the same CPU or OS as an existing Rust target should use the same name for that CPU or OS. Targets should normally use the same names and naming conventions as used elsewhere in the broader ecosystem beyond Rust (such as in other toolchains), unless they have a very good reason to diverge. Changing the name of a target can be highly disruptive, especially once the target reaches a higher tier, so getting the name right is important even for a tier 3 target.
 - Target names should not introduce undue confusion or ambiguity unless absolutely necessary to maintain ecosystem compatibility. For example, if the name of the target makes people extremely likely to form incorrect beliefs about what it targets, the name should be changed or augmented to disambiguate it.
 - If possible, use only letters, numbers, dashes and underscores for the name. Periods (.) are known to cause issues in Cargo.
- Tier 3 targets may have unusual requirements to build or use, but must not create legal issues or impose onerous legal terms for the Rust project or for Rust developers or users.
 - The target must not introduce license incompatibilities.

- Anything added to the Rust repository must be under the standard Rust license (MIT OR Apache-2.0).
- The target must not cause the Rust tools or libraries built for any other host (even when supporting cross-compilation to the target) to depend on any new dependency less permissive than the Rust licensing policy. This applies whether the dependency is a Rust crate that would require adding new license exceptions (as specified by the `tidy` tool in the rust-lang/rust repository), or whether the dependency is a native library or binary. In other words, the introduction of the target must not cause a user installing or running a version of Rust or the Rust tools to be subject to any new license requirements.
- Compiling, linking, and emitting functional binaries, libraries, or other code for the target (whether hosted on the target itself or cross-compiling from another target) must not depend on proprietary (non-FOSS) libraries. Host tools built for the target itself may depend on the ordinary runtime libraries supplied by the platform and commonly used by other applications built for the target, but those libraries must not be required for code generation for the target; cross-compilation to the target must not require such libraries at all. For instance, `rustc` built for the target may depend on a common proprietary C runtime library or console output library, but must not depend on a proprietary code generation library or code optimization library. Rust's license permits such combinations, but the Rust project has no interest in maintaining such combinations within the scope of Rust itself, even at tier 3.
- "onerous" here is an intentionally subjective term. At a minimum, "onerous" legal/licensing terms include but are *not* limited to: non-disclosure requirements, non-compete requirements, contributor license agreements (CLAs) or equivalent, "non-commercial"/"research-only"/etc terms, requirements conditional on the employer or employment of any particular Rust developers, revocable terms, any requirements that create liability for the Rust project or its developers or users, or any requirements that adversely affect the livelihood or prospects of the Rust project or its developers or users.
- Neither this policy nor any decisions made regarding targets shall create any binding agreement or estoppel by any party. If any member of an approving Rust team serves as one of the maintainers of a target, or has any legal or employment requirement (explicit or implicit) that might affect their decisions regarding a target, they must recuse themselves from any approval decisions regarding the target's tier status, though they may otherwise participate in discussions.
 - This requirement does not prevent part or all of this policy from being cited in an

explicit contract or work agreement (e.g. to implement or maintain support for a target). This requirement exists to ensure that a developer or team responsible for reviewing and approving a target does not face any legal threats or obligations that would prevent them from freely exercising their judgment in such approval, even if such judgment involves subjective matters or goes beyond the letter of these requirements.

- Tier 3 targets should attempt to implement as much of the standard libraries as possible and appropriate (`core` for most targets, `alloc` for targets that can support dynamic memory allocation, `std` for targets with an operating system or equivalent layer of system-provided functionality), but may leave some code unimplemented (either unavailable or stubbed out as appropriate), whether because the target makes it impossible to implement or challenging to implement. The authors of pull requests are not obligated to avoid calling any portions of the standard library on the basis of a tier 3 target not implementing those portions.
- The target must provide documentation for the Rust community explaining how to build for the target, using cross-compilation if possible. If the target supports running binaries, or running tests (even if they do not pass), the documentation must explain how to run such binaries or tests for the target, using emulation if possible or dedicated hardware if necessary.
- Tier 3 targets must not impose burden on the authors of pull requests, or other developers in the community, to maintain the target. In particular, do not post comments (automated or manual) on a PR that derail or suggest a block on the PR based on a tier 3 target. Do not send automated messages or notifications (via any medium, including via @) to a PR author or others involved with a PR regarding a tier 3 target, unless they have opted into such messages.
 - Backlinks such as those generated by the issue/PR tracker when linking to an issue or PR are not considered a violation of this policy, within reason. However, such messages (even on a separate repository) must not generate notifications to anyone involved with a PR who has not requested such notifications.
- Patches adding or updating tier 3 targets must not break any existing tier 2 or tier 1 target, and must not knowingly break another tier 3 target without approval of either the compiler team or the maintainers of the other tier 3 target.
 - In particular, this may come up when working on closely related targets, such as variations of the same architecture with different features. Avoid introducing unconditional uses of features that another variation of the target may not have; use conditional compilation or runtime detection, as appropriate, to let each target run code supported by that target.

- Tier 3 targets must be able to produce assembly using at least one of rustc's supported backends from any host target. (Having support in a fork of the backend is not sufficient, it must be upstream.)

If a tier 3 target stops meeting these requirements, or the target maintainers no longer have interest or time, or the target shows no signs of activity and has not built for some time, or removing the target would improve the quality of the Rust codebase, we may post a PR to remove it; any such PR will be CCed to the target maintainers (and potentially other people who have previously worked on the target), to check potential interest in improving the situation.

Tier 2 target policy

At this tier, the Rust project guarantees that a target builds, and will reject patches that fail to build on a target. Thus, we place requirements that ensure the target will not block forward progress of the Rust project.

A proposed new tier 2 target must be reviewed and approved by the compiler team based on these requirements. Such review and approval may occur via a [Major Change Proposal \(MCP\)](#).

In addition, the infrastructure team must approve the integration of the target into Continuous Integration (CI), and the tier 2 CI-related requirements. This review and approval may take place in a PR adding the target to CI, or simply by an infrastructure team member reporting the outcome of a team discussion.

- A tier 2 target must have value to people other than its maintainers. (It may still be a niche target, but it must not be exclusively useful for an inherently closed group.)
- A tier 2 target must have a designated team of developers (the "target maintainers") available to consult on target-specific build-breaking issues, or if necessary to develop target-specific language or library implementation details. This team must have at least 2 developers.
 - The target maintainers should not only fix target-specific issues, but should use any such issue as an opportunity to educate the Rust community about portability to their target, and enhance documentation of the target.
- The target must not place undue burden on Rust developers not specifically concerned

with that target. Rust developers are expected to not gratuitously break a tier 2 target, but are not expected to become experts in every tier 2 target, and are not expected to provide target-specific implementations for every tier 2 target.

- The target must provide documentation for the Rust community explaining how to build for the target using cross-compilation, and explaining how to run tests for the target. If at all possible, this documentation should show how to run Rust programs and tests for the target using emulation, to allow anyone to do so. If the target cannot be feasibly emulated, the documentation should explain how to obtain and work with physical hardware, cloud systems, or equivalent.
- The target must document its baseline expectations for the features or versions of CPUs, operating systems, libraries, runtime environments, and similar.
- If introducing a new tier 2 or higher target that is identical to an existing Rust target except for the baseline expectations for the features or versions of CPUs, operating systems, libraries, runtime environments, and similar, then the proposed target must document to the satisfaction of the approving teams why the specific difference in baseline expectations provides sufficient value to justify a separate target.
 - Note that in some cases, based on the usage of existing targets within the Rust community, Rust developers or a target's maintainers may wish to modify the baseline expectations of a target, or split an existing target into multiple targets with different baseline expectations. A proposal to do so will be treated similarly to the analogous promotion, demotion, or removal of a target, according to this policy, with the same team approvals required.
 - For instance, if an OS version has become obsolete and unsupported, a target for that OS may raise its baseline expectations for OS version (treated as though removing a target corresponding to the older versions), or a target for that OS may split out support for older OS versions into a lower-tier target (treated as though demoting a target corresponding to the older versions, and requiring justification for a new target at a lower tier for the older OS versions).
- Tier 2 targets must not leave any significant portions of `core` or the standard library unimplemented or stubbed out, unless they cannot possibly be supported on the target.
 - The right approach to handling a missing feature from a target may depend on whether the target seems likely to develop the feature in the future. In some cases, a target may be co-developed along with Rust support, and Rust may gain new features on the target as that target gains the capabilities to support those features.

- As an exception, a target identical to an existing tier 1 target except for lower baseline expectations for the OS, CPU, or similar, may propose to qualify as tier 2 (but not higher) without support for `std` if the target will primarily be used in `no_std` applications, to reduce the support burden for the standard library. In this case, evaluation of the proposed target's value will take this limitation into account.
- The code generation backend for the target should not have deficiencies that invalidate Rust safety properties, as evaluated by the Rust compiler team. (This requirement does not apply to arbitrary security enhancements or mitigations provided by code generation backends, only to those properties needed to ensure safe Rust code cannot cause undefined behavior or other unsoundness.) If this requirement does not hold, the target must clearly and prominently document any such limitations as part of the target's entry in the target tier list, and ideally also via a failing test in the testsuite. The Rust compiler team must be satisfied with the balance between these limitations and the difficulty of implementing the necessary features.
 - For example, if Rust relies on a specific code generation feature to ensure that safe code cannot overflow the stack, the code generation for the target should support that feature.
 - If the Rust compiler introduces new safety properties (such as via new capabilities of a compiler backend), the Rust compiler team will determine if they consider those new safety properties a best-effort improvement for specific targets, or a required property for all Rust targets. In the latter case, the compiler team may require the maintainers of existing targets to either implement and confirm support for the property or update the target tier list with documentation of the missing property.
- If the target supports C code, and the target has an interoperable calling convention for C code, the Rust target must support that C calling convention for the platform via `extern "C"`. The C calling convention does not need to be the default Rust calling convention for the target, however.
- The target must build reliably in CI, for all components that Rust's CI considers mandatory.
- The approving teams may additionally require that a subset of tests pass in CI, such as enough to build a functional "hello world" program, `./x.py test --no-run`, or equivalent "smoke tests". In particular, this requirement may apply if the target builds host tools, or if the tests in question provide substantial value via early detection of critical problems.
- Building the target in CI must not take substantially longer than the current slowest

target in CI, and should not substantially raise the maintenance burden of the CI infrastructure. This requirement is subjective, to be evaluated by the infrastructure team, and will take the community importance of the target into account.

- Tier 2 targets should, if at all possible, support cross-compiling. Tier 2 targets should not require using the target as the host for builds, even if the target supports host tools.
- In addition to the legal requirements for all targets (specified in the tier 3 requirements), because a tier 2 target typically involves the Rust project building and supplying various compiled binaries, incorporating the target and redistributing any resulting compiled binaries (e.g. built libraries, host tools if any) must not impose any onerous license requirements on any members of the Rust project, including infrastructure team members and those operating CI systems. This is a subjective requirement, to be evaluated by the approving teams.
 - As an exception to this, if the target's primary purpose is to build components for a Free and Open Source Software (FOSS) project licensed under "copyleft" terms (terms which require licensing other code under compatible FOSS terms), such as kernel modules or plugins, then the standard libraries for the target may potentially be subject to copyleft terms, as long as such terms are satisfied by Rust's existing practices of providing full corresponding source code. Note that anything added to the Rust repository itself must still use Rust's standard license terms.
- Tier 2 targets must not impose burden on the authors of pull requests, or other developers in the community, to ensure that tests pass for the target. In particular, do not post comments (automated or manual) on a PR that derail or suggest a block on the PR based on tests failing for the target. Do not send automated messages or notifications (via any medium, including via @) to a PR author or others involved with a PR regarding the PR breaking tests on a tier 2 target, unless they have opted into such messages.
 - Backlinks such as those generated by the issue/PR tracker when linking to an issue or PR are not considered a violation of this policy, within reason. However, such messages (even on a separate repository) must not generate notifications to anyone involved with a PR who has not requested such notifications.
- The target maintainers should regularly run the testsuite for the target, and should fix any test failures in a reasonably timely fashion.
- All requirements for tier 3 apply.

A tier 2 target may be demoted or removed if it no longer meets these requirements. Any

proposal for demotion or removal will be CCed to the target maintainers, and will be communicated widely to the Rust community before being dropped from a stable release. (The amount of time between such communication and the next stable release may depend on the nature and severity of the failed requirement, the timing of its discovery, whether the target has been part of a stable release yet, and whether the demotion or removal can be a planned and scheduled action.)

In some circumstances, especially if the target maintainers do not respond in a timely fashion, Rust teams may land pull requests that temporarily disable some targets in the nightly compiler, in order to implement a feature not yet supported by those targets. (As an example, this happened when introducing the 128-bit types `u128` and `i128`.) Such a pull request will include notification and coordination with the maintainers of such targets, and will ideally happen towards the beginning of a new development cycle to give maintainers time to update their targets. The maintainers of such targets will then be expected to implement the corresponding target-specific support in order to re-enable the target. If the maintainers of such targets cannot provide such support in time for the next stable release, this may result in demoting or removing the targets.

Tier 2 with host tools

Some tier 2 targets may additionally have binaries built to run on them as a host (such as `rustc` and `cargo`). This allows the target to be used as a development platform, not just a compilation target.

A proposed new tier 2 target with host tools must be reviewed and approved by the compiler team based on these requirements. Such review and approval may occur via a [Major Change Proposal \(MCP\)](#).

In addition, the infrastructure team must approve the integration of the target's host tools into Continuous Integration (CI), and the CI-related requirements for host tools. This review and approval may take place in a PR adding the target's host tools to CI, or simply by an infrastructure team member reporting the outcome of a team discussion.

- Depending on the target, its capabilities, its performance, and the likelihood of use for any given tool, the host tools provided for a tier 2 target may include only `rustc` and `cargo`, or may include additional tools such as `clippy` and `rustfmt`.
- Approval of host tools will take into account the additional time required to build the

host tools, and the substantial additional storage required for the host tools.

- The host tools must have direct value to people other than the target's maintainers. (It may still be a niche target, but the host tools must not be exclusively useful for an inherently closed group.) This requirement will be evaluated independently from the corresponding tier 2 requirement.
 - The requirement to provide "direct value" means that it does not suffice to argue that having host tools will help the target's maintainers more easily provide the target to others. The tools themselves must provide value to others.
- There must be a reasonable expectation that the host tools will be used, for purposes other than to prove that they can be used.
- The host tools must build and run reliably in CI (for all components that Rust's CI considers mandatory), though they may or may not pass tests.
- Building host tools for the target must not take substantially longer than building host tools for other targets, and should not substantially raise the maintenance burden of the CI infrastructure.
- The host tools must provide a substantively similar experience as on other targets, subject to reasonable target limitations.
 - Adding a substantively different interface to an existing tool, or a target-specific interface to the functionality of an existing tool, requires design and implementation approval (e.g. RFC/MCP) from the appropriate approving teams for that tool.
 - Such an interface should have a design that could potentially work for other targets with similar properties.
 - This should happen separately from the review and approval of the target, to simplify the target review and approval processes, and to simplify the review and approval processes for the proposed new interface.
 - By way of example, a target that runs within a sandbox may need to modify the handling of files, tool invocation, and similar to meet the expectations and conventions of the sandbox, but must not introduce a separate "sandboxed compilation" interface separate from the CLI interface without going through the normal approval process for such an interface. Such an interface should take into account potential other targets with similar sandboxes.
- If the host tools for the platform would normally be expected to be signed or equivalent (e.g. if running unsigned binaries or similar involves a "developer mode" or an additional prompt), it must be possible for the Rust project's automated builds to apply the appropriate signature process, without any manual intervention by either Rust developers, target maintainers, or a third party. This process must meet the

approval of the infrastructure team.

- This process may require one-time or semi-regular manual steps by the infrastructure team, such as registration or renewal of a signing key. Any such manual process must meet the approval of the infrastructure team.
- This process may require the execution of a legal agreement with the signature provider. Such a legal agreement may be revocable, and may potentially require a nominal fee, but must not be otherwise onerous. Any such legal agreement must meet the approval of the infrastructure team. (The infrastructure team is not expected or required to sign binding legal agreements on behalf of the Rust project; this review and approval exists to ensure no terms are onerous or cause problems for infrastructure, especially if such terms may impose requirements or obligations on people who have access to target-specific infrastructure.)
- Changes to this process, or to any legal agreements involved, may cause a target to stop meeting this requirement.
- This process involved must be available under substantially similar non-onerous terms to the general public. Making it available exclusively to the Rust project does not suffice.
- This requirement exists to ensure that Rust builds, including nightly builds, can meet the necessary requirements to allow users to smoothly run the host tools.
- Providing host tools does not exempt a target from requirements to support cross-compilation if at all possible.
- All requirements for tier 2 apply.

A target may be promoted directly from tier 3 to tier 2 with host tools if it meets all the necessary requirements, but doing so may introduce substantial additional complexity. If in doubt, the target should qualify for tier 2 without host tools first.

Tier 1 target policy

At this tier, the Rust project guarantees that a target builds and passes all tests, and will reject patches that fail to build or pass the testsuite on a target. We hold tier 1 targets to our highest standard of requirements.

A proposed new tier 1 target must be reviewed and approved by the compiler team based on these requirements. In addition, the release team must approve the viability and value of

supporting the target. For a tier 1 target, this will typically take place via a full RFC proposing the target, to be jointly reviewed and approved by the compiler team and release team.

In addition, the infrastructure team must approve the integration of the target into Continuous Integration (CI), and the tier 1 CI-related requirements. This review and approval may take place in a PR adding the target to CI, by an infrastructure team member reporting the outcome of a team discussion, or by including the infrastructure team in the RFC proposing the target.

- Tier 1 targets must have substantial, widespread interest within the developer community, and must serve the ongoing needs of multiple production users of Rust across multiple organizations or projects. These requirements are subjective, and determined by consensus of the approving teams. A tier 1 target may be demoted or removed if it becomes obsolete or no longer meets this requirement.
- The target maintainer team must include at least 3 developers.
- The target must build and pass tests reliably in CI, for all components that Rust's CI considers mandatory.
 - The target must not disable an excessive number of tests or pieces of tests in the testsuite in order to do so. This is a subjective requirement.
 - If the target does not have host tools support, or if the target has low performance, the infrastructure team may choose to have CI cross-compile the testsuite from another platform, and then run the compiled tests either natively or via accurate emulation. However, the approving teams may take such performance considerations into account when determining the viability of the target or of its host tools.
- The target must provide as much of the Rust standard library as is feasible and appropriate to provide. For instance, if the target can support dynamic memory allocation, it must provide an implementation of `alloc` and the associated data structures.
- Building the target and running the testsuite for the target must not take substantially longer than other targets, and should not substantially raise the maintenance burden of the CI infrastructure.
 - In particular, if building the target takes a reasonable amount of time, but the target cannot run the testsuite in a timely fashion due to low performance of either native code or accurate emulation, that alone may prevent the target from qualifying as tier 1.
- If running the testsuite requires additional infrastructure (such as physical systems

running the target), the target maintainers must arrange to provide such resources to the Rust project, to the satisfaction and approval of the Rust infrastructure team.

- Such resources may be provided via cloud systems, via emulation, or via physical hardware.
- If the target requires the use of emulation to meet any of the tier requirements, the approving teams for those requirements must have high confidence in the accuracy of the emulation, such that discrepancies between emulation and native operation that affect test results will constitute a high-priority bug in either the emulation or the implementation of the target.
- If it is not possible to run the target via emulation, these resources must additionally be sufficient for the Rust infrastructure team to make them available for access by Rust team members, for the purposes of development and testing. (Note that the responsibility for doing target-specific development to keep the target well maintained remains with the target maintainers. This requirement ensures that it is possible for other Rust developers to test the target, but does not obligate other Rust developers to make target-specific fixes.)
- Resources provided for CI and similar infrastructure must be available for continuous exclusive use by the Rust project. Resources provided for access by Rust team members for development and testing must be available on an exclusive basis when in use, but need not be available on a continuous basis when not in use.
- Tier 1 targets must not have a hard requirement for signed, verified, or otherwise "approved" binaries. Developers must be able to build, run, and test binaries for the target on systems they control, or provide such binaries for others to run. (Doing so may require enabling some appropriate "developer mode" on such systems, but must not require the payment of any additional fee or other consideration, or agreement to any onerous legal agreements.)
 - The Rust project may decide to supply appropriately signed binaries if doing so provides a smoother experience for developers using the target, and a tier 2 target with host tools already requires providing appropriate mechanisms that enable our infrastructure to provide such signed binaries. However, this additional tier 1 requirement ensures that Rust developers can develop and test Rust software for the target (including Rust itself), and that development or testing for the target is not limited.
- All requirements for tier 2 apply.

A tier 1 target may be demoted if it no longer meets these requirements but still meets the

requirements for a lower tier. Any proposal for demotion of a tier 1 target requires a full RFC process, with approval by the compiler and release teams. Any such proposal will be communicated widely to the Rust community, both when initially proposed and before being dropped from a stable release. A tier 1 target is highly unlikely to be directly removed without first being demoted to tier 2 or tier 3. (The amount of time between such communication and the next stable release may depend on the nature and severity of the failed requirement, the timing of its discovery, whether the target has been part of a stable release yet, and whether the demotion or removal can be a planned and scheduled action.)

Raising the baseline expectations of a tier 1 target (such as the minimum CPU features or OS version required) requires the approval of the compiler and release teams, and should be widely communicated as well, but does not necessarily require a full RFC.

Tier 1 with host tools

Some tier 1 targets may additionally have binaries built to run on them as a host (such as `rustc` and `cargo`). This allows the target to be used as a development platform, not just a compilation target.

A proposed new tier 1 target with host tools must be reviewed and approved by the compiler team based on these requirements. In addition, the release team must approve the viability and value of supporting host tools for the target. For a tier 1 target, this will typically take place via a full RFC proposing the target, to be jointly reviewed and approved by the compiler team and release team.

In addition, the infrastructure team must approve the integration of the target's host tools into Continuous Integration (CI), and the CI-related requirements for host tools. This review and approval may take place in a PR adding the target's host tools to CI, by an infrastructure team member reporting the outcome of a team discussion, or by including the infrastructure team in the RFC proposing the target.

- Tier 1 targets with host tools should typically include all of the additional tools such as `clippy` and `rustfmt`, unless there is a target-specific reason why a tool cannot possibly make sense for the target.
 - Unlike with tier 2, for tier 1 we will not exclude specific tools on the sole basis of them being less likely to be used; rather, we'll take that into account when considering whether the target should be at tier 1 with host tools. In general, on

any tier 1 target with host tools, people should be able to expect to find and install all the same components that they would for any other tier 1 target with host tools.

- Approval of host tools will take into account the additional time required to build the host tools, and the substantial additional storage required for the host tools.
- Host tools for the target must have substantial, widespread interest within the developer community, and must serve the ongoing needs of multiple production users of Rust across multiple organizations or projects. These requirements are subjective, and determined by consensus of the approving teams. This requirement will be evaluated independently from the corresponding tier 1 requirement; it is possible for a target to have sufficient interest for cross-compilation, but not have sufficient interest for native compilation. The host tools may be dropped if they no longer meet this requirement, even if the target otherwise qualifies as tier 1.
- The host tools must build, run, and pass tests reliably in CI, for all components that Rust's CI considers mandatory.
 - The target must not disable an excessive number of tests or pieces of tests in the testsuite in order to do so. This is a subjective requirement.
- Building the host tools and running the testsuite for the host tools must not take substantially longer than other targets, and should not substantially raise the maintenance burden of the CI infrastructure.
 - In particular, if building the target's host tools takes a reasonable amount of time, but the target cannot run the testsuite in a timely fashion due to low performance of either native code or accurate emulation, that alone may prevent the target from qualifying as tier 1 with host tools.
- Providing host tools does not exempt a target from requirements to support cross-compilation if at all possible.
- All requirements for tier 2 targets with host tools apply.
- All requirements for tier 1 apply.

A target seeking promotion to tier 1 with host tools should typically either be tier 2 with host tools or tier 1 without host tools, to reduce the number of requirements to simultaneously review and approve.

In addition to the general process for demoting a tier 1 target, a tier 1 target with host tools may be demoted (including having its host tools dropped, or being demoted to tier 2 with host tools) if it no longer meets these requirements but still meets the requirements for a lower tier. Any proposal for demotion of a tier 1 target (with or without host tools) requires a

full RFC process, with approval by the compiler and release teams. Any such proposal will be communicated widely to the Rust community, both when initially proposed and before being dropped from a stable release.

target-name-here

Tier: 3

One-sentence description of the target (e.g. CPU, OS)

Target maintainers

- Some Person, <https://github.com/...>

Requirements

Does the target support host tools, or only cross-compilation? Does the target support `std`, or `alloc` (either with a default allocator, or if the user supplies an allocator)?

Document the expectations of binaries built for the target. Do they assume specific minimum features beyond the baseline of the CPU/environment/etc? What version of the OS or environment do they expect?

Are there notable `#[target_feature(...)]` or `-C target-feature=` values that programs may wish to use?

What calling convention does `extern "C"` use on the target?

What format do binaries use by default? ELF, PE, something else?

Building the target

If Rust doesn't build the target by default, how can users build it? Can users just add it to the `target` list in `config.toml`?

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target. To compile for this target, you will either need to build Rust with the target enabled (see "Building the target" above), or build your own copy of `core` by using `build-std` or similar.

Testing

Does the target support running binaries, or do binaries have varying expectations that prevent having a standard way to run them? If users can run binaries, can they do so in some common emulator, or do they need native hardware? Does the target support running the Rust testsuite?

Cross-compilation toolchains and C code

Does the target support C code? If so, what toolchain target should users use to build compatible C code? (This may match the target triple, or it may be a toolchain for a different target triple, potentially with specific options or caveats.)

arm64ec-pc-windows-msvc

Tier: 2

Arm64EC ("Emulation Compatible") for mixed architecture (AArch64 and x86_64) applications on AArch64 Windows 11. See <https://learn.microsoft.com/en-us/windows/arm/arm64ec>.

Target maintainers

- [@dpaoliello](#)

Requirements

Builds Arm64EC static and dynamic libraries and executables which can be run on AArch64 Windows 11 devices. Arm64EC static libraries can also be linked into Arm64X dynamic libraries and executables.

Only supported backend is LLVM 18 or above:

- 18.1.0 added initial support for Arm64EC.
- 18.1.2 fixed import library generation (required for `raw-dylib` support).
- 18.1.4 fixed linking issue for some intrinsics implemented in `compiler_builtins`.

Visual Studio 2022 (or above) with the "ARM64/ARM64EC built tools" component and the Windows 11 SDK are required.

Reusing code from other architectures - x86_64 or AArch64?

Arm64EC uses `arm64ec` as its `target_arch`, but it is possible to reuse existing architecture-specific code in most cases. The best mental model for deciding which architecture to reuse is to think of Arm64EC as an x86_64 process that happens to use the AArch64

instruction set (with some caveats) and has a completely custom ABI.

To put this in practice:

- Arm64EC interacts with the operating system, other processes and other DLLs as x86_64.
 - For example, in `backtrace` we use the x86_64 versions of `CONTEXT` and `RtlVirtualUnwind`.
 - If you are configuring a search path to find DLLs (e.g., to load plugins or addons into your application), you should use the same path as the x86_64 version of your application, not the AArch64 path (since Arm64EC (i.e., x86_64) processes cannot load native AArch64 DLLs).
- Arm64EC uses AArch64 intrinsics.
 - For example, <https://github.com/rust-lang/portable-simd/commit/ca4033f49b1f6019561b8b161b4097b4a07f2e1b> and <https://github.com/rust-lang/stdarch/commit/166ef7ba22b6a1d908d4b29a36e68ceca324808a>.
- Assembly for AArch64 might be reusable for Arm64EC, but there are many caveats. For full details see [Microsoft's documentation on the Arm64EC ABI](#) but in brief:
 - Arm64EC uses a subset of AArch64 registers.
 - Arm64EC uses a different name mangling scheme than AArch64.
 - Arm64EC requires entry and exit thunks be generated for some functions.
 - Indirect calls must be done via a call checker.
 - Control Flow Guard and stack checks use different functions than AArch64.

Building the target

You can build Rust with support for the targets by adding it to the `target` list in `config.toml`:

```
[build]
target = [ "arm64ec-pc-windows-msvc" ]
```

Building Rust programs

These targets are distributed through `rustup`, and otherwise require no special configuration.

Testing

Tests can be run on AArch64 Windows 11 devices.

Cross-compilation toolchains and C code

C code can be built using the Arm64-targeting MSVC or Clang toolchain.

To compile:

```
cl /arm64EC /c ...
```

To link:

```
link /MACHINE:ARM64EC ...
```

Further reading: <https://learn.microsoft.com/en-us/windows/arm/arm64ec-build>

*-apple-darwin

Apple macOS targets.

Tier: 1

- `x86_64-apple-darwin` : macOS on 64-bit x86.

Tier: 2 (with Host Tools)

- `aarch64-apple-darwin` : macOS on ARM64 (M1-family or later Apple Silicon CPUs).

Target maintainers

- [@thomcc](#)
- [@madsmtm](#)

Requirements

OS version

The minimum supported version is macOS 10.12 Sierra on x86, and macOS 11.0 Big Sur on ARM64.

This version can be raised per-binary by changing the [deployment target](#), which might yield more performance optimizations. `rustc` respects the common environment variables used by Xcode to do so, in this case `MACOSX_DEPLOYMENT_TARGET`.

The current default deployment target for `rustc` can be retrieved with `rustc --print=deployment-target`.

Binary format

The default binary format is Mach-O, the executable format used on Apple's platforms.

Building

These targets are distributed through `rustup`, and otherwise require no special configuration.

Testing

There are no special requirements for testing and running this target.

x86 binaries can be run on Apple Silicon by using Rosetta.

Cross-compilation toolchains and C code

Cross-compilation of these targets are supported using Clang, but may require Xcode or the macOS SDK (`MacOSX.sdk`) to be available to compile C code and to link.

The path to the SDK can be passed to `rustc` using the common `SDKROOT` environment variable.

i686-apple-darwin

Apple macOS on 32-bit x86.

Target maintainers

- [@thomcc](#)
- [@madsmtm](#)

Requirements

See the docs on [*-apple-darwin](#) for general macOS requirements.

Building the target

You'll need the macOS 10.13 SDK shipped with Xcode 9. The location of the SDK can be passed to `rustc` using the common `SDKROOT` environment variable.

Once you have that, you can build Rust with support for the target by adding it to the target list in `config.toml`:

```
[build]
target = ["i686-apple-darwin"]
```

Using the unstable `-Zbuild-std` with a nightly Cargo may also work.

Building Rust programs

Rust [no longer](#) ships pre-compiled artifacts for this target. To compile for this target, you will

either need to build Rust with the target enabled (see "Building the target" above), or build your own copy using `build-std` or similar.

Testing

Running this target requires an Intel Macbook running macOS 10.14 or earlier, as later versions removed support for running 32-bit binaries.

x86_64h-apple-darwin

Tier: 3

Target for macOS on late-generation `x86_64` Apple chips, usable as the `x86_64h` entry in universal binaries, and equivalent to LLVM's `x86_64h-apple-macosx*` targets.

Target maintainers

- Thom Chiovoloni `thom@shift.click` <https://github.com/thomcc>

Requirements

This target is an `x86_64` target that only supports Apple's late-gen (Haswell-compatible) Intel chips. It enables a set of target features available on these chips (AVX2 and similar), and MachO binaries built with this target may be used as the `x86_64h` entry in universal binaries ("fat" MachO binaries), and will fail to load on machines that do not support this.

It should support the full standard library (`std` and `alloc` either with default or user-defined allocators). This target is probably most useful when targeted via cross-compilation (including from `x86_64-apple-darwin`), but if built manually, the host tools work.

It is similar to `x86_64-apple-darwin` in nearly all respects.

Building the target

Users on Apple targets can build this by adding it to the `target` list in `config.toml`, or with `-Zbuild-std`.

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target. To compile for this target, you will either need to build Rust with the target enabled (see "Building the target" above), or build your own copy of `core` by using `build-std` or similar.

Testing

Code built with this target can be run on the set of Intel macOS machines that support running `x86_64h` binaries (relatively recent Intel macs). The Rust test suite seems to work.

Cross-compilation toolchains and C code

Cross-compilation to this target from Apple hosts should generally work without much configuration, so long as XCode and the CommandLineTools are installed. Targeting it from non-Apple hosts is difficult, but no more so than targeting `x86_64-apple-darwin`.

When compiling C code for this target, either the "`x86_64h-apple-macosx*`" LLVM targets should be used, or an argument like `-arch x86_64h` should be passed to the C compiler.

arm64e-apple-darwin

Tier: 3 (with Host Tools)

ARM64e macOS (11.0+, Big Sur+)

Target maintainers

- Artyom Tetyukhin (@arttet)

Requirements

Target for macOS on late-generation M series Apple chips.

See the docs on [*-apple-darwin](#) for general macOS requirements.

Building the target

You can build Rust with support for the targets by adding it to the `target` list in `config.toml`:

```
[build]
target = [ "arm64e-apple-darwin" ]
```

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target. To compile for this target, you will need to build Rust with the target enabled (see [Building the target](#) above).

Testing

The target does support running binaries on macOS platforms with `arm64e` architecture.

Cross-compilation toolchains and C code

The targets do support `c` code. To build compatible `c` code, you have to use XCode with the same compiler and flags.

*-apple-ios

Apple iOS / iPadOS targets.

Tier: 2 (without Host Tools)

- `aarch64-apple-ios` : Apple iOS on ARM64.
- `aarch64-apple-ios-sim` : Apple iOS Simulator on ARM64.
- `x86_64-apple-ios` : Apple iOS Simulator on 64-bit x86.

Tier: 3

- `armv7s-apple-ios` : Apple iOS on Armv7-A.
- `i386-apple-ios` : Apple iOS Simulator on 32-bit x86.

Target maintainers

- [@badboy](#)
- [@deg4uss3r](#)
- [@madsmtm](#)

Requirements

These targets are cross-compiled, and require the corresponding iOS SDK (`iPhoneOS.sdk` or `iPhoneSimulator.sdk`), as provided by Xcode. To build the ARM64 targets, Xcode 12 or higher is required.

The path to the SDK can be passed to `rustc` using the common `SDKROOT` environment variable.

OS version

The minimum supported version is iOS 10.0.

This can be raised per-binary by changing the deployment target. `rustc` respects the common environment variables used by Xcode to do so, in this case

```
IPHONEOS_DEPLOYMENT_TARGET .
```

Building the target

The tier 2 targets are distributed through `rustup`, and can be installed using one of:

```
$ rustup target add aarch64-apple-ios
$ rustup target add aarch64-apple-ios-sim
$ rustup target add x86_64-apple-ios
```

The tier 3 targets can be built by enabling them for a `rustc` build in `config.toml`, by adding, for example:

```
[build]
target = ["armv7s-apple-ios", "i386-apple-ios"]
```

Using the unstable `-Zbuild-std` with a nightly Cargo may also work.

Building Rust programs

Rust programs can be built for these targets by specifying `--target`, if `rustc` has been built with support for them. For example:

```
$ rustc --target aarch64-apple-ios your-code.rs
```

Testing

There is no support for running the Rust or standard library testsuite at the moment. Testing has mostly been done manually with builds of static libraries embedded into applications called from Xcode or a simulator.

It hopefully will be possible to improve this in the future.

*-apple-ios-macabi

Apple Mac Catalyst targets.

Tier: 3

- `aarch64-apple-ios-macabi` : Mac Catalyst on ARM64.
- `x86_64-apple-ios-macabi` : Mac Catalyst on 64-bit x86.

Target maintainers

- [@badboy](#)
- [@BlackHoleFox](#)
- [@madsmtm](#)

Requirements

These targets are cross-compiled, and require the corresponding macOS SDK (`MacOSX.sdk`) which contain `./System/iOSSupport` headers to allow linking to iOS-specific headers, as provided by Xcode 11 or higher.

The path to the SDK can be passed to `rustc` using the common `SDKROOT` environment variable.

OS version

The minimum supported version is iOS 13.1.

This can be raised per-binary by changing the deployment target. `rustc` respects the common environment variables used by Xcode to do so, in this case `IPHONEOS_DEPLOYMENT_TARGET` .

Building the target

The targets can be built by enabling them for a `rustc` build in `config.toml`, by adding, for example:

```
[build]
target = ["aarch64-apple-ios-macabi", "x86_64-apple-ios-macabi"]
```

Using the unstable `-Zbuild-std` with a nightly Cargo may also work.

Building Rust programs

Rust programs can be built for these targets by specifying `--target`, if `rustc` has been built with support for them. For example:

```
$ rustc --target aarch64-apple-ios-macabi your-code.rs
```

Testing

Mac Catalyst binaries can be run directly on macOS 10.15 Catalina or newer.

x86 binaries can be run on Apple Silicon by using Rosetta.

Note that using certain UIKit functionality requires the binary to be bundled.

arm64e-apple-ios

Tier: 3

ARM64e iOS (12.0+)

Target maintainers

- Artyom Tetyukhin ([@arttet](#))

Requirements

See the docs on [*-apple-ios](#) for general iOS requirements.

Building the target

You can build Rust with support for the targets by adding it to the `target` list in `config.toml`:

```
[build]
target = [ "arm64e-apple-ios" ]
```

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target. To compile for this target, you will need to build Rust with the target enabled (see [Building the target](#) above).

Testing

The target does support running binaries on iOS platforms with `arm64e` architecture.

Cross-compilation toolchains and C code

The targets do support `c` code. To build compatible `c` code, you have to use XCode with the same compiler and flags.

*-apple-tvos

Apple tvOS targets.

Tier: 3

- `aarch64-apple-tvos` : Apple tvOS on ARM64.
- `aarch64-apple-tvos-sim` : Apple tvOS Simulator on ARM64.
- `x86_64-apple-tvos` : Apple tvOS Simulator on x86_64.

Target maintainers

- [@thomcc](#)
- [@madsmtm](#)

Requirements

These targets are cross-compiled, and require the corresponding tvOS SDK (`AppleTVOS.sdk` or `AppleTVSimulator.sdk`), as provided by Xcode. To build the ARM64 targets, Xcode 12 or higher is required.

The path to the SDK can be passed to `rustc` using the common `SDKROOT` environment variable.

OS version

The minimum supported version is tvOS 10.0, although the actual minimum version you can target may be newer than this, for example due to the versions of Xcode and your SDKs.

The version can be raised per-binary by changing the deployment target. `rustc` respects the common environment variables used by Xcode to do so, in this case

`TVOS_DEPLOYMENT_TARGET` .

Incompletely supported library functionality

The targets support most of the standard library including the allocator to the best of my knowledge, however they are very new, not yet well-tested, and it is possible that there are various bugs.

The following APIs are currently known to have missing or incomplete support:

- `std::process::Command`'s API will return an error if it is configured in a manner which cannot be performed using `posix_spawn` -- this is because the more flexible `fork / exec`-based approach is prohibited on these platforms in favor of `posix_spawn{,p}` (which still probably will get you rejected from app stores, so is likely sideloading-only). A concrete set of cases where this will occur is difficult to enumerate (and would quickly become stale), but in some cases it may be worked around by tweaking the manner in which `Command` is invoked.

Building the target

The targets can be built by enabling them for a `rustc` build in `config.toml`, by adding, for example:

```
[build]
build-stage = 1
target = ["aarch64-apple-tvos", "aarch64-apple-tvos-sim"]
```

Using the unstable `-Zbuild-std` with a nightly Cargo may also work.

Building Rust programs

Rust programs can be built for these targets by specifying `--target`, if `rustc` has been built with support for them. For example:

```
$ rustc --target aarch64-apple-tvos your-code.rs
```

Testing

There is no support for running the Rust or standard library testsuite at the moment. Testing has mostly been done manually with builds of static libraries embedded into applications called from Xcode or a simulator.

It hopefully will be possible to improve this in the future.

*-apple-watchos

Apple watchOS targets.

Tier: 3

- `aarch64-apple-watchos` : Apple WatchOS on ARM64.
- `aarch64-apple-watchos-sim` : Apple WatchOS Simulator on ARM64.
- `x86_64-apple-watchos-sim` : Apple WatchOS Simulator on 64-bit x86.
- `arm64_32-apple-watchos` : Apple WatchOS on Arm 64_32.
- `armv7k-apple-watchos` : Apple WatchOS on Armv7k.

Target maintainers

- [@deg4uss3r](#)
- [@vladimir-ea](#)
- [@leohowell](#)
- [@madsmtm](#)

Requirements

These targets are cross-compiled, and require the corresponding watchOS SDK (`WatchOS.sdk` or `WatchSimulator.sdk`), as provided by Xcode. To build the ARM64 targets, Xcode 12 or higher is required.

The path to the SDK can be passed to `rustc` using the common `SDKROOT` environment variable.

OS version

The minimum supported version is watchOS 5.0.

This can be raised per-binary by changing the deployment target. `rustc` respects the common environment variables used by Xcode to do so, in this case `WATCHOS_DEPLOYMENT_TARGET`.

Building the target

The targets can be built by enabling them for a `rustc` build in `config.toml`, by adding, for example:

```
[build]
build-stage = 1
target = ["aarch64-apple-watchos", "aarch64-apple-watchos-sim"]
```

Using the unstable `-Zbuild-std` with a nightly Cargo may also work.

Building Rust programs

Rust programs can be built for these targets by specifying `--target`, if `rustc` has been built with support for them. For example:

```
$ rustc --target aarch64-apple-watchos-sim your-code.rs
```

Testing

There is no support for running the Rust or standard library testsuite at the moment. Testing has mostly been done manually with builds of static libraries embedded into applications called from Xcode or a simulator.

It hopefully will be possible to improve this in the future.

*-apple-visionos

Apple visionOS / xrOS targets.

Tier: 3

- `aarch64-apple-visionos` : Apple visionOS on arm64.
- `aarch64-apple-visionos-sim` : Apple visionOS Simulator on arm64.

Target maintainers

- [@agg23](#)
- [@madsmtm](#)

Requirements

These targets are cross-compiled, and require the corresponding visionOS SDK (`xrOS.sdk` or `xrSimulator.sdk`), as provided by Xcode 15 or newer.

The path to the SDK can be passed to `rustc` using the common `SDKROOT` environment variable.

OS version

The minimum supported version is visionOS 1.0.

This can be raised per-binary by changing the deployment target. `rustc` respects the common environment variables used by Xcode to do so, in this case `XR0S_DEPLOYMENT_TARGET` .

Building the target

The targets can be built by enabling them for a `rustc` build in `config.toml`, by adding, for example:

```
[build]
target = ["aarch64-apple-visionos", "aarch64-apple-visionos-sim"]
```

Using the unstable `-Zbuild-std` with a nightly Cargo may also work.

Note: Currently, a newer version of `libc` and `cc` may be required, this will be fixed in [#124560](#).

Building Rust programs

Rust programs can be built for these targets by specifying `--target`, if `rustc` has been built with support for them. For example:

```
$ rustc --target aarch64-apple-visionos-sim your-code.rs
```

Testing

There is no support for running the Rust or standard library testsuite at the moment. Testing has mostly been done manually with builds of static libraries embedded into applications called from Xcode or a simulator.

It hopefully will be possible to improve this in the future.

Cross-compilation toolchains and C code

The Clang target is suffixed with `-xros` for historical reasons.

LLVM 18 or newer is required to build this target.

aarch64-nintendo-switch-freestanding

Tier: 3

Nintendo Switch with pure-Rust toolchain.

Designated Developers

- @leo60228
- @jam1garner

Requirements

This target is cross-compiled. It has no special requirements for the host.

Building

The target can be built by enabling it for a `rustc` build:

```
[build]
build-stage = 1
target = ["aarch64-nintendo-switch-freestanding"]
```

Cross-compilation

This target can be cross-compiled from any host.

Testing

Currently there is no support to run the rustc test suite for this target.

Building Rust programs

If `rustc` has support for that target and the library artifacts are available, then Rust programs can be built for that target:

```
rustc --target aarch64-nintendo-switch-freestanding your-code.rs
```

To generate binaries in the NRO format that can be easily run on-device, you can use [cargo-nx](#):

```
cargo nx --triple=aarch64-nintendo-switch-freestanding
```

armeb-unknown-linux-gnueabi

Tier: 3

Target for cross-compiling Linux user-mode applications targeting the Arm BE8 architecture.

Overview

BE8 architecture retains the same little-endian ordered code-stream used by conventional little endian Arm systems, however the data accesses are in big-endian. BE8 is used primarily in high-performance networking applications where the ability to read packets in their native "Network Byte Order" is important (many network protocols transmit data in big-endian byte order for their wire formats).

History

BE8 architecture is the default big-endian architecture for Arm since [Armv6](#). It's predecessor, used for Armv4 and Armv5 devices was [BE32](#). On Armv6 architecture, endianness can be configured via [system registers](#). However, BE32 was withdrawn for [Armv7](#) onwards.

Target Maintainers

- [@WorksButNotTested](#)

Requirements

The target is cross-compiled. This target supports `std` in the normal way (indeed only nominal changes are required from the standard Arm configuration).

Target definition

The target definition can be seen [here](#). In particular, it should be noted that the `features` specify that this target is built for the Armv8 core. Though this can likely be modified as required.

Building the target

Because it is Tier 3, rust does not yet ship pre-compiled artifacts for this target.

Therefore, you can build Rust with support for the target by adding it to the target list in `config.toml`, a sample configuration is shown below. It is expected that the user already have a working GNU compiler toolchain and update the paths accordingly.

```

[llvm]
download-ci-llvm = false
optimize = true
ninja = true
targets = "ARM;X86"
clang = false

[build]
target = ["x86_64-unknown-linux-gnu", "armeb-unknown-linux-gnueabi"]
docs = false
docs-minification = false
compiler-docs = false
[install]
prefix = "/home/user/x-tools/rust/"

[rust]
debug-logging=true
backtrace = true
incremental = true

[target.x86_64-unknown-linux-gnu]

[dist]

[target.armeb-unknown-linux-gnueabi]
cc = "/home/user/x-tools/armeb-unknown-linux-gnueabi/bin/armeb-unknown-linux-gnueabi-gcc"
cxx = "/home/user/x-tools/armeb-unknown-linux-gnueabi/bin/armeb-unknown-linux-gnueabi-g++"
ar = "/home/user/x-tools/armeb-unknown-linux-gnueabi/bin/armeb-unknown-linux-gnueabi-ar"
ranlib = "/home/user/x-tools/armeb-unknown-linux-gnueabi/bin/armeb-unknown-linux-gnueabi-ranlib"
linker = "/home/user/x-tools/armeb-unknown-linux-gnueabi/bin/armeb-unknown-linux-gnueabi-gcc"
llvm-config = "/home/user/x-tools/clang/bin/llvm-config"
llvm-filecheck = "/home/user/x-tools/clang/bin/FileCheck"

```

Building Rust programs

The following `.cargo/config` is needed inside any project directory to build for the BE8 target:

```
[build]
target = "armeb-unknown-linux-gnueabi"

[target.armeb-unknown-linux-gnueabi]
linker = "armeb-unknown-linux-gnueabi-gcc"
```

Note that it is expected that the user has a suitable linker from the GNU toolchain.

{arm, thumb}*–none–eabi(hf)?

Common Target Details

This documentation covers details that apply to a range of bare-metal targets for 32-bit Arm CPUs. The `arm-none-eabi` flavor of the GNU compiler toolchain is often used to assist compilation to these targets.

Details that apply only to only a specific target in this group are covered in their own document.

Tier 2 Target List

- Arm A-Profile Architectures
 - `armv7a-none-eabi`
- Arm R-Profile Architectures
 - `armv7r-none-eabi` and `armv7r-none-eabihf`
 - `armebv7r-none-eabi` and `armebv7r-none-eabihf`
- Arm M-Profile Architectures
 - `thumbv6m-none-eabi`
 - `thumbv7m-none-eabi`
 - `thumbv7em-none-eabi` and `thumbv7em-none-eabihf`
 - `thumbv8m.base-none-eabi`
 - `thumbv8m.main-none-eabi` and `thumbv8m.main-none-eabihf`
- *Legacy* Arm Architectures
 - None

Tier 3 Target List

- Arm A-Profile Architectures
 - `armv7a-none-eabihf`
- Arm R-Profile Architectures

- `armv8r-none-eabihf`
- Arm M-Profile Architectures
 - None
- Legacy Arm Architectures
 - `armv4t-none-eabi` and `thumbv4t-none-eabi`
 - `armv5te-none-eabi` and `thumbv5te-none-eabi`

Instruction Sets

There are two 32-bit instruction set architectures (ISAs) defined by Arm:

- The [A32 ISA](#), with fixed-width 32-bit instructions. Previously known as the *Arm* ISA, this originated with the original Arm1 of 1985 and has been updated by various revisions to the architecture specifications ever since.
- The [T32 ISA](#), with a mix of 16-bit and 32-bit width instructions. Note that this term includes both the original 16-bit width *Thumb* ISA introduced with the Armv4T architecture in 1994, and the later 16/32-bit sized *Thumb-2* ISA introduced with the Armv6T2 architecture in 2003.

Again, these ISAs have been revised by subsequent revisions to the relevant Arm architecture specifications.

There is also a 64-bit ISA with fixed-width 32-bit instructions called the *A64 ISA*, but targets which implement that instruction set generally start with `aarch64*` and are discussed elsewhere.

Rust targets starting with `arm*` generate Arm (A32) code by default, whilst targets named `thumb*` generate Thumb (T32) code by default. Most Arm chips support both Thumb mode and Arm mode, with the notable exception that M-profile processors (`thumbv*m*-none-eabi*` targets) *only* support Thumb-mode.

Rust targets ending with `eabi` use the so-called *soft-float ABI*: functions which take `f32` or `f64` as arguments will have those values packed into integer registers. This means that an FPU is not required from an ABI perspective, but within a function floating-point instructions may still be used if the code is compiled with a `target-cpu` or `target-feature` option that enables FPU support.

Rust targets ending in `eabihf` use the so-called *hard-float ABI*: functions which take `f32` or `f64` as arguments will have them passed via FPU registers. These targets therefore require the availability of an FPU and will assume some baseline level of floating-point support is available (which can vary depending on the target). More advanced floating-point instructions may be generated if the code is compiled with a `target-cpu` or `target-feature` option that enables such additional FPU support. For example, if a given hard-float target has baseline *single-precision* (`f32`) support in hardware, there may be `target-cpu` or `target-feature` options that tell LLVM to assume your processor in fact also has *double-precision* (`f64`) support.

You may of course use the `f32` and `f64` types in your code, regardless of the ABI being used, or the level of support your processor has for performing such operations in hardware. Any floating-point operations that LLVM assumes your processor cannot support will be lowered to library calls (like `__aeabi_dadd`) which perform the floating-point operation in software using integer instructions.

Target CPU and Target Feature options

It is possible to tell Rust (or LLVM) that you have a specific model of Arm processor, using the `-C target-cpu` option. You can also control whether Rust (or LLVM) will include instructions that target optional hardware features, e.g. hardware floating-point, or Advanced SIMD operations, using `-C target-feature` .

It is important to note that selecting a *target-cpu* will typically enable *all* the optional features available from Arm on that model of CPU and your particular implementation of that CPU may not have those features available. In that case, you can use `-C target-feature=-option` to turn off the specific CPU features you do not have available, leaving you with the optimized instruction scheduling and support for the features you do have. More details are available in the detailed target-specific documentation.

Note: Many target-features are currently unstable and subject to change, and if you use them you should disassemble the compiler output and manually inspect it to ensure only appropriate instructions for your CPU have been generated.

If you wish to use the *target-cpu* and *target-feature* options, you can add them to your

`.cargo/config.toml` file alongside any other flags your project uses (likely linker related ones):

```
rustflags = [
  # Usual Arm bare-metal linker setup
  "-Clink-arg=-Tlink.x",
  "-Clink-arg=--nmagic",
  # tell Rust we have a Cortex-M55
  "-Ctarget-cpu=cortex-m55",
  # tell Rust our Cortex-M55 doesn't have Floating-Point M-Profile Vector
  # Extensions (but it does have everything else a Cortex-M55 could have).
  "-Ctarget-feature=-mve.fp"
]

[build]
target = "thumbv8m.main-none-eabi-hf"
```

Requirements

These targets are cross-compiled and use static linking.

By default, the `ld` linker included with Rust will be used; however, you may want to use the GNU linker instead. This can be obtained for Windows/Mac/Linux from the [Arm Developer Website](#), or possibly from your OS's package manager. To use it, add the following to your `.cargo/config.toml`:

```
[target.<your-target>]
linker = "arm-none-eabi-ld"
```

The GNU linker can also be used by specifying `arm-none-eabi-gcc` as the linker. This is needed when using GCC's link time optimization.

These targets don't provide a linker script, so you'll need to bring your own according to the specific device you are using. Pass `-Clink-arg=-Tyour_script.ld` as a rustc argument to make the linker use `your_script.ld` during linking.

For the `arm*` targets, Thumb-mode code generation can be enabled by using `-C target-feature=+thumb-mode`. Using the unstable `#![feature(arm_target_feature)]`, the attribute

`#[target_feature(enable = "thumb-mode")]` can be applied to individual `unsafe` functions to cause those functions to be compiled to Thumb-mode code.

Building Rust Programs

For the Tier 3 targets in this family, rust does not ship pre-compiled artifacts.

Just use the `build-std` nightly cargo feature to build the `core` library. You can pass this as a command line argument to cargo, or your `.cargo/config.toml` file might include the following lines:

```
[unstable]
build-std = ["core"]
```

Most of `core` should work as expected, with the following notes:

- Floating-point operations are emulated in software unless LLVM is told to enable FPU support (either by using an `eabihf` target, specifying a `target-cpu` with FPU support, or using a `target-feature` to support for a specific kind of FPU)
- Integer division is also emulated in software on some targets, depending on the target, `target-cpu` and `target-feature` S.
- Older Arm architectures (e.g. Armv4, Armv5TE and Armv6-M) are limited to basic `load` and `store` operations, and not more advanced operations like `fetch_add` or `compare_exchange`.

`alloc` is also supported, as long as you provide your own global allocator.

Rust programs are output as ELF files.

Testing

This is a cross-compiled target that you will need to emulate during testing.

The exact emulator that you'll need depends on the specific device you want to run your

code on.

Cross-compilation toolchains and C code

The target supports C code compiled with the `arm-none-eabi` target triple and a suitable `-march` or `-mcpu` flag.

`gcc` or `clang` can be used, but note that `gcc` uses `-fshort-enums` by default for `arm-none*` targets, while `clang` does not. `rustc` matches the `gcc` behavior, i.e., the size of a `# [repr(C)] enum` in Rust can be as little as 1 byte, rather than 4, as they are on `arm-linux` targets.

armv4t-none-eabi / thumbv4t-none-eabi

Tier 3

These two targets are part of the [arm-none-eabi](#) target group, and all the information there applies.

Both of these targets can be used on the Game Boy Advance (GBA), among other things. On the GBA, one should usually use the `thumb` target to get the best overall performance.

Target Maintainers

- [@Lokathor](#)
- [@corwinkuiper](#)

Testing

This is a cross-compiled target that you will need to emulate during testing.

Because this is a device-agnostic target, and the exact emulator that you'll need depends on the specific device you want to run your code on.

- When building for the GBA, [mgba-test-runner](#) can be used to make a normal set of rust tests be run within the `mgba` emulator.

armv5te-none-eabi

Tier: 3

Bare-metal target for any cpu in the Armv5TE architecture family, supporting ARM/Thumb code interworking (aka A32 / T32), with A32 code as the default code generation.

The `thumbv5te-none-eabi` target is the same as this one, but the instruction set defaults to T32 .

See [arm-none-eabi](#) for information applicable to all `arm-none-eabi` targets.

Target Maintainers

- [@QuinnPainter](#)

Testing

This is a cross-compiled target that you will need to emulate during testing.

Because this is a device-agnostic target, and the exact emulator that you'll need depends on the specific device you want to run your code on.

For example, when programming for the DS, you can use one of the several available DS emulators, such as [melonDS](#).

arm(eb)?v7r-none-eabi(hf)?

Tier: 2

Bare-metal target for CPUs in the Armv7-R architecture family, supporting dual ARM/Thumb mode, with ARM mode as the default.

Processors in this family include the [Arm Cortex-R4, 5, 7, and 8](#).

The `eb` versions of this target generate code for big-endian processors.

See [arm-none-eabi](#) for information applicable to all `arm-none-eabi` targets.

Target maintainers

- [Chris Copeland](#), chris@chrisnc.net

Requirements

When using the big-endian version of this target, note that some variants of the Cortex-R have both big-endian instructions and data. This configuration is known as BE-32, while data-only big-endianness is known as BE-8. To build programs for BE-32 processors, the GNU linker must be used with the `-mbe32` option. See [ARM Cortex-R Series Programmer's Guide: Endianness](#) for more details about different endian modes.

When using the hardfloat targets, the minimum floating-point features assumed are those of the `vfpv3-d16`, which includes single- and double-precision, with 16 double-precision registers. This floating-point unit appears in Cortex-R4F and Cortex-R5F processors. See [VFP in the Cortex-R processors](#) for more details on the possible FPU variants.

If your processor supports a different set of floating-point features than the default expectations of `vfpv3-d16`, then these should also be enabled or disabled as needed with `-C target-feature=(+/-)`.

Cross-compilation toolchains and C code

This target supports C code compiled with the `arm-none-eabi` target triple and `-march=armv7-r` or a suitable `-mcpu` flag.

armv8r-none-eabihf

Tier: 3

Bare-metal target for CPUs in the Armv8-R architecture family, supporting dual ARM/Thumb mode, with ARM mode as the default.

Processors in this family include the Arm [Cortex-R52](#) and [Cortex-R52+](#).

See [arm-none-eabi](#) for information applicable to all `arm-none-eabi` targets.

Target maintainers

- [Chris Copeland](#), chris@chrisnc.net

Requirements

The Cortex-R52 family always includes a floating-point unit, so there is no non-`hf` version of this target. The floating-point features assumed by this target are those of the single-precision-only config of the Cortex-R52, which has 16 double-precision registers, accessible as 32 single-precision registers. The other variant of Cortex-R52 includes double-precision, 32 double-precision registers, and Advanced SIMD (Neon).

The manual refers to this as the "Full Advanced SIMD config". To compile code for this variant, use: `-C target-feature=+fp64,+d32,+neon`. See the [Advanced SIMD and floating-point support](#) section of the Cortex-R52 Processor Technical Reference Manual for more details.

Cross-compilation toolchains and C code

This target supports C code compiled with the `arm-none-eabi` target triple and `-`

`march=armv8-r` or a suitable `-mcpu` flag.

thumbv6m-none-eabi

Tier: 2

Bare-metal target for CPUs in the [Armv6-M](#) architecture family, supporting a subset of the T32 ISA.

Processors in this family include the:

- [Arm Cortex-M0](#)
- [Arm Cortex-M0+](#)
- [Arm Cortex-M1](#)

See [arm-none-eabi](#) for information applicable to all `arm-none-eabi` targets.

This target uses the soft-float ABI: functions which take `f32` or `f64` as arguments will have those values packed into integer registers. This is the only option because there is no FPU support in [Armv6-M](#).

Target maintainers

- [Rust Embedded Devices Working Group Cortex-M Team](#), `cortex-m@teams.rust-embedded.org`

Target CPU and Target Feature options

See [the bare-metal Arm docs](#) for details on how to use these flags.

Table of supported CPUs

CPU	FPU	Target CPU	Target Features
Cortex-M0	No	<code>cortex-m0</code>	None

Cortex-M0+	No	cortex-m0plus	None
Cortex-M1	No	cortex-m1	None

Arm Cortex-M0

The target CPU option is `cortex-m0` .

There are no relevant feature flags, and the FPU is not available.

Arm Cortex-M0+

The target CPU option is `cortex-m0plus` .

There are no relevant feature flags, and the FPU is not available.

Arm Cortex-M1

The target CPU option is `cortex-m1` .

There are no relevant feature flags, and the FPU is not available.

thumbv7em-none-eabi and thumbv7em-none-eabihf

Tier: 2

Bare-metal target for CPUs in the [Armv7E-M](#) architecture family, supporting a subset of the T32 ISA.

Processors in this family include the:

- [Arm Cortex-M4](#) and Arm Cortex-M4F
- [Arm Cortex-M7](#) and Arm Cortex-M7F

See [arm-none-eabi](#) for information applicable to all `arm-none-eabi` targets, in particular the difference between the `eabi` and `eabihf` ABI.

Target maintainers

- [Rust Embedded Devices Working Group Cortex-M Team](#), `cortex-m@teams.rust-embedded.org`

Target CPU and Target Feature options

See [the bare-metal Arm docs](#) for details on how to use these flags.

Table of supported CPUs for **thumbv7em-none-eabi**

CPU	FPU	DSP	Target CPU	Target Features
Any	No	Yes	None	None
Cortex-M4	No	Yes	cortex-m4	+soft-float

Cortex-M4F	SP	Yes	cortex-m4	None
Cortex-M7	No	Yes	cortex-m7	+soft-float
Cortex-M7F	SP	Yes	cortex-m7	-fp64
Cortex-M7F	DP	Yes	cortex-m7	None

Table of supported CPUs for thumbv7em-none-eabihf

CPU	FPU	DSP	Target CPU	Target Features
Any	SP	Yes	None	None
Cortex-M4F	SP	Yes	cortex-m4	None
Cortex-M7F	SP	Yes	cortex-m7	-fp64
Cortex-M7F	DP	Yes	cortex-m7	None

Arm Cortex-M4 and Arm Cortex-M4F

The target CPU is cortex-m4 .

- All Cortex-M4 have DSP extensions
 - support is controlled by the dsp target-feature
 - enabled by default with this target
- Cortex-M4F has a single precision FPU
 - support is enabled by default with this target-cpu
 - disable support using the +soft-float feature (eabi only)

Arm Cortex-M7 and Arm Cortex-M7F

The target CPU is cortex-m7 .

- All Cortex-M7 have DSP extensions
 - support is controlled by the dsp target-feature
 - enabled by default with this target
- Cortex-M7F have either a single-precision or double-precision FPU

- double-precision support is enabled by default with this *target-cpu*
 - opt-out by using the `-f64` *target-feature*
- disable support entirely using the `+soft-float` feature (*eabi* only)

thumbv7m-none-eabi

Tier: 2

Bare-metal target for CPUs in the [Armv7-M](#) architecture family, supporting a subset of the T32 ISA.

Processors in this family include the:

- [Arm Cortex-M3](#)

See [arm-none-eabi](#) for information applicable to all `arm-none-eabi` targets.

This target uses the soft-float ABI: functions which take `f32` or `f64` as arguments will have those values packed into integer registers. This is the only option because there is no FPU support in [Armv7-M](#).

Target maintainers

- [Rust Embedded Devices Working Group Cortex-M Team](#), `cortex-m@teams.rust-embedded.org`

Target CPU and Target Feature options

See [the bare-metal Arm docs](#) for details on how to use these flags.

Table of supported CPUs

CPU	FPU	Target CPU	Target Features
Cortex-M3	No	cortex-m3	None

Arm Cortex-M3

The target CPU option is `cortex-m3` .

There are no relevant feature flags, and the FPU is not available.

thumbv8m.base-none-eabi

Tier: 2

Bare-metal target for CPUs in the Baseline [Armv8-M](#) architecture family, supporting a subset of the [T32 ISA](#).

Processors in this family include the:

- [Arm Cortex-M23](#)

See [arm-none-eabi](#) for information applicable to all `arm-none-eabi` targets.

This target uses the soft-float ABI: functions which take `f32` or `f64` as arguments will have those values packed into integer registers. This is the only option because there is no FPU support in [Armv8-M](#) Baseline.

Target maintainers

- [Rust Embedded Devices Working Group Cortex-M Team](#), `cortex-m@teams.rust-embedded.org`

Target CPU and Target Feature options

See [the bare-metal Arm docs](#) for details on how to use these flags.

Table of supported CPUs

CPU	FPU	Target CPU	Target Features
Cortex-M23	No	<code>cortex-m23</code>	None

Arm Cortex-M23

The target CPU option is `cortex-m23` .

There are no relevant feature flags, and the FPU is not available.

thumbv8m.main-none-eabi and thumbv8m.main-none-eabihf

Tier: 2

Bare-metal target for CPUs in the Mainline [Armv8-M](#) architecture family, supporting a subset of the [T32 ISA](#).

Processors in this family include the:

- [Arm Cortex-M33](#)
- [Arm Cortex-M35P](#)
- [Arm Cortex-M55](#)
- [Arm Cortex-M85](#)

See [arm-none-eabi](#) for information applicable to all `arm-none-eabi` targets, in particular the difference between the `eabi` and `eabihf` ABI.

Target maintainers

- [Rust Embedded Devices Working Group Cortex-M Team](#), `cortex-m@teams.rust-embedded.org`

Target CPU and Target Feature options

See [the bare-metal Arm docs](#) for details on how to use these flags.

Table of supported CPUs for `thumbv8m.main-none-eabi`

CPU	FPU	DSP	MVE	Target CPU	Target Features
Unspecified	No	No	No	None	None

Cortex-M33	No	No	No	cortex-m33	+soft-float,-dsp
Cortex-M33	No	Yes	No	cortex-m33	+soft-float
Cortex-M33	SP	No	No	cortex-m33	-dsp
Cortex-M33	SP	Yes	No	cortex-m33	None
Cortex-M35P	No	No	No	cortex-m35p	+soft-float,-dsp
Cortex-M35P	No	Yes	No	cortex-m35p	+soft-float
Cortex-M35P	SP	No	No	cortex-m35p	-dsp
Cortex-M35P	SP	Yes	No	cortex-m35p	None
Cortex-M55	No	Yes	No	cortex-m55	+soft-float,-mve
Cortex-M55	DP	Yes	No	cortex-m55	-mve
Cortex-M55	No	Yes	Int	cortex-m55	+soft-float,-mve.fp
Cortex-M55	DP	Yes	Int	cortex-m55	-mve.fp
Cortex-M55	DP	Yes	Int+Float	cortex-m55	None
Cortex-M85	No	Yes	No	cortex-m85	+soft-float,-mve
Cortex-M85	DP	Yes	No	cortex-m85	-mve
Cortex-M85	No	Yes	Int	cortex-m85	+soft-float,-mve.fp
Cortex-M85	DP	Yes	Int	cortex-m85	-mve.fp
Cortex-M85	DP	Yes	Int+Float	cortex-m85	None

Table of supported CPUs for thumbv8m.main-none-eabihf

CPU	FPU	DSP	MVE	Target CPU	Target Features
Unspecified	SP	No	No	None	None
Cortex-M33	SP	No	No	cortex-m33	-dsp
Cortex-M33	SP	Yes	No	cortex-m33	None

Cortex-M33P	SP	No	No	cortex-m35p	-dsp
Cortex-M33P	SP	Yes	No	cortex-m35p	None
Cortex-M55	DP	Yes	No	cortex-m55	-mve
Cortex-M55	DP	Yes	Int	cortex-m55	-mve.fp
Cortex-M55	DP	Yes	Int+Float	cortex-m55	None
Cortex-M85	DP	Yes	No	cortex-m85	-mve
Cortex-M85	DP	Yes	Int	cortex-m85	-mve.fp
Cortex-M85	DP	Yes	Int+Float	cortex-m85	None

Arm Cortex-M33

The target CPU is `cortex-m33` .

- Has optional DSP extensions
 - support is controlled by the `dsp target-feature`
 - enabled by default with this `target-cpu`
- Has an optional single precision FPU
 - support is enabled by default with this `target-cpu`
 - disable support using the `+soft-float` feature (`eabi` only)

Arm Cortex-M35P

The target CPU is `cortex-m35p` .

- Has optional DSP extensions
 - support is controlled by the `dsp target-feature`
 - enabled by default with this `target-cpu`
- Has an optional single precision FPU
 - support is enabled by default with this `target-cpu`
 - disable support using the `+soft-float` feature (`eabi` only)

Arm Cortex-M55

The target CPU is `cortex-m55`.

- Has DSP extensions
 - support is controlled by the `dsp target-feature`
 - enabled by default with this `target-cpu`
- Has an optional double-precision FPU that also supports half-precision FP16 values
 - support is enabled by default with this `target-cpu`
 - disable support using the `+soft-float` feature (`eabi` only)
- Has optional support for M-Profile Vector Extensions
 - Also known as *Helium Technology*
 - Available with only integer support, or both integer/float support
 - The appropriate feature for the MVE is either `mve` (integer) or `mve.fp` (float)
 - `mve.fp` is enabled by default on this target CPU
 - disable using `-mve.fp` (disable float MVE) or `-mve` (disable all MVE)

Arm Cortex-M85

The target CPU is `cortex-m85`.

- Has DSP extensions
 - support is controlled by the `dsp target-feature`
 - enabled by default with this `target-cpu`
- Has an optional double-precision FPU that also supports half-precision FP16 values
 - support is enabled by default with this `target-cpu`
 - disable support using the `+soft-float` feature (`eabi` only)
- Has optional support for M-Profile Vector Extensions
 - Also known as *Helium Technology*
 - Available with only integer support, or both integer/float support
 - The appropriate feature for the MVE is either `mve` (integer) or `mve.fp` (float)
 - `mve.fp` is enabled by default on this target CPU
 - disable using `-mve.fp` (disable float MVE) or `-mve` (disable all MVE)

armv6k-nintendo-3ds

Tier: 3

The Nintendo 3DS platform, which has an Armv6k processor, and its associated operating system (`horizon`).

Rust support for this target is not affiliated with Nintendo, and is not derived from nor used with any official Nintendo SDK.

Target maintainers

This target is maintained by members of the [@rust3ds](#) organization:

- [@Meziu](#)
- [@AzureMarker](#)
- [@ian-h-chamberlain](#)

Requirements

This target is cross-compiled. Dynamic linking is not supported.

`#![no_std]` crates can be built using `build-std` to build `core` and optionally `alloc`, and either `panic_abort` or `panic_unwind`.

`std` is partially supported, but mostly works. Some APIs are unimplemented and will simply return an error, such as `std::process`. An allocator is provided by default.

In order to support some APIs, binaries must be linked against `libc` written for the target, using a linker for the target. These are provided by the devkitARM toolchain. See [Cross-compilation toolchains and C code](#) for more details.

Additionally, some helper crates provide implementations of some `libc` functions used by `std` that may otherwise be missing. These, or an alternate implementation of the relevant

functions, are required to use `std`:

- `pthread-3ds` provides pthread APIs for `std::thread`.
- `shim-3ds` fulfills some other missing libc APIs (e.g. `getrandom`).

Binaries built for this target should be compatible with all variants of the 3DS (and 2DS) hardware and firmware, but testing is limited and some versions may not work correctly.

This target generates binaries in the ELF format.

Building the target

You can build Rust with support for the target by adding it to the `target` list in `config.toml` and providing paths to the devkitARM toolchain.

```
[build]
build-stage = 1
target = ["armv6k-nintendo-3ds"]

[target.armv6k-nintendo-3ds]
cc = "/opt/devkitpro/devkitARM/bin/arm-none-eabi-gcc"
cxx = "/opt/devkitpro/devkitARM/bin/arm-none-eabi-g++"
ar = "/opt/devkitpro/devkitARM/bin/arm-none-eabi-ar"
ranlib = "/opt/devkitpro/devkitARM/bin/arm-none-eabi-ranlib"
linker = "/opt/devkitpro/devkitARM/bin/arm-none-eabi-gcc"
```

Also, to build `compiler_builtins` for the target, export these flags before building the Rust toolchain:

```
export CFLAGS_armv6k_nintendo_3ds="-mfloat-abi=hard -mtune=mpcore -mtp=soft -march=armv6k"
```

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target.

The recommended way to build binaries is by using the [cargo-3ds](#) tool, which uses `build-std` and provides commands that work like the usual `cargo run`, `cargo build`, etc. The `cargo 3ds new` will automatically set up a new project with the dependencies needed to build a simple binary.

You can also build Rust with the target enabled (see [Building the target](#) above).

As mentioned in [Requirements](#), programs that use `std` must link against both the devkitARM toolchain and libraries providing the `libc` APIs used in `std`. There is a general-purpose utility crate for working with nonstandard APIs provided by the OS: [ctru-rs](#). Add it to `Cargo.toml` to use it in your program:

```
[dependencies]
ctru-rs = { git = "https://github.com/rust3ds/ctru-rs.git" }
```

Depending on `ctru-rs` ensures that all the necessary symbols are available at link time.

Testing

Binaries built for this target can be run in an emulator (most commonly [Citra](#)), or sent to a device through the use of a tool like devkitARM's `3dslink`. They may also simply be copied to an SD card to be inserted in the device.

The `cargo-3ds` tool mentioned in [Building Rust programs](#) supports the use of `3dslink` with `cargo 3ds run`. The default Rust test runner is not supported, but [custom test frameworks](#) can be used with `cargo 3ds test` to run unit tests on a device.

The Rust test suite for `library/std` is not yet supported.

Cross-compilation toolchains and C code

C code can be built for this target using the [devkitARM toolchain](#). This toolchain provides `arm-none-eabi-gcc` as the linker used to link Rust programs as well.

The toolchain also provides a `libc` implementation, which is required by `std` for many of its APIs, and a helper library `libctru` which is used by several of the helper crates listed in [Requirements](#). This toolchain does not, however, include all of the APIs expected by `std`, and the remaining APIs are implemented by `pthread-3ds` and `linker-fix-3ds`.

armv7-sony-vita-newlibeabihf

Tier: 3

This tier supports the ARM Cortex A9 processor running on a PlayStation Vita console.

Rust support for this target is not affiliated with Sony, and is not derived from nor used with any official Sony SDK.

Target maintainers

- [@nikarh](#)
- [@pheki](#)
- [@ZetaNumbers](#)

Requirements

This target is cross-compiled, and requires installing [VITASDK](#) toolchain on your system. Dynamic linking is not supported.

`#![no_std]` crates can be built using `build-std` to build `core`, and optionally `alloc`, and `panic_abort`.

`std` is partially supported, but mostly works. Some APIs are unimplemented and will simply return an error, such as `std::process`.

This target generates binaries in the ELF format with thumb ISA by default.

Binaries are linked with `arm-vita-eabi-gcc` provided by VITASDK toolchain.

Building the target

Rust does not ship pre-compiled artifacts for this target. You can use `build-std` flag to build ELF binaries with `std`:

```
cargo build -Z build-std=std,panic_abort --target=armv7-sony-vita-newlibabi --release
```

Building Rust programs

The recommended way to build artifacts that can be installed and run on PlayStation Vita is by using the [cargo-vita](#) tool. This tool uses `build-std` and VITASDK toolchain to build artifacts runnable on Vita.

To install the tool run:

```
cargo install cargo-vita
```

[VITASDK](#) toolchain must be installed, and the `VITASDK` environment variable must be set to its location, e.g.:

```
export VITASDK=/opt/vitasdk
```

Add the following section to your project's `Cargo.toml`:

```
[package.metadata.vita]
# A unique 9 character alphanumeric identifier of the app.
title_id = "RUSTAPP01"
# A title that will be used for the app. Optional, name will be used if not
defined
title_name = "My application"
```

To build a VPK with ELF in the release profile, run:

```
cargo vita build vpk --release
```

After building a `*.vpk` file it can be uploaded to a PlayStation Vita and installed, or used with a [Vita3K](#) emulator.

Testing

The default Rust test runner is supported, and tests can be compiled to an elf and packed to a *.vpk file using `cargo-vita` tool. Filtering tests is not currently supported since passing command-line arguments to the executable is not supported on Vita, so the runner will always execute all tests.

The Rust test suite for `library/std` is not yet supported.

Cross-compilation

This target can be cross-compiled from `x86_64` on Windows, MacOS or Linux systems. Other hosts are not supported for cross-compilation.

armv7-unknown-linux-ucLibcabi

Tier: 3

This target supports Armv7-A softfloat CPUs and uses the uclibc-ng standard library. This is a common configuration on many consumer routers (e.g., Netgear R7000, Asus RT-AC68U).

Target maintainers

- [@lancethepants](#)

Requirements

This target is cross compiled, and requires a cross toolchain.

This target supports host tools and std.

Building the target

You will need to download or build a 'c' cross toolchain that targets Armv7-A softfloat and that uses the uclibc-ng standard library. If your target hardware is something like a router or an embedded device, keep in mind that manufacturer supplied SDKs for this class of CPU could be outdated and potentially unsuitable for bootstrapping rust.

[Here](#) is a sample toolchain that is built using [buildroot](#). It uses modern toolchain components, older thus universal kernel headers (2.6.36.4), and is used for a project called [Tomatoware](#). This toolchain is patched so that its sysroot is located at /mmc (e.g., /mmc/bin, /mmc/lib, /mmc/include). This is useful in scenarios where the root filesystem is read-only but you are able attach external storage loaded with user applications. Tomatoware is an example of this that even allows you to run various compilers and developer tools natively on the target device.

Utilizing the Tomatoware toolchain this target can be built for cross compilation and native compilation (host tools) with project

[rust-bootstrap-armv7-unknown-linux-uclibceabi](#).

Here is a sample config if using your own toolchain.

```
[build]
build-stage = 2
target = ["armv7-unknown-linux-uclibceabi"]

[target.armv7-unknown-linux-uclibceabi]
cc = "/path/to/arm-unknown-linux-uclibcgnueabi-gcc"
cxx = "/path/to/arm-unknown-linux-uclibcgnueabi-g++"
ar = "path/to/arm-unknown-linux-uclibcgnueabi-ar"
ranlib = "path/to/arm-unknown-linux-uclibcgnueabi-ranlib"
linker = "/path/to/arm-unknown-linux-uclibcgnueabi-gcc"
```

Building Rust programs

The following assumes you are using the Tomatoware toolchain and environment. Adapt if you are using your own toolchain.

Native compilation

Since this target supports host tools, you can natively build rust applications directly on your target device. This can be convenient because it removes the complexities of cross compiling and you can immediately test and deploy your binaries. One downside is that compiling on your Armv7-A CPU will probably be much slower than cross compilation on your x86 machine.

To setup native compilation:

- Download Tomatoware to your device using the latest nightly release found [here](#).
- Extract tar zxvf arm-soft-mmc.tgz -C /mmc
- Add /mmc/bin:/mmc/sbin/ to your PATH, or source /mmc/etc/profile
- apt update && apt install rust

If you bootstrap rust on your own using the project above, it will create a .deb file that you then can install with

```
dpkg -i rust_1.xx.x-x_arm.deb
```

After completing these steps you can use rust normally in a native environment.

Cross Compilation

To cross compile, you'll need to:

- Build the rust cross toolchain using [rust-bootstrap-armv7-unknown-linux-ucLibcabi](#) or your own built toolchain.
- Link your built toolchain with

```
rustup toolchain link stage2 \  
${HOME}/rust-bootstrap-armv7-unknown-linux-  
ucLibcabi/src/rust/rust/build/x86_64-unknown-linux-gnu/stage2
```

- Build with:


```
CC_armv7_unknown_linux_uclibceabi=/opt/tomatoWARE/arm-soft-mmc/bin/arm-
linux-gcc \
CXX_armv7_unknown_linux_uclibceabi=/opt/tomatoWARE/arm-soft-mmc/bin/arm-
linux-g++ \
AR_armv7_unknown_linux_uclibceabi=/opt/tomatoWARE/arm-soft-mmc/bin/arm-
linux-ar \
CFLAGS_armv7_unknown_linux_uclibceabi="-march=armv7-a -mtune=cortex-a9" \
CXXFLAGS_armv7_unknown_linux_uclibceabi="-march=armv7-a -mtune=cortex-a9" \
CARGO_TARGET_ARMV7_UNKNOWN_LINUX_UCLIBCEABI_LINKER=/opt/tomatoWARE/arm-
soft-mmc/bin/arm-linux-gcc \
CARGO_TARGET_ARMV7_UNKNOWN_LINUX_UCLIBCEABI_RUSTFLAGS='-Clink-arg=-s -
Clink-arg=-Wl,--dynamic-linker=/mmc/lib/ld-uClibc.so.1 -Clink-arg=-Wl,-
rpath,/mmc/lib' \
cargo +stage2 \
build \
--target armv7-unknown-linux-uclibceabi \
--release
```

- Copy the binary to your target device and run.

We specify `CC`, `CXX`, `AR`, `CFLAGS`, and `CXXFLAGS` environment variables because sometimes a project or a subproject requires the use of your 'c' cross toolchain. Since Tomatoware has a modified sysroot we also pass via `RUSTFLAGS` the location of the dynamic-linker and `rpath`.

Test with QEMU

To test a cross-compiled binary on your build system follow the instructions for `Cross Compilation`, install `qemu-arm-static`, and run with the following.

```
CC_armv7_unknown_linux_uclibceabi=/opt/tomatoWARE/arm-soft-mmc/bin/arm-linux-
gcc \
CXX_armv7_unknown_linux_uclibceabi=/opt/tomatoWARE/arm-soft-mmc/bin/arm-linux-
g++ \
AR_armv7_unknown_linux_uclibceabi=/opt/tomatoWARE/arm-soft-mmc/bin/arm-linux-ar
\
CFLAGS_armv7_unknown_linux_uclibceabi="-march=armv7-a -mtune=cortex-a9" \
CXXFLAGS_armv7_unknown_linux_uclibceabi="-march=armv7-a -mtune=cortex-a9" \
CARGO_TARGET_ARMV7_UNKNOWN_LINUX_UCLIBCEABI_LINKER=/opt/tomatoWARE/arm-soft-
mmc/bin/arm-linux-gcc \
CARGO_TARGET_ARMV7_UNKNOWN_LINUX_UCLIBCEABI_RUNNER="qemu-arm-static -L
/opt/tomatoWARE/arm-soft-mmc/arm-tomatoWARE-linux-uclibcgnueabi/sysroot/" \
cargo +stage2 \
run \
--target armv7-unknown-linux-uclibceabi \
--release
```

Run in a chroot

It's also possible to build in a chroot environment. This is a convenient way to work without needing to access the target hardware.

To build the chroot:

- `sudo debootstrap --arch armel bullseye $HOME/debian`
- `sudo chroot $HOME/debian/ /bin/bash`
- `mount proc /proc -t proc`
- `mount -t sysfs /sys sys/`
- `export PATH=/mmc/bin:/mmc/sbin:$PATH`

From here you can setup your environment (e.g., add user, install wget).

- Download Tomatoware to the chroot environment using the latest nightly release found [here](#).
- Extract `tar zxvf arm-soft-mmc.tgz -C /mmc`
- Add `/mmc/bin:/mmc/sbin/` to your PATH, or `source /mmc/etc/profile`
- `sudo /mmc/bin/apt update && sudo /mmc/bin/apt install rust`

After completing these steps you can use rust normally in a chroot environment.

Remember when using `sudo` the root user's PATH could differ from your user's PATH.

armv7-unknown-linux-uclibceabihf

Tier: 3

This tier supports the Armv7-A processor running a Linux kernel and uClibc-ng standard library. It provides full support for rust and the rust standard library.

Designated Developers

- [@skrap](#)

Requirements

This target is cross compiled, and requires a cross toolchain. You can find suitable pre-built toolchains at [bootlin](#) or build one yourself via [buildroot](#).

Building

Get a C toolchain

Compiling rust for this target has been tested on `x86_64` linux hosts. Other host types have not been tested, but may work, if you can find a suitable cross compilation toolchain for them.

If you don't already have a suitable toolchain, download one [here](#), and unpack it into a directory.

Configure rust

The target can be built by enabling it for a `rustc` build, by placing the following in `config.toml`:

```
[build]
target = ["armv7-unknown-linux-ucLibceabiHf"]
stage = 2

[target.armv7-unknown-linux-ucLibceabiHf]
# ADJUST THIS PATH TO POINT AT YOUR TOOLCHAIN
cc = "/TOOLCHAIN_PATH/bin/arm-buildroot-linux-ucLibcgnueabiHf-gcc"
```

Build

```
# in rust dir
./x.py build --stage 2
```

Building and Running Rust Programs

To test cross-compiled binaries on a `x86_64` system, you can use the `qemu-arm` [userspace emulation](#) program. This avoids having a full emulated ARM system by doing dynamic binary translation and dynamic system call translation. It lets you run ARM programs directly on your `x86_64` kernel. It's very convenient!

To use:

- Install `qemu-arm` according to your distro.
- Link your built toolchain via:
 - `rustup toolchain link stage2 ${RUST}/build/x86_64-unknown-linux-gnu/stage2`
- Create a test program

```
cargo new hello_world
cd hello_world
```

- Build and run

```
CARGO_TARGET_ARMV7_UNKNOWN_LINUX_UCLIBCABIHF_RUNNER="qemu-arm -L  
${TOOLCHAIN}/arm-buildroot-linux-uclibcgnueabi/f/sysroot/" \  
CARGO_TARGET_ARMV7_UNKNOWN_LINUX_UCLIBCABIHF_LINKER=${TOOLCHAIN}/bin/arm-  
buildroot-linux-uclibcgnueabi/f/gcc \  
cargo +stage2 run --target armv7-unknown-linux-uclibceabi/f
```

*-linux-android and *-linux-androideabi

Tier: 2

[Android](#) is a mobile operating system built on top of the Linux kernel.

Target maintainers

- Chris Wailes ([@chriswailes](#))
- Matthew Maurer ([@maurer](#))
- Martin Geisler ([@mgeisler](#))

Requirements

This target is cross-compiled from a host environment. Development may be done from the [source tree](#) or using the Android NDK.

Android targets support std. Generated binaries use the ELF file format.

NDK/API Update Policy

Rust will support the most recent Long Term Support (LTS) Android Native Development Kit (NDK). By default Rust will support all API levels supported by the NDK, but a higher minimum API level may be required if deemed necessary.

Building the target

To build Rust binaries for Android you'll need a copy of the most recent LTS edition of the [Android NDK](#). Supported Android targets are:

- `aarch64-linux-android`
- `arm-linux-androideabi`
- `armv7-linux-androideabi`
- `i686-linux-android`
- `thumbv7neon-linux-androideabi`
- `x86_64-linux-android`

The `riscv64-linux-android` target is supported as a Tier 3 target.

A list of all supported targets can be found [here](#)

Architecture Notes

`riscv64-linux-android`

Currently the `riscv64-linux-android` target requires the following architecture features/extensions:

- `a` (atomics)
- `d` (double-precision floating-point)
- `c` (compressed instruction set)
- `f` (single-precision floating-point)
- `m` (multiplication and division)
- `v` (vector)
- `zba` (address calculation instructions)
- `zbb` (base instructions)
- `zbs` (single-bit instructions)

*-unknown-linux-ohos

Tier: 2

Targets for the [OpenHarmony](#) operating system.

Target maintainers

- Amanieu d'Antras ([@Amanieu](#))
- Lu Binglun ([@lubinglun](#))

Setup

The OpenHarmony SDK doesn't currently support Rust compilation directly, so some setup is required.

First, you must obtain the OpenHarmony SDK from [this page](#). Select the version of OpenHarmony you are developing for and download the "Public SDK package for the standard system".

Create the following shell scripts that wrap Clang from the OpenHarmony SDK:

```
aarch64-unknown-linux-ohos-clang.sh
```

```
#!/bin/sh
exec /path/to/ohos-sdk/linux/native/llvm/bin/clang \
  -target aarch64-linux-ohos \
  --sysroot=/path/to/ohos-sdk/linux/native/sysroot \
  -D__MUSL__ \
  "$@"
```

```
aarch64-unknown-linux-ohos-clang++.sh
```

```
#!/bin/sh
exec /path/to/ohos-sdk/linux/native/llvm/bin/clang++ \
  -target aarch64-linux-ohos \
  --sysroot=/path/to/ohos-sdk/linux/native/sysroot \
  -D__MUSL__ \
  "$@"
```

armv7-unknown-linux-ohos-clang.sh

```
#!/bin/sh
exec /path/to/ohos-sdk/linux/native/llvm/bin/clang \
  -target arm-linux-ohos \
  --sysroot=/path/to/ohos-sdk/linux/native/sysroot \
  -D__MUSL__ \
  -march=armv7-a \
  -mfloat-abi=softfp \
  -mtune=generic-armv7-a \
  -mthumb \
  "$@"
```

armv7-unknown-linux-ohos-clang++.sh

```
#!/bin/sh
exec /path/to/ohos-sdk/linux/native/llvm/bin/clang++ \
  -target arm-linux-ohos \
  --sysroot=/path/to/ohos-sdk/linux/native/sysroot \
  -D__MUSL__ \
  -march=armv7-a \
  -mfloat-abi=softfp \
  -mtune=generic-armv7-a \
  -mthumb \
  "$@"
```

x86_64-unknown-linux-ohos-clang.sh

```
#!/bin/sh
exec /path/to/ohos-sdk/linux/native/llvm/bin/clang \
  -target x86_64-linux-ohos \
  --sysroot=/path/to/ohos-sdk/linux/native/sysroot \
  -D__MUSL__ \
  "$@"
```

```
x86_64-unknown-linux-ohos-clang++.sh
```

```
#!/bin/sh
exec /path/to/ohos-sdk/linux/native/llvm/bin/clang++ \
  -target x86_64-linux-ohos \
  --sysroot=/path/to/ohos-sdk/linux/native/sysroot \
  -D__MUSL__ \
  "$@"
```

Future versions of the OpenHarmony SDK will avoid the need for this process.

Building Rust programs

Rustup ships pre-compiled artifacts for this target, which you can install with:

```
rustup target add aarch64-unknown-linux-ohos
rustup target add armv7-unknown-linux-ohos
rustup target add x86_64-unknown-linux-ohos
```

You will need to configure the linker to use in `~/.cargo/config.toml`:

```
[target.aarch64-unknown-linux-ohos]
ar = "/path/to/ohos-sdk/linux/native/llvm/bin/llvm-ar"
linker = "/path/to/aarch64-unknown-linux-ohos-clang.sh"

[target.armv7-unknown-linux-ohos]
ar = "/path/to/ohos-sdk/linux/native/llvm/bin/llvm-ar"
linker = "/path/to/armv7-unknown-linux-ohos-clang.sh"

[target.x86_64-unknown-linux-ohos]
ar = "/path/to/ohos-sdk/linux/native/llvm/bin/llvm-ar"
linker = "/path/to/x86_64-unknown-linux-ohos-clang.sh"
```

Building the target from source

Instead of using `rustup`, you can instead build a rust toolchain from source. Create a

config.toml with the following contents:

```
profile = "compiler"
change-id = 115898

[build]
sanitizers = true
profiler = true

[target.aarch64-unknown-linux-ohos]
cc = "/path/to/aarch64-unknown-linux-ohos-clang.sh"
cxx = "/path/to/aarch64-unknown-linux-ohos-clang++.sh"
ar = "/path/to/ohos-sdk/linux/native/llvm/bin/llvm-ar"
ranlib = "/path/to/ohos-sdk/linux/native/llvm/bin/llvm-ranlib"
linker = "/path/to/aarch64-unknown-linux-ohos-clang.sh"

[target.armv7-unknown-linux-ohos]
cc = "/path/to/armv7-unknown-linux-ohos-clang.sh"
cxx = "/path/to/armv7-unknown-linux-ohos-clang++.sh"
ar = "/path/to/ohos-sdk/linux/native/llvm/bin/llvm-ar"
ranlib = "/path/to/ohos-sdk/linux/native/llvm/bin/llvm-ranlib"
linker = "/path/to/armv7-unknown-linux-ohos-clang.sh"

[target.x86_64-unknown-linux-ohos]
cc = "/path/to/x86_64-unknown-linux-ohos-clang.sh"
cxx = "/path/to/x86_64-unknown-linux-ohos-clang++.sh"
ar = "/path/to/ohos-sdk/linux/native/llvm/bin/llvm-ar"
ranlib = "/path/to/ohos-sdk/linux/native/llvm/bin/llvm-ranlib"
linker = "/path/to/x86_64-unknown-linux-ohos-clang.sh"
```

Testing

Running the Rust testsuite is possible, but currently difficult due to the way the OpenHarmony emulator is set up (no networking).

Cross-compilation toolchains and C code

You can use the shell scripts above to compile C code for the target.

i686-unknown-hurd-gnu

Tier: 3

[GNU/Hurd] is the GNU Hurd is the GNU project's replacement for the Unix kernel.

Target maintainers

- Samuel Thibault, samuel.thibault@ens-lyon.org, <https://github.com/sthibaul/>

Requirements

The target supports host tools.

The GNU/Hurd target supports `std` and uses the standard ELF file format.

Building the target

This target can be built by adding `i686-unknown-hurd-gnu` as target in the rustc list.

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target. To compile for this target, you will either need to build Rust with the target enabled (see "Building the target" above), or build your own copy of `core` by using `build-std` or similar.

Testing

Tests can be run in the same way as a regular binary.

Cross-compilation toolchains and C code

The target supports C code, the GNU toolchain calls the target `i686-unknown-gnu`.

aarch64-unknown-teeos

Tier: 3

Target for the TEEOS operating system.

TEEOS is a mini os run in TrustZone, for trusted/security apps. The kernel of TEEOS is HongMeng/ChCore micro kernel. The libc for TEEOS is a part of musl. It's very small that there is no RwLock, no network, no stdin, and no file system for apps in TEEOS.

Some abbreviation:

Abbreviation	The full text	Description
TEE	Trusted Execution Environment	ARM TrustZone divides the system into two worlds/modes -- the secure world/mode and the normal world/mode. TEE is in the secure world/mode.
REE	Rich Execution Environment	The normal world. for example, Linux for Android phone is in REE side.
TA	Trusted Application	The app run in TEE side system.
CA	Client Application	The progress run in REE side system.

TEEOS is open source in progress. [MORE about](#)

Target maintainers

- Petrochenkov Vadim
- Sword-Destiny

Setup

We use OpenHarmony SDK for TEEOS.

The OpenHarmony SDK doesn't currently support Rust compilation directly, so some setup is required.

First, you must obtain the OpenHarmony SDK from [this page](#). Select the version of OpenHarmony you are developing for and download the "Public SDK package for the standard system".

Create the following shell scripts that wrap Clang from the OpenHarmony SDK:

```
aarch64-unknown-teeos-clang.sh
```

```
#!/bin/sh
exec /path/to/ohos-sdk/linux/native/llvm/bin/clang \
  -target aarch64-linux-gnu \
  "$@"
```

```
aarch64-unknown-teeos-clang++.sh
```

```
#!/bin/sh
exec /path/to/ohos-sdk/linux/native/llvm/bin/clang++ \
  -target aarch64-linux-gnu \
  "$@"
```

Building the target

To build a rust toolchain, create a `config.toml` with the following contents:


```

profile = "compiler"
change-id = 115898

[build]
sanitizers = true
profiler = true
target = ["x86_64-unknown-linux-gnu", "aarch64-unknown-teeos"]
submodules = false
compiler-docs = false
extended = true

[install]
bindir = "bin"
libdir = "lib"

[target.aarch64-unknown-teeos]
cc = "/path/to/scripts/aarch64-unknown-teeos-clang.sh"
cxx = "/path/to/scripts/aarch64-unknown-teeos-clang.sh"
linker = "/path/to/scripts/aarch64-unknown-teeos-clang.sh"
ar = "/path/to/ohos-sdk/linux/native/llvm/bin/llvm-ar"
ranlib = "/path/to/ohos-sdk/linux/native/llvm/bin/llvm-ranlib"
llvm-config = "/path/to/ohos-sdk/linux/native/llvm/bin/llvm-config"

```

note: You need to insert "/usr/include/x86_64-linux-gnu/" into environment variable: \$C_INCLUDE_PATH

if some header files like bits/xxx.h not found.

note: You can install gcc-aarch64-linux-gnu,g++-aarch64-linux-gnu if some files like crt1.o not found.

note: You may need to install libc6-dev-i386 libc6-dev if "gnu/stubs-32.h" not found.

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target. To compile for this target, you will either need to build Rust with the target enabled (see "Building the target" above), or build your own copy of `core` by using `build-std` or similar.

You will need to configure the linker to use in `~/.cargo/config`:

```
[target.aarch64-unknown-teeos]  
linker = "/path/to/aarch64-unknown-teeos-clang.sh" # or aarch64-linux-gnu-ld
```

Testing

Running the Rust testsuite is not possible now.

More information about how to test CA/TA. [See here](#)

★-espidf

Tier: 3

Targets for the [ESP-IDF](#) development framework running on RISC-V and Xtensa CPUs.

Target maintainers

- Ivan Markov [@ivmarkov](#)
- Scott Mabin [@MabezDev](#)
- Sergio Gasquez [@SergioGasquez](#)

Requirements

The target names follow this format: `$ARCH-esp-espidf`, where `$ARCH` specifies the target processor architecture. The following targets are currently defined:

Target name	Target CPU(s)	Minimum ESP-IDF version
<code>riscv32imc-esp-espidf</code>	ESP32-C2	v5.0
<code>riscv32imc-esp-espidf</code>	ESP32-C3	v4.4
<code>riscv32imac-esp-espidf</code>	ESP32-C6	v5.1
<code>riscv32imac-esp-espidf</code>	ESP32-H2	v5.1
<code>riscv32imafc-esp-espidf</code>	ESP32-P4	v5.2
<code>xtensa-esp32-espidf</code>	ESP32	v4.4
<code>xtensa-esp32s2-espidf</code>	ESP32-S2	v4.4
<code>xtensa-esp32s3-espidf</code>	ESP32-S3	v4.4

It is recommended to use the latest ESP-IDF stable release if possible.

Building the target

The target can be built by enabling it for a `rustc` build. The `build-std` feature is required to build the standard library for ESP-IDF. `ldproxy` is also required for linking, it can be installed from crates.io.

```
[build]
target = ["$ARCH-esp-espidf"]

[target.$ARCH-esp-espidf]
linker = "ldproxy"

[unstable]
build-std = ["std", "panic_abort"]
```

The `esp-idf-sys` crate will handle the compilation of ESP-IDF, including downloading the relevant toolchains for the build.

Cross-compilation toolchains and C code

`esp-idf-sys` exposes the toolchain used in the compilation of ESP-IDF, see the crate [documentation for build output propagation](#) for more information.

aarch64-unknown-fuchsia and x86_64-unknown-fuchsia

Tier: 2

[Fuchsia](#) is a modern open source operating system that's simple, secure, updatable, and performant.

Target maintainers

The [Fuchsia team](#):

- Tyler Mandry ([@tmandry](#))
- David Koloski ([@djkoloski](#))
- Julia Ryan ([@P1n3appl3](#))
- Erick Tryzelaar ([@erickt](#))

As the team evolves over time, the specific members listed here may differ from the members reported by the API. The API should be considered to be authoritative if this occurs. Instead of pinging individual members, use `@rustbot ping fuchsia` to contact the team on GitHub.

Table of contents

1. Requirements
2. Walkthrough structure
3. Compiling a Rust binary targeting Fuchsia
 1. Targeting Fuchsia with rustup and cargo
 2. Targeting Fuchsia with a compiler built from source
4. Creating a Fuchsia package
 1. Creating a Fuchsia component
 2. Building a Fuchsia package

5. Publishing a Fuchsia package
 1. Creating a Fuchsia package repository
 2. Publishing Fuchsia package to repository
6. Running a Fuchsia component on an emulator
 1. Starting the Fuchsia emulator
 2. Watching emulator logs
 3. Serving a Fuchsia package
 4. Running a Fuchsia component
7. `.gitignore` extensions
8. Testing
 1. Running unit tests
 2. Running the compiler test suite
9. Debugging
 1. `zxdb`
 2. Attaching `zxdb`
 3. Using `zxdb`
 4. Displaying source code in `zxdb`

Requirements

This target is cross-compiled from a host environment. You will need a recent copy of the [Fuchsia SDK](#), which provides the tools, libraries, and binaries required to build and link programs for Fuchsia.

Development may also be done from the [source tree](#).

Fuchsia targets support `std` and follow the `sysv64` calling convention on `x86_64`. Fuchsia binaries use the ELF file format.

Walkthrough structure

This walkthrough will cover:

1. Compiling a Rust binary targeting Fuchsia.

2. Building a Fuchsia package.
3. Publishing and running a Fuchsia package to a Fuchsia emulator.

For the purposes of this walkthrough, we will only target `x86_64-unknown-fuchsia`.

Compiling a Rust binary targeting Fuchsia

Today, there are two main ways to build a Rust binary targeting Fuchsia using the Fuchsia SDK:

1. Allow `rustup` to handle the installation of Fuchsia targets for you.
2. Build a toolchain locally that can target Fuchsia.

Targeting Fuchsia with `rustup` and `cargo`

The easiest way to build a Rust binary targeting Fuchsia is by allowing `rustup` to handle the installation of Fuchsia targets for you. This can be done by issuing the following commands:

```
rustup target add x86_64-unknown-fuchsia
rustup target add aarch64-unknown-fuchsia
```

After installing our Fuchsia targets, we can now compile a Rust binary that targets Fuchsia.

To create our Rust project, we can use `cargo` as follows:

From base working directory

```
cargo new hello_fuchsia
```

The rest of this walkthrough will take place from `hello_fuchsia`, so we can change into that directory now:

```
cd hello_fuchsia
```

Note: From this point onwards, all commands will be issued from the `hello_fuchsia/` directory,

and all `hello_fuchsia/` prefixes will be removed from references for sake of brevity.

We can edit our `src/main.rs` to include a test as follows:

`src/main.rs`

```
fn main() {
    println!("Hello Fuchsia!");
}

#[test]
fn it_works() {
    assert_eq!(2 + 2, 4);
}
```

In addition to the standard workspace created, we will want to create a `.cargo/config.toml` file to link necessary libraries during compilation:

`.cargo/config.toml`

```
[target.x86_64-unknown-fuchsia]

rustflags = [
    "-Lnative=<SDK_PATH>/arch/x64/lib",
    "-Lnative=<SDK_PATH>/arch/x64/sysroot/lib"
]
```

Note: Make sure to fill out `<SDK_PATH>` with the path to the downloaded [Fuchsia SDK](#).

These options configure the following:

- `-Lnative=${SDK_PATH}/arch/${ARCH}/lib` : Link against Fuchsia libraries from the SDK
- `-Lnative=${SDK_PATH}/arch/${ARCH}/sysroot/lib` : Link against Fuchsia sysroot libraries from the SDK

In total, our new project will look like:

Current directory structure


```
hello_fuchsia/  
├── src/  
│   └── main.rs  
├── Cargo.toml  
└── .cargo/  
    └── config.toml
```

Finally, we can build our rust binary as:

```
cargo build --target x86_64-unknown-fuchsia
```

Now we have a Rust binary at `target/x86_64-unknown-fuchsia/debug/hello_fuchsia`, targeting our desired Fuchsia target.

Current directory structure

```
hello_fuchsia/  
├── src/  
│   └── main.rs  
├── target/  
│   └── x86_64-unknown-fuchsia/  
│       └── debug/  
│           └── hello_fuchsia  
├── Cargo.toml  
└── .cargo/  
    └── config.toml
```

Targeting Fuchsia with a compiler built from source

An alternative to the first workflow is to target Fuchsia by using `rustc` built from source.

Before building Rust for Fuchsia, you'll need a clang toolchain that supports Fuchsia as well. A recent version (14+) of clang should be sufficient to compile Rust for Fuchsia.

x86-64 and AArch64 Fuchsia targets can be enabled using the following configuration in `config.toml`:

```
[build]
target = [<host_platform>, "aarch64-unknown-fuchsia", "x86_64-unknown-fuchsia"]

[rust]
lld = true

[llvm]
download-ci-llvm = false

[target.x86_64-unknown-fuchsia]
cc = "clang"
cxx = "clang++"

[target.aarch64-unknown-fuchsia]
cc = "clang"
cxx = "clang++"
```

Though not strictly required, you may also want to use `clang` for your host target as well:

```
[target.<host_platform>]
cc = "clang"
cxx = "clang++"
```

By default, the Rust compiler installs itself to `/usr/local` on most UNIX systems. You may want to install it to another location (e.g. a local `install` directory) by setting a custom prefix in `config.toml`:

```
[install]
# Make sure to use the absolute path to your install directory
prefix = "<RUST_SRC_PATH>/install"
```

Next, the following environment variables must be configured. For example, using a script we name `config-env.sh`:

```
# Configure this environment variable to be the path to the downloaded SDK
export SDK_PATH="<SDK path goes here>"

export CFLAGS_aarch64_unknown_fuchsia="--target=aarch64-unknown-fuchsia --
sysroot=${SDK_PATH}/arch/arm64/sysroot -I${SDK_PATH}/pkg/fdio/include"
export CXXFLAGS_aarch64_unknown_fuchsia="--target=aarch64-unknown-fuchsia --
sysroot=${SDK_PATH}/arch/arm64/sysroot -I${SDK_PATH}/pkg/fdio/include"
export LDFLAGS_aarch64_unknown_fuchsia="--target=aarch64-unknown-fuchsia --
sysroot=${SDK_PATH}/arch/arm64/sysroot -L${SDK_PATH}/arch/arm64/lib"
export CARGO_TARGET_AARCH64_UNKNOWN_FUCHSIA_RUSTFLAGS="-C link-arg=--
sysroot=${SDK_PATH}/arch/arm64/sysroot -
Lnative=${SDK_PATH}/arch/arm64/sysroot/lib -Lnative=${SDK_PATH}/arch/arm64/lib"
export CFLAGS_x86_64_unknown_fuchsia="--target=x86_64-unknown-fuchsia --
sysroot=${SDK_PATH}/arch/x64/sysroot -I${SDK_PATH}/pkg/fdio/include"
export CXXFLAGS_x86_64_unknown_fuchsia="--target=x86_64-unknown-fuchsia --
sysroot=${SDK_PATH}/arch/x64/sysroot -I${SDK_PATH}/pkg/fdio/include"
export LDFLAGS_x86_64_unknown_fuchsia="--target=x86_64-unknown-fuchsia --
sysroot=${SDK_PATH}/arch/x64/sysroot -L${SDK_PATH}/arch/x64/lib"
export CARGO_TARGET_X86_64_UNKNOWN_FUCHSIA_RUSTFLAGS="-C link-arg=--
sysroot=${SDK_PATH}/arch/x64/sysroot -Lnative=${SDK_PATH}/arch/x64/sysroot/lib
-Lnative=${SDK_PATH}/arch/x64/lib"
```

Finally, the Rust compiler can be built and installed:

```
(source config-env.sh && ./x.py install)
```

Once `rustc` is installed, we can create a new working directory to work from, `hello_fuchsia` along with `hello_fuchsia/src`:

```
mkdir hello_fuchsia
cd hello_fuchsia
mkdir src
```

Note: From this point onwards, all commands will be issued from the `hello_fuchsia/` directory, and all `hello_fuchsia/` prefixes will be removed from references for sake of brevity.

There, we can create a new file named `src/hello_fuchsia.rs`:

src/hello_fuchsia.rs

```
fn main() {
    println!("Hello Fuchsia!");
}

#[test]
fn it_works() {
    assert_eq!(2 + 2, 4);
}
```

Current directory structure

```
hello_fuchsia/
└─ src/
   └─ hello_fuchsia.rs
```

Using your freshly installed `rustc`, you can compile a binary for Fuchsia using the following options:

- `--target x86_64-unknown-fuchsia / --target aarch64-unknown-fuchsia`: Targets the Fuchsia platform of your choice
- `-Lnative ${SDK_PATH}/arch/${ARCH}/lib`: Link against Fuchsia libraries from the SDK
- `-Lnative ${SDK_PATH}/arch/${ARCH}/sysroot/lib`: Link against Fuchsia sysroot libraries from the SDK

Putting it all together:

```
# Configure these for the Fuchsia target of your choice
TARGET_ARCH="<x86_64-unknown-fuchsia|aarch64-unknown-fuchsia>"
ARCH="<x64|aarch64>"

rustc \
    --target ${TARGET_ARCH} \
    -Lnative=${SDK_PATH}/arch/${ARCH}/lib \
    -Lnative=${SDK_PATH}/arch/${ARCH}/sysroot/lib \
    --out-dir bin src/hello_fuchsia.rs
```

Current directory structure

```

hello_fuchsia/
├── src/
│   └── hello_fuchsia.rs
└── bin/
    └── hello_fuchsia

```

Creating a Fuchsia package

Before moving on, double check your directory structure:

Current directory structure

```

hello_fuchsia/
├── src/                                (if using rustc)
│   └── hello_fuchsia.rs                ...
├── bin/                                ...
│   └── hello_fuchsia                   ...
├── src/                                (if using cargo)
│   └── main.rs                         ...
├── target/                             ...
│   └── x86_64-unknown-fuchsia/         ...
│       ├── debug/                      ...
│       └── hello_fuchsia                ...

```

With our Rust binary built, we can move to creating a Fuchsia package. On Fuchsia, a package is the unit of distribution for software. We'll need to create a new package directory where we will place files like our finished binary and any data it may need.

To start, make the `pkg`, and `pkg/meta` directories:

```

mkdir pkg
mkdir pkg/meta

```

Current directory structure

```

hello_fuchsia/
└── pkg/
    └── meta/

```

Now, create the following files inside:

pkg/meta/package

```
{
  "name": "hello_fuchsia",
  "version": "0"
}
```

The `package` file describes our package's name and version number. Every package must contain one.

pkg/hello_fuchsia.manifest if using cargo

```
bin/hello_fuchsia=target/x86_64-unknown-fuchsia/debug/hello_fuchsia
lib/ld.so.1=<SDK_PATH>/arch/x64/sysroot/dist/lib/ld.so.1
lib/libfdio.so=<SDK_PATH>/arch/x64/dist/libfdio.so
meta/package=pkg/meta/package
meta/hello_fuchsia.cm=pkg/meta/hello_fuchsia.cm
```

pkg/hello_fuchsia.manifest if using rustc

```
bin/hello_fuchsia=bin/hello_fuchsia
lib/ld.so.1=<SDK_PATH>/arch/x64/sysroot/dist/lib/ld.so.1
lib/libfdio.so=<SDK_PATH>/arch/x64/dist/libfdio.so
meta/package=pkg/meta/package
meta/hello_fuchsia.cm=pkg/meta/hello_fuchsia.cm
```

Note: Relative manifest paths are resolved starting from the working directory of `ffx`. Make sure to fill out `<SDK_PATH>` with the path to the downloaded SDK.

The `.manifest` file will be used to describe the contents of the package by relating their location when installed to their location on the file system. The `bin/hello_fuchsia=` entry will be different depending on how your Rust binary was built, so choose accordingly.

Current directory structure

```

hello_fuchsia/
└─ pkg/
    └─ meta/
        └─ package
    └─ hello_fuchsia.manifest

```

Creating a Fuchsia component

On Fuchsia, components require a component manifest written in Fuchsia's markup language called CML. The Fuchsia devsite contains an [overview of CML](#) and a [reference for the file format](#). Here's a basic one that can run our single binary:

pkg/hello_fuchsia.cml

```

{
  include: [ "syslog/client.shard.cml" ],
  program: {
    runner: "elf",
    binary: "bin/hello_fuchsia",
  },
}

```

Current directory structure

```

hello_fuchsia/
└─ pkg/
    └─ meta/
        └─ package
    └─ hello_fuchsia.manifest
    └─ hello_fuchsia.cml

```

Now we can compile that CML into a component manifest:

```

${SDK_PATH}/tools/${ARCH}/cmc compile \
  pkg/hello_fuchsia.cml \
  --includepath ${SDK_PATH}/pkg \
  -o pkg/meta/hello_fuchsia.cm

```

Note: `--includepath` tells the compiler where to look for `include`s from our CML. In our case,

we're only using `syslog/client.shard.cml`.

Current directory structure

```
hello_fuchsia/  
└─ pkg/  
   └─ meta/  
      └─ package  
         └─ hello_fuchsia.cm  
      └─ hello_fuchsia.manifest  
      └─ hello_fuchsia.cml
```

Building a Fuchsia package

Next, we'll build a package manifest as defined by our manifest:

```
${SDK_PATH}/tools/${ARCH}/ffx package build \  
  --api-level $((${SDK_PATH}/tools/${ARCH}/ffx --machine json version | jq  
.tool_version.api_level) \  
  --out pkg/hello_fuchsia_manifest \  
  pkg/hello_fuchsia.manifest
```

This will produce `pkg/hello_fuchsia_manifest/` which is a package manifest we can publish directly to a repository.

Current directory structure

```
hello_fuchsia/  
└─ pkg/  
   └─ meta/  
      └─ package  
         └─ hello_fuchsia.cm  
      └─ hello_fuchsia_manifest/  
         └─ ...  
      └─ hello_fuchsia.manifest  
      └─ hello_fuchsia.cml  
      └─ hello_fuchsia_package_manifest
```

We are now ready to publish the package.

Publishing a Fuchsia package

With our package and component manifests setup, we can now publish our package. The first step will be to create a Fuchsia package repository to publish to.

Creating a Fuchsia package repository

We can set up our repository with:

```
${SDK_PATH}/tools/${ARCH}/ffx repository create pkg/repo
```

Current directory structure

```
hello_fuchsia/
├── pkg/
│   ├── meta/
│   │   ├── package
│   │   └── hello_fuchsia.cm
│   ├── hello_fuchsia_manifest/
│   │   └── ...
│   ├── repo/
│   │   └── ...
│   ├── hello_fuchsia.manifest
│   ├── hello_fuchsia.cml
│   └── hello_fuchsia_package_manifest
```

Publishing Fuchsia package to repository

We can publish our new package to that repository with:

```
${SDK_PATH}/tools/${ARCH}/ffx repository publish \
  --package pkg/hello_fuchsia_package_manifest \
  pkg/repo
```

Then we can add the repository to ffx 's package server as hello-fuchsia using:

```
${SDK_PATH}/tools/${ARCH}/ffx repository add-from-pm \
  --repository hello-fuchsia \
  pkg/repo
```

Running a Fuchsia component on an emulator

At this point, we are ready to run our Fuchsia component. For reference, our final directory structure will look like:

Final directory structure

```
hello_fuchsia/
├── src/                                (if using rustc)
│   └── hello_fuchsia.rs                ...
├── bin/                                ...
│   └── hello_fuchsia                   ...
├── src/                                (if using cargo)
│   └── main.rs                         ...
├── target/                             ...
│   └── x86_64-unknown-fuchsia/         ...
│       ├── debug/                       ...
│       └── hello_fuchsia                 ...
└── pkg/
    ├── meta/
    │   ├── package
    │   └── hello_fuchsia.cm
    ├── hello_fuchsia_manifest/
    │   └── ...
    ├── repo/
    │   └── ...
    ├── hello_fuchsia.manifest
    ├── hello_fuchsia.cml
    └── hello_fuchsia_package_manifest
```

Starting the Fuchsia emulator

Start a Fuchsia emulator in a new terminal using:

```
${SDK_PATH}/tools/${ARCH}/ffx product-bundle get workstation_eng.qemu-${ARCH}  
${SDK_PATH}/tools/${ARCH}/ffx emu start workstation_eng.qemu-${ARCH} --headless
```

Watching emulator logs

Once the emulator is running, open a separate terminal to watch the emulator logs:

In separate terminal

```
${SDK_PATH}/tools/${ARCH}/ffx log \  
--since now
```

Serving a Fuchsia package

Now, start a package repository server to serve our package to the emulator:

```
${SDK_PATH}/tools/${ARCH}/ffx repository server start
```

Once the repository server is up and running, register it with the target Fuchsia system running in the emulator:

```
${SDK_PATH}/tools/${ARCH}/ffx target repository register \  
--repository hello-fuchsia
```

Running a Fuchsia component

Finally, run the component:

```
${SDK_PATH}/tools/${ARCH}/ffx component run \  
/core/ffx-laboratory:hello_fuchsia \  
fuchsia-pkg://hello-fuchsia/hello_fuchsia_manifest#meta/hello_fuchsia.cm
```

On reruns of the component, the `--recreate` argument may also need to be passed.

```
${SDK_PATH}/tools/${ARCH}/ffx component run \  
--recreate \  
/core/ffx-laboratory:hello_fuchsia \  
fuchsia-pkg://hello-fuchsia/hello_fuchsia_manifest#meta/hello_fuchsia.cm
```

.gitignore extensions

Optionally, we can create/extend our `.gitignore` file to ignore files and directories that are not helpful to track:

```
pkg/repo  
pkg/meta/hello_fuchsia.cm  
pkg/hello_fuchsia_manifest  
pkg/hello_fuchsia_package_manifest
```

Testing

Running unit tests

Tests can be run in the same way as a regular binary.

- If using `cargo`, you can simply pass `test --no-run` to the `cargo` invocation and then repackage and rerun the Fuchsia package. From our previous example, this would look like `cargo test --target x86_64-unknown-fuchsia --no-run`, and moving the executable binary path found from the line `Executable unittests src/main.rs (target/x86_64-unknown-fuchsia/debug/deps/hello_fuchsia-<HASH>)` into `pkg/hello_fuchsia.manifest`.
- If using the compiled `rustc`, you can simply pass `--test` to the `rustc` invocation and then repackage and rerun the Fuchsia package.

The test harness will run the applicable unit tests.

Often when testing, you may want to pass additional command line arguments to your binary. Additional arguments can be set in the component manifest:

pkg/hello_fuchsia.cml

```
{
  include: [ "syslog/client.shard.cml" ],
  program: {
    runner: "elf",
    binary: "bin/hello_fuchsia",
    args: ["it_works"],
  },
}
```

This will pass the argument `it_works` to the binary, filtering the tests to only those tests that match the pattern. There are many more configuration options available in CML including environment variables. More documentation is available on the [Fuchsia devsite](#).

Running the compiler test suite

The commands in this section assume that they are being run from inside your local Rust source checkout:

```
cd ${RUST_SRC_PATH}
```

To run the Rust test suite on an emulated Fuchsia device, you'll also need to download a copy of the Fuchsia SDK. The current minimum supported SDK version is [20.20240412.3.1](#).

Fuchsia's test runner interacts with the Fuchsia emulator and is located at `src/ci/docker/scripts/fuchsia-test-runner.py`. First, add the following variables to your existing `config-env.sh`:

```
# TEST_TOOLCHAIN_TMP_DIR can point anywhere, but it:
# - must be less than 108 characters, otherwise qemu can't handle the path
# - must be consistent across calls to this file (don't use `mktemp -d` here)
export TEST_TOOLCHAIN_TMP_DIR="/tmp/rust-tmp"

# Keep existing contents of `config-env.sh` from earlier, including SDK_PATH
```

We can then use the script to start our test environment with:

```
( \
  source config-env.sh &&
  src/ci/docker/scripts/fuchsia-test-runner.py start
  --rust-build ${RUST_SRC_PATH}/build
  --sdk ${SDK_PATH}
  --target {x86_64-unknown-fuchsia|aarch64-unknown-fuchsia}
  --verbose
)
```

Where `${RUST_SRC_PATH}/build` is the `build-dir` set in `config.toml`.

Once our environment is started, we can run our tests using `x.py` as usual. The test runner script will run the compiled tests on an emulated Fuchsia device. To run the full `tests/ui` test suite:

```
( \
  source config-env.sh &&
  ./x.py
  --config config.toml
  --stage=2
  test tests/ui
  --target x86_64-unknown-fuchsia
  --run=always
  --test-args --target-rustcflags
  --test-args -Lnative=${SDK_PATH}/arch/{x64|arm64}/sysroot/lib
  --test-args --target-rustcflags
  --test-args -Lnative=${SDK_PATH}/arch/{x64|arm64}/lib
  --test-args --target-rustcflags
  --test-args -Clink-arg=--undefined-version
  --test-args --remote-test-client
  --test-args src/ci/docker/scripts/fuchsia-test-runner.py
)
```

By default, `x.py` compiles test binaries with `panic=unwind`. If you built your Rust toolchain with `-Cpanic=abort`, you need to tell `x.py` to compile test binaries with `panic=abort` as well:

```
--test-args --target-rustcflags
--test-args -Cpanic=abort
--test-args --target-rustcflags
--test-args -Zpanic_abort_tests
```

```
\
\
\
\
```

When finished testing, the test runner can be used to stop the test environment:

```
src/ci/docker/scripts/fuchsia-test-runner.py stop
```

Debugging

zxdb

Debugging components running on a Fuchsia emulator can be done using the console-mode debugger: [zxdb](#). We will demonstrate attaching necessary symbol paths to debug our `hello-fuchsia` component.

Attaching zxdb

In a separate terminal, issue the following command from our `hello_fuchsia` directory to launch `zxdb`:

In separate terminal

```
${SDK_PATH}/tools/${ARCH}/ffx debug connect -- \
  --symbol-path target/x86_64-unknown-fuchsia/debug
```

- `--symbol-path` gets required symbol paths, which are necessary for stepping through your program.

The "[displaying source code in zxdb](#)" section describes how you can display Rust and/or Fuchsia source code in your debugging session.

Using zxdb

Once launched, you will be presented with the window:

```
Connecting (use "disconnect" to cancel)...  
Connected successfully.  
👉 To get started, try "status" or "help".  
[zxdb]
```

To attach to our program, we can run:

```
[zxdb] attach hello_fuchsia
```

Expected output

```
Waiting for process matching "hello_fuchsia".  
Type "filter" to see the current filters.
```

Next, we can create a breakpoint at main using "b main":

```
[zxdb] b main
```

Expected output

```
Created Breakpoint 1 @ main
```

Finally, we can re-run the "hello_fuchsia" component from our original terminal:

```
${SDK_PATH}/tools/${ARCH}/ffx component run \  
--recreate \  
fuchsia-pkg://hello-fuchsia/hello_fuchsia_manifest#meta/hello_fuchsia.cm
```

Once our component is running, our zxdb window will stop execution in our main as desired:

Expected output


```
Breakpoint 1 now matching 1 addrs for main
● on bp 1 hello_fuchsia::main() • main.rs:2
  1 fn main() {
  ▶ 2     println!("Hello Fuchsia!");
    3 }
    4
[zxdb]
```

zxdb has similar commands to other debuggers like [gdb](#). To list the available commands, run "help" in the zxdb window or visit [the zxdb documentation](#).

```
[zxdb] help
```

Expected output

```
Help!
```

```
Type "help <command>" for command-specific help.
```

```
Other help topics (see "help <topic>")
```

```
...
```

Displaying source code in zxdb

By default, the debugger will not be able to display source code while debugging. For our user code, we displayed source code by pointing our debugger to our debug binary via the `--symbol-path` arg. To display library source code in the debugger, you must provide paths to the source using `--build-dir`. For example, to display the Rust and Fuchsia source code:

```
${SDK_PATH}/tools/${ARCH}/ffx debug connect -- \
  --symbol-path target/x86_64-unknown-fuchsia/debug \
  --build-dir ${RUST_SRC_PATH}/rust \
  --build-dir ${FUCHSIA_SRC_PATH}/fuchsia/out/default
```

- `--build-dir` links against source code paths, which are not strictly necessary for debugging, but is a nice-to-have for displaying source code in zxdb.

Linking to a Fuchsia checkout can help with debugging Fuchsia libraries, such as [fdio](#).

Debugging the compiler test suite

Debugging the compiler test suite requires some special configuration:

First, we have to properly configure `zxdb` so it will be able to find debug symbols and source information for our test. The test runner can do this for us with:

```
src/ci/docker/scripts/fuchsia-test-runner.py debug \
  --rust-src ${RUST_SRC_PATH} \
  --fuchsia-src ${FUCHSIA_SRC_PATH} \
  --test ${TEST}
```

where `${TEST}` is relative to Rust's `tests` directory (e.g. `ui/abi/...`).

This will start a `zxdb` session that is properly configured for the specific test being run. All three arguments are optional, so you can omit `--fuchsia-src` if you don't have it downloaded. Now is a good time to set any desired breakpoints, like `b main`.

Next, we have to tell `x.py` not to optimize or strip debug symbols from our test suite binaries. We can do this by passing some new arguments to `rustc` through our `x.py` invocation. The full invocation is:

```
( \
  source config-env.sh && \
  ./x.py \
  --config config.toml \
  --stage=2 \
  test tests/${TEST} \
  --target x86_64-unknown-fuchsia \
  --run=always \
  --test-args --target-rustcflags \
  --test-args -Lnative=${SDK_PATH}/arch/{x64|arm64}/sysroot/lib \
  --test-args --target-rustcflags \
  --test-args -Lnative=${SDK_PATH}/arch/{x64|arm64}/lib \
  --test-args --target-rustcflags \
  --test-args -Clink-arg=--undefined-version \
  --test-args --target-rustcflags \
  --test-args -Cdebuginfo=2 \
  --test-args --target-rustcflags \
  --test-args -Copt-level=0 \
  --test-args --target-rustcflags \
  --test-args -Cstrip=none \
  --test-args --remote-test-client \
  --test-args src/ci/docker/scripts/fuchsia-test-runner.py \
)
```

If you built your Rust toolchain with `panic=abort`, make sure to include the previous flags so your test binaries are also compiled with `panic=abort`.

Upon running this command, the test suite binary will be run and `zxdb` will attach and load any relevant debug symbols.

-kmc-solid_

Tier: 3

[SOLID](#) embedded development platform by Kyoto Microcomputer Co., Ltd.

The target names follow this format: `$ARCH-kmc-solid_$KERNEL-$ABI` , where `$ARCH` specifies the target processor architecture, `$KERNEL` the base kernel, and `$ABI` the target ABI (optional). The following targets are currently defined:

Target name	target_arch	target_vendor	target
aarch64-kmc-solid_asp3	aarch64	kmc	solid_a
armv7a-kmc-solid_asp3-eabi	arm	kmc	solid_a
armv7a-kmc-solid_asp3-eabihf	arm	kmc	solid_a

Designated Developers

- [@kawadakk](#)

Requirements

This target is cross-compiled. A platform-provided C compiler toolchain is required, though it can be substituted by [GNU Arm Embedded Toolchain](#) for the purpose of building Rust and functional binaries.

Building

The target can be built by enabling it for a `rustc` build.

```
[build]
target = ["aarch64-kmc-solid_asp3"]
```

Make sure `aarch64-kmc-elf-gcc` is included in `$PATH`. Alternatively, you can use GNU Arm Embedded Toolchain by adding the following to `config.toml`:

```
[target.aarch64-kmc-solid_asp3]
cc = "arm-none-eabi-gcc"
```

Cross-compilation

This target can be cross-compiled from any hosts.

Testing

Currently there is no support to run the rustc test suite for this target.

Building Rust programs

Building executables is not supported yet.

If `rustc` has support for that target and the library artifacts are available, then Rust static libraries can be built for that target:

```
$ rustc --target aarch64-kmc-solid_asp3 your-code.rs --crate-type staticlib
$ ls libyour_code.a
```

On Rust Nightly it's possible to build without the target artifacts available:

```
cargo build -Z build-std --target aarch64-kmc-solid_asp3
```

csky-unknown-linux-gnuabiv2

Tier: 3

This target supports C-SKY CPUs with abi v2 and glibc .

target	std	host	notes
csky-unknown-linux-gnuabiv2	✓		C-SKY abiv2 Linux (little endian)
csky-unknown-linux-gnuabiv2hf	✓		C-SKY abiv2 Linux, hardfloat (little endian)

Reference:

- CSKY ABI Manual
- csky-linux-gnuabiv2-toolchain
- csky-linux-gnuabiv2-qemu

other links:

- https://c-sky.github.io/
- https://gitlab.com/c-sky/

Target maintainers

- @Dirreke

Requirements

Building the target

Get a C toolchain

Compiling rust for this target has been tested on `x86_64` linux hosts. Other host types have not been tested, but may work, if you can find a suitable cross compilation toolchain for them.

If you don't already have a suitable toolchain, you can download from [here](#), and unpack it into a directory.

Configure rust

The target can be built by enabling it for a `rustc` build, by placing the following in `config.toml`:

```
[build]
target = ["x86_64-unknown-linux-gnu", "csky-unknown-linux-gnuabi2", "csky-unknown-linux-gnuabi2hf"]
stage = 2

[target.csky-unknown-linux-gnuabi2]
# ADJUST THIS PATH TO POINT AT YOUR TOOLCHAIN
cc = "${TOOLCHAIN_PATH}/bin/csky-linux-gnuabi2-gcc"

[target.csky-unknown-linux-gnuabi2hf]
# ADJUST THIS PATH TO POINT AT YOUR TOOLCHAIN
cc = "${TOOLCHAIN_PATH}/bin/csky-linux-gnuabi2-gcc"
```

Build

```
# in rust dir
./x.py build --stage 2
```

Building and Running Rust programs

To test cross-compiled binaries on a `x86_64` system, you can use the `qemu-cskyv2`. This

avoids having a full emulated ARM system by doing dynamic binary translation and dynamic system call translation. It lets you run CSKY programs directly on your `x86_64` kernel. It's very convenient!

To use:

- Install `qemu-cskyv2` (If you don't already have a qemu, you can download from [here](#), and unpack it into a directory.)
- Link your built toolchain via:
 - `rustup toolchain link stage2 ${RUST}/build/x86_64-unknown-linux-gnu/stage2`
- Create a test program

```
cargo new hello_world
cd hello_world
```

- Build and run

```
CARGO_TARGET_CSKY_UNKNOWN_LINUX_GNUABIV2_RUNNER=${QEMU_PATH}/bin/qemu-cskyv2 -L
${TOOLCHAIN_PATH}/csky-linux-gnuabiv2/libc \
CARGO_TARGET_CSKY_UNKNOWN_LINUX_GNUABIV2_LINKER=${TOOLCHAIN_PATH}/bin/csky-
linux-gnuabiv2-gcc \
RUSTFLAGS="-C target-features=+crt-static" \
cargo +stage2 run --target csky-unknown-linux-gnuabiv2
```

Attention: The dynamic-linked program may not be run by `qemu-cskyv2` but can be run on the target.

hexagon-unknown-linux-musl

Tier: 3

Target for cross-compiling Linux user-mode applications targeting the Hexagon DSP architecture.

Target	Descriptions
hexagon-unknown-linux-musl	Hexagon 32-bit Linux

Target maintainers

- [Brian Cain](#), bcain@quicinc.com

Requirements

The target is cross-compiled. This target supports `std`. By default, code generated with this target should run on Hexagon DSP hardware.

- `-Ctarget-cpu=hexagonv73` adds support for instructions defined up to Hexagon V73.

Binaries can be run using QEMU user emulation. On Debian-based systems, it should be sufficient to install the package `qemu-user-static` to be able to run simple static binaries:

```
# apt install qemu-user-static
# qemu-hexagon-static ./hello
```

In order to build linux programs with Rust, you will require a linker capable of targeting hexagon. You can use `clang` / `lld` from the [hexagon toolchain using exclusively public open source repos](#).

Also included in that toolchain is the C library that can be used when creating dynamically linked executables.

```
# /opt/clang+llvm-18.1.0-cross-hexagon-unknown-linux-musl/x86_64-linux-gnu/bin/qemu-hexagon -L /opt/clang+llvm-18.1.0-cross-hexagon-unknown-linux-musl/x86_64-linux-gnu/target/hexagon-unknown-linux-musl/usr/ ./hello
```

Building the target

Because it is Tier 3, rust does not yet ship pre-compiled artifacts for this target.

Therefore, you can build Rust with support for the target by adding it to the target list in `config.toml`, a sample configuration is shown below.

```
[build]
target = [ "hexagon-unknown-linux-musl" ]

[target.hexagon-unknown-linux-musl]

cc = "hexagon-unknown-linux-musl-clang"
cxx = "hexagon-unknown-linux-musl-clang++"
linker = "hexagon-unknown-linux-musl-clang"
ar = "hexagon-unknown-linux-musl-ar"
ranlib = "hexagon-unknown-linux-musl-ranlib"
musl-root = "/opt/clang+llvm-18.1.0-cross-hexagon-unknown-linux-musl/x86_64-linux-gnu/target/hexagon-unknown-linux-musl/usr"
llvm-libunwind = 'in-tree'
qemu-rootfs = "/opt/clang+llvm-18.1.0-cross-hexagon-unknown-linux-musl/x86_64-linux-gnu/target/hexagon-unknown-linux-musl/usr"
```

Testing

Currently there is no support to run the rustc test suite for this target.

Building Rust programs

Download and install the hexagon open source toolchain from

https://github.com/quic/toolchain_for_hexagon/releases

The following `.cargo/config` is needed inside any project directory to build for the Hexagon Linux target:

```
[build]
target = "hexagon-unknown-linux-musl"

[target.hexagon-unknown-linux-musl]
linker = "hexagon-unknown-linux-musl-clang"
ar = "hexagon-unknown-linux-musl-ar"
runner = "qemu-hexagon -L /opt/clang+llvm-18.1.0-cross-hexagon-unknown-linux-musl/x86_64-linux-gnu/target/hexagon-unknown-linux-musl/usr"
```

Edit the "runner" in `.cargo/config` to point to the path to your toolchain's C library.

```
...
runner = "qemu-hexagon -L /path/to/my/inst/clang+llvm-18.1.0-cross-hexagon-unknown-linux-musl/x86_64-linux-gnu/target/hexagon-unknown-linux-musl/usr"
...
```

Build/run your rust program with `qemu-hexagon` in your `PATH`:

```
export PATH=/path/to/my/inst/clang+llvm-18.1.0-cross-hexagon-unknown-linux-musl/x86_64-linux-gnu/bin/:$PATH
cargo run -Zbuild-std -Zbuild-std-features=llvm-libunwind
```

hexagon-unknown-none-elf

Tier: 3

Rust for baremetal Hexagon DSPs.

Target	Descriptions
hexagon-unknown-none-elf	Hexagon 32-bit (freestanding, hardfloat)

Target maintainers

- [Brian Cain](#), bcain@quicinc.com

Requirements

This target is cross-compiled. There is no support for `std`. There is no default allocator, but it's possible to use `alloc` by supplying an allocator.

By default, code generated with this target should run on Hexagon DSP hardware.

- `-Ctarget-cpu=hexagonv73` adds support for instructions defined up to Hexagon V73.

Functions marked `extern "C"` use the [Hexagon architecture calling convention](#).

This target generates PIC ELF binaries.

Building the target

You can build Rust with support for the target by adding it to the `target` list in `config.toml`:

```
[build]
build-stage = 1
host = ["<target for your host>"]
target = ["<target for your host>", "hexagon-unknown-none-elf"]

[target.hexagon-unknown-none-elf]

cc = "hexagon-unknown-none-elf-clang"
cxx = "hexagon-unknown-none-elf-clang++"
linker = "hexagon-unknown-none-elf-clang"
ranlib = "hexagon-unknown-none-elf-ranlib"
ar = "hexagon-unknown-none-elf-ar"
llvm-libunwind = 'in-tree'
```

Replace `<target for your host>` with `x86_64-unknown-linux-gnu` or whatever else is appropriate for your host machine.

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target. To compile for this target, you will either need to build Rust with the target enabled (see "Building the target" above), or build your own copy of `core` by using `build-std` or similar.

Testing

Since `hexagon-unknown-none-elf` supports a variety of different environments and does not support `std`, this target does not support running the Rust test suite.

Cross-compilation toolchains and C code

This target has been tested using `qemu-system-hexagon`.

A common use case for `hexagon-unknown-none-elf` is building libraries that link against C

code and can be used in emulation or on a device with a Hexagon DSP.

The Hexagon SDK has libraries which are useful to link against when running on a device.

Standalone OS

The script below will build an executable against "hexagon standalone OS" which is suitable for emulation or bare-metal on-device testing.

First, run `cargo new --bin demo1_hexagon` then add the source below as `src/main.rs`. This program demonstrates the console output via semihosting.

```
#![no_std]
#![no_main]

extern "C" {
    fn putchar(ch: i32);
    fn _exit(code: i32) -> !;
}

#[no_mangle]
extern "C" fn main() -> i32 {
    let message = "Hello, this is Rust!";
    for b in message.bytes() {
        unsafe {
            putchar(b as i32);
        }
    }
    0
}

#[panic_handler]
fn panic(_panic: &core::panic::PanicInfo) -> ! {
    unsafe {
        _exit(1);
    }
}
```

Next, save the script below as `build.sh` and edit it to suit your environment.

- `hex_toolchain` below refers to the [hexagon toolchain using exclusively public open source repos](#).

- `cc` below refers to `clang`. You can use `clang` from your distribution, as long as it's at least `clang-17`. Or you can use `hexagon-unknown-none-elf-clang` from one of the [hexagon open source toolchain releases](#).

```
# Hexagon SDK, required for target libraries:
hex_sdk_root=/local/mnt/workspace/Qualcomm/Hexagon_SDK/5.3.0.0
hex_sdk_toolchain=${hex_sdk_root}/tools/HEXAGON_Tools/8.6.06

sdk_libs=${hex_sdk_toolchain}/Tools/target/hexagon/lib
q6_arch=v65
g0_lib_path=${sdk_libs}/${q6_arch}/G0
pic_lib_path=${sdk_libs}/${q6_arch}/G0/pic

build_cfg=release
cargo build --target=hexagon-unknown-none-elf -Zbuild-std --release

# Builds an executable against "hexagon standalone OS" suitable for emulation:
${cc} --target=hexagon-unknown-none-elf -o testit \
    -fuse-ld=lld \
    -m${q6_arch} \
    -nodefaultlibs \
    -nostartfiles \
    ${g0_lib_path}/crt0_standalone.o \
    ${g0_lib_path}/crt0.o \
    ${g0_lib_path}/init.o \
    -L${sdk_libs}/${q6_arch}/ \
    -L${sdk_libs}/ \
    wrap.c \
    target/hexagon-unknown-none-elf/${build_cfg}/libdemo1_hexagon.rlib \
    target/hexagon-unknown-none-elf/${build_cfg}/deps/libcore-*.rlib \
    target/hexagon-unknown-none-elf/${build_cfg}/deps/libcompiler_builtins-
*.rlib \
    -Wl,--start-group \
    -Wl,--defsym,_SDA_BASE_=0,--defsym,__sbss_start=0,--defsym,__sbss_end=0 \
    ${g0_lib_path}/libstandalone.a \
    ${g0_lib_path}/libc.a \
    -lgcc \
    -lc_eh \
    -Wl,--end-group \
    ${g0_lib_path}/fini.o \

${hex_toolchain}/x86_64-linux-gnu/bin/qemu-system-hexagon -monitor none -
display none -kernel ./testit
```

QuRT OS

First, run `cargo new --lib demo2_hexagon` then add the source below as `src/lib.rs`. This program demonstrates inline assembly and console output via semihosting.

```
#![no_std]
#![no_main]
#![feature(lang_items)]
#![feature(asm_experimental_arch)]

use core::arch::asm;

extern "C" {
    fn putchar(ch: i32);
    fn _exit(code: i32) -> !;
}

fn hexagon_specific() {
    let mut buffer = [0_u8; 128];

    unsafe {
        let mut x = &buffer;
        asm!(
            "{{\n\t",
            "  v0=vmem({addr}+#0)\n\t",
            "  {tmp} = and({tmp}, #1)\n\t",
            "}}\n\t",
            addr = in(reg) x,
            tmp = out(reg) _,
        );
    }
}

#[no_mangle]
extern "C" fn hello() -> i32 {
    let message = "Hello, this is Rust!\n";
    for b in message.bytes() {
        unsafe {
            putchar(b as i32);
        }
    }
    hexagon_specific();
    0
}
```



```
#[panic_handler]
fn panic(_panic: &core::panic::PanicInfo) -> ! {
    unsafe {
        _exit(1);
    }
}

#[lang = "eh_personality"]
fn rust_eh_personality() {}
```

Next, create a C program as an entry point, save the content below as `wrap.c` :

```
int hello();

int main() {
    hello();
}
```

Then, save the script below as `build.sh` and edit it to suit your environment. The script below will build a shared object against the QuRT RTOS which is suitable for emulation or on-device testing when loaded via the `fastrpc-shell`.

```

# Hexagon SDK, required for target libraries:
hex_sdk_root=/local/mnt/workspace/Qualcomm/Hexagon_SDK/5.3.0.0
hex_sdk_toolchain=${hex_sdk_root}/tools/HEXAGON_Tools/8.6.06

sdk_libs=${hex_sdk_toolchain}/Tools/target/hexagon/lib
q6_arch=v65
g0_lib_path=${sdk_libs}/${q6_arch}/G0
pic_lib_path=${sdk_libs}/${q6_arch}/G0/pic
runelf=${hex_sdk_root}/rtos/qurt/compute/v65/sdksim_bin/runelf.pbn
rmohs=${hex_sdk_root}/libs/run_main_on_hexagon/ship/hexagon_toolv86_${q6_arch}/
run_main_on_hexagon_sim

# Builds a library suitable for loading into "run_main_on_hexagon_sim" for
# emulation or frpc shell on real target:
${cc} --target=hexagon-unknown-none-elf -o testit.so \
    -fuse-ld=lld \
    -fPIC -shared \
    -nostdlib \
    -Wl,-Bsymbolic \
    -Wl,--wrap=malloc \
    -Wl,--wrap=calloc \
    -Wl,--wrap=free \
    -Wl,--wrap=realloc \
    -Wl,--wrap=memalign \
    -m${q6_arch} \
    wrap.c \
    target/hexagon-unknown-none-elf/${build_cfg}/libdemo2_hexagon.rlib \
    target/hexagon-unknown-none-elf/${build_cfg}/deps/libcore-*.rlib \
    target/hexagon-unknown-none-elf/${build_cfg}/deps/libcompiler_builtins-
*.rlib \
    -Wl,-soname=testit \
    ${pic_lib_path}/libc.so

# -Bsymbolic above for memory alloc funcs is necessary to access the heap on
# target, but otherwise not required.

# multi-stage loader: runelf => run_main_on_hexagon_sim => testit.so{`main`}
${hex_toolchain}/x86_64-linux-gnu/bin/qemu-system-hexagon \
    -monitor none \
    -display none \
    -kernel ${runelf} \
    -append "${rmohs} -- ./testit.so"

```

loongarch*-unknown-linux-*

Tier: 2 (with Host Tools)

[LoongArch](#) Linux targets. LoongArch is a RISC ISA developed by Loongson Technology Corporation Limited.

Target	Description
loongarch64-unknown-linux-gnu	LoongArch64 Linux, LP64D ABI (kernel 5.19, glibc 2.36)
loongarch64-unknown-linux-musl	LoongArch64 Linux, LP64D ABI (kernel 5.19, musl 1.2.5)

These support both native and cross builds, and have full support for `std`.

Reference material:

- [LoongArch ISA manuals](#)
- [Application Binary Interface for the LoongArch™ Architecture](#)

Target maintainers

- [WANG Rui](#) wangrui@loongson.cn
- [ZHAI Xiang](#) zhaixiang@loongson.cn
- [ZHAI Xiaojuan](#) zhaixiaojuan@loongson.cn
- [WANG Xuerui](#) git@xen0n.name

Requirements

OS Version

The minimum supported Linux version is 5.19.

Some Linux distributions, mostly commercial ones, may provide forked Linux kernels that has a version number less than 5.19 for their LoongArch ports. Such kernels may still get patched to be compatible with the upstream Linux 5.19 UAPI, therefore supporting the targets described in this document, but this is not always the case. The `rustup` installer contains a check for this, and will abort if incompatibility is detected.

Host toolchain

The targets require a reasonably up-to-date LoongArch toolchain on the host. Currently the following components are used by the Rust CI to build the target, and the versions can be seen as the minimum requirement:

- GNU Binutils 2.40
- GCC 13.x
- glibc 2.36
- linux-headers 5.19

Of these, glibc and linux-headers are at their respective earliest versions with mainline LoongArch support, so it is impossible to use older versions of these. Older versions of Binutils and GCC will not work either, due to lack of support for newer LoongArch ELF relocation types, among other features.

Recent LLVM/Clang toolchains may be able to build the targets, but are not currently being actively tested.

Building

These targets are distributed through `rustup`, and otherwise require no special configuration.

If you need to build your own Rust for some reason though, the targets can be simply enabled in `config.toml`. For example:

```
[build]
target = ["loongarch64-unknown-linux-gnu"]
```

Make sure the LoongArch toolchain binaries are reachable from `$PATH`. Alternatively, you can explicitly configure the paths in `config.toml`:

```
[target.loongarch64-unknown-linux-gnu]
# Adjust the paths to point at your toolchain
# Suppose the toolchain is placed at /TOOLCHAIN_PATH, and the cross prefix is
# "loongarch64-unknown-linux-gnu-":
cc = "/TOOLCHAIN_PATH/bin/loongarch64-unknown-linux-gnu-gcc"
cxx = "/TOOLCHAIN_PATH/bin/loongarch64-unknown-linux-gnu-g++"
ar = "/TOOLCHAIN_PATH/bin/loongarch64-unknown-linux-gnu-ar"
ranlib = "/TOOLCHAIN_PATH/bin/loongarch64-unknown-linux-gnu-ranlib"
linker = "/TOOLCHAIN_PATH/bin/loongarch64-unknown-linux-gnu-gcc"
```

Cross-compilation

This target can be cross-compiled on a `x86_64-unknown-linux-gnu` host. Other hosts are also likely to work, but not actively tested.

You can test the cross build directly on the host, thanks to QEMU linux-user emulation. An example is given below:

```
# Suppose the cross toolchain is placed at $TOOLCHAIN_PATH, with a cross prefix
# of "loongarch64-unknown-linux-gnu-".
export CC_loongarch64_unknown_linux_gnu="$TOOLCHAIN_PATH"/bin/loongarch64-
unknown-linux-gnu-gcc
export CXX_loongarch64_unknown_linux_gnu="$TOOLCHAIN_PATH"/bin/loongarch64-
unknown-linux-gnu-g++
export AR_loongarch64_unknown_linux_gnu="$TOOLCHAIN_PATH"/bin/loongarch64-
unknown-linux-gnu-gcc-ar
export
CARGO_TARGET_LOONGARCH64_UNKNOWN_LINUX_GNU_LINKER="$TOOLCHAIN_PATH"/bin/loongar
ch64-unknown-linux-gnu-gcc

# Point qemu-loongarch64 to the LoongArch sysroot.
# Suppose the sysroot is located at "sysroot" below the toolchain root:
export CARGO_TARGET_LOONGARCH64_UNKNOWN_LINUX_GNU_RUNNER="qemu-loongarch64 -L
$TOOLCHAIN_PATH/sysroot"
# Or alternatively, if binfmt_misc is set up for running LoongArch binaries
# transparently:
export QEMU_LD_PREFIX="$TOOLCHAIN_PATH"/sysroot

cargo run --target loongarch64-unknown-linux-gnu --release
```

Testing

There are no special requirements for testing and running the targets. For testing cross builds on the host, please refer to the "Cross-compilation" section above.

Building Rust programs

As the targets are available through `rustup`, it is very easy to build Rust programs for these targets: same as with other architectures. Note that you will need a LoongArch C/C++ toolchain for linking, or if you want to compile C code along with Rust (such as for Rust crates with C dependencies).

```
rustup target add loongarch64-unknown-linux-gnu
cargo build --target loongarch64-unknown-linux-gnu
```

Availability of pre-built artifacts through `rustup` are as follows:

- `loongarch64-unknown-linux-gnu` : since Rust 1.71;
- `loongarch64-unknown-linux-musl` : since Rust 1.81.

loongarch*-unknown-none*

Tier: 2

Freestanding/bare-metal LoongArch64 binaries in ELF format: firmware, kernels, etc.

Target	Description
loongarch64-unknown-none	LoongArch 64-bit, LP64D ABI (freestanding, hard-float)
loongarch64-unknown-none-softfloat	LoongArch 64-bit, LP64S ABI (freestanding, soft-float)

Target maintainers

- [WANG Rui](#) `wangrui@loongson.cn`
- [WANG Xuerui](#) `git@xen0n.name`

Requirements

This target is cross-compiled. There is no support for `std`. There is no default allocator, but it's possible to use `alloc` by supplying an allocator.

The `*-softfloat` target does not assume existence of FPU or any other LoongArch ISA extension, and does not make use of any non-GPR register. This allows the generated code to run in environments, such as kernels, which may need to avoid the use of such registers or which may have special considerations about the use of such registers (e.g. saving and restoring them to avoid breaking userspace code using the same registers). You can change code generation to use additional CPU features via the `-C target-feature=` codegen options to rustc, or via the `#[target_feature]` mechanism within Rust code.

By default, code generated with the soft-float target should run on any LoongArch64 hardware, with the hard-float target additionally requiring an FPU; enabling additional target

features may raise this baseline.

Code generated with the targets will use the `small` code model by default. You can change this using the `-C code-model=` option to `rustc`.

On `loongarch64-unknown-none*`, extern "C" uses the [architecture's standard calling convention](#).

The targets generate binaries in the ELF format. Any alternate formats or special considerations for binary layout will require linker options or linker scripts.

Building the target

You can build Rust with support for the targets by adding them to the `target` list in `config.toml`:

```
[build]
build-stage = 1
target = [
    "loongarch64-unknown-none",
    "loongarch64-unknown-none-softfloat",
]
```

Testing

As the targets support a variety of different environments and do not support `std`, they do not support running the Rust test suite.

Building Rust programs

Starting with Rust 1.74, precompiled artifacts are provided via `rustup`:

```
# install cross-compile toolchain
rustup target add loongarch64-unknown-none
# target flag may be used with any cargo or rustc command
cargo build --target loongarch64-unknown-none
```

Cross-compilation toolchains and C code

For cross builds, you will need an appropriate LoongArch C/C++ toolchain for linking, or if you want to compile C code along with Rust (such as for Rust crates with C dependencies).

Rust *may* be able to use an `loongarch64-unknown-linux-gnu-` toolchain with appropriate standalone flags to build for this toolchain (depending on the assumptions of that toolchain, see below), or you may wish to use a separate `loongarch64-unknown-none` toolchain.

On some LoongArch hosts that use ELF binaries, you *may* be able to use the host C toolchain, if it does not introduce assumptions about the host environment that don't match the expectations of a standalone environment. Otherwise, you may need a separate toolchain for standalone/freestanding development, just as when cross-compiling from a non-LoongArch platform.

m68k-unknown-linux-gnu

Tier: 3

Motorola 680x0 Linux

Designated Developers

- @glaubit
- @ricky26

Requirements

This target requires a Linux/m68k build environment for cross-compilation which is available on Debian and Debian-based systems, openSUSE and other distributions.

On Debian, it should be sufficient to install a g++ cross-compiler for the m68k architecture which will automatically pull in additional dependencies such as the glibc cross development package:

```
# apt install g++-m68k-linux-gnu
```

Binaries can be run using QEMU user emulation. On Debian-based systems, it should be sufficient to install the package `qemu-user-static` to be able to run simple static binaries:

```
# apt install qemu-user-static
```

To run more complex programs, it will be necessary to set up a Debian/m68k chroot with the help of the command `debootstrap` :

```
# apt install debootstrap debian-ports-archive-keyring
# debootstrap --keyring=/usr/share/keyrings/debian-ports-archive-keyring.gpg --
arch=m68k unstable debian-68k http://ftp.ports.debian.org/debian-ports
```

This chroot can then seamlessly entered using the normal `chroot` command thanks to QEMU user emulation:

```
# chroot /path/to/debian-68k
```

To get started with native builds, which are currently untested, a native Debian/m68k system can be installed either on real hardware such as 68k-based Commodore Amiga or Atari systems or emulated environments such as QEMU version 4.2 or newer or ARAnyM.

ISO images for installation are provided by the Debian Ports team and can be obtained from the Debian CD image server available at:

<https://cdimage.debian.org/cdimage/ports/current>

Documentation for Debian/m68k is available on the Debian Wiki at:

<https://wiki.debian.org/M68k>

Support is available either through the `debian-68k` mailing list:

<https://lists.debian.org/debian-68k/>

or the `#debian-68k` IRC channel on OFTC network.

Building

The codegen for this target should be built by default. However, core and std are currently missing but are being worked on and should become available in the near future.

Cross-compilation

This target can be cross-compiled from a standard Debian or Debian-based, openSUSE or any other distribution which has a basic m68k cross-toolchain available.

Testing

Currently there is no support to run the rustc test suite for this target.

Building Rust programs

Rust programs can be built for that target:

```
rustc --target m68k-unknown-linux-gnu your-code.rs
```

Very simple programs can be run using the `qemu-m68k-static` program:

```
$ qemu-m68k-static your-code
```

For more complex applications, a chroot or native (emulated) Debian/m68k system are required for testing.

mips64-openwrt-linux-musl

Tier: 3

Target maintainers

- Donald Hoskins grommish@gmail.com, <https://github.com/ltus-shield>

Requirements

This target is cross-compiled. There is no support for `std`. There is no default allocator, but it's possible to use `alloc` by supplying an allocator.

By default, Rust code generated for this target uses `-msoft-float` and is dynamically linked.

This target generated binaries in the ELF format.

Building the target

This target is built exclusively within the `openWrt` build system via the `rust-lang` HOST package

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target. To compile for this target, you will either need to build Rust with the target enabled (see "Building the target" above).

Testing

As `mips64-openwrt-linux-musl` supports a variety of different environments and does not support `std`, this target does not support running the Rust testsuite at this time.

mipsel-sony-psx

Tier: 3

Sony PlayStation 1 (psx)

Designated Developer

- [@ayrtonm](#)

Requirements

This target is cross-compiled. It has no special requirements for the host.

Building

The target can be built by enabling it for a `rustc` build:

```
[build]
build-stage = 1
target = ["mipsel-sony-psx"]
```

Cross-compilation

This target can be cross-compiled from any host.

Testing

Currently there is no support to run the rustc test suite for this target.

Building Rust programs

Since it is Tier 3, rust doesn't ship pre-compiled artifacts for this target.

Just use the `build-std` nightly cargo feature to build the `core` and `alloc` libraries:

```
cargo build -Zbuild-std=core,alloc --target mipsel-sony-psx
```

The command above generates an ELF. To generate binaries in the PSEXE format that emulators run, you can use [cargo-psx](#):

```
cargo psx build
```

or use `-Clink-arg=--oformat=binary` to produce a flat binary.

mipsisa*r6*-unknown-linux-gnu*

Tier: 3

MIPS Release 6, or simply MIPS R6, is the latest iteration of the MIPS instruction set architecture (ISA).

MIPS R6 is experimental in nature, as there is not yet real hardware. However, Qemu emulation is available and we have two Linux distros maintained for development and evaluation purposes. This documentation describes the Rust support for MIPS R6 targets under `mipsisa*r6*-unknown-linux-gnu*`.

The target name follow this format: `<machine>-<vendor>-<os><abi_suffix>`, where `<machine>` specifies the CPU family/model, `<vendor>` specifies the vendor and `<os>` the operating system name. The `<abi_suffix>` denotes the base ABI (32/n32/64/o64).

ABI suffix	Description
abi64	Uses the 64-bit (64) ABI
abin32	Uses the n32 ABI
N/A	Uses the (assumed) 32-bit (32) ABI

Target Maintainers

- Xuan Chen henry.chen@oss.cipunited.com
- Walter Ji walter.ji@oss.cipunited.com
- Xinhui Yang cyan@oss.cipunited.com
- Lain Yang lain.yang@oss.cipunited.com

Requirements

C/C++ Toolchain

A GNU toolchain for one of the MIPS R6 target is required. [AOSC OS](#) provides working native and cross-compiling build environments. You may also supply your own a toolchain consisting of recent versions of GCC and Binutils.

Target libraries

A minimum set of libraries is required to perform dynamic linking:

- GNU glibc
- OpenSSL
- Zlib
- Linux API Headers

This set of libraries should be installed to make up minimal target sysroot.

For AOSC OS, You may install such a sysroot with the following commands:

```
cd /tmp

# linux+api, glibc, and file system structure are included in the toolchain.
sudo apt install gcc+cross-mips64r6el binutils+cross-mips64r6el

# Download and extract required libraries.
wget https://repo.aosc.io/debs/pool/stable/main/z/zlib_1.2.13-0_mips64r6el.deb
-O zlib.deb
wget https://repo.aosc.io/debs/pool/stable/main/o/openssl_1.1.1q-
1_mips64r6el.deb -O openssl.deb

# Extract them to your desired location.
for i in zlib openssl ; do
    sudo dpkg-deb -vx $i.deb /var/ab/cross-root/mips64r6el
done

# Workaround a possible ld bug when using -Wl,-Bdynamic.
sudo sed -i 's|/usr|=/usr|g' /var/ab/cross-root/mips64r6el/usr/lib/libc.so
```

For other distros, you may build them manually.

Building

The following procedure outlines the build process for the MIPS64 R6 target with 64-bit (64) ABI (`mipsisa64r6el-unknown-linux-gnuabi64`).

Prerequisite: Disable debuginfo

An LLVM bug makes rustc crash if debug or debug info generation is enabled. You need to edit `config.toml` to disable this:

```
[rust]
debug = false
debug-info-level = 0
```

Prerequisite: Enable rustix's libc backend

The crate `rustix` may try to link itself against MIPS R2 assembly, resulting in linkage error. To avoid this, you may force `rustix` to use its fallback `libc` backend by setting relevant `RUSTFLAGS` :

```
export RUSTFLAGS="--cfg rustix_use_libc"
```

This will trigger warnings during build, as `-D warnings` is enabled by default. Disable `-D warnings` by editing `config.toml` to append the following:

```
[rust]
deny-warnings = false
```

Prerequisite: Supplying OpenSSL

As a Tier 3 target, `openssl_sys` lacks the vendored OpenSSL library for this target. You will need to provide a prebuilt OpenSSL library to link `cargo` . Since we have a pre-configured sysroot, we can point to it directly:

```
export MIPSISA64R6EL_UNKNOWN_LINUX_GNUABI64_OPENSSL_NO_VENDOR=y
export MIPSISA64R6EL_UNKNOWN_LINUX_GNUABI64_OPENSSL_DIR="/var/ab/cross-
root/mips64r6el/usr"
```

On Debian, you may need to provide library path and include path separately:

```
export MIPSISA64R6EL_UNKNOWN_LINUX_GNUABI64_OPENSSL_NO_VENDOR=y
export
MIPSISA64R6EL_UNKNOWN_LINUX_GNUABI64_OPENSSL_LIB_DIR="/usr/lib/mipsisa64r6el-
linux-gnuabi64/"
export MIPSISA64R6EL_UNKNOWN_LINUX_GNUABI64_OPENSSL_INCLUDE_DIR="/usr/include"
```

Launching x.py

```
[build]
target = ["mipsisa64r6el-unknown-linux-gnuabi64"]
```

Make sure that `mipsisa64r6el-unknown-linux-gnuabi64-gcc` is available from your executable search path (`$PATH`).

Alternatively, you can specify the directories to all necessary toolchain executables in `config.toml`:

```
[target.mipsisa64r6el-unknown-linux-gnuabi64]
# Adjust the paths below to point to your toolchain installation prefix.
cc = "/toolchain_prefix/bin/mipsisa64r6el-unknown-linux-gnuabi64-gcc"
cxx = "/toolchain_prefix/bin/mipsisa64r6el-unknown-linux-gnuabi64-g++"
ar = "/toolchain_prefix/bin/mipsisa64r6el-unknown-linux-gnuabi64-gcc-ar"
ranlib = "/toolchain_prefix/bin/mipsisa64r6el-unknown-linux-gnuabi64-ranlib"
linker = "/toolchain_prefix/bin/mipsisa64r6el-unknown-linux-gnuabi64-gcc"
```

Or, you can specify your cross compiler toolchain with an environment variable:

```
export CROSS_COMPILE="/opt/abcross/mips64r6el/bin/mipsisa64r6el-aosc-linux-
gnuabi64-"
```

Finally, launch the build script:

```
./x.py build
```

Tips

- Avoid setting `cargo-native-static` to `false`, as this will result in a redundant artifact error while building clippy:

duplicate artifacts found when compiling a tool, this typically means that something was recompiled because a transitive dependency has different features activated than in a previous build:

the following dependencies have different features:

```
syn 2.0.8 (registry+https://github.com/rust-lang/crates.io-index)
`clippy-driver` additionally enabled features {"full"} at ...
`cargo` additionally enabled features {} at ...
```

to fix this you will probably want to edit the local `src/tools/rustc-workspace-hack/Cargo.toml` crate, as that will update the dependency graph to ensure that these crates all share the same feature set

thread 'main' panicked at 'tools should not compile multiple copies of the same crate', tool.rs:250:13

note: run with ``RUST_BACKTRACE=1`` environment variable to display a backtrace

Building Rust programs

To build Rust programs for MIPS R6 targets, for instance, the `mipsisa64r6el-unknown-linux-gnuabi64` target:

```
cargo build --target mipsisa64r6el-unknown-linux-gnuabi64
```

Testing

To test a cross-compiled binary on your build system, install the Qemu user emulator that support the MIPS R6 architecture (`qemu-user-mipsel` or `qemu-user-mips64el`). GCC runtime libraries (`libgcc_s`) for the target architecture should be present in target sysroot to run the program.

```
env \
CARGO_TARGET_MIPSISA64R6EL_UNKNOWN_LINUX_GNUABI64_LINKER="/opt/abcross/mips64r6el/bin/mipsisa64r6el-aosc-linux-gnuabi64-gcc" \
  CARGO_TARGET_MIPSISA64R6EL_UNKNOWN_LINUX_GNUABI64_RUNNER="qemu-mips64el-static -L /var/ab/cross-root/mips64r6el" \
  cargo run --release \
    --target mipsisa64r6el-unknown-linux-gnuabi64
```

Tips for building Rust programs for MIPS R6

- Until we finalize a fix, please make sure the aforementioned workarounds for `rustix` crate and LLVM are always applied. This can be achieved by setting the relevant environment variables, and editing `Cargo.toml` before building.

nvptx64-nvidia-cuda

Tier: 2

This is the target meant for deploying code for Nvidia® accelerators based on their CUDA platform.

Target maintainers

- Riccardo D'Ambrosio, <https://github.com/RDambrosio016>
- Kjetil Kjeksa, <https://github.com/kjetilkjeksa>

Designated maintainers

`powerpc-unknown-openbsd` is not maintained by OpenBSD developers and there are currently no active rustc maintainers.

powerpc64-ibm-aix

Tier: 3

Rust for AIX operating system, currently only 64-bit PowerPC is supported.

Target maintainers

- QIU Chaofan qiucofan@cn.ibm.com, <https://github.com/ecnelises>
- Kai LUO, lkail@cn.ibm.com, <https://github.com/bzEq>

Requirements

This target supports host tools, std and alloc. This target cannot be cross-compiled as for now, mainly because of the unavailability of system linker on other platforms.

Binary built for this target is expected to run on Power7 or newer CPU, and AIX 7.2 or newer version.

Binary format of this platform is [XCOFF](#). Archive file format is '[AIX big format](#)'.

Testing

This target supports running test suites natively, but it's not available to cross-compile and execute in emulator.

Interoperability with C code

This target supports C code. C code compiled by XL, Open XL and Clang are compatible with

Rust. Typical triple of AIX on 64-bit PowerPC of these compilers are also `powerpc64-ibm-aix` .

riscv32im-risc0-zkvm-elf

Tier: 3

RISC Zero's Zero Knowledge Virtual Machine (zkVM) implementing the RV32IM instruction set.

Target maintainers

- Frank Laub, frank@risczero.com, <https://github.com/flaub>
- Jeremy Bruestle, jeremy@risczero.com, <https://github.com/jbruestle>
- Erik Kaneda, erik@risczero.com, <https://github.com/SchmErik>

Background

This target is an execution environment to produce a proof of execution of a RISC-V ELF binary and any output that the developer of the binary wishes to display publicly. In order to do this, the target will execute the ELF to generate a receipt containing the output of the computation along with a cryptographic seal. This receipt can be verified to ensure the integrity of the computation and its result. This target is implemented as software only; it has no hardware implementation.

We have a cargo extension called [cargo-risczero](#) that allow users to generate project templates, install tools for improved user experience, build the binary using a docker environment and test programs.

Requirements

The target only supports cross compilation and no host tools. The target supports `alloc` with a default allocator and has experimental support for `std`. The target expects the binaries to be in ELF.

The target's execution environment is single threaded, non-preemptive, and does not support any privileged instructions, nor unaligned accesses. At the time of writing the VM has 192 MB of memory and text/data, heap, and stack need to be within the address range `0x400 - 0x0C000000`. The binaries themselves expect no operating system and can be thought of as running on bare-metal. The target does not use `#[target_feature(...)]` or `-C target-feature=` values.

Calling `extern "C"` on the target uses the C calling convention outlined in the [RISC-V specification](#).

Building for the zkVM

Programs for the zkVM could be built by adding it to the `target` list in `config.toml`. However, we recommend building programs in our starter template generated by the [cargo-risczero](#) utility and the [risc0-build](#) crate. This crate calls `rustc` with `-C "link-arg=-Ttext="` so that it maps the text in the appropriate location as well as generating variables that represent the ELF and a unique ID associated with the ELF. The starter template provides developers with system calls that are useful to zero knowledge computing such as writing to the public output, hashing using sha256, and multiply big integers.

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target. To compile for this target, you will either need to build Rust with the target enabled (see "Building the target" above). We do not recommend using `build-std` as we have run into issues building core in the past on our starter template. An alternate solution is to download the risc0 tool chain by running `cargo risczero install`.

Testing

Note: the target is implemented as a software emulator called the zkVM and there is no

hardware implementation of the target.

The most practical way to test the target program is to use our starter template that can be generated by using the `cargo risczero new` command. The template generates a sample "host" and "guest" code. The guest code compiled to the target (which is RV32IM) whereas the "host" code is compiled to run on the programmer's machine running either a Linux distribution or macOS. The host program is responsible for running the guest binary on the zkVM and retrieving its public output.

The target currently does not support running the Rust test suite.

Cross-compilation toolchains and C code

Compatible C code can be built for this target on any compiler that has a RV32IM target. On clang and ld.lld linker, it can be generated using the `-march=rv32im`, `-mabi=ilp32` with llvm features flag `features=+m` and llvm target `riscv32-unknown-none`.

riscv32imac-unknown-xous-elf

Tier: 3

Xous microkernel, message-based operating system that powers devices such as Precursor and Betrusted. The operating system is written entirely in Rust, so no additional software is required to compile programs for Xous.

Target maintainers

- [@xobs](#)

Requirements

Building the target itself requires a RISC-V compiler that is supported by `cc-rs`. For example, you can use the prebuilt [xPack](#) toolchain.

Cross-compiling programs does not require any additional software beyond the toolchain. Prebuilt versions of the toolchain are available [from Betrusted](#).

Building the target

The target can be built by enabling it for a `rustc` build.

```
[build]
target = ["riscv32imac-unknown-xous-elf"]
```

Make sure your C compiler is included in `$PATH`, then add it to the `config.toml`:

```
[target.riscv32imac-unknown-xous-elf]  
cc = "riscv-none-elf-gcc"  
ar = "riscv-none-elf-ar"
```

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target. To compile for this target, you will need to do one of the following:

- Build Rust with the target enabled (see "Building the target" above)
- Build your own copy of `core` by using `build-std` or similar
- Download a prebuilt toolchain [from Betrusted](#)

Cross-compilation

This target can be cross-compiled from any host.

Testing

Currently there is no support to run the rustc test suite for this target.

`riscv32{i,im,ima,imc,imac,imafc}-unknown-none-elf`

Tier: 2

Bare-metal target for RISC-V CPUs with the RV32I, RV32IM, RV32IMC, RV32IMAFc and RV32IMAC ISAs.

Tier: 3

Bare-metal target for RISC-V CPUs with the RV32IMA ISA.

Target maintainers

- Rust Embedded Working Group, [RISC-V team](#)

Requirements

The target is cross-compiled, and uses static linking. No external toolchain is required and the default `rust-lld` linker works, but you must specify a linker script. The [riscv-rt](#) crate provides a suitable one. The [riscv-rust-quickstart](#) repository gives an example of an RV32 project.

Building the target

This target is included in Rust and can be installed via `rustup`.

Testing

This is a cross-compiled `no-std` target, which must be run either in a simulator or by programming them onto suitable hardware. It is not possible to run the Rust test-suite on this target.

Cross-compilation toolchains and C code

This target supports C code. If interlinking with C or C++, you may need to use `riscv64-unknown-elf-gcc` as a linker instead of `rust-ld`.

sparc-unknown-none-elf

Tier: 3

Rust for bare-metal 32-bit SPARC V7 and V8 systems, e.g. the Gaisler LEON3.

Target	Descriptions
sparc-unknown-none-elf	SPARC V7 32-bit (freestanding, hardfloat)

Target maintainers

- Jonathan Pallant, jonathan.pallant@ferrous-systems.com, <https://ferrous-systems.com>

Requirements

This target is cross-compiled. There is no support for `std`. There is no default allocator, but it's possible to use `alloc` by supplying an allocator.

By default, code generated with this target should run on any SPARC hardware; enabling additional target features may raise this baseline.

- `-Ctarget-cpu=v8` adds the extra SPARC V8 instructions.
- `-Ctarget-cpu=leon3` adds the SPARC V8 instructions and sets up scheduling to suit the Gaisler Leon3.

Functions marked `extern "C"` use the [standard SPARC architecture calling convention](#).

This target generates ELF binaries. Any alternate formats or special considerations for binary layout will require linker options or linker scripts.

Building the target

You can build Rust with support for the target by adding it to the `target` list in `config.toml`:

```
[build]
build-stage = 1
host = ["<target for your host>"]
target = ["<target for your host>", "sparc-unknown-none-elf"]
```

Replace `<target for your host>` with `x86_64-unknown-linux-gnu` or whatever else is appropriate for your host machine.

Building Rust programs

To build with this target, pass it to the `--target` argument, like:

```
cargo build --target sparc-unknown-none-elf
```

This target uses GCC as a linker, and so you will need an appropriate GCC compatible `sparc-unknown-none` toolchain. The default linker binary is `sparc-elf-gcc`, but you can override this in your project configuration, as follows:

`.cargo/config.toml`:

```
[target.sparc-unknown-none-elf]
linker = "sparc-custom-elf-gcc"
```

Testing

As `sparc-unknown-none-elf` supports a variety of different environments and does not support `std`, this target does not support running the Rust test suite.

Cross-compilation toolchains and C code

This target was initially tested using [BCC2](#) from Gaisler, along with the TSIM Leon3 processor simulator. Both [BCC2](#) GCC and [BCC2](#) Clang have been shown to work. To work with these tools, your project configuration should contain something like:

```
.cargo/config.toml:
```

```
[target.sparc-unknown-none-elf]
linker = "sparc-gaisler-elf-gcc"
runner = "tsim-leon3"

[build]
target = ["sparc-unknown-none-elf"]
rustflags = ["-Ctarget-cpu=leon3"]
```

With this configuration, running `cargo run` will compile your code for the SPARC V8 compatible Gaisler Leon3 processor and then start the `tsim-leon3` simulator. The `libcore` was pre-compiled as part of the `rustc` compilation process using the SPARC V7 baseline, but if you are using a nightly toolchain you can use the `-Z build-std=core` option to rebuild `libcore` from source. This may be useful if you want to compile it for SPARC V8 and take advantage of the extra instructions.

```
.cargo/config.toml:
```

```
[target.sparc-unknown-none-elf]
linker = "sparc-gaisler-elf-gcc"
runner = "tsim-leon3"

[build]
target = ["sparc-unknown-none-elf"]
rustflags = ["-Ctarget-cpu=leon3"]

[unstable]
build-std = ["core"]
```

Either way, once the simulator is running, simply enter the command `run` to start the code executing in the simulator.

The default C toolchain libraries are linked in, so with the Gaisler [BCC2](#) toolchain, and using

its default Leon3 BSP, you can use call the C `putchar` function and friends to output to the simulator console. The default linker script is also appropriate for the Leon3 simulator, so no linker script is required.

Here's a complete example using the above config file:

```
#![no_std]
#![no_main]

extern "C" {
    fn putchar(ch: i32);
    fn _exit(code: i32) -> !;
}

#[no_mangle]
extern "C" fn main() -> i32 {
    let message = "Hello, this is Rust!";
    for b in message.bytes() {
        unsafe {
            putchar(b as i32);
        }
    }
    0
}

#[panic_handler]
fn panic(_panic: &core::panic::PanicInfo) -> ! {
    unsafe {
        _exit(1);
    }
}
```

```
$ cargo run --target=sparc-unknown-none-elf
  Compiling sparc-demo-rust v0.1.0 (/work/sparc-demo-rust)
  Finished dev [unoptimized + debuginfo] target(s) in 3.44s
  Running `tsim-leon3 target/sparc-unknown-none-elf/debug/sparc-demo-rust`
```

TSIM3 LEON3 SPARC simulator, version 3.1.9 (evaluation version)

Copyright (C) 2023, Frontgrade Gaisler - all rights reserved.
This software may only be used with a valid license.
For latest updates, go to <https://www.gaisler.com/>
Comments or bug-reports to support@gaisler.com

This TSIM evaluation version will expire 2023-11-28

```
Number of CPUs: 2
system frequency: 50.000 MHz
icache: 1 * 4 KiB, 16 bytes/line (4 KiB total)
dcache: 1 * 4 KiB, 16 bytes/line (4 KiB total)
Allocated 8192 KiB SRAM memory, in 1 bank at 0x40000000
Allocated 32 MiB SDRAM memory, in 1 bank at 0x60000000
Allocated 8192 KiB ROM memory at 0x00000000
section: .text, addr: 0x40000000, size: 20528 bytes
section: .rodata, addr: 0x40005030, size: 128 bytes
section: .data, addr: 0x400050b0, size: 1176 bytes
read 347 symbols
```

```
tsim> run
  Initializing and starting from 0x40000000
Hello, this is Rust!
```

```
  Program exited normally on CPU 0.
tsim>
```

*-pc-windows-gnullvm

Tier: 2 (without host tools)

Windows targets similar to `*-pc-windows-gnu` but using UCRT as the runtime and various LLVM tools/libraries instead of GCC/Binutils.

Target triples available so far:

- `aarch64-pc-windows-gnullvm`
- `i686-pc-windows-gnullvm`
- `x86_64-pc-windows-gnullvm`

Target maintainers

- [@mati865](#)
- [@thomcc](#)

Requirements

The easiest way to obtain these targets is cross-compilation, but native build from `x86_64-pc-windows-gnu` is possible with few hacks which I don't recommend. Std support is expected to be on par with `*-pc-windows-gnu`.

Binaries for this target should be at least on par with `*-pc-windows-gnu` in terms of requirements and functionality.

Those targets follow Windows calling convention for `extern "C"`.

Like with any other Windows target, created binaries are in PE format.

Building the target

These targets can be easily cross-compiled using [llvm-mingw](#) toolchain or [MSYS2 CLANG*](#) environments. Just fill `[target.*]` sections for both build and resulting compiler and set installation prefix in `config.toml`. Then run `./x.py install`. In my case I had ran `./x.py install --host x86_64-pc-windows-gnu --target x86_64-pc-windows-gnu` inside MSYS2 MINGW64 shell so `x86_64-pc-windows-gnu` was my build toolchain.

Native bootstrapping is doable in two ways:

- cross-compile gnullvm host toolchain and use it as build toolchain for the next build,
- copy libunwind libraries and rename them to mimic libgcc like here:
<https://github.com/msys2/MINGW-packages/blob/68e640756df2df6df6afa60f025e3f936e7b977c/mingw-w64-rust/PKGBUILD#L108-L109>, stage0 compiler will be mostly broken but good enough to build the next stage.

The second option might stop working anytime, so it's not recommended.

Building Rust programs

Rust does ship a pre-compiled std library for those targets. That means one can easily cross-compile for those targets from other hosts if C proper toolchain is installed.

Alternatively full toolchain can be built as described in the previous section.

Testing

Created binaries work fine on Windows or Wine using native hardware. Testing AArch64 on x86_64 is problematic though and requires spending some time with QEMU. Most of x86_64 testsuite does pass when cross-compiling, with exception for `rustdoc` and `ui-fulldeps` that fail with an error regarding a missing library, they do pass in native builds though. The only failing test is std's `process::tests::test_proc_thread_attributes` for unknown reason.

Cross-compilation toolchains and C code

Compatible C code can be built with Clang's `aarch64-pc-windows-gnu`, `i686-pc-windows-gnu` and `x86_64-pc-windows-gnu` targets as long as LLVM-based C toolchains are used. Those include:

- [llvm-mingw](#)
- [MSYS2 with CLANG*](#) environment

nto-qnx

Tier: 3

QNX® Neutrino (nto) Real-time operating system. The support has been implemented jointly by [Elektrobit Automotive GmbH](#) and [Blackberry QNX](#).

Target maintainers

- Florian Bartels, Florian.Bartels@elektrobit.com , <https://github.com/flba-eb>
- Tristan Roach, TRoach@blackberry.com , <https://github.com/gh-tr>
- Jonathan Pallant Jonathan.Pallant@ferrous-systems.com , <https://github.com/jonathanpallant>
- Jorge Aparicio Jorge.Aparicio@ferrous-systems.com , <https://github.com/japartic>

Requirements

Currently, the following QNX Neutrino versions and compilation targets are supported:

QNX Neutrino Version	Target Architecture	Full support	no_std support
7.1	AArch64	✓	✓
7.1	x86_64	✓	✓
7.0	x86		✓

Adding other architectures that are supported by QNX Neutrino is possible.

In the table above, 'full support' indicates support for building Rust applications with the full standard library. 'no_std support' indicates that only `core` and `alloc` are available.

For building or using the Rust toolchain for QNX Neutrino, the [QNX Software Development Platform \(SDP\)](#) must be installed and initialized. Initialization is usually done by sourcing

`qnxsdp-env.sh` (this will be installed as part of the SDP, see also installation instruction provided with the SDP). Afterwards `qcc` (QNX C/C++ compiler) should be available (in the `$PATH` variable). `qcc` will be called e.g. for linking executables.

When linking `no_std` applications, they must link against `libc.so` (see example). This is required because applications always link against the `crt` library and `crt` depends on `libc.so`. This is done automatically when using the standard library.

Small example application

Small `no_std` example is shown below. Applications using the standard library work as well.

```
#![no_std]
#![no_main]
#![feature(lang_items)]

// We must always link against libc, even if no external functions are used
// "extern C" - Block can be empty but must be present
#[link(name = "c")]
extern "C" {
    pub fn printf(format: *const core::ffi::c_char, ...) -> core::ffi::c_int;
}

#[no_mangle]
pub extern "C" fn main(_argc: isize, _argv: *const *const u8) -> isize {
    const HELLO: &'static str = "Hello World, the answer is %d\n\0";
    unsafe {
        printf(HELLO.as_ptr() as *const _, 42);
    }
    0
}

use core::panic::PanicInfo;

#[panic_handler]
fn panic(_panic: &PanicInfo<'_>) -> ! {
    loop {}
}

#[lang = "eh_personality"]
#[no_mangle]
pub extern "C" fn rust_eh_personality() {}
```

The QNX Neutrino support of Rust has been tested with QNX Neutrino 7.0 and 7.1.

There are no further known requirements.

Conditional compilation

For conditional compilation, following QNX Neutrino specific attributes are defined:

- `target_os = "nto"`
- `target_env = "nto71"` (for QNX Neutrino 7.1)
- `target_env = "nto70"` (for QNX Neutrino 7.0)

Building the target

1. Create a `config.toml`

Example content:

```
profile = "compiler"
change-id = 115898
```

2. Compile the Rust toolchain for an `x86_64-unknown-linux-gnu` host (for both `aarch64` and `x86_64` targets)

Compiling the Rust toolchain requires the same environment variables used for compiling C binaries. Refer to the [QNX developer manual](#).

To compile for QNX Neutrino (`aarch64` and `x86_64`) and Linux (`x86_64`):

```
export build_env='
  CC_aarch64-unknown-nto-qnx710=qcc
  CFLAGS_aarch64-unknown-nto-qnx710=-Vgcc_ntoaarch64le_cxx
  CXX_aarch64-unknown-nto-qnx710=qcc
  AR_aarch64-unknown-nto-qnx710=ntoaarch64-ar
  CC_x86_64-pc-nto-qnx710=qcc
  CFLAGS_x86_64-pc-nto-qnx710=-Vgcc_ntox86_64_cxx
  CXX_x86_64-pc-nto-qnx710=qcc
  AR_x86_64-pc-nto-qnx710=ntox86_64-ar'

env $build_env \
  ./x.py build \
    --target aarch64-unknown-nto-qnx710,x86_64-pc-nto-qnx710,x86_64-
unknown-linux-gnu \
    rustc library/core library/alloc library/std
```

Running the Rust test suite

The test suites of the Rust compiler and standard library can be executed much like other Rust targets. The environment for testing should match the one used during compiler compilation (refer to `build_env` and `qcc / PATH` above) with the addition of the `TEST_DEVICE_ADDR` environment variable. The `TEST_DEVICE_ADDR` variable controls the remote runner and should point to the target, despite localhost being shown in the following example. Note that some tests are failing which is why they are currently excluded by the target maintainers which can be seen in the following example.

To run all tests on a x86_64 QNX Neutrino target:

```

export TEST_DEVICE_ADDR="localhost:12345" # must address the test target, can
be a SSH tunnel
export build_env='
    CC_aarch64-unknown-nto-qnx710=qcc
    CFLAGS_aarch64-unknown-nto-qnx710=-Vgcc_ntoaarch64le_cxx
    CXX_aarch64-unknown-nto-qnx710=qcc
    AR_aarch64-unknown-nto-qnx710=ntoaarch64-ar
    CC_x86_64-pc-nto-qnx710=qcc
    CFLAGS_x86_64-pc-nto-qnx710=-Vgcc_ntox86_64_cxx
    CXX_x86_64-pc-nto-qnx710=qcc
    AR_x86_64-pc-nto-qnx710=ntox86_64-ar'

# Disable tests that only work on the host or don't make sense for this target.
# See also:
# - src/ci/docker/host-x86_64/i686-gnu/Dockerfile
# - https://rust-lang.zulipchat.com/#narrow/stream/182449-t-
  compiler.2Fhelp/topic/Running.20tests.20on.20remote.20target
# - .github/workflows/ci.yml
export exclude_tests='
    --exclude src/bootstrap
    --exclude src/tools/error_index_generator
    --exclude src/tools/linkchecker
    --exclude tests/ui-fulldeps
    --exclude rustc
    --exclude rustdoc'

env $build_env \
    ./x.py test \
        $exclude_tests \
        --stage 1 \
        --target x86_64-pc-nto-qnx710

```

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target. To compile for this target, you must either build Rust with the target enabled (see "Building the target" above), or build your own copy of `core` by using `build-std` or similar.

Testing

Compiled executables can run directly on QNX Neutrino.

Rust std library test suite

The target needs sufficient resources to execute all tests. The commands below assume that a QEMU image is used.

- Ensure that the temporary directory used by `remote-test-server` has enough free space and inodes. 5GB of free space and 40000 inodes are known to be sufficient (the test will create more than 32k files). To create a QEMU image in an empty directory, run this command inside the directory:

```
mkqnximage --type=qemu --ssh-ident=$HOME/.ssh/id_ed25519.pub --data-size=5000 --data-inodes=40000
```

`/data` should have enough free resources. Set the `TMPDIR` environment variable accordingly when running `remote-test-server`, e.g.:

```
TMPDIR=/data/tmp/rust remote-test-server --bind 0.0.0.0:12345
```

- Ensure the TCP stack can handle enough parallel connections (default is 200, should be 300 or higher). After creating an image (see above), edit the file `output/build/startup.sh`:

1. Search for `io-pkt-v6-hc`
2. Add the parameter `-ptcpip threads_max=300`, e.g.:

```
io-pkt-v6-hc -U 33:33 -d e1000 -ptcpip threads_max=300
```

3. Update the image by running `mkqnximage` again with the same parameters as above for creating it.

- Running and stopping the virtual machine

To start the virtual machine, run inside the directory of the VM:


```
mkqnximage --run=-h
```

To stop the virtual machine, run inside the directory of the VM:

```
mkqnximage --stop
```

- Ensure local networking

Ensure that 'localhost' is getting resolved to 127.0.0.1. If you can't ping the localhost, some tests may fail. Ensure it's appended to /etc/hosts (if first `ping` command fails). Commands have to be executed inside the virtual machine!

```
$ ping localhost
ping: Cannot resolve "localhost" (Host name lookup failure)
```

```
$ echo "127.0.0.1 localhost" >> /etc/hosts
```

```
$ ping localhost
PING localhost (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=255 time=1 ms
```

Cross-compilation toolchains and C code

Compiling C code requires the same environment variables to be set as compiling the Rust toolchain (see above), to ensure `qcc` is used with proper arguments. To ensure compatibility, do not specify any further arguments that for example change calling conventions or memory layout.

*-unikraft-linux-musl

Tier: 3

Targets for the [Unikraft](#) Unikernel Development Kit (with musl).

Target triplets available so far:

- `x86_64-unikraft-linux-musl`

Target maintainers

- Martin Kröning ([@mkroening](#))

Requirements

These targets only support cross-compilation. The targets do support std.

Unikraft pretends to behave exactly like Linux. How much of that functionality is available depends on the individual unikernel configuration. For example, the basic Unikraft + musl config does not support `poll` or networking out of the box. That functionality requires enabling [LIBPOSIX_EVENT](#) or [lwIP](#) respectively.

The Unikraft targets follow Linux's `extern "C"` calling convention.

For these targets, `rustc` does not perform the final linking step. Instead, the Unikraft build system will produce the final Unikernel image for the selected platform (e.g., KVM, Linux user space, and Xen).

Building the targets

You can build Rust with support for the targets by adding it to the `target` list in

config.toml:

```
[build]
build-stage = 1
target = [ "x86_64-unikraft-linux-musl" ]
```

Building Rust programs

Rust does not yet ship pre-compiled artifacts for these targets. To compile for these targets, you will either need to build Rust with the targets enabled (see “Building the targets” above), or build your own copy of `core` by using `build-std` or similar.

Linking requires a [KraftKit](#) shim. See [unikraft/kraftkit#612](#) for more information.

Testing

The targets do support running binaries in the form of unikernel images. How the unikernel image is run depends on the specific platform (e.g., KVM, Linux user space, and Xen). The targets do not support running the Rust test suite.

Cross-compilation toolchains and C code

The targets do support C code. To build compatible C code, you have to use the same compiler and flags as does the Unikraft build system for your specific configuration. The easiest way to achieve that, is to build the C code with the Unikraft build system when building your unikernel image.

*-unknown-hermit

Tier: 3

The [Hermit](#) unikernel target allows compiling your applications into self-contained, specialized unikernel images that can be run in small virtual machines.

Target triplets available so far:

- `x86_64-unknown-hermit`
- `aarch64-unknown-hermit`
- `riscv64gc-unknown-hermit`

Target maintainers

- Stefan Lankes ([@stlankes](#))
- Martin Kröning ([@mkroening](#))

Requirements

These targets only support cross-compilation. The targets do support std.

When building binaries for this target, the Hermit unikernel is built from scratch. The application developer themselves specializes the target and sets corresponding expectations.

The Hermit targets follow Linux's `extern "C"` calling convention.

Hermit binaries have the ELF format.

Building the target

You can build Rust with support for the targets by adding it to the `target` list in `config.toml`. To run the Hermit build scripts, you also have to enable your host target. The build scripts rely on `llvm-tools` and binaries are linked using `rust-ld`, so those have to be enabled as well.

```
[build]
build-stage = 1
target = [
    "<HOST_TARGET>",
    "x86_64-unknown-hermit",
    "aarch64-unknown-hermit",
    "riscv64gc-unknown-hermit",
]

[rust]
ld = true
llvm-tools = true
```

Building Rust programs

Rust does not yet ship pre-compiled artifacts for these targets. To compile for these targets, you will either need to build Rust with the targets enabled (see “Building the targets” above), or build your own copy of `core` by using `build-std` or similar.

As all Hermit programs are unikernels, building a Rust program also requires including the operating system code. A guide for doing so is provided in our starter [hermit-rs-template](#).

Testing

The targets support running binaries in the form of self-contained unikernel images. These images can be chainloaded by Hermit's [loader](#) or hypervisor ([Uhyve](#)). QEMU can be used to boot Hermit binaries using the loader on any architecture. The targets do not support running the Rust test suite.

Cross-compilation toolchains and C code

The targets do not yet support C code and Rust code at the same time.

*-unknown-netbsd

Tier: 3

NetBSD multi-platform 4.4BSD-based UNIX-like operating system.

The target names follow this format: `$ARCH-unknown-netbsd{- $\$SUFFIX$ }` , where `$ARCH` specifies the target processor architecture and `- $\$SUFFIX$` (optional) might indicate the ABI. The following targets are currently defined running NetBSD:

Target name	NetBSD Platform
<code>x86_64-unknown-netbsd</code>	amd64 / x86_64 systems
<code>armv7-unknown-netbsd-eabihf</code>	32-bit ARMv7 systems with hard-float
<code>armv6-unknown-netbsd-eabihf</code>	32-bit ARMv6 systems with hard-float
<code>aarch64-unknown-netbsd</code>	64-bit ARM systems, little-endian
<code>aarch64_be-unknown-netbsd</code>	64-bit ARM systems, big-endian
<code>i586-unknown-netbsd</code>	32-bit i386, restricted to Pentium
<code>i686-unknown-netbsd</code>	32-bit i386 with SSE
<code>mipsel-unknown-netbsd</code>	32-bit mips, requires mips32 cpu support
<code>powerpc-unknown-netbsd</code>	Various 32-bit PowerPC systems, e.g. MacPPC
<code>riscv64gc-unknown-netbsd</code>	64-bit RISC-V
<code>sparc64-unknown-netbsd</code>	Sun UltraSPARC systems

All use the "native" `stdc++` library which goes along with the natively supplied GNU C++ compiler for the given OS version. Many of the bootstraps are built for NetBSD 9.x, although some exceptions exist (some are built for NetBSD 8.x but also work on newer OS versions).

Designated Developers

- [@he32](#), he@NetBSD.org
- [NetBSD/pkgsrc-wip's rust](#) maintainer (see MAINTAINER variable). This package is part of "pkgsrc work-in-progress" and is used for deployment and testing of new versions of

rust

- [NetBSD's pkgsrc lang/rust](#) for the "proper" package in pkgsrc.
- [NetBSD's pkgsrc lang/rust-bin](#) which re-uses the bootstrap kit as a binary distribution and therefore avoids the rather protracted native build time of rust itself

Fallback to pkgsrc-users@NetBSD.org, or fault reporting via NetBSD's bug reporting system.

Requirements

The `x86_64-unknown-netbsd` artifacts is being distributed by the rust project.

The other targets are built by the designated developers (see above), and the targets are initially cross-compiled, but many if not most of them are also built natively as part of testing.

Building

The default build mode for the packages is a native build.

Cross-compilation

These targets can be cross-compiled, and we do that via the pkgsrc package(s).

Cross-compilation typically requires the "tools" and "dest" trees resulting from a normal cross-build of NetBSD itself, ref. our main build script, `build.sh`.

See e.g. [do-cross.mk](#) Makefile for the Makefile used to cross-build all the above NetBSD targets (except for the `amd64` target).

The major option for the rust build is whether to build rust with the LLVM rust carries in its distribution, or use the LLVM package installed from pkgsrc. The `PKG_OPTIONS.rust` option is `rust-internal-llvm`, ref. [the rust package's options.mk make fragment](#). It defaults to

being set for a few of the above platforms, for various reasons (see comments), but is otherwise unset and therefore indicates use of the pkgsrc LLVM.

Testing

The Rust testsuite could presumably be run natively.

For the systems where the maintainer can build natively, the rust compiler itself is re-built natively. This involves the rust compiler being re-built with the newly self-built rust compiler, so exercises the result quite extensively.

Additionally, for some systems we build `librsvg`, and for the more capable systems we build and test `firefox` (amd64, i386, aarch64).

Building Rust programs

Rust ships pre-compiled artifacts for the `x86_64-unknown-netbsd` target.

For the other systems mentioned above, using the `pkgsrc` route is probably the easiest, possibly via the `rust-bin` package to save time, see the `RUST_TYPE` variable from the `rust.mk` Makefile fragment.

The `pkgsrc` rust package has a few files to assist with building `pkgsrc` packages written in rust, ref. the `rust.mk` and `cargo.mk` Makefile fragments in the `lang/rust` package.

*-unknown-openbsd

Tier: 3

OpenBSD multi-platform 4.4BSD-based UNIX-like operating system.

The target names follow this format: `$ARCH-unknown-openbsd` , where `$ARCH` specifies the target processor architecture. The following targets are currently defined:

Target name	C++ library	OpenBSD Platform
aarch64-unknown-openbsd	libc++	64-bit ARM systems
i686-unknown-openbsd	libc++	Standard PC and clones based on the Intel i386 architecture and compatible processors
powerpc64-unknown-openbsd	libc++	IBM POWER-based PowerNV systems
riscv64gc-unknown-openbsd	libc++	64-bit RISC-V systems
sparc64-unknown-openbsd	estdc++	Sun UltraSPARC and Fujitsu SPARC64 systems
x86_64-unknown-openbsd	libc++	AMD64-based systems

Note that all OS versions are *major* even if using X.Y notation (`6.8` and `6.9` are different major versions) and could be binary incompatibles (with breaking changes).

Designated Developers

- [@semarie](#), `semarie@openbsd.org`
- [lang/rust](#) maintainer (see MAINTAINER variable)

Fallback to `ports@openbsd.org`, OpenBSD third parties public mailing-list (with `openbsd` developers readers)

Requirements

These targets are natively compiled and could be cross-compiled. C compiler toolchain is required for the purpose of building Rust and functional binaries.

Building

The target can be built by enabling it for a `rustc` build.

```
[build]
target = ["$ARCH-unknown-openbsd"]
```

```
[target.$ARCH-unknown-openbsd]
cc = "$ARCH-openbsd-cc"
```

Cross-compilation

These targets can be cross-compiled, but LLVM might not build out-of-box.

Testing

The Rust testsuite could be run natively.

Building Rust programs

Rust does not yet ship pre-compiled artifacts for these targets.

★-unknown-redox

Tier: 2/3

Targets for the [Redox OS](#) operating system.

Target triplets available so far:

- `x86_64-unknown-redox` (tier 2)
- `aarch64-unknown-redox` (tier 3)
- `i686-unknown-redox` (tier 3)

Target maintainers

- Jeremy Soller ([@jackpot51](#))

Requirements

These targets are natively compiled and can be cross-compiled. Std is fully supported.

The targets are only expected to work with the latest version of Redox OS as the ABI is not yet stable.

`extern "C"` uses the official calling convention of the respective architectures.

Redox OS binaries use ELF as file format.

Building the target

You can build Rust with support for the targets by adding it to the `target` list in `config.toml`. In addition a copy of [relibc](#) needs to be present in the linker search path.

```
[build]
build-stage = 1
target = [
    "<HOST_TARGET>",
    "x86_64-unknown-redox",
    "aarch64-unknown-redox",
    "i686-unknown-redox",
]
```

Building Rust programs and testing

Rust does not yet ship pre-compiled artifacts for Redox OS except for x86_64-unknown-redox.

The easiest way to build and test programs for Redox OS is using [redoxer](#) which sets up the required compiler toolchain for building as well as runs programs inside a Redox OS VM using QEMU.

Cross-compilation toolchains and C code

The target supports C code. Pre-compiled C toolchains can be found at <https://static.redox-os.org/toolchain/>.

★-unknown-uefi

Tier: 2

Unified Extensible Firmware Interface (UEFI) targets for application, driver, and core UEFI binaries.

Available targets:

- `aarch64-unknown-uefi`
- `i686-unknown-uefi`
- `x86_64-unknown-uefi`

Target maintainers

- David Rheinsberg ([@dvdhrm](#))
- Nicholas Bishop ([@nicholasbishop](#))

Requirements

All UEFI targets can be used as `no-std` environments via cross-compilation. Support for `std` is present, but incomplete and extremely new. `alloc` is supported if an allocator is provided by the user or if using `std`. No host tools are supported.

The UEFI environment resembles the environment for Microsoft Windows, with some minor differences. Therefore, cross-compiling for UEFI works with the same tools as cross-compiling for Windows. The target binaries are PE32+ encoded, the calling convention is different for each architecture, but matches what Windows uses (if the architecture is supported by Windows). The special `efiapi` Rust calling-convention chooses the right ABI for the target platform (`extern "C"` is incorrect on Intel targets at least). The specification has an elaborate section on the different supported calling-conventions, if more details are desired.

MMX, SSE, and other FP-units are disabled by default, to allow for compilation of core UEFI code that runs before they are set up. This can be overridden for individual compilations via rustc command-line flags. Not all firmwares correctly configure those units, though, so careful inspection is required.

As native to PE32+, binaries are position-dependent, but can be relocated at runtime if their desired location is unavailable. The code must be statically linked. Dynamic linking is not supported. Code is shared via UEFI interfaces, rather than dynamic linking. Additionally, UEFI forbids running code on anything but the boot CPU/thread, nor is interrupt-usage allowed (apart from the timer interrupt). Device drivers are required to use polling methods.

UEFI uses a single address-space to run all code in. Multiple applications can be loaded simultaneously and are dispatched via cooperative multitasking on a single stack.

By default, the UEFI targets use the `link`-flavor of the LLVM linker `lld` to link binaries into the final PE32+ file suffixed with `*.efi`. The PE subsystem is set to `EFI_APPLICATION`, but can be modified by passing `/subsystem:<...>` to the linker. Similarly, the entry-point is set to `efi_main` but can be changed via `/entry:<...>`. The panic-strategy is set to `abort`,

The UEFI specification is available online for free: [UEFI Specification Directory](#)

Building rust for UEFI targets

Rust can be built for the UEFI targets by enabling them in the `rustc` build configuration. Note that you can only build the standard libraries. The compiler and host tools currently cannot be compiled for UEFI targets. A sample configuration would be:

```
[build]
build-stage = 1
target = ["x86_64-unknown-uefi"]
```

Building Rust programs

Starting with Rust 1.67, precompiled artifacts are provided via `rustup`. For example, to use

x86_64-unknown-uefi :

```
# install cross-compile toolchain
rustup target add x86_64-unknown-uefi
# target flag may be used with any cargo or rustc command
cargo build --target x86_64-unknown-uefi
```

Building a driver

There are three types of UEFI executables: application, boot service driver, and runtime driver. All of Rust's UEFI targets default to producing applications. To build a driver instead, pass a `subsystem` linker flag with a value of `efi_boot_service_driver` or `efi_runtime_driver`.

Example:

```
# In .cargo/config.toml:
[build]
rustflags = ["-C", "link-args=/subsystem:efi_runtime_driver"]
```

Testing

UEFI applications can be copied into the ESP on any UEFI system and executed via the firmware boot menu. The `qemu` suite allows emulating UEFI systems and executing UEFI applications as well. See its documentation for details.

The `uefi-run` rust tool is a simple wrapper around `qemu` that can spawn UEFI applications in `qemu`. You can install it via `cargo install uefi-run` and execute `qemu` applications as `uefi-run ./application.efi`.

Cross-compilation toolchains and C code

There are 3 common ways to compile native C code for UEFI targets:

- Use the official SDK by Intel: [Tianocore/EDK2](#). This supports a multitude of platforms, comes with the full specification transposed into C, lots of examples and build-system integrations. This is also the only officially supported platform by Intel, and is used by many major firmware implementations. Any code compiled via the SDK is compatible to rust binaries compiled for the UEFI targets. You can link them directly into your rust binaries, or call into each other via UEFI protocols.
- Use the **GNU-EFI** suite. This approach is used by many UEFI applications in the Linux/OSS ecosystem. The GCC compiler is used to compile ELF binaries, and linked with a pre-loader that converts the ELF binary to PE32+ **at runtime**. You can combine such binaries with the rust UEFI targets only via UEFI protocols. Linking both into the same executable will fail, since one is an ELF executable, and one a PE32+. If linking to **GNU-EFI** executables is desired, you must compile your rust code natively for the same GNU target as **GNU-EFI** and use their pre-loader. This requires careful consideration about which calling-convention to use when calling into native UEFI protocols, or calling into linked **GNU-EFI** code (similar to how these differences need to be accounted for when writing **GNU-EFI** C code).
- Use native Windows targets. This means compiling your C code for the Windows platform as if it was the UEFI platform. This works for static libraries, but needs adjustments when linking into an UEFI executable. You can, however, link such static libraries seamlessly into rust code compiled for UEFI targets. Be wary of any includes that are not specifically suitable for UEFI targets (especially the C standard library includes are not always compatible). Freestanding compilations are recommended to avoid incompatibilities.

Ecosystem

The rust language has a long history of supporting UEFI targets. Many crates have been developed to provide access to UEFI protocols and make UEFI programming more ergonomic in rust. The following list is a short overview (in alphabetical ordering):

- **efi**: *Ergonomic Rust bindings for writing UEFI applications*. Provides *rustified* access to UEFI protocols, implements allocators and a safe environment to write UEFI applications.
- **r-efi**: *UEFI Reference Specification Protocol Constants and Definitions*. A pure transpose of the UEFI specification into rust. This provides the raw definitions from the specification, without any extended helpers or *rustification*. It serves as baseline to implement any

more elaborate rust UEFI layers.

- **uefi-rs**: *Safe and easy-to-use wrapper for building UEFI apps*. An elaborate library providing safe abstractions for UEFI protocols and features. It implements allocators and provides an execution environment to UEFI applications written in rust.
- **uefi-run**: *Run UEFI applications*. A small wrapper around *qemu* to spawn UEFI applications in an emulated `x86_64` machine.

Example: Freestanding

The following code is a valid UEFI application returning immediately upon execution with an exit code of 0. A panic handler is provided. This is executed by rust on panic. For simplicity, we simply end up in an infinite loop.

This example can be compiled as binary crate via `cargo` :

```
cargo build --target x86_64-unknown-uefi
```

```
#![no_main]
#![no_std]

#[panic_handler]
fn panic_handler(_info: &core::panic::PanicInfo) -> ! {
    loop {}
}

#[export_name = "efi_main"]
pub extern "C" fn main(_h: *mut core::ffi::c_void, _st: *mut core::ffi::c_void)
-> usize {
    0
}
```

Example: Hello World

This is an example UEFI application that prints "Hello World!", then waits for key input before it exits. It serves as base example how to write UEFI applications without any helper modules

other than the standalone UEFI protocol definitions provided by the `r-efi` crate.

This extends the "Freestanding" example and builds upon its setup. See there for instruction how to compile this as binary crate.

Note that UEFI uses UTF-16 strings. Since rust literals are UTF-8, we have to use an open-coded, zero-terminated, UTF-16 array as argument to `output_string()`. Similarly to the panic handler, real applications should rather use UTF-16 modules.

```

#![no_main]
#![no_std]

use r_efi::efi;

#[panic_handler]
fn panic_handler(_info: &core::panic::PanicInfo) -> ! {
    loop {}
}

#[export_name = "efi_main"]
pub extern "C" fn main(_h: efi::Handle, st: *mut efi::SystemTable) ->
efi::Status {
    let s = [
        0x0048u16, 0x0065u16, 0x006cu16, 0x006cu16, 0x006fu16, // "Hello"
        0x0020u16, // " "
        0x0057u16, 0x006fu16, 0x0072u16, 0x006cu16, 0x0064u16, // "World"
        0x0021u16, // "!"
        0x000au16, // "\n"
        0x0000u16, // NUL
    ];

    // Print "Hello World!".
    let r =
        unsafe { ((*st).con_out).output_string((*st).con_out, s.as_ptr() as
*mut efi::Char16) };
    if r.is_error() {
        return r;
    }

    // Wait for key input, by waiting on the `wait_for_key` event hook.
    let r = unsafe {
        let mut x: usize = 0;
        ((*st).boot_services).wait_for_event(1, &mut (*
(*st).con_in).wait_for_key, &mut x)
    };
    if r.is_error() {
        return r;
    }

    efi::Status::SUCCESS
}

```

Rust std for UEFI

This section contains information on how to use std on UEFI.

Build std

The building std part is pretty much the same as the official [docs](#). The linker that should be used is `rust-lld`. Here is a sample `config.toml`:

```
[rust]
lld = true
```

Then just build using `x.py`:

```
./x.py build --target x86_64-unknown-uefi --stage 1
```

Alternatively, it is possible to use the `build-std` feature. However, you must use a toolchain which has the UEFI std patches. Then just build the project using the following command:

```
cargo build --target x86_64-unknown-uefi -Zbuild-std=std,panic_abort
```

Implemented features

alloc

- Implemented using `EFI_BOOT_SERVICES.AllocatePool()` and `EFI_BOOT_SERVICES.FreePool()`.
- Passes all the tests.
- Currently uses `EfiLoaderData` as the `EFI_ALLOCATE_POOL->PoolType`.

cmath

- Provided by compiler-builtins.

env

- Just some global constants.

locks

- The provided locks should work on all standard single-threaded UEFI implementations.

os_str

- While the strings in UEFI should be valid UCS-2, in practice, many implementations just do not care and use UTF-16 strings.
- Thus, the current implementation supports full UTF-16 strings.

stdio

- Uses `Simple Text Input Protocol` and `Simple Text Output Protocol`.
- Note: UEFI uses CRLF for new line. This means Enter key is registered as CR instead of LF.

args

- Uses `EFI_LOADED_IMAGE_PROTOCOL->LoadOptions`

Example: Hello World With std

The following code features a valid UEFI application, including `stdio` and `alloc (OsString and Vec)`:

This example can be compiled as binary crate via `cargo` using the toolchain compiled from the above source (named `custom`):

```
cargo +custom build --target x86_64-unknown-uefi
```

```

#![feature(uefi_std)]

use r_efi::{efi, protocols::simple_text_output};
use std::{
    ffi::OsString,
    os::uefi::{env, ffi::OsStrExt}
};

pub fn main() {
    println!("Starting Rust Application...");

    // Use System Table Directly
    let st = env::system_table().as_ptr() as *mut efi::SystemTable;
    let mut s: Vec<u16> = OsString::from("Hello
World!\n").encode_wide().collect();
    s.push(0);
    let r =
        unsafe {
            let con_out: *mut simple_text_output::Protocol = (*st).con_out;
            let output_string: extern "efiapi" fn(_: *mut
simple_text_output::Protocol, *mut u16) -> efi::Status =
(*con_out).output_string;
            output_string(con_out, s.as_ptr() as *mut efi::Char16)
        };
    assert!(!r.is_error())
}

```

BootServices

The current implementation of std makes `BootServices` unavailable once `ExitBootServices` is called. Refer to [Runtime Drivers](#) for more information regarding how to handle switching from using physical addresses to using virtual addresses.

Note: It should be noted that it is up to the user to drop all allocated memory before `ExitBootServices` is called.

wasm32-wasip1

Tier: 2

The `wasm32-wasip1` target is a WebAssembly compilation target which assumes that the [WASIp1] (aka "WASI preview1") set of "syscalls" are available for use in the standard library. Historically this target in the Rust compiler was one of the first for WebAssembly where Rust and C code are explicitly intended to interoperate as well.

There's a bit of history to the target and current development which is also worth explaining before going much further. Historically this target was originally called `wasm32-wasi` in both rustc and Clang. This was first added to Rust in 2019. In the intervening years leading up to 2024 the WASI standard continued to be developed and was eventually "rebased" on top of the [Component Model](#). This was a large change to the WASI specification and was released as 0.2.0 ("WASIp2" colloquially) in January 2024. The previous target's name in rustc, `wasm32-wasi`, was then renamed to `wasm32-wasip1`, to avoid confusion with this new target to be added to rustc as `wasm32-wasip2`. Some more context can be found in these MCPs:

- [Rename wasm32-wasi target to wasm32-wasip1](#)
- [Smooth the renaming transition of wasm32-wasi](#)

At this point the `wasm32-wasip1` target is intended for historical compatibility with the first version of the WASI standard. As of now (January 2024) the 0.2.0 target of WASI ("WASIp2") is relatively new. The state of WASI will likely change in few years after which point this documentation will probably receive another update.

Today the `wasm32-wasip1` target will generate core WebAssembly modules which will import functions from the `wasi_snapshot_preview1` module for OS-related functionality (e.g. printing).

Target maintainers

When this target was added to the compiler platform-specific documentation here was not maintained at that time. This means that the list below is not exhaustive and there are more interested parties in this target. That being said since when this document was last updated

those interested in maintaining this target are:

- Alex Crichton, <https://github.com/alexcrichton>

Requirements

This target is cross-compiled. The target includes support for `std` itself, but not all of the standard library works. For example spawning a thread will always return an error (see the `wasm32-wasip1-threads` target for example). Another example is that spawning a process will always return an error. Operations such as opening a file, however, will be implemented by calling WASI-defined APIs.

The WASI targets for Rust are explicitly intended to interoperate with other languages compiled to WebAssembly, for example C/C++. Any ABI differences or mismatches are considered bugs that need to be fixed.

By default the WASI targets in Rust ship in `rustup` with a precompiled copy of `wasi-libc` meaning that a WebAssembly-targeting-Clang is not required to use the WASI targets from Rust. If there is no actual interoperation with C then `rustup target add wasm32-wasip1` is all that's needed to get started with WASI.

Note that this behavior can be controlled with `-Clinker` and `-Clink-self-contained`, however. By specifying `clang` as a linker and disabling the `link-self-contained` option an external version of `libc.a` can be used instead.

Building the target

To build this target first acquire a copy of `wasi-sdk`. At this time version 22 is the minimum needed.

Next configure the `WASI_SDK_PATH` environment variable to point to where this is installed. For example:

```
export WASI_SDK_PATH=/path/to/wasi-sdk-22.0
```

Next be sure to enable LLD when building Rust from source as LLVM's `wasm-ld` driver for LLD is required when linking WebAssembly code together. Rust's build system will automatically pick up any necessary binaries and programs from `WASI_SDK_PATH`.

Building Rust programs

The `wasm32-wasip1` target is shipped with rustup so users can install the target with:

```
rustup target add wasm32-wasip1
```

Note: the `wasm32-wasip1` target is new and may only be available on nightly by the time you're reading this. If `wasm32-wasip1` isn't available on stable Rust then `wasm32-wasi` should be available instead.

Rust programs can be built for that target:

```
rustc --target wasm32-wasip1 your-code.rs
```

Cross-compilation

This target can be cross-compiled from any hosts.

Testing

This target is tested in rust-lang/rust CI on all merges. A subset of tests are run in the `test-various` builder such as the UI tests and libcore tests. This can be tested locally, for example, with:

```
./x.py test --target wasm32-wasip1 tests/ui
```

Conditionally compiling code

It's recommended to conditionally compile code for this target with:

```
#[cfg(all(target_os = "wasi", target_env = "p1"))]
```

Note that the `target_env = "p1"` condition first appeared in Rust 1.80. Prior to Rust 1.80 the `target_env` condition was not set.

wasm32-wasip1-threads

Tier: 2

The `wasm32-wasip1-threads` target is a new and still (as of July 2023) an experimental target. This target is an extension to `wasm32-wasip1` target, originally known as `wasm32-wasi`. It extends the original target with a standardized set of syscalls that are intended to empower WebAssembly binaries with native multi threading capabilities.

Note: Prior to March 2024 this target was known as `wasm32-wasi-preview1-threads`, and even longer before that it was known as `wasm32-wasi-threads`.

Target maintainers

- Georgii Rylov, <https://github.com/g0djan>
- Alex Crichton, <https://github.com/alexcrichton>
- Andrew Brown, <https://github.com/abrown>
- Marcin Kolny, <https://github.com/loganek>

Requirements

This target is cross-compiled. The target supports `std` fully.

The Rust target definition here is interesting in a few ways. We want to serve two use cases here with this target:

- First, we want Rust usage of the target to be as hassle-free as possible, ideally avoiding the need to configure and install a local `wasm32-wasip1-threads` toolchain.
- Second, one of the primary use cases of LLVM's new `wasm` backend and the `wasm` support in LLD is that any compiled language can interoperate with any other. The `wasm32-wasip1-threads` target is the first with a viable C standard library and `sysroot`

common definition, so we want Rust and C/C++ code to interoperate when compiled to `wasm32-unknown-unknown`.

You'll note, however, that the two goals above are somewhat at odds with one another. To attempt to solve both use cases in one go we define a target that (ab)uses the `crt-static` target feature to indicate which one you're in.

No interop with C required

By default the `crt-static` target feature is enabled, and when enabled this means that the bundled version of `libc.a` found in `liblibc.rlib` is used. This isn't intended really for interoperation with a C because it may be the case that Rust's bundled C library is incompatible with a foreign-compiled C library. In this use case, though, we use `rust-ld` and some copied crt startup object files to ensure that you can download the wasi target for Rust and you're off to the races, no further configuration necessary. All in all, by default, no external dependencies are required. You can compile `wasm32-wasip1-threads` binaries straight out of the box. You can't, however, reliably interoperate with C code in this mode (yet).

Interop with C required

For the second goal we repurpose the `target-feature` flag, meaning that you'll need to do a few things to have C/Rust code interoperate.

1. All Rust code needs to be compiled with `-C target-feature=-crt-static`, indicating that the bundled C standard library in the Rust sysroot will not be used.
2. If you're using rustc to build a linked artifact then you'll need to specify `-C linker` to a `clang` binary that supports `wasm32-wasip1-threads` and is configured with the `wasm32-wasip1-threads` sysroot. This will cause Rust code to be linked against the `libc.a` that the specified `clang` provides.
3. If you're building a staticlib and integrating Rust code elsewhere, then compiling with `-C target-feature=-crt-static` is all you need to do.

All in all, by default, no external dependencies are required. You can compile `wasm32-wasip1-threads` binaries straight out of the box. You can't, however, reliably interoperate

with C code in this mode (yet).

Also note that at this time the `wasm32-wasip1-threads` target assumes the presence of other merged wasm proposals such as (with their LLVM feature flags):

- [Bulk memory](#) - `+bulk-memory`
- Mutable imported globals - `+mutable-globals`
- Atomics - `+atomics`

LLVM 16 is required for this target. The reason is related to linker flags: prior to LLVM 16, `--import-memory` and `--export-memory` were not allowed together. The reason both are needed is an artifact of how WASI currently does things; see <https://github.com/WebAssembly/WASI/issues/502> for more details.

The target intends to match the corresponding Clang target for its "C" ABI.

Note: due to the relatively early-days nature of this target when working with this target you may encounter LLVM bugs. If an assertion hit or a bug is found it's recommended to open an issue either with `rust-lang/rust` or ideally with LLVM itself.

Platform requirements

The runtime should support the same set of APIs as any other supported wasi target for interacting with the host environment through the WASI standard. The runtime also should have implementation of [wasi-threads proposal](#).

This target is not a stable target. This means that there are a few engines which implement the `wasi-threads` feature and if they do they're likely behind a flag, for example:

- Wasmtime - `--wasm-features=threads --wasi-modules=experimental-wasi-threads`
- [WAMR](#) - needs to be built with `WAMR_BUILD_LIB_WASI_THREADS=1`

Building the target

Users need to install or built wasi-sdk since release 20.0

<https://github.com/WebAssembly/wasi-sdk/releases/tag/wasi-sdk-20> and specify path to `wasi-root` `config.toml`

```
[target.wasm32-wasip1-threads]
wasi-root = ".../wasi-libc/sysroot"
```

After that users can build this by adding it to the `target` list in `config.toml`, or with `-Zbuild-std`.

Building Rust programs

From Rust Nightly 1.71.1 (2023-08-03) on the artifacts are shipped pre-compiled:

```
rustup target add wasm32-wasip1-threads --toolchain nightly
```

Rust programs can be built for that target:

```
rustc --target wasm32-wasip1-threads your-code.rs
```

Cross-compilation

This target can be cross-compiled from any hosts.

Testing

Currently testing is not well supported for `wasm32-wasip1-threads` and the Rust project doesn't run any tests for this target. However the UI testsuite can be run manually following this instructions:

0. Ensure [wamr](#), [wasmtime](#) or another engine that supports `wasi-threads` is installed

and can be found in the `$PATH` env variable.

1. Clone master branch.
2. Apply such [a change](#) with an engine from the step 1.
3. Run `./x.py test --target wasm32-wasip1-threads tests/ui` and save the list of failed tests.
4. Checkout branch with your changes.
5. Apply such [a change](#) with an engine from the step 1.
6. Run `./x.py test --target wasm32-wasip1-threads tests/ui` and save the list of failed tests.
7. For both lists of failed tests run `cat list | sort > sorted_list` and compare it with `diff sorted_list1 sorted_list2`.

Conditionally compiling code

It's recommended to conditionally compile code for this target with:

```
#[cfg(all(target_os = "wasi", target_env = "p1", target_feature = "atomics"))]
```

Prior to Rust 1.80 the `target_env = "p1"` key was not set. Currently the `target_feature = "atomics"` is Nightly-only. Note that the precise `#[cfg]` necessary to detect this target may change as the target becomes more stable.

wasm32-wasip2

Tier: 3

The `wasm32-wasip2` target is a new and still (as of January 2024) an experimental target. This target is an extension to `wasm32-wasip1` target, originally known as `wasm32-wasi`. It is the next evolution in the development of wasi (the [WebAssembly System Interface](#)) that uses the WebAssembly [component model](#) to allow for a standardized set of syscalls that are intended to empower WebAssembly binaries with native host capabilities.

Target maintainers

- Alex Crichton, <https://github.com/alexcrichton>
- Ryan Levick, <https://github.com/rylev>

Requirements

This target is cross-compiled. The target supports `std` fully.

Platform requirements

The WebAssembly runtime should support the wasi preview 2 API set. Runtimes also are required to support components since this target outputs a component as opposed to a core wasm module. As of the time of this writing Wasmtime 17 and above is able to run this target natively with no extra flags.

Building the target

To build this target first acquire a copy of `wasi-sdk`. At this time version 22 is the minimum needed.

Next configure the `WASI_SDK_PATH` environment variable to point to where this is installed. For example:

```
export WASI_SDK_PATH=/path/to/wasi-sdk-22.0
```

Next be sure to enable LLD when building Rust from source as LLVM's `wasm-ld` driver for LLD is required when linking WebAssembly code together. Rust's build system will automatically pick up any necessary binaries and programs from `WASI_SDK_PATH`.

Testing

This target is not tested in CI at this time. Locally it can be tested with a `wasmtime` binary in `PATH` like so:

```
./x.py test --target wasm32-wasip2 tests/ui
```

Conditionally compiling code

It's recommended to conditionally compile code for this target with:

```
#[cfg(all(target_os = "wasi", target_env = "p2"))]
```

wasm64-unknown-unknown

Tier: 3

WebAssembly target which uses 64-bit memories, relying on the [memory64](#) WebAssembly proposal.

Target maintainers

- Alex Crichton, <https://github.com/alexcrichton>

Requirements

This target is cross-compiled. The target supports `std` in the same manner as the `wasm32-unknown-unknown` target which is to say that it comes with the standard library but many I/O functions such as `std::fs` and `std::net` will simply return error. Additionally I/O operations like `println!` don't actually do anything and the prints aren't routed anywhere. This is the same as the `wasm32-unknown-unknown` target. This target comes by default with an allocator, currently [dlmalloc](#) which is [ported to rust](#).

The difference of this target with `wasm32-unknown-unknown` is that it's compiled for 64-bit memories instead of 32-bit memories. This means that `usize` is 8-bytes large as well as pointers. The tradeoff, though, is that the maximum memory size is now the full 64-bit address space instead of the 4GB as limited by the 32-bit address space for `wasm32-unknown-unknown`.

This target is not a stable target. The [memory64](#) WebAssembly proposal is still in-progress and not standardized. This means that there are not many engines which implement the `memory64` feature and if they do they're likely behind a flag, for example:

- Nodejs - `--experimental-wasm-memory64`
- Wasmtime - `--wasm-features memory64`

Also note that at this time the `wasm64-unknown-unknown` target assumes the presence of other merged wasm proposals such as (with their LLVM feature flags):

- [Bulk memory](#) - `+bulk-memory`
- Mutable imported globals - `+mutable-globals`
- [Sign-extending operations](#) - `+sign-ext`
- [Non-trapping fp-to-int operations](#) - `+nontrapping-fptoint`

The `wasm64-unknown-unknown` target intends to match the default Clang targets for its "C" ABI, which is likely to be the same as Clang's `wasm32-unknown-unknown` largely.

Note: due to the relatively early-days nature of this target when working with this target you may encounter LLVM bugs. If an assertion hit or a bug is found it's recommended to open an issue either with `rust-lang/rust` or ideally with LLVM itself.

This target does not support `panic=unwind` at this time.

Building the target

You can build Rust with support for the target by adding it to the `target` list in `config.toml`, and the target also requires `lld` to be built to work.

```
[build]
target = ["wasm64-unknown-unknown"]
```

```
[rust]
lld = true
```

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target. To compile for this target, you will either need to build Rust with the target enabled (see "Building the target" above), or

build your own copy of `std` by using `build-std` or similar.

Note that the following `cfg` directives are set for `wasm64-unknown-unknown`:

- `cfg(target_arch = "wasm64")`
- `cfg(target_family = "wasm")`

Testing

Currently testing is not well supported for `wasm64-unknown-unknown` and the Rust project doesn't run any tests for this target. Testing support sort of works but without `println!` it's not the most exciting tests to run.

Cross-compilation toolchains and C code

Compiling Rust code with C code for `wasm64-unknown-unknown` is theoretically possible, but there are no known toolchains to do this at this time. At the time of this writing there is no known "libc" for `wasm` that works with `wasm64-unknown-unknown`, which means that mixing C & Rust with this target effectively cannot be done.

*-win7-windows-msvc

Tier: 3

Windows targets continuing support of windows7.

Target maintainers

- @roblabla

Requirements

This target supports all of core, alloc, std and test. This is automatically tested every night on private infrastructure hosted by the maintainer. Host tools may also work, though those are not currently tested.

Those targets follow Windows calling convention for extern "C".

Like any other Windows target, the created binaries are in PE format.

Building the target

You can build Rust with support for the targets by adding it to the target list in config.toml:

```
[build]
build-stage = 1
target = [ "x86_64-win7-windows-msvc" ]
```

Building Rust programs

Rust does not ship pre-compiled artifacts for this target. To compile for this target, you will either need to build Rust with the target enabled (see "Building the target" above), or build your own copy by using `build-std` or similar.

Testing

Created binaries work fine on Windows or Wine using native hardware. Remote testing is possible using the `remote-test-server` described [here](#).

Cross-compilation toolchains and C code

Compatible C code can be built with either MSVC's `cl.exe` or LLVM's `clang-cl`.

Cross-compilation is possible using `clang-cl/lld-link`. It also requires the Windows SDK, which can be acquired using [xwin](#).

- Install `clang-cl` and `lld-link` on your machine, and make sure they are in your `$PATH`.
- Install `xwin`: `cargo install xwin`
- Use `xwin` to install the Windows SDK: `xwin splat --output winsdk`
- Create an `xwin-lld-link` script with the following content:

```
#!/usr/bin/env bash
set -e
XWIN=/path/to/winsdk
lld-link "$@" /libpath:$XWIN/crt/lib/x86_64
/libpath:$XWIN/sdk/lib/um/x86_64 /libpath:$XWIN/sdk/lib/ucrt/x86_64
```

- Create an `xwin-clang-cl` script with the following content:

```
#!/usr/bin/env bash
set -e
XWIN=/path/to/winsdk
clang-cl /imsvc "$XWIN/crt/include" /imsvc "$XWIN/sdk/include/ucrt" /imsvc
"$XWIN/sdk/include/um" /imsvc "$XWIN/sdk/include/shared" --target="x86_64-
pc-windows-msvc" "$@"
```

- In your config.toml, add the following lines:

```
[target.x86_64-win7-windows-msvc]
linker = "path/to/xwin-ld-link"
cc = "path/to/xwin-clang-cl"
```

You should now be able to cross-compile the Rust std, and any rust program.

x86_64-fortanix-unknown-sgx

Tier: 2

Secure enclaves using [Intel Software Guard Extensions \(SGX\)](#) based on the ABI defined by Fortanix for the [Enclave Development Platform \(EDP\)](#).

Target maintainers

The [EDP team](#) at Fortanix.

- Jethro Beekman [@jethrogb](#)
- Raoul Strackx [@raoulstrackx](#)
- Mohsen Zohrevandi [@mzohreva](#)

Requirements

The target supports `std` with a default allocator. Only cross compilation is supported.

Binaries support all CPUs that include Intel SGX. Only 64-bit mode is supported.

Not all `std` features are supported, see [Using Rust's std](#) for details.

The `extern "C"` calling convention is the System V AMD64 ABI.

The supported ABI is the [fortanix-sgx-abi](#).

The compiler output is ELF, but the native format for the platform is the SGX stream (SGXS) format. A converter like [ftxsgx-elf2sgxs](#) is needed.

Programs in SGXS format adhering to the Fortanix SGX ABI can be run with any compatible runner, such as [ftxsgx-runner](#).

See the [EDP installation guide](#) for recommendations on how to setup a development and runtime environment.

Building the target

As a tier 2 target, the target is built by the Rust project.

You can configure bootstrap like so:

```
[build]
build-stage = 1
target = ["x86_64-fortanix-unknown-sgx"]
```

Building Rust programs

Standard build flows using `cargo` or `rustc` should work.

Testing

The Rust test suite as well as custom unit and integration tests will run on hardware that has Intel SGX enabled if a cargo runner is configured correctly, see the requirements section.

Cross-compilation toolchains and C code

C code is not generally supported, as there is no `libc`. C code compiled for x86-64 in freestanding mode using the System V AMD64 ABI may work. The [rs-libc](#) crate contains a subset of `libc` that's known to work with this target.

x86_64-unknown-linux-none

Tier: 3

Freestanding x86-64 linux binary with no dependency on libc.

Target maintainers

- [morrOne](#)

Requirements

This target is cross compiled and can be built from any host.

This target has no support for host tools, std, or alloc.

Building the target

The target can be built by enabling it for a `rustc` build:

```
[build]
build-stage = 1
target = ["x86_64-unknown-linux-none"]
```

Building Rust programs

Rust does not yet ship pre-compiled artifacts for this target. To compile for this target, you will either need to build Rust with the target enabled (see "Building the target" above), or build your own copy of `core` by using `build-std` or similar.

Testing

Created binaries will run on linux without any external requirements

Cross-compilation toolchains and C code

Support for C code is currently untested

x86_64-unknown-none

Tier: 2

Freestanding/bare-metal x86-64 binaries in ELF format: firmware, kernels, etc.

Target maintainers

- Harald Hoyer harald@profian.com, <https://github.com/haraldh>
- Mike Leany, <https://github.com/mikeleany>

Requirements

This target is cross-compiled. There is no support for `std`. There is no default allocator, but it's possible to use `alloc` by supplying an allocator.

By default, Rust code generated for this target does not use any vector or floating-point registers (e.g. SSE, AVX). This allows the generated code to run in environments, such as kernels, which may need to avoid the use of such registers or which may have special considerations about the use of such registers (e.g. saving and restoring them to avoid breaking userspace code using the same registers). You can change code generation to use additional CPU features via the `-C target-feature=` codegen options to rustc, or via the `# [target_feature]` mechanism within Rust code.

By default, code generated with this target should run on any `x86_64` hardware; enabling additional target features may raise this baseline.

Code generated with this target will use the `kernel` code model by default. You can change this using the `-C code-model=` option to rustc.

On `x86_64-unknown-none`, `extern "C"` uses the [standard System V calling convention](#), without red zones.

This target generates binaries in the ELF format. Any alternate formats or special

considerations for binary layout will require linker options or linker scripts.

Building the target

You can build Rust with support for the target by adding it to the `target` list in `config.toml`:

```
[build]
build-stage = 1
target = ["x86_64-unknown-none"]
```

Building Rust programs

Starting with Rust 1.62, precompiled artifacts are provided via `rustup`:

```
# install cross-compile toolchain
rustup target add x86_64-unknown-none
# target flag may be used with any cargo or rustc command
cargo build --target x86_64-unknown-none
```

Testing

As `x86_64-unknown-none` supports a variety of different environments and does not support `std`, this target does not support running the Rust test suite.

Cross-compilation toolchains and C code

If you want to compile C code along with Rust (such as for Rust crates with C dependencies), you will need an appropriate `x86_64` toolchain.

Rust *may* be able to use an `x86_64-linux-gnu-` toolchain with appropriate standalone flags to build for this toolchain (depending on the assumptions of that toolchain, see below), or you may wish to use a separate `x86_64-unknown-none` (or `x86_64-elf-`) toolchain.

On some `x86_64` hosts that use ELF binaries, you *may* be able to use the host C toolchain, if it does not introduce assumptions about the host environment that don't match the expectations of a standalone environment. Otherwise, you may need a separate toolchain for standalone/freestanding development, just as when cross-compiling from a non-`x86_64` platform.

xtensa-*-none-elf

Tier: 3

Targets for Xtensa CPUs.

Target maintainers

- Scott Mabin [@MabezDev](#)
- Sergio Gasquez [@SergioGasquez](#)

Requirements

The target names follow this format: `xtensa- $\text{\$CPU}$` , where `$\text{\$CPU}$` specifies the target chip. The following targets are currently defined:

Target name	Target CPU(s)
<code>xtensa-esp32-none-elf</code>	ESP32
<code>xtensa-esp32s2-none-elf</code>	ESP32-S2
<code>xtensa-esp32s3-none-elf</code>	ESP32-S3

Xtensa targets that support `std` are documented in the [ESP-IDF platform support document](#)

Building the targets

The targets can be built by installing the [Xtensa enabled Rust channel](#). See instructions in the [RISC-V and Xtensa Targets](#) section of the [The Rust on ESP Book](#).

Targets

`rustc` is a cross-compiler by default. This means that you can use any compiler to build for any architecture. The list of *targets* are the possible architectures that you can build for. See the [Platform Support](#) page for a detailed list of targets, or [Built-in Targets](#) for instructions on how to view what is available for your version of `rustc`.

To see all the options that you can set with a target, see the docs [here](#).

To compile to a particular target, use the `--target` flag:

```
$ rustc src/main.rs --target=wasm32-unknown-unknown
```

Target Features

`x86`, and `ARMv8` are two popular CPU architectures. Their instruction sets form a common baseline across most CPUs. However, some CPUs extend these with custom instruction sets, e.g. vector (`AVX`), bitwise manipulation (`BMI`) or cryptographic (`AES`).

Developers, who know on which CPUs their compiled code is going to run can choose to add (or remove) CPU specific instruction sets via the `-C target-feature=val` flag.

Please note, that this flag is generally considered as unsafe. More details can be found in [this section](#).

Built-in Targets

`rustc` ships with the ability to compile to many targets automatically, we call these "built-in" targets, and they generally correspond to targets that the team is supporting directly. To see the list of built-in targets, you can run `rustc --print target-list`.

Typically, a target needs a compiled copy of the Rust standard library to work. If using [rustup](#), then check out the documentation on [Cross-compilation](#) on how to download a pre-built standard library built by the official Rust distributions. Most targets will need a system linker, and possibly other things.

Custom Targets

If you'd like to build for a target that is not yet supported by `rustc`, you can use a "custom target specification" to define a target. These target specification files are JSON. To see the JSON for the host target, you can run:

```
rustc +nightly -Z unstable-options --print target-spec-json
```

To see it for a different target, add the `--target` flag:

```
rustc +nightly -Z unstable-options --target=wasm32-unknown-unknown --print  
target-spec-json
```

To use a custom target, see the (unstable) [build-std](#) feature of `cargo`.

Known Issues

This section informs you about known "gotchas". Keep in mind, that this section is (and always will be) incomplete. For suggestions and amendments, feel free to [contribute](#) to this guide.

Target Features

Most target-feature problems arise, when mixing code that have the target-feature *enabled* with code that have it *disabled*. If you want to avoid undefined behavior, it is recommended to build *all code* (including the standard library and imported crates) with a common set of target-features.

By default, compiling your code with the `-C target-feature` flag will not recompile the entire standard library and/or imported crates with matching target features. Therefore, target features are generally considered as unsafe. Using `#[target_feature]` on individual functions makes the function unsafe.

Examples:

Target-Feature	Issue	Seen on	Description	Details
<code>+soft-float</code> and <code>-sse</code>	Segfaults and ABI mismatches	x86 and x86-64	The x86 and x86_64 architecture uses SSE registers (aka <code>xmm</code>) for floating point operations. Using software emulated floats ("soft-floats") disables usage of <code>xmm</code> registers, but parts of Rust's core libraries (e.g. <code>std::f32</code> or <code>std::f64</code>) are compiled without soft-	#63466

			floats and expect parameters to be passed in <code>xmm</code> registers. This leads to ABI mismatches. Attempting to compile with disabled SSE causes the same error, too.	
--	--	--	--	--

Profile-guided Optimization

`rustc` supports doing profile-guided optimization (PGO). This chapter describes what PGO is, what it is good for, and how it can be used.

What Is Profiled-Guided Optimization?

The basic concept of PGO is to collect data about the typical execution of a program (e.g. which branches it is likely to take) and then use this data to inform optimizations such as inlining, machine-code layout, register allocation, etc.

There are different ways of collecting data about a program's execution. One is to run the program inside a profiler (such as `perf`) and another is to create an instrumented binary, that is, a binary that has data collection built into it, and run that. The latter usually provides more accurate data and it is also what is supported by `rustc`.

Usage

Generating a PGO-optimized program involves following a workflow with four steps:

1. Compile the program with instrumentation enabled (e.g. `rustc -Cprofile-generate=/tmp/pgo-data main.rs`)
2. Run the instrumented program (e.g. `./main`) which generates a `default_<id>.profrac` file
3. Convert the `.profrac` file into a `.profdata` file using LLVM's `llvm-profdata` tool
4. Compile the program again, this time making use of the profiling data (for example `rustc -Cprofile-use=merged.profdata main.rs`)

An instrumented program will create one or more `.profrac` files, one for each instrumented binary. E.g. an instrumented executable that loads two instrumented dynamic libraries at runtime will generate three `.profrac` files. Running an instrumented binary multiple times, on the other hand, will re-use the respective `.profrac` files, updating them

in place.

These `.profrac` files have to be post-processed before they can be fed back into the compiler. This is done by the `llvm-profdata` tool. This tool is most easily installed via

```
rustup component add llvm-tools-preview
```

Note that installing the `llvm-tools-preview` component won't add `llvm-profdata` to the `PATH`. Rather, the tool can be found in:

```
~/.rustup/toolchains/<toolchain>/lib/rustlib/<target-triple>/bin/
```

Alternatively, an `llvm-profdata` coming with a recent LLVM or Clang version usually works too.

The `llvm-profdata` tool merges multiple `.profrac` files into a single `.profdata` file that can then be fed back into the compiler via `-Cprofile-use`:

```
# STEP 1: Compile the binary with instrumentation
rustc -Cprofile-generate=/tmp/pgo-data -O ./main.rs

# STEP 2: Run the binary a few times, maybe with common sets of args.
#         Each run will create or update `.profrac` files in /tmp/pgo-data
./main mydata1.csv
./main mydata2.csv
./main mydata3.csv

# STEP 3: Merge and post-process all the `.profrac` files in /tmp/pgo-data
llvm-profdata merge -o ./merged.profdata /tmp/pgo-data

# STEP 4: Use the merged `.profdata` file during optimization. All `rustc`
#         flags have to be the same.
rustc -Cprofile-use=./merged.profdata -O ./main.rs
```

A Complete Cargo Workflow

Using this feature with Cargo works very similar to using it with `rustc` directly. Again, we generate an instrumented binary, run it to produce data, merge the data, and feed it back into the compiler. Some things of note:

- We use the `RUSTFLAGS` environment variable in order to pass the PGO compiler flags to the compilation of all crates in the program.
- We pass the `--target` flag to Cargo, which prevents the `RUSTFLAGS` arguments to be passed to Cargo build scripts. We don't want the build scripts to generate a bunch of `.profraw` files.
- We pass `--release` to Cargo because that's where PGO makes the most sense. In theory, PGO can also be done on debug builds but there is little reason to do so.
- It is recommended to use *absolute paths* for the argument of `-Cprofile-generate` and `-Cprofile-use`. Cargo can invoke `rustc` with varying working directories, meaning that `rustc` will not be able to find the supplied `.profdata` file. With absolute paths this is not an issue.
- It is good practice to make sure that there is no left-over profiling data from previous compilation sessions. Just deleting the directory is a simple way of doing so (see `STEP 0` below).

This is what the entire workflow looks like:

```
# STEP 0: Make sure there is no left-over profiling data from previous runs
rm -rf /tmp/pgo-data
```

```
# STEP 1: Build the instrumented binaries
RUSTFLAGS="-Cprofile-generate=/tmp/pgo-data" \
  cargo build --release --target=x86_64-unknown-linux-gnu
```

```
# STEP 2: Run the instrumented binaries with some typical data
./target/x86_64-unknown-linux-gnu/release/myprogram mydata1.csv
./target/x86_64-unknown-linux-gnu/release/myprogram mydata2.csv
./target/x86_64-unknown-linux-gnu/release/myprogram mydata3.csv
```

```
# STEP 3: Merge the `.profraw` files into a `.profdata` file
llvm-profdata merge -o /tmp/pgo-data/merged.profdata /tmp/pgo-data
```

```
# STEP 4: Use the `.profdata` file for guiding optimizations
RUSTFLAGS="-Cprofile-use=/tmp/pgo-data/merged.profdata" \
  cargo build --release --target=x86_64-unknown-linux-gnu
```


Troubleshooting

- It is recommended to pass `-Cllvm-args=-pgo-warn-missing-function` during the `-fprofile-use` phase. LLVM by default does not warn if it cannot find profiling data for a given function. Enabling this warning will make it easier to spot errors in your setup.
- There is a [known issue](#) in Cargo prior to version 1.39 that will prevent PGO from working correctly. Be sure to use Cargo 1.39 or newer when doing PGO.

Further Reading

`rustc`'s PGO support relies entirely on LLVM's implementation of the feature and is equivalent to what Clang offers via the `-fprofile-generate` / `-fprofile-use` flags. The [Profile Guided Optimization](#) section in Clang's documentation is therefore an interesting read for anyone who wants to use PGO with Rust.

Community Maintained Tools

As an alternative to directly using the compiler for Profile-Guided Optimization, you may choose to go with `cargo-pgo`, which has an intuitive command-line API and saves you the trouble of doing all the manual work. You can read more about it in their repository accessible from this link: <https://github.com/Kobzol/cargo-pgo>

For the sake of completeness, here are the corresponding steps using `cargo-pgo`:

```
# Install if you haven't already
cargo install cargo-pgo

cargo pgo build
cargo pgo optimize
```

These steps will do the following just as before:

1. Build an instrumented binary from the source code.
2. Run the instrumented binary to gather PGO profiles.

-
-
3. Use the gathered PGO profiles from the last step to build an optimized binary.

Instrumentation-based Code Coverage

Introduction

The Rust compiler includes two code coverage implementations:

- A GCC-compatible, gcov-based coverage implementation, enabled with `-Z profile`, which derives coverage data based on `DebugInfo`.
- A source-based code coverage implementation, enabled with `-C instrument-coverage`, which uses LLVM's native, efficient coverage instrumentation to generate very precise coverage data.

This document describes how to enable and use the LLVM instrumentation-based coverage, via the `-C instrument-coverage` compiler flag.

How it works

When `-C instrument-coverage` is enabled, the Rust compiler enhances rust-based libraries and binaries by:

- Automatically injecting calls to an LLVM intrinsic (`llvm.instrprof.increment`), at functions and branches in compiled code, to increment counters when conditional sections of code are executed.
- Embedding additional information in the data section of each library and binary (using the [LLVM Code Coverage Mapping Format Version 5](#), if compiling with LLVM 12, or *Version 6*, if compiling with LLVM 13 or higher), to define the code regions (start and end positions in the source code) being counted.

When running a coverage-instrumented program, the counter values are written to a `profw` file at program termination. LLVM bundles tools that read the counter results, combine those results with the coverage map (embedded in the program binary), and generate coverage reports in multiple formats.

Note: `-C instrument-coverage` also automatically enables `-C symbol-mangling-version=v0` (tracking issue [#60705](#)). The `v0` symbol mangler is strongly recommended. The `v0` demangler can be overridden by explicitly adding `-Z unstable-options -C symbol-mangling-version=legacy`.

Enable coverage profiling in the Rust compiler

Rust's source-based code coverage requires the Rust "profiler runtime". Without it, compiling with `-C instrument-coverage` generates an error that the profiler runtime is missing.

The Rust `nightly` distribution channel includes the profiler runtime, by default.

Important: If you are building the Rust compiler from the source distribution, the profiler runtime is *not* enabled in the default `config.example.toml`. Edit your `config.toml` file and ensure the `profiler` feature is set it to `true` (either under the `[build]` section, or under the settings for an individual `[target.<triple>]`):

```
# Build the profiler runtime (required when compiling with options that
# depend
# on this runtime, such as `'-C profile-generate' or `'-C instrument-
# coverage`').
profiler = true
```

Building the demangler

LLVM coverage reporting tools generate results that can include function names and other symbol references, and the raw coverage results report symbols using the compiler's "mangled" version of the symbol names, which can be difficult to interpret. To work around this issue, LLVM coverage tools also support a user-specified symbol name demangler.

One option for a Rust demangler is [rustfilt](#), which can be installed with:

```
cargo install rustfilt
```

Compiling with coverage enabled

Set the `-C instrument-coverage` compiler flag in order to enable LLVM source-based code coverage profiling.

The default option generates coverage for all functions, including unused (never called) functions and generics. The compiler flag supports an optional value to tailor this behavior. (See `-C instrument-coverage=<options>`, below.)

With `cargo`, you can instrument your program binary *and* dependencies at the same time.

For example (if your project's Cargo.toml builds a binary by default):

```
$ cd your-project
$ cargo clean
$ RUSTFLAGS="-C instrument-coverage" cargo build
```

If `cargo` is not configured to use your profiler-enabled version of `rustc`, set the path explicitly via the `RUSTC` environment variable. Here is another example, using a `stage1` build of `rustc` to compile an `example` binary (from the `json5format` crate):

```
$ RUSTC=$HOME/rust/build/x86_64-unknown-linux-gnu/stage1/bin/rustc \
  RUSTFLAGS="-C instrument-coverage" \
  cargo build --example formatjson5
```

Note: that some compiler options, combined with `-C instrument-coverage`, can produce LLVM IR and/or linked binaries that are incompatible with LLVM coverage maps. For example, coverage requires references to actual functions in LLVM IR. If any covered function is optimized out, the coverage tools may not be able to process the coverage results. If you need to pass additional options, with coverage enabled, test them early, to confirm you will get the coverage results you expect.

Running the instrumented binary to generate raw coverage profiling data

In the previous example, `cargo` generated the coverage-instrumented binary `formatjson5` :

```
$ echo "{some: 'thing'}" | target/debug/examples/formatjson5 -
```

```
{
  some: "thing",
}
```

After running this program, a new file named like `default_11699812450447639123_0_20944` should be in the current working directory. A new, unique file name will be generated each time the program is run to avoid overwriting previous data.

```
$ echo "{some: 'thing'}" | target/debug/examples/formatjson5 -
...
$ ls default_*.profrac
default_11699812450447639123_0_20944.profrac
```

You can also set a specific file name or path for the generated `.profrac` files by using the environment variable `LLVM_PROFILE_FILE` :

```
$ echo "{some: 'thing'}" \
  | LLVM_PROFILE_FILE="formatjson5.profrac" target/debug/examples/formatjson5
-
...
$ ls formatjson5.profrac
formatjson5.profrac
```

If `LLVM_PROFILE_FILE` contains a path to a nonexistent directory, the missing directory structure will be created. Additionally, the following special pattern strings are rewritten:

- `%p` - The process ID.
- `%h` - The hostname of the machine running the program.
- `%t` - The value of the `TMPDIR` environment variable.
- `%Nm` - the instrumented binary's signature: The runtime creates a pool of `N` raw profiles, used for on-line profile merging. The runtime takes care of selecting a raw

profile from the pool, locking it, and updating it before the program exits. `N` must be between 1 and 9, and defaults to 1 if omitted (with simply `%m`).

- `%c` - Does not add anything to the filename, but enables a mode (on some platforms, including Darwin) in which profile counter updates are continuously synced to a file. This means that if the instrumented program crashes, or is killed by a signal, perfect coverage information can still be recovered.

In the first example above, the value `11699812450447639123_0` in the generated filename is the instrumented binary's signature, which replaced the `%m` pattern and the value `20944` is the process ID of the binary being executed.

Installing LLVM coverage tools

LLVM's supplies two tools—`llvm-profdata` and `llvm-cov`—that process coverage data and generate reports. There are several ways to find and/or install these tools, but note that the coverage mapping data generated by the Rust compiler requires LLVM version 12 or higher, and processing the *raw* data may require exactly the LLVM version used by the compiler. (`llvm-cov --version` typically shows the tool's LLVM version number, and `rustc --verbose --version` shows the version of LLVM used by the Rust compiler.)

- You can install compatible versions of these tools via the `rustup` component `llvm-tools-preview`. This component is the recommended path, though the specific tools available and their interface is not currently subject to Rust's usual stability guarantees. In this case, you may also find `cargo-binutils` useful as a wrapper around these tools.
- You can install a compatible version of LLVM tools from your operating system distribution, or from your distribution of LLVM.
- If you are building the Rust compiler from source, you can optionally use the bundled LLVM tools, built from source. Those tool binaries can typically be found in your build platform directory at something like: `rust/build/x86_64-unknown-linux-gnu/llvm/bin/llvm-*`.

The examples in this document show how to use the `llvm` tools directly.

Creating coverage reports

Raw profiles have to be indexed before they can be used to generate coverage reports. This is done using `llvm-profdata merge`, which can combine multiple raw profiles and index them at the same time:

```
$ llvm-profdata merge -sparse formatjson5.profrac -o formatjson5.profdata
```

Finally, the `.profdata` file is used, in combination with the coverage map (from the program binary) to generate coverage reports using `llvm-cov report`, for a coverage summaries; and `llvm-cov show`, to see detailed coverage of lines and regions (character ranges) overlaid on the original source code.

These commands have several display and filtering options. For example:

```
$ llvm-cov show -Xdemangler=rustfilt target/debug/examples/formatjson5 \
  -instr-profile=formatjson5.profdata \
  -show-line-counts-or-regions \
  -show-instantiations \
  -name=add_quoted_string
```



```

<json5format::parser::Parser>::add_quoted_string:
439| 110|         match captured {
440| 110|             Some(unquoted) => {
441| 110|                 if self.is_in_object()
442| 110|                     && !self.with_object(|object| object.has_pending_property()) {
443|     |                     ^34                                     ^34^0
444| 0|                     {
445| 0|                         let captured = self.colon_capturer.capture(self.remaining);
446| 0|                         if self.consume_if_matched(captured) {
447| 0|                             if matches_unquoted_property_name(&unquoted) {
448| 0|                                 self.set_pending_property(unquoted);
449| 0|                             } else {
450| 0|                                 self.set_pending_property(&format!("{}", quote, &unquoted, quote));
451| 0|                             }
452| 0|                         } else {
453| 0|                             return Err(self.error("Property name separator (:) missing"));
454| 0|                         }
455| 110|                     } else {
456| 110|                         let comments = self.take_pending_comments();
457| 110|                         self.add_value(Primitive::new(
458| 110|                             format!("{}", quote, &unquoted, quote),
459| 110|                             comments,
460| 0|                         ))
461| 0|                     }
462| 0|                 }
463| 110|             }
464| 110|         }
465| 34|         && !self.with_object(|object| object.has_pending_property())?
<json5format::parser::Parser>::with_object::<<json5format::parser::Parser>::add_quoted_string::<closure#0>, bool>:
290| 34|         match &mut *self.current_scope().borrow_mut() {
291| 34|             Value::Object { val, .. } => f(val),
292| 0|             unexpected => Err(self.error(format!(
293| 0|                 "Invalid Object token found while parsing an {:?} (mismatched braces?)",
294| 0|                 unexpected
295| 0|             ))),
296| 0|         }
297| 34|     }

```

Some of the more notable options in this example include:

- `--Xdemangler=rustfilt` - the command name or path used to demangle Rust symbols (rustfilt in the example)
- `target/debug/examples/formatjson5` - the instrumented binary (from which to extract the coverage map)
- `--instr-profile=<path-to-file>.profdata` - the location of the .profdata file created by `llvm-profdata merge` (from the .profrw file generated by the instrumented binary)
- `--name=<exact-function-name>` - to show coverage for a specific function (or, consider using another filter option, such as `--name-regex=<pattern>`)

Note: Coverage can also be disabled on an individual function by annotating the function with the `[coverage(off)]` attribute (which requires the feature flag `#!`)

```
[feature(coverage)] ).
```

Interpreting reports

There are four statistics tracked in a coverage summary:

- Function coverage is the percentage of functions that have been executed at least once. A function is considered to be executed if any of its instantiations are executed.
- Instantiation coverage is the percentage of function instantiations that have been executed at least once. Generic functions and functions generated from macros are two kinds of functions that may have multiple instantiations.
- Line coverage is the percentage of code lines that have been executed at least once. Only executable lines within function bodies are considered to be code lines.
- Region coverage is the percentage of code regions that have been executed at least once. A code region may span multiple lines: for example, in a large function body with no control flow. In other cases, a single line can contain multiple code regions: `return x || (y && z)` has countable code regions for `x` (which may resolve the expression, if `x` is `true`), `|| (y && z)` (executed only if `x` was `false`), and `return` (executed in either situation).

Of these four statistics, function coverage is usually the least granular while region coverage is the most granular. The project-wide totals for each statistic are listed in the summary.

Test coverage

A typical use case for coverage analysis is test coverage. Rust's source-based coverage tools can both measure your tests' code coverage as percentage, and pinpoint functions and branches not tested.

The following example (using the [json5format](#) crate, for demonstration purposes) show how to generate and analyze coverage results for all tests in a crate.

Since `cargo test` both builds and runs the tests, we set the additional `RUSTFLAGS`, to add

the `-C instrument-coverage` flag.

```
$ RUSTFLAGS="-C instrument-coverage" \  
  cargo test --tests
```

Note: The default for `LLVM_PROFILE_FILE` is `default_%m_%p.profraw`. Versions prior to 1.65 had a default of `default.profraw`, so if using those earlier versions, it is recommended to explicitly set `LLVM_PROFILE_FILE="default_%m_%p.profraw"` to avoid having multiple tests overwrite the `.profraw` files.

Make note of the test binary file paths, displayed after the word "Running" in the test output:

```
...  
Compiling json5format v0.1.3 ($HOME/json5format)  
Finished test [unoptimized + debuginfo] target(s) in 14.60s  
  
    Running target/debug/deps/json5format-fececd4653271682  
running 25 tests  
...  
test result: ok. 25 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out  
  
    Running target/debug/deps/lib-30768f9c53506dc5  
running 31 tests  
...  
test result: ok. 31 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

You should have one or more `.profraw` files now, one for each test binary. Run the `profdata` tool to merge them:

```
$ llvm-profdata merge -sparse default_*.profraw -o json5format.profdata
```

Then run the `cov` tool, with the `profdata` file and all test binaries:

```
$ llvm-cov report \  
  --use-color --ignore-filename-regex='/.cargo/registry' \  
  --instr-profile=json5format.profddata \  
  --object target/debug/deps/lib-30768f9c53506dc5 \  
  --object target/debug/deps/json5format-fececd4653271682  
$ llvm-cov show \  
  --use-color --ignore-filename-regex='/.cargo/registry' \  
  --instr-profile=json5format.profddata \  
  --object target/debug/deps/lib-30768f9c53506dc5 \  
  --object target/debug/deps/json5format-fececd4653271682 \  
  --show-instantiations --show-line-counts-or-regions \  
  --Xdemangler=rustfilt | less -R
```

Note: If overriding the default `profdraw` file name via the `LLVM_PROFILE_FILE` environment variable, it's highly recommended to use the `%m` and `%p` special pattern strings to generate unique file names in the case of more than a single test binary being executed.

Note: The command line option `--ignore-filename-regex=/.cargo/registry`, which excludes the sources for dependencies from the coverage results.

Tips for listing the binaries automatically

For `bash` users, one suggested way to automatically complete the `cov` command with the list of binaries is with a command like:

```
$ llvm-cov report \
  $( \
    for file in \
      $( \
        RUSTFLAGS="-C instrument-coverage" \
        cargo test --tests --no-run --message-format=json \
          | jq -r "select(.profile.test == true) | .filenames[]" \
          | grep -v dSYM - \
        ); \
    do \
      printf "%s %s " -object $file; \
    done \
  ) \
  --instr-profile=json5format.profdata --summary-only # and/or other options
```

Adding `--no-run --message-format=json` to the *same* `cargo test` command used to run the tests (including the same environment variables and flags) generates output in a JSON format that `jq` can easily query.

The `printf` command takes this list and generates the `--object <binary>` arguments for each listed test binary.

Including doc tests

The previous examples run `cargo test` with `--tests`, which excludes doc tests.¹

To include doc tests in the coverage results, drop the `--tests` flag, and apply the `-c instrument-coverage` flag, and some doc-test-specific options in the `RUSTDOCFLAGS` environment variable. (The `llvm-profdata` command does not change.)

```
$ RUSTFLAGS="-C instrument-coverage" \
  RUSTDOCFLAGS="-C instrument-coverage -Z unstable-options --persist-doctests \
  target/debug/doctestbins" \
  cargo test
$ llvm-profdata merge -sparse default_*.profdump -o json5format.profdump
```

The `-Z unstable-options --persist-doctests` flag is required, to save the test binaries (with their coverage maps) for `llvm-cov`.

```
$ llvm-cov report \
  $( \
    for file in \
      $( \
        RUSTFLAGS="-C instrument-coverage" \
        RUSTDOCFLAGS="-C instrument-coverage -Z unstable-options --persist-
doctests target/debug/doctestbins" \
        cargo test --no-run --message-format=json \
          | jq -r "select(.profile.test == true) | .filenames[]" \
          | grep -v dSYM - \
        ) \
        target/debug/doctestbins/*/rust_out; \
    do \
      [[ -x $file ]] && printf "%s %s " -object $file; \
    done \
  ) \
  --instr-profile=json5format.profdata --summary-only # and/or other options
```

Note: The differences in this `llvm-cov` invocation, compared with the version without doc tests, include:

- The `cargo test ... --no-run` command is updated with the same environment variables and flags used to *build* the tests, *including* the doc tests.
- The file glob pattern `target/debug/doctestbins/*/rust_out` adds the `rust_out` binaries generated for doc tests (note, however, that some `rust_out` files may not be executable binaries).
- `[[ -x $file ]] &&` filters the files passed on to the `printf`, to include only executable binaries.

¹ There is ongoing work to resolve a known issue ([#79417](#)) that doc test coverage generates incorrect source line numbers in `llvm-cov show` results.

-C instrument-coverage=<options>

- `-C instrument-coverage=no` (or `n` / `off` / `false`): Don't enable coverage instrumentation. No functions will be instrumented for coverage.

- This is the same as not using the `-C instrument-coverage` flag at all.
- `-C instrument-coverage=yes` (or `y / on / true`): Enable coverage instrumentation with the default behaviour. Currently this instruments all functions, including unused functions and unused generics.
 - This is the same as `-C instrument-coverage` with no value.

Other values

- `-C instrument-coverage=all`: Currently an alias for `yes`, but may behave differently in the future if more fine-grained coverage options are added. Using this value is currently not recommended.

`-Z coverage-options=<options>`

This unstable option is described in the [Unstable Book](#).

Other references

Rust's implementation and workflow for source-based code coverage is based on the same library and tools used to implement [source-based code coverage in Clang](#). (This document is partially based on the Clang guide.)

Linker-plugin-based LTO

The `-C linker-plugin-lto` flag allows for deferring the LTO optimization to the actual linking step, which in turn allows for performing interprocedural optimizations across programming language boundaries if all the object files being linked were created by LLVM based toolchains. The prime example here would be linking Rust code together with Clang-compiled C/C++ code.

Usage

There are two main cases how linker plugin based LTO can be used:

- compiling a Rust `staticlib` that is used as a C ABI dependency
- compiling a Rust binary where `rustc` invokes the linker

In both cases the Rust code has to be compiled with `-C linker-plugin-lto` and the C/C++ code with `-flto` or `-flto=thin` so that object files are emitted as LLVM bitcode.

Rust `staticlib` as dependency in C/C++ program

In this case the Rust compiler just has to make sure that the object files in the `staticlib` are in the right format. For linking, a linker with the LLVM plugin must be used (e.g. LLD).

Using `rustc` directly:

```
# Compile the Rust staticlib
rustc --crate-type=staticlib -Clinker-plugin-lto -Copt-level=2 ./lib.rs
# Compile the C code with '-flto=thin'
clang -c -O2 -flto=thin -o cmain.o ./cmain.c
# Link everything, making sure that we use an appropriate linker
clang -flto=thin -fuse-ld=lld -L . -l"name-of-your-rust-lib" -o main -O2
./cmain.o
```

Using `cargo`:


```
# Compile the Rust staticlib
RUSTFLAGS="-Clinker-plugin-lto" cargo build --release
# Compile the C code with '-flto=thin'
clang -c -O2 -flto=thin -o cmain.o ./cmain.c
# Link everything, making sure that we use an appropriate linker
clang -flto=thin -fuse-ld=lld -L . -l"name-of-your-rust-lib" -o main -O2
./cmain.o
```

C/C++ code as a dependency in Rust

In this case the linker will be invoked by `rustc`. We again have to make sure that an appropriate linker is used.

Using `rustc` directly:

```
# Compile C code with '-flto'
clang ./clib.c -flto=thin -c -o ./clib.o -O2
# Create a static library from the C code
ar crus ./libxyz.a ./clib.o

# Invoke 'rustc' with the additional arguments
rustc -Clinker-plugin-lto -L. -Copt-level=2 -Clinker=clang -Clink-arg=-fuse-ld=lld ./main.rs
```

Using `cargo` directly:

```
# Compile C code with '-flto'
clang ./clib.c -flto=thin -c -o ./clib.o -O2
# Create a static library from the C code
ar crus ./libxyz.a ./clib.o

# Set the linking arguments via RUSTFLAGS
RUSTFLAGS="-Clinker-plugin-lto -Clinker=clang -Clink-arg=-fuse-ld=lld" cargo
build --release
```

Explicitly specifying the linker plugin to be used by `rustc`

If one wants to use a linker other than LLD, the LLVM linker plugin has to be specified

explicitly. Otherwise the linker cannot read the object files. The path to the plugin is passed as an argument to the `-Clinker-plugin-lto` option:

```
rustc -Clinker-plugin-lto="/path/to/LLVMgold.so" -L. -Copt-level=2 ./main.rs
```

Usage with clang-cl and x86_64-pc-windows-msvc

Cross language LTO can be used with the `x86_64-pc-windows-msvc` target, but this requires using the `clang-cl` compiler instead of the MSVC `cl.exe` included with Visual Studio Build Tools, and linking with `lld-link`. Both `clang-cl` and `lld-link` can be downloaded from [LLVM's download page](#). Note that most crates in the ecosystem are likely to assume you are using `cl.exe` if using this target and that some things, like for example `vcpkg`, [don't work very well with clang-cl](#).

You will want to make sure your rust major LLVM version matches your installed LLVM tooling version, otherwise it is likely you will get linker errors:

```
rustc -V --verbose  
clang-cl --version
```

If you are compiling any proc-macros, you will get this error:

```
error: Linker plugin based LTO is not supported together with `-C prefer-  
dynamic` when  
targeting Windows-like targets
```

This is fixed if you explicitly set the target, for example `cargo build --target x86_64-pc-windows-msvc`. Without an explicit `--target` the flags will be passed to all compiler invocations (including build scripts and proc macros), see [cargo docs on rustflags](#)

If you have dependencies using the `cc` crate, you will need to set these environment variables:

```
set CC=clang-cl
set CXX=clang-cl
set CFLAGS=/clang:-flto=thin /clang:-fuse-ld=lld-link
set CXXFLAGS=/clang:-flto=thin /clang:-fuse-ld=lld-link
REM Needed because msvc's lib.exe crashes on LLVM LTO .obj files
set AR=llvm-lib
```

If you are specifying lld-link as your linker by setting `linker = "lld-link.exe"` in your cargo config, you may run into issues with some crates that compile code with separate cargo invocations. You should be able to get around this problem by setting `-Clinker=lld-link` in RUSTFLAGS

Toolchain Compatibility

In order for this kind of LTO to work, the LLVM linker plugin must be able to handle the LLVM bitcode produced by both `rustc` and `clang`.

Best results are achieved by using a `rustc` and `clang` that are based on the exact same version of LLVM. One can use `rustc -vV` in order to view the LLVM used by a given `rustc` version. Note that the version number given here is only an approximation as Rust sometimes uses unstable revisions of LLVM. However, the approximation is usually reliable.

The following table shows known good combinations of toolchain versions.

Rust Version	Clang Version
1.34 - 1.37	8
1.38 - 1.44	9
1.45 - 1.46	10
1.47 - 1.51	11
1.52 - 1.55	12
1.56 - 1.59	13
1.60 - 1.64	14
1.65 - 1.69	15
1.70 - 1.72	16

1.73 - 1.77	17
1.78	18

Note that the compatibility policy for this feature might change in the future.

Checking conditional configurations

`rustc` supports checking that every *reachable*¹ `#[cfg]` matches a list of the expected config names and values.

This can help with verifying that the crate is correctly handling conditional compilation for different target platforms or features. It ensures that the `cfg` settings are consistent between what is intended and what is used, helping to catch potential bugs or errors early in the development process.

In order to accomplish that goal, `rustc` accepts the `--check-cfg` flag, which specifies whether to check conditions and how to check them.

Note: For interacting with this through Cargo, see [Cargo Specifics](#) page.

¹ `rustc` promises to at least check reachable `#[cfg]`, and while non-reachable `#[cfg]` are not currently checked, they may well be checked in the future without it being a breaking change.

Specifying expected names and values

To specify expected names and values, the *check cfg specification* provides the `cfg(...)` option which enables specifying for an expected config name and its expected values.

Note: No implicit expectation is added when using `--cfg`. Users are expected to pass all expected names and values using the *check cfg specification*.

It has this basic form:

```
rustc --check-cfg 'cfg(name, values("value1", "value2", ... "valueN"))'
```

where `name` is a bare identifier (has no quotes) and each `"value"` term is a quoted literal string. `name` specifies the name of the condition, such as `feature` or `my_cfg`. `"value"`

specify one of the value of that condition name.

When the `cfg(...)` option is specified, `rustc` will check every¹:

- `#[cfg(name = "value")]` attribute
- `#[cfg_attr(name = "value")]` attribute
- `#[link(name = "a", cfg(name = "value"))]` attribute
- `cfg!(name = "value")` macro call

The command line `--cfg` arguments are currently NOT checked but may very well be checked in the future.

`rustc` will check that the "value" specified is present in the list of expected values. If "value" is not in it, then `rustc` will report an `unexpected_cfgs` lint diagnostic. The default diagnostic level for this lint is `Warn`.

To check for the *none* value (ie `#[cfg(foo)]`) one can use the `none()` predicate inside `values()`: `values(none())`. It can be followed or preceded by any number of "value".

To enable checking of values, but to provide an *none*/empty set of expected values (ie. expect `#[cfg(name)]`), use these forms:

```
rustc --check-cfg 'cfg(name)'
rustc --check-cfg 'cfg(name, values(none()))'
```

To enable checking of name but not values, use one of these forms:

- No expected values (*will lint on every value of name*):

```
rustc --check-cfg 'cfg(name, values())'
```

- Unknown expected values (*will never lint on value of name*):

```
rustc --check-cfg 'cfg(name, values(any()))'
```

To avoid repeating the same set of values, use this form:

```
rustc --check-cfg 'cfg(name1, ..., nameN, values("value1", "value2", ...  
"valueN"))'
```

To enable checking without specifying any names or values, use this form:

```
rustc --check-cfg 'cfg()'
```

The `--check-cfg cfg(...)` option can be repeated, both for the same condition name and for different names. If it is repeated for the same condition name, then the sets of values for that condition are merged together (precedence is given to `values(any())`).

To help out an equivalence table between `--cfg` arguments and `--check-cfg` is available [down below](#).

Well known names and values

`rustc` maintains a list of well-known names and their corresponding values in order to avoid the need to specify them manually.

Well known names and values are implicitly added as long as at least one `--check-cfg` argument is present.

As of 2024-05-06T, the list of known names is as follows:

- `clippy`
- `debug_assertions`
- `doc`
- `doctest`
- `miri`
- `overflow_checks`
- `panic`
- `proc_macro`
- `relocation_model`
- `rustfmt`

- `sanitize`
- `sanitizer_cfi_generalize_pointers`
- `sanitizer_cfi_normalize_integers`
- `target_abi`
- `target_arch`
- `target_endian`
- `target_env`
- `target_family`
- `target_feature`
- `target_has_atomic`
- `target_has_atomic_equal_alignment`
- `target_has_atomic_load_store`
- `target_os`
- `target_pointer_width`
- `target_thread_local`
- `target_vendor`
- `test`
- `ub_checks`
- `unix`
- `windows`

Like with `values(any())` , well known names checking can be disabled by passing `cfg(any())` as argument to `--check-cfg` .

Equivalence table with `--cfg`

This table describe the equivalence between a `--cfg` argument to a `--check-cfg` argument.

<code>--cfg</code>	<code>--check-cfg</code>
<i>nothing</i>	<i>nothing</i> or <code>--check-cfg=cfg()</code> (to enable the checking)
<code>--cfg foo</code>	<code>--check-cfg=cfg(foo)</code> OR <code>--check-cfg=cfg(foo, values())</code>
<code>--cfg foo=""</code>	<code>--check-cfg=cfg(foo, values(""))</code>

<code>--cfg foo="bar"</code>	<code>--check-cfg=cfg(foo, values("bar"))</code>
<code>--cfg foo="1" --cfg foo="2"</code>	<code>--check-cfg=cfg(foo, values("1", "2"))</code>
<code>--cfg foo="1" --cfg bar="2"</code>	<code>--check-cfg=cfg(foo, values("1")) --check-cfg=</code>
<code>--cfg foo --cfg foo="bar"</code>	<code>--check-cfg=cfg(foo, values(none(), "bar"))</code>

Examples

Example: Cargo-like **feature** example

Consider this command line:

```
rustc --check-cfg 'cfg(feature, values("lion", "zebra"))' \
      --cfg 'feature="lion"' example.rs
```

This command line indicates that this crate has two features: `lion` and `zebra`. The `lion` feature is enabled, while the `zebra` feature is disabled.

```

#[cfg(feature = "lion")]    // This condition is expected, as "lion" is an
                           // expected value of `feature`

fn tame_lion(lion: Lion) {}

#[cfg(feature = "zebra")]    // This condition is expected, as "zebra" is an
                           // expected
                           // value of `feature` but the condition will
                           // evaluate
                           // to false since only --cfg feature="lion" was
                           // passed
fn ride_zebra(z: Zebra) {}

#[cfg(feature = "platypus")] // This condition is UNEXPECTED, as "platypus" is
                           // NOT
                           // an expected value of `feature` and will cause a
                           // the compiler to emit the `unexpected_cfgs` lint

fn poke_platypus() {}

#[cfg(feechure = "lion")]    // This condition is UNEXPECTED, as 'feechure' is
                           // NOT
                           // a expected condition name, no `cfg(feechure,
                           // ...)`.
                           // was passed in `--check-cfg`

fn tame_lion() {}

#[cfg(windows = "unix")]    // This condition is UNEXPECTED, as the well known
                           // 'windows' cfg doesn't expect any values

fn tame_windows() {}

```

Example: Multiple names and values

```

rustc --check-cfg 'cfg(is_embedded, has_feathers)' \
      --check-cfg 'cfg(feature, values("zapping", "lasers"))' \
      --cfg has_feathers --cfg 'feature="zapping"'

```

```

#[cfg(is_embedded)]           // This condition is expected, as 'is_embedded' was
                              // provided in --check-cfg and doesn't take any
value
fn do_embedded() {}

#[cfg(has_feathers)]          // This condition is expected, as 'has_feathers'
was                             // provided in --check-cfg and doesn't take any
value
fn do_features() {}

#[cfg(has_mumble_frotz)]      // This condition is UNEXPECTED, as
'has_mumble_frotz'           // was NEVER provided in any --check-cfg arguments
fn do_mumble_frotz() {}

#[cfg(feature = "lasers")]    // This condition is expected, as "lasers" is an
                              // expected value of `feature`
fn shoot_lasers() {}

#[cfg(feature = "monkeys")]   // This condition is UNEXPECTED, as "monkeys" is
NOT                             // an expected value of `feature`
fn write_shakespeare() {}

```

Example: Condition names without values

```

rustc --check-cfg 'cfg(is_embedded, has_feathers, values(any()))' \
      --cfg has_feathers

```

```
#[cfg(is_embedded)]           // This condition is expected, as 'is_embedded' was
                               // provided in --check-cfg as condition name
fn do_embedded() {}

#[cfg(has_feathers)]          // This condition is expected, as "has_feathers" was
                               // provided in --check-cfg as condition name
fn do_features() {}

#[cfg(has_feathers = "zapping")] // This condition is expected, as
"has_feathers"
                               // was provided and because *any* values is
                               // expected for 'has_feathers' no
                               // warning is emitted for the value "zapping"
fn do_zapping() {}

#[cfg(has_mumble_frotz)]      // This condition is UNEXPECTED, as 'has_mumble_frotz'
                               // was not provided in any --check-cfg arguments
fn do_mumble_frotz() {}
```

Cargo Specifics - Checking Conditional Configurations

This document is intended to summarize the principal ways Cargo interacts with the `unexpected_cfgs` lint and `--check-cfg` flag. It is not intended to provide individual details, for that refer to the [--check-cfg documentation](#) and to the [Cargo book](#).

Cargo feature

See the [\[features\]](#) section in the *Cargo book* for more details.

With the `[features]` table, Cargo provides a mechanism to express conditional compilation and optional dependencies. Cargo *automatically* declares corresponding `cfgs` for every feature as expected.

`Cargo.toml` :

```
[features]
serde = ["dep:serde"]
my_feature = []
```

check-cfg in [lints.rust] table

See the [\[lints\]](#) section in the *Cargo book* for more details.

When using a statically known custom config (i.e., not dependent on a build-script), Cargo provides the custom lint config `check-cfg` under `[lints.rust.unexpected_cfgs]`.

It can be used to set custom static `--check-cfg` args, it is mainly useful when the list of expected `cfgs` is known in advance.

`Cargo.toml` :

```
[lints.rust]
unexpected_cfgs = { level = "warn", check-cfg = ['cfg(has_foo)'] }
```

`cargo::rustc-check-cfg` for `build.rs/build-script`

See the `cargo::rustc-check-cfg` section in the Cargo book for more details.

When setting a custom config with `cargo::rustc-cfg`, Cargo provides the corollary instruction: `cargo::rustc-check-cfg` to expect custom configs.

`build.rs`:

```
fn main() {
    println!("cargo::rustc-check-cfg=cfg(has_foo)");
    // ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ new with Cargo 1.80
    if has_foo() {
        println!("cargo::rustc-cfg=has_foo");
    }
}
```

Exploit Mitigations

This chapter documents the exploit mitigations supported by the Rust compiler, and is by no means an extensive survey of the Rust programming language's security features.

This chapter is for software engineers working with the Rust programming language, and assumes prior knowledge of the Rust programming language and its toolchain.

Introduction

The Rust programming language provides memory[1] and thread[2] safety guarantees via its ownership[3], references and borrowing[4], and slice types[5] features. However, Unsafe Rust[6] introduces unsafe blocks, unsafe functions and methods, unsafe traits, and new types that are not subject to the borrowing rules.

Parts of the Rust standard library are implemented as safe abstractions over unsafe code (and historically have been vulnerable to memory corruption[7]). Furthermore, the Rust code and documentation encourage creating safe abstractions over unsafe code. This can cause a false sense of security if unsafe code is not properly reviewed and tested.

Unsafe Rust introduces features that do not provide the same memory and thread safety guarantees. This causes programs or libraries to be susceptible to memory corruption (CWE-119)[8] and concurrency issues (CWE-557)[9]. Modern C and C++ compilers provide exploit mitigations to increase the difficulty to exploit vulnerabilities resulting from these issues. Therefore, the Rust compiler must also support these exploit mitigations in order to mitigate vulnerabilities resulting from the use of Unsafe Rust. This chapter documents these exploit mitigations and how they apply to Rust.

This chapter does not discuss the effectiveness of these exploit mitigations as they vary greatly depending on several factors besides their design and implementation, but rather describe what they do, so their effectiveness can be understood within a given context.

Exploit mitigations

This section documents the exploit mitigations applicable to the Rust compiler when building programs for the Linux operating system on the AMD64 architecture and equivalent.¹ All examples in this section were built using nightly builds of the Rust compiler on Debian testing.

The Rust Programming Language currently has no specification. The Rust compiler (i.e., rustc) is the language reference implementation. All references to “the Rust compiler” in this chapter refer to the language reference implementation.

Table I
Summary of exploit mitigations supported by the Rust compiler when building programs for the Linux operating system on the AMD64 architecture and equivalent.

Exploit mitigation	Supported and enabled by default	Since
Position-independent executable	Yes	0.12.0 (2014-10-09)
Integer overflow checks	Yes (enabled when debug assertions are enabled, and disabled when debug assertions are disabled)	1.1.0 (2015-06-25)
Non-executable memory regions	Yes	1.8.0 (2016-04-14)
Stack clashing protection	Yes	1.20.0 (2017-08-31)
Read-only relocations and immediate binding	Yes	1.21.0 (2017-10-12)
Heap corruption protection	Yes	1.32.0 (2019-01-17) (via operating system default or specified allocator)
Stack smashing protection	Yes	Nightly
Forward-edge		

control flow protection	Yes	Nightly
Backward-edge control flow protection (e.g., shadow and safe stack)	Yes	Nightly

1. See https://github.com/rust-lang/rust/tree/master/compiler/rustc_target/src/spec for a list of targets and their default options. ↩

Position-independent executable

Position-independent executable increases the difficulty of the use of code reuse exploitation techniques, such as return-oriented programming (ROP) and variants, by generating position-independent code for the executable, and instructing the dynamic linker to load it similarly to a shared object at a random load address, thus also benefiting from address-space layout randomization (ASLR). This is also referred to as “full ASLR”.

The Rust compiler supports position-independent executable, and enables it by default since version 0.12.0 (2014-10-09)[10]–[13].

```
$ readelf -h target/release/hello-rust | grep Type:
Type:                                DYN (Shared object file)
```

Fig. 1. Checking if an executable is a position-independent executable.

An executable with an object type of `ET_DYN` (i.e., shared object) and not `ET_EXEC` (i.e., executable) is a position-independent executable (see Fig. 1).

Integer overflow checks

Integer overflow checks protects programs from undefined and unintended behavior (which may cause vulnerabilities) by checking for results of signed and unsigned integer computations that cannot be represented in their type, resulting in an overflow or wraparound.

The Rust compiler supports integer overflow checks, and enables it when debug assertions are enabled since version 1.0.0 (2015-05-15)[14]–[17], but support for it was not completed until version 1.1.0 (2015-06-25)[16]. An option to control integer overflow checks was later stabilized in version 1.17.0 (2017-04-27)[18]–[20].

```
fn main() {
    let u: u8 = 255;
    println!("u: {}", u + 1);
}
```

Fig. 2. hello-rust-integer program.

```
$ cargo run
  Compiling hello-rust-integer v0.1.0 (/home/rcvalle/hello-rust-integer)
  Finished dev [unoptimized + debuginfo] target(s) in 0.23s
  Running `target/debug/hello-rust-integer`
thread 'main' panicked at 'attempt to add with overflow', src/main.rs:3:23
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace.
```

Fig. 3. Build and execution of hello-rust-integer with debug assertions enabled.

```
$ cargo run --release
  Compiling hello-rust-integer v0.1.0 (/home/rcvalle/hello-rust-integer)
  Finished release [optimized] target(s) in 0.23s
  Running `target/release/hello-rust-integer`
u: 0
```

Fig. 4. Build and execution of hello-rust-integer with debug assertions disabled.

Integer overflow checks are enabled when debug assertions are enabled (see Fig. 3), and disabled when debug assertions are disabled (see Fig. 4). To enable integer overflow checks independently, use the option to control integer overflow checks, scoped attributes, or explicit checking methods such as `checked_add` ².

It is recommended that explicit wrapping methods such as `wrapping_add` be used when wrapping semantics are intended, and that explicit checking and wrapping methods always be used when using Unsafe Rust.

2. See [the u32 docs](#) for more information on the checked, overflowing, saturating, and wrapping methods (using u32 as an example). ↩

Non-executable memory regions

Non-executable memory regions increase the difficulty of exploitation by limiting the memory regions that can be used to execute arbitrary code. Most modern processors provide support for the operating system to mark memory regions as non executable, but it was previously emulated by software, such as in grsecurity/PaX's [PAGEEXEC](#) and [SEGMEXEC](#), on processors that did not provide support for it. This is also known as “No Execute (NX) Bit”, “Execute Disable (XD) Bit”, “Execute Never (XN) Bit”, and others.

The Rust compiler supports non-executable memory regions, and enables it by default since its initial release, version 0.1 (2012-01-20)[21], [22], but has regressed since then[23]–[25], and enforced by default since version 1.8.0 (2016-04-14)[25].

```
$ readelf -l target/release/hello-rust | grep -A 1 GNU_STACK
GNU_STACK      0x0000000000000000 0x0000000000000000 0x0000000000000000
                0x0000000000000000 0x0000000000000000 RW      0x10
```

Fig. 5. Checking if non-executable memory regions are enabled for a given binary.

The presence of an element of type `PT_GNU_STACK` in the program header table with the `PF_X` (i.e., executable) flag unset indicates non-executable memory regions³ are enabled for a given binary (see Fig. 5). Conversely, the presence of an element of type `PT_GNU_STACK` in the program header table with the `PF_X` flag set or the absence of an element of type `PT_GNU_STACK` in the program header table indicates non-executable memory regions are not enabled for a given binary.

3. See the Appendix section for more information on why it affects other memory regions besides the stack. ↩

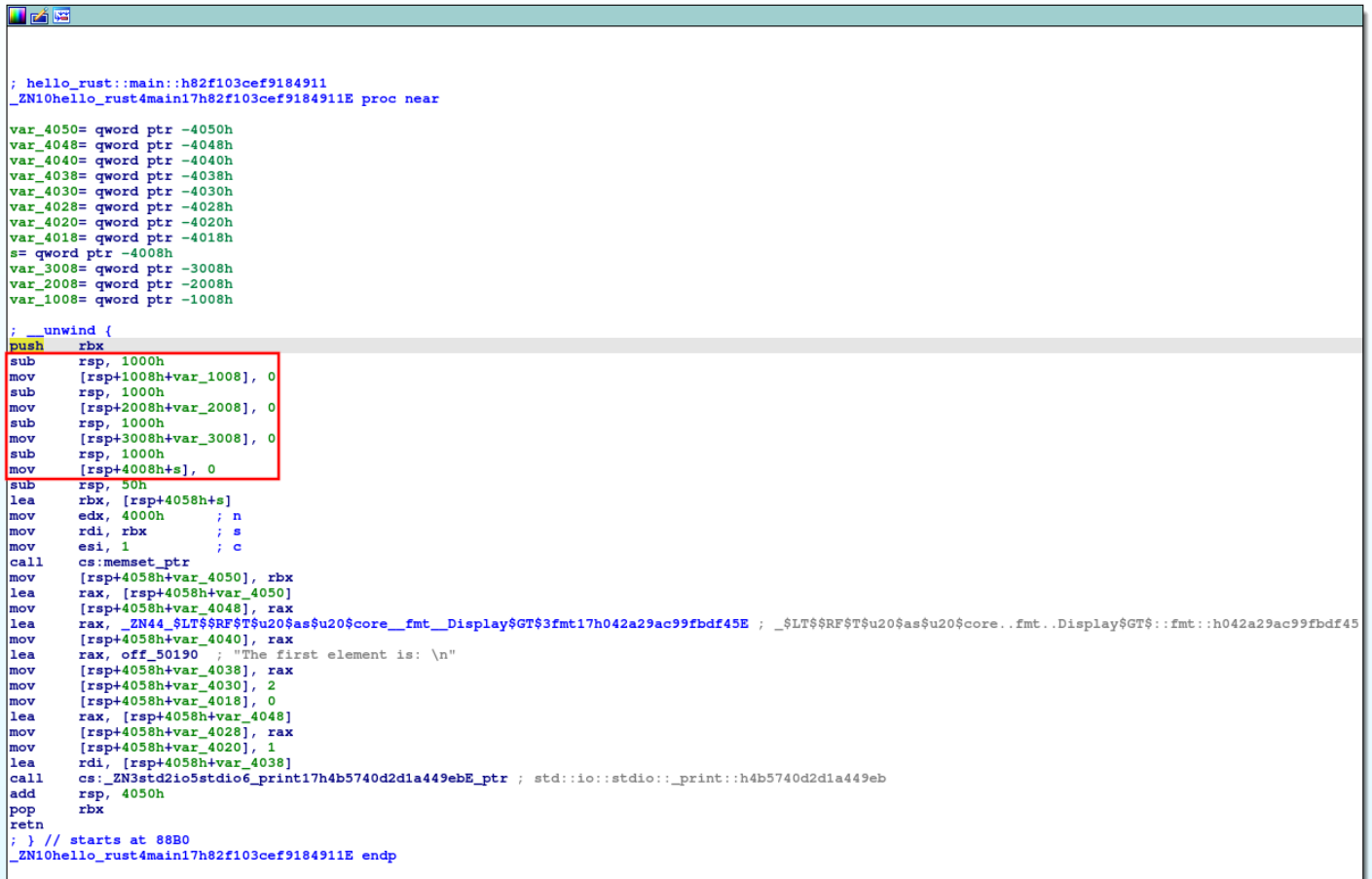
Stack clashing protection

Stack clashing protection protects the stack from overlapping with another memory region—allowing arbitrary data in both to be overwritten using each other—by reading from the stack pages as the stack grows to cause a page fault when attempting to read from the guard page/region. This is also referred to as “stack probes” or “stack probing”.

The Rust compiler supports stack clashing protection via stack probing, and enables it by default since version 1.20.0 (2017-08-31)[26]–[29].

```
fn main() {
    let v: [u8; 16384] = [1; 16384];
    let first = &v[0];
    println!("The first element is: {first}");
}
```

Fig. 6. hello-rust-stack-probe-1 program.



```
; hello_rust::main::h82f103cef9184911
_ZN10hello_rust4main17h82f103cef9184911E proc near
var_4050= qword ptr -4050h
var_4048= qword ptr -4048h
var_4040= qword ptr -4040h
var_4038= qword ptr -4038h
var_4030= qword ptr -4030h
var_4028= qword ptr -4028h
var_4020= qword ptr -4020h
var_4018= qword ptr -4018h
s= qword ptr -4008h
var_3008= qword ptr -3008h
var_2008= qword ptr -2008h
var_1008= qword ptr -1008h

; __unwind {
push     rbp
sub      rsp, 1000h
mov      [rsp+1008h+var_1008], 0
sub      rsp, 1000h
mov      [rsp+2008h+var_2008], 0
sub      rsp, 1000h
mov      [rsp+3008h+var_3008], 0
sub      rsp, 1000h
mov      [rsp+4008h+s], 0
sub      rsp, 50h
lea      rbp, [rsp+4058h+s]
mov      edx, 4000h ; n
mov      rdi, rbp ; s
mov      esi, 1 ; c
call     cs:memset_ptr
mov      [rsp+4058h+var_4050], rbp
lea      rax, [rsp+4058h+var_4050]
mov      [rsp+4058h+var_4048], rax
lea      rax, 2N44_$LT$RF$T$u20$as$u20$core__fmt__Display$GT$3fmt17h042a29ac99fbdf45E ; _$LT$RF$T$u20$as$u20$core__fmt__Display$GT$::fmt::h042a29ac99fbdf45
mov      [rsp+4058h+var_4040], rax
lea      rax, off_50190 ; "The first element is: \n"
mov      [rsp+4058h+var_4038], rax
mov      [rsp+4058h+var_4030], 2
mov      [rsp+4058h+var_4018], 0
lea      rax, [rsp+4058h+var_4048]
mov      [rsp+4058h+var_4028], rax
mov      [rsp+4058h+var_4020], 1
lea      rdi, [rsp+4058h+var_4038]
call     cs:_ZN3std2io5stdio6_print17h4b5740d2dia449ebE_ptr ; std::io::stdio::_print::h4b5740d2dia449eb
add      rsp, 4050h
pop      rbp
retn
; } // starts at 88B0
_ZN10hello_rust4main17h82f103cef9184911E endp
```

Fig. 7. The "unrolled loop" stack probe variant in modified hello-rust.

```
fn main() {
    let v: [u8; 65536] = [1; 65536];
    let first = &v[0];
    println!("The first element is: {first}");
}
```

Fig. 8. hello-rust-stack-probe-2 program.

```

; hello_rust::main::h82f103cef9184911
_ZN10hello_rust4main17h82f103cef9184911E proc near

var_1050= qword ptr -1050h
var_1048= qword ptr -1048h
var_1040= qword ptr -1040h
var_1038= qword ptr -1038h
var_1030= qword ptr -1030h
var_1028= qword ptr -1028h
var_1020= qword ptr -1020h
var_1018= qword ptr -1018h
s= qword ptr -1008h

; __unwind {
push    rbx
mov     r11, rsp
sub     r11, 10000h

```

```

loc_88BB:
sub     rsp, 1000h
mov     [rsp+1008h+s], 0
cmp     rsp, r11
jnz     short loc_88BB

```

```

sub     rsp, 50h
lea     rbx, [rsp+1058h+s]
mov     edx, 10000h ; n
mov     rdi, rbx ; s
mov     esi, 1 ; c
call    cs:memset_ptr
mov     [rsp+1058h+var_1050], rbx
lea     rax, [rsp+1058h+var_1050]
mov     [rsp+1058h+var_1048], rax
lea     rax, _ZN4_$LT$$SRF$T$u20$as$u20$core__fmt_Display$GT$3fmt17h042a29ac99fbdf45E ; _$LT$$SRF$T$u20$as$u20$core__fmt__Display$GT$::fmt::h042a29ac99fbdf45
mov     [rsp+1058h+var_1048], rax
lea     rax, off_50190 ; "The first element is: \n"
mov     [rsp+1058h+var_1038], rax
mov     [rsp+1058h+var_1030], 2
mov     [rsp+1058h+var_1018], 0
lea     rax, [rsp+1058h+var_1048]
mov     [rsp+1058h+var_1028], rax
mov     [rsp+1058h+var_1020], 1
lea     rdi, [rsp+1058h+var_1038]
call    cs:_ZN3std2io5stdio6_print17h4b5740d2d1a449ebE_ptr ; std::io::stdio::_print::h4b5740d2d1a449eb
add     rsp, 10050h
pop     rbx
retn
; } // starts at 88B0
_ZN10hello_rust4main17h82f103cef9184911E endp ; sp-analysis failed

```

Fig. 9. The "standard loop" stack probe variant in modified hello-rust.

To check if stack clashing protection is enabled for a given binary, look for any of the two stack probe variants in the prologue of functions whose stack size is larger than a page size (see Figs. 6–9).

Read-only relocations and immediate binding

Read-only relocations protect segments containing relocations and relocation information (i.e., `.init_array`, `.fini_array`, `.dynamic`, and `.got`) from being overwritten by marking these segments read only. This is also referred to as “partial RELRO”.

The Rust compiler supports read-only relocations, and enables it by default since version 1.21.0 (2017-10-12)[30], [31].

```
$ readelf -l target/release/hello-rust | grep GNU_RELRO
GNU_RELRO      0x00000000000002ee00 0x00000000000002fe00 0x00000000000002fe00
```

Fig. 9. Checking if read-only relocations is enabled for a given binary.

The presence of an element of type `PT_GNU_RELRO` in the program header table indicates read-only relocations are enabled for a given binary (see Fig. 9). Conversely, the absence of an element of type `PT_GNU_RELRO` in the program header table indicates read-only relocations are not enabled for a given binary.

Immediate binding protects additional segments containing relocations (i.e., `.got.plt`) from being overwritten by instructing the dynamic linker to perform all relocations before transferring control to the program during startup, so all segments containing relocations can be marked read only (when combined with read-only relocations). This is also referred to as “full RELRO”.

The Rust compiler supports immediate binding, and enables it by default since version 1.21.0 (2017-10-12)[30], [31].

```
$ readelf -d target/release/hello-rust | grep BIND_NOW
0x000000000000001e (FLAGS)          BIND_NOW
```

Fig. 10. Checking if immediate binding is enabled for a given binary.

The presence of an element with the `DT_BIND_NOW` tag and the `DF_BIND_NOW` flag⁴ in the dynamic section indicates immediate binding is enabled for a given binary (see Fig. 10). Conversely, the absence of an element with the `DT_BIND_NOW` tag and the `DF_BIND_NOW` flag in the dynamic section indicates immediate binding is not enabled for a given binary.

The presence of both an element of type `PT_GNU_RELRO` in the program header table and of an element with the `DT_BIND_NOW` tag and the `DF_BIND_NOW` flag in the dynamic section indicates full RELRO is enabled for a given binary (see Figs. 9–10).

4. And the `DF_1_NOW` flag for some link editors. ↩

Heap corruption protection

Heap corruption protection protects memory allocated dynamically by performing several checks, such as checks for corrupted links between list elements, invalid pointers, invalid sizes, double/multiple “frees” of the same memory allocated, and many corner cases of these. These checks are implementation specific, and vary per allocator.

[ARM Memory Tagging Extension \(MTE\)](#), when available, will provide hardware assistance for a probabilistic mitigation to detect memory safety violations by tagging memory allocations, and automatically checking that the correct tag is used on every memory access.

Rust’s default allocator has historically been [jemalloc](#), and it has long been the cause of issues and the subject of much discussion[32]–[38]. Consequently, it has been removed as the default allocator in favor of the operating system’s standard C library default allocator⁵ since version 1.32.0 (2019-01-17)[39].

```
fn main() {
    let mut x = Box::new([0; 1024]);

    for i in 0..1026 {
        unsafe {
            let elem = x.get_unchecked_mut(i);
            *elem = 0x4141414141414141u64;
        }
    }
}
```

Fig. 11. hello-rust-heap program.

```
$ cargo run
  Compiling hello-rust-heap v0.1.0 (/home/rcvalle/hello-rust-heap)
  Finished dev [unoptimized + debuginfo] target(s) in 0.25s
  Running `target/debug/hello-rust-heap`
free(): invalid next size (normal)
Aborted
```

Fig. 12. Build and execution of hello-rust-heap with debug assertions enabled.

```
$ cargo run --release
   Compiling hello-rust-heap v0.1.0 (/home/rcvalle/hello-rust-heap)
   Finished release [optimized] target(s) in 0.25s
   Running `target/release/hello-rust-heap`
free(): invalid next size (normal)
Aborted
```

Fig. 13. Build and execution of hello-rust-heap with debug assertions disabled.

Heap corruption checks are performed when using the default allocator (i.e., the GNU Allocator) (see Figs. 12–13).

5. Linux's standard C library default allocator is the GNU Allocator, which is derived from ptmalloc (pthreads malloc) by Wolfram Gloger, which in turn is derived from dlmalloc (Doug Lea malloc) by Doug Lea. ↩

Stack smashing protection

Stack smashing protection protects programs from stack-based buffer overflows by inserting a random guard value between local variables and the saved return instruction pointer, and checking if this value has changed when returning from a function. This is also known as “Stack Protector” or “Stack Smashing Protector (SSP)”.

The Rust compiler supports stack smashing protection on nightly builds[40].

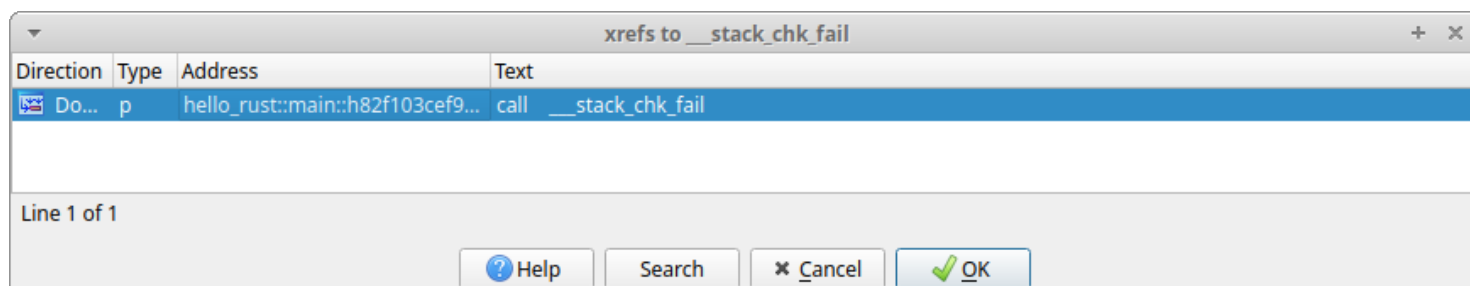


Fig. 14. IDA Pro listing cross references to `__stack_chk_fail` in hello-rust.

To check if stack smashing protection is enabled for a given binary, search for cross references to `__stack_chk_fail` (see Fig. 14).

Forward-edge control flow protection

Forward-edge control flow protection protects programs from having its control flow changed/hijacked by performing checks to ensure that destinations of indirect branches are one of their valid destinations in the control flow graph. The comprehensiveness of these checks vary per implementation. This is also known as “forward-edge control flow integrity (CFI)”.

Newer processors provide hardware assistance for forward-edge control flow protection, such as ARM Branch Target Identification (BTI), ARM Pointer Authentication, and Intel Indirect Branch Tracking (IBT) as part of Intel Control-flow Enforcement Technology (CET). However, ARM BTI and Intel IBT -based implementations are less comprehensive than software-based implementations such as [LLVM ControlFlowIntegrity \(CFI\)](#), and the commercially available [grsecurity/PaX Reuse Attack Protector \(RAP\)](#).

The Rust compiler supports forward-edge control flow protection on nightly builds[41]-[42] 6.

```
$ readelf -s -W target/release/hello-rust | grep "\.cfi"
      5: 000000000000006480    657 FUNC      LOCAL  DEFAULT  15
      _ZN10hello_rust4main17h4e359f1dcd627c83E.cfi
```

Fig. 15. Checking if LLVM CFI is enabled for a given binary.

The presence of symbols suffixed with ".cfi" or the `__cfi_init` symbol (and references to `__cfi_check`) indicates that LLVM CFI (i.e., forward-edge control flow protection) is enabled for a given binary. Conversely, the absence of symbols suffixed with ".cfi" or the `__cfi_init` symbol (and references to `__cfi_check`) indicates that LLVM CFI is not enabled for a given binary (see Fig. 15).

6. It also supports Control Flow Guard (CFG) on Windows (see <https://github.com/rust-lang/rust/issues/68793>).



Backward-edge control flow protection

Shadow stack protects saved return instruction pointers from being overwritten by storing a copy of them on a separate (shadow) stack, and using these copies as authoritative values when returning from functions. This is also known as “ShadowCallStack” and “Return Flow Guard”, and is considered an implementation of backward-edge control flow protection (or

“backward-edge CFI”).

Safe stack protects not only the saved return instruction pointers, but also register spills and some local variables from being overwritten by storing unsafe variables, such as large arrays, on a separate (unsafe) stack, and using these unsafe variables on the separate stack instead. This is also known as “SafeStack”, and is also considered an implementation of backward-edge control flow protection.

Both shadow and safe stack are intended to be a more comprehensive alternatives to stack smashing protection as they protect the saved return instruction pointers (and other data in the case of safe stack) from arbitrary writes and non-linear out-of-bounds writes.

Newer processors provide hardware assistance for backward-edge control flow protection, such as ARM Pointer Authentication, and Intel Shadow Stack as part of Intel CET.

The Rust compiler supports shadow stack for the AArch64 architecture⁷ on nightly builds[43]-[44], and also supports safe stack on nightly builds[45]-[46].

```
$ readelf -s target/release/hello-rust | grep __safestack_init
678: 00000000000008c80 426 FUNC GLOBAL DEFAULT 15 __safestack_init
```

Fig. 16. Checking if LLVM SafeStack is enabled for a given binary.

The presence of the `__safestack_init` symbol indicates that LLVM SafeStack is enabled for a given binary. Conversely, the absence of the `__safestack_init` symbol indicates that LLVM SafeStack is not enabled for a given binary (see Fig. 16).

7. The shadow stack implementation for the AMD64 architecture and equivalent in LLVM was removed due to performance and security issues. ↩

Appendix

As of the latest version of the [Linux Standard Base \(LSB\) Core Specification](#), the `PT_GNU_STACK` program header indicates whether the stack should be executable, and the absence of this header indicates that the stack should be executable. However, the Linux kernel currently sets the `READ_IMPLIES_EXEC` personality upon loading any executable with

the `PT_GNU_STACK` program header and the `PF_X` flag set or with the absence of this header, resulting in not only the stack, but also all readable virtual memory mappings being executable.

An attempt to fix this [was made in 2012](#), and another [was made in 2020](#). The former never landed, and the latter partially fixed it, but introduced other issues—the absence of the `PT_GNU_STACK` program header still causes not only the stack, but also all readable virtual memory mappings to be executable in some architectures, such as IA-32 and equivalent (or causes the stack to be non-executable in some architectures, such as AMD64 and equivalent, contradicting the LSB).

The `READ_IMPLIES_EXEC` personality needs to be completely separated from the `PT_GNU_STACK` program header by having a separate option for it (or `setarch -X` could just be used whenever `READ_IMPLIES_EXEC` is needed), and the absence of the `PT_GNU_STACK` program header needs to have more secure defaults (unrelated to `READ_IMPLIES_EXEC`).

References

1. D. Hosfelt. “Fearless security: memory safety.” Mozilla Hacks.
<https://hacks.mozilla.org/2019/01/fearless-security-memory-safety/>.
2. D. Hosfelt. “Fearless security: thread safety.” Mozilla Hacks.
<https://hacks.mozilla.org/2019/02/fearless-security-thread-safety/>.
3. S. Klabnik and C. Nichols. “What Is Ownership?.” The Rust Programming Language.
<https://doc.rust-lang.org/book/ch04-01-what-is-ownership.html>.
4. S. Klabnik and C. Nichols. “References and Borrowing.” The Rust Programming Language. <https://doc.rust-lang.org/book/ch04-02-references-and-borrowing.html>.
5. S. Klabnik and C. Nichols. “The Slice Type.” The Rust Programming Language.
<https://doc.rust-lang.org/book/ch04-03-slices.html>.
6. S. Klabnik and C. Nichols. “Unsafe Rust.” The Rust Programming Language.
<https://doc.rust-lang.org/book/ch19-01-unsafe-rust.html>.
7. S. Davidoff. “How Rust’s standard library was vulnerable for years and nobody noticed.”

- Medium. <https://medium.com/@shnatsel/how-rusts-standard-library-was-vulnerable-for-years-and-nobody-noticed-aebf0503c3d6>.
8. “Improper restriction of operations within the bounds of a memory buffer (CWE-119).” MITRE CWE List. <https://cwe.mitre.org/data/definitions/119.html>.
 9. “Concurrency issues (CWE-557).” MITRE CWE List. <https://cwe.mitre.org/data/definitions/557.html>.
 10. K. McAllister. “Memory exploit mitigations #15179.” GitHub. <https://github.com/rust-lang/rust/issues/15179>.
 11. K. McAllister. “RFC: Memory exploit mitigation #145.” GitHub. <https://github.com/rust-lang/rfcs/pull/145>.
 12. K. McAllister. “RFC: Memory exploit mitigation.” GitHub. <https://github.com/kmcallister/rfcs/blob/hardening/active/0000-memory-exploit-mitigation.md>.
 13. D. Micay. “Enable PIE by default on Linux for full ASLR #16340.” GitHub. <https://github.com/rust-lang/rust/pull/16340>.
 14. N. Matsakis. “Integer overflow #560.” GitHub. <https://github.com/rust-lang/rfcs/pull/560>.
 15. G. Lehel and N. Matsakis. “Integer overflow.” GitHub. <https://rust-lang.github.io/rfcs/0560-integer-overflow.html>.
 16. A. Turon. “Tracking issue for integer overflow (RFC 560) #22020.” GitHub. <https://github.com/rust-lang/rust/issues/22020>.
 17. H. Wilson. “Myths and legends about integer overflow in Rust.” Huon on the Internet. <http://huonw.github.io/blog/2016/04/myths-and-legends-about-integer-overflow-in-rust/>.
 18. B. Anderson. “Stabilize -C overflow-checks #1535.” GitHub. <https://github.com/rust-lang/rfcs/pull/1535>.
 19. B. Anderson. “Stable overflow checks.” GitHub. <https://github.com/brson/rfcs/blob/overflow/text/0000-stable-overflow-checks.md>.

20. N. Froyd. “Add -C overflow-checks option #40037.” GitHub. <https://github.com/rust-lang/rust/pull/40037>.
21. R. Á. de Espíndola. “rustc requires executable stack #798.” GitHub. <https://github.com/rust-lang/rust/issues/798>.
22. A. Seipp. “Make sure librustrt.so is linked with a non-executable stack. #1066.” GitHub. <https://github.com/rust-lang/rust/pull/1066>.
23. D. Micay. “Rust binaries should not have an executable stack #5643.” GitHub. <https://github.com/rust-lang/rust/issues/5643>.
24. D. Micay. “Mark the assembly object stacks as non-executable #5647.” GitHub. <https://github.com/rust-lang/rust/pull/5647>.
25. A. Clark. “Explicitly disable stack execution on linux and bsd #30859.” GitHub. <https://github.com/rust-lang/rust/pull/30859>.
26. Zoxc. “Replace stack overflow checking with stack probes #16012.” GitHub. <https://github.com/rust-lang/rust/issues/16012>.
27. A. Crichton. “rustc: Implement stack probes for x86 #42816.” GitHub. <https://github.com/rust-lang/rust/pull/42816>.
28. A. Crichton. “Add __rust_probestack intrinsic #175.” GitHub. <https://github.com/rust-lang/compiler-builtins/pull/175>.
29. S. Guelton, S. Ledru, J. Stone. “Bringing Stack Clash Protection to Clang / X86 — the Open Source Way.” The LLVM Project Blog. <https://blog.llvm.org/posts/2021-01-05-stack-clash-protection/>.
30. B. Anderson. “Consider applying -Wl,-z,relro or -Wl,-z,relro,-z,now by default #29877.” GitHub. <https://github.com/rust-lang/rust/issues/29877>.
31. J. Löthberg. “Add support for full RELRO #43170.” GitHub. <https://github.com/rust-lang/rust/pull/43170>.
32. N. Matsakis. “Allocators in Rust.” Baby Steps. <http://smallcultfollowing.com/babysteps/blog/2014/11/14/allocators-in-rust/>.
33. A. Crichton. “RFC: Allow changing the default allocator #1183.” GitHub.

- <https://github.com/rust-lang/rfcs/pull/1183>.
34. A. Crichton. "RFC: Swap out jemalloc." GitHub. <https://rust-lang.github.io/rfcs/1183-swap-out-jemalloc.html>.
 35. A. Crichton. "Tracking issue for changing the global, default allocator (RFC 1974) #27389." GitHub. <https://github.com/rust-lang/rust/issues/27389>.
 36. S. Fackler. "Prepare global allocators for stabilization #1974." GitHub. <https://github.com/rust-lang/rfcs/pull/1974>.
 37. A. Crichton. "RFC: Global allocators." GitHub. <https://rust-lang.github.io/rfcs/1974-global-allocators.html>.
 38. B. Anderson. "Switch the default global allocator to System, remove alloc_jemalloc, use jemallocator in rustc #36963." GitHub. <https://github.com/rust-lang/rust/issues/36963>.
 39. A. Crichton. "Remove the alloc_jemalloc crate #55238." GitHub. <https://github.com/rust-lang/rust/pull/55238>.
 40. bbjornse. "Add codegen option for using LLVM stack smash protection #84197." GitHub. <https://github.com/rust-lang/rust/pull/84197>
 41. R. de C. Valle. "Tracking Issue for LLVM Control Flow Integrity (CFI) Support for Rust #89653." GitHub. <https://github.com/rust-lang/rust/issues/89653>.
 42. "ControlFlowIntegrity." The Rust Unstable Book. <https://doc.rust-lang.org/unstable-book/compiler-flags/sanitizer.html#controlflowintegrity>.
 43. I. Lozano. "Add support for LLVM ShadowCallStack #98208." GitHub. <https://github.com/rust-lang/rust/pull/98208>.
 44. "ShadowCallStack." The Rust Unstable Book. <https://doc.rust-lang.org/unstable-book/compiler-flags/sanitizer.html#shadowcallstack>.
 45. W. Wiser. "Add support for LLVM SafeStack #112000" GitHub. <https://github.com/rust-lang/rust/pull/112000>
 46. "SafeStack." The Rust Unstable Book. <https://doc.rust-lang.org/unstable-book/compiler-flags/sanitizer.html#safestack>.

Symbol Mangling

Symbol name mangling is used by `rustc` to encode a unique name for symbols that are used during code generation. The encoded names are used by the linker to associate the name with the thing it refers to.

The method for mangling the names can be controlled with the `-C symbol-mangling-version` option.

Per-item control

The `#[no_mangle]` attribute can be used on items to disable name mangling on that item.

The `#[export_name]` attribute can be used to specify the exact name that will be used for a function or static.

Items listed in an `extern block` use the identifier of the item without mangling to refer to the item. The `#[link_name]` attribute can be used to change that name.

Decoding

The encoded names may need to be decoded in some situations. For example, debuggers and other tooling may need to demangle the name so that it is more readable to the user. Recent versions of `gdb` and `lldb` have built-in support for demangling Rust identifiers. In situations where you need to do your own demangling, the `rustc-demangle` crate can be used to programmatically demangle names. `rustfilt` is a CLI tool which can demangle names.

An example of running `rustfilt`:

```
$ rustfilt _RNVcSkwGfYPst2Cb_3foo16example_function
foo::example_function
```

Mangling versions

`rustc` supports different mangling versions which encode the names in different ways. The legacy version (which is currently the default) is not described here. The "v0" mangling scheme addresses several limitations of the legacy format, and is described in the [v0 Symbol Format](#) chapter.

v0 Symbol Format

The v0 mangling format was introduced in [RFC 2603](#). It has the following properties:

- It provides an unambiguous string encoding for everything that can end up in a binary's symbol table.
- It encodes information about generic parameters in a reversible way.
- The mangled symbols are *decodable* such that the demangled form should be easily identifiable as some concrete instance of e.g. a polymorphic function.
- It has a consistent definition that does not rely on pretty-printing certain language constructs.
- Symbols can be restricted to only consist of the characters `A-Z`, `a-z`, `0-9`, and `_`. This helps ensure that it is platform-independent, where other characters might have special meaning in some context (e.g. `.` for MSVC `DEF` files). Unicode symbols are optionally supported.
- It tries to stay efficient, avoiding unnecessarily long names, and avoiding computationally expensive operations to demangle.

The v0 format is not intended to be compatible with other mangling schemes (such as C++).

The v0 format is not presented as a stable ABI for Rust. This format is currently intended to be well-defined enough that a demangler can produce a reasonable human-readable form of the symbol. There are several implementation-defined portions that result in it not being possible to entirely predict how a given Rust entity will be encoded.

The sections below define the encoding of a v0 symbol. There is no standardized demangled form of the symbols, though suggestions are provided for how to demangle a symbol. Implementers may choose to demangle in different ways.

Extensions

This format may be extended in the future to add new tags as Rust is extended with new language items. To be forward compatible, demanglers should gracefully handle symbols that have encodings where it encounters a tag character not described in this document. For example, they may fall back to displaying the mangled symbol. The format may be extended

anywhere there is a tag character, such as the `type` rule. The meaning of existing tags and encodings will not be changed.

Grammar notation

The format of an encoded symbol is illustrated as a context free grammar in an extended BNF-like syntax. A consolidated summary can be found in the [Symbol grammar summary](#).

Name	Syntax	Example	Description
Rule	<code>→</code>	<code>A → B C</code>	A production.
Concatenation	whitespace	<code>A → B C D</code>	Individual elements in sequence left-to-right.
Alternative	<code> </code>	<code>A → B C</code>	Matches either one or the other.
Grouping	<code>()</code>	<code>A → B (C D) E</code>	Groups multiple elements as one.
Repetition	<code>{ }</code>	<code>A → {B}</code>	Repeats the enclosed zero or more times.
Option	<code>opt</code>	<code>A → B_{opt} C</code>	An optional element.
Literal	<code>monospace</code>	<code>A → G</code>	A terminal matching the exact characters case-sensitive.

Symbol name

```
symbol-name → _R decimal-numberopt path instantiating-crateopt vendor-specific-suffixopt
```

A mangled symbol starts with the two characters `_R` which is a prefix to identify the symbol as a Rust symbol. The prefix can optionally be followed by a `decimal-number` which specifies

the encoding version. This number is currently not used, and is never present in the current encoding. Following that is a *path* which encodes the path to an entity. The path is followed by an optional *instantiating-crate* which helps to disambiguate entities which may be instantiated multiple times in separate crates. The final part is an optional *vendor-specific-suffix*.

Recommended Demangling

A *symbol-name* should be displayed as the *path*. The *instantiating-crate* and the *vendor-specific-suffix* usually need not be displayed.

Example:

```
std::path::PathBuf::new();
```

The symbol for `PathBuf::new` in crate `mycrate` is:

The diagram illustrates the demangling of the symbol `_RNvMsr_NtCs3ssYzQotkvD_3std4pathNtB5_7PathBuf3newCs15kBYyAo9fc_7mycrate`. The symbol is divided into three segments by brackets:

- instantiating-crate**: `mycrate`
- path**: `std::path::PathBuf::new`
- symbol prefix**: `_R``

Arrows indicate the mapping from the symbol segments to their respective components. The `path` segment is further labeled as `path to`.

Recommended demangling: `<std::path::PathBuf>::new`

Symbol path

path →
crate-root

- | [inherent-impl](#)
- | [trait-impl](#)
- | [trait-definition](#)
- | [nested-path](#)
- | [generic-args](#)
- | [backref](#)

A *path* represents a variant of a [Rust path](#) to some entity. In addition to typical Rust path segments using identifiers, it uses extra elements to represent unnameable entities (like an `impl`) or generic arguments for monomorphized items.

The initial tag character can be used to determine which kind of path it represents:

Tag	Rule	Description
C	crate-root	The root of a crate path.
M	inherent-impl	An inherent implementation.
X	trait-impl	A trait implementation.
Y	trait-definition	A trait definition.
N	nested-path	A nested path.
I	generic-args	Generic arguments.
B	backref	A back reference.

Path: Crate root

`crate-root` → `c` [identifier](#)

A *crate-root* indicates a path referring to the root of a crate's module tree. It consists of the character `c` followed by the crate name as an [identifier](#).

The crate name is the name as seen from the defining crate. Since Rust supports linking multiple crates with the same name, the [disambiguator](#) is used to make the name unique across the crate graph.

Recommended Demangling

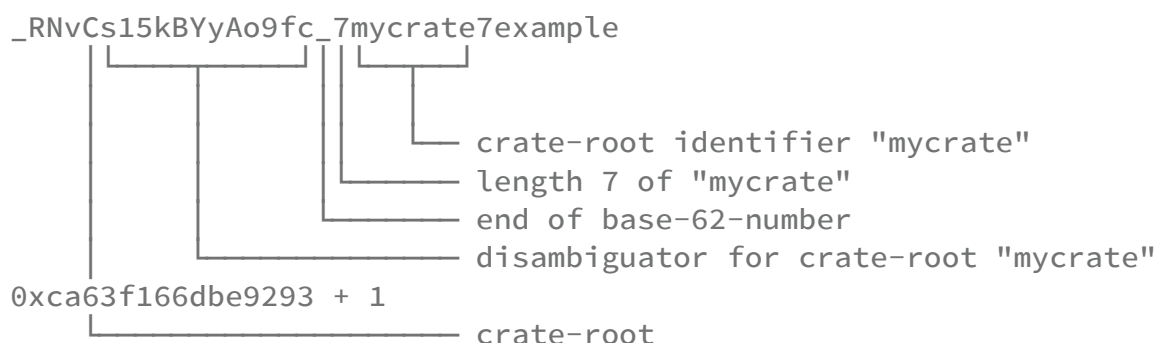
A *crate-root* can be displayed as the identifier such as `mycrate` .

Usually the disambiguator in the identifier need not be displayed, but as an alternate form the disambiguator can be shown in hex such as `mycrate[ca63f166dbe9294]` .

Example:

```
fn example() {}
```

The symbol for `example` in crate `mycrate` is:



Recommended demangling: `mycrate::example`

Note: The compiler may re-use the *crate-root* form to express arbitrary unscoped, undisambiguated identifiers, such as for new basic types that have not been added to the grammar yet. To achieve that, it will emit a *crate-root* without an explicit disambiguator, relying on the fact that such an undisambiguated crate name cannot occur in practice. For example, the basic type `f128` would be encoded as `c4f128` . For this to have the desired effect, demanglers are expected to never render zero disambiguators of crate roots. I.e. `c4f128` is expected to be displayed as `f128` and not `f128[0]` .

Path: Inherent impl

inherent-impl → $\mathbb{M}$ *impl-path type*

An *inherent-impl* indicates a path to an [inherent implementation](#). It consists of the character $\mathbb{M}$ followed by an *impl-path*, which uniquely identifies the impl block the item is defined in. Following that is a *type* representing the `self` type of the impl.

Recommended Demangling

An *inherent-impl* can be displayed as a qualified path segment to the *type* within angled brackets. The *impl-path* usually need not be displayed.

Example:

```
struct Example;
impl Example {
    fn foo() {}
}
```

The symbol for `foo` in the impl for `Example` is:

`_RNVms_Cs4Cv8Wi1oAIB_7mycrateNtB4_7Example3foo`

```

graph LR
    A["_RNVms_Cs4Cv8Wi1oAIB_7mycrateNtB4_7Example3foo"]
    B["inherent-impl"]
    C["disambiguator 1"]
    D["path to the impl's parent 'mycrate'"]
    E["Self type 'Example'"]
    A --- B
    A --- C
    A --- D
    A --- E
  
```

Recommended demangling: `<mycrate::Example>::foo`

Path: Trait impl

trait-impl → $\mathbb{X}$ *impl-path type path*

A *trait-impl* indicates a path to a [trait implementation](#). It consists of the character `x` followed by an *impl-path* to the impl's parent followed by the *type* representing the `self` type of the impl followed by a *path* to the trait.

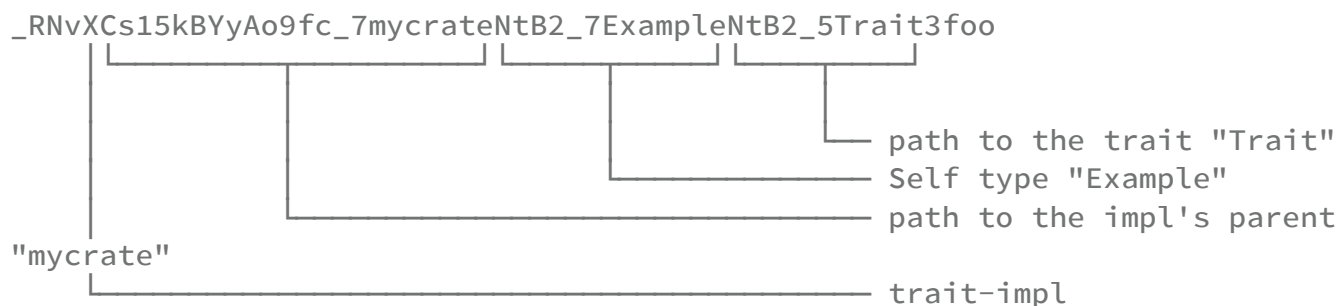
Recommended Demangling

A *trait-impl* can be displayed as a qualified path segment using the `< type as path >` syntax. The *impl-path* usually need not be displayed.

Example:

```
struct Example;
trait Trait {
    fn foo();
}
impl Trait for Example {
    fn foo() {}
}
```

The symbol for `foo` in the trait impl for `Example` is:



Recommended demangling: `<mycrate::Example as mycrate::Trait>::foo`

Path: Impl

`impl-path` → *disambiguator*_{opt} *path*

An *impl-path* is a path used for *inherent-impl* and *trait-impl* to indicate the path to parent of an *implementation*. It consists of an optional *disambiguator* followed by a *path*. The *path* is the path to the parent that contains the impl. The *disambiguator* can be used to distinguish between multiple impls within the same parent.

Recommended Demangling

An *impl-path* usually need not be displayed (unless the location of the impl is desired).

Example:

```
struct Example;
impl Example {
    fn foo() {}
}
impl Example {
    fn bar() {}
}
```

The symbol for `foo` in the impl for `Example` is:

```
_RNVmCs7qp2U7fqm6G_7mycrateNtB2_7Example3foo
```

path to the impl's parent crate-root "mycrate"

The symbol for `bar` is similar, though it has a disambiguator to indicate it is in a different impl block.

```
_RNVmS_Cs7qp2U7fqm6G_7mycrateNtB4_7Example3bar
```

path to the impl's parent crate-root "mycrate"

disambiguator 1

Recommended demangling:

- `foo : <mycrate::Example>::foo`
- `bar : <mycrate::Example>::bar`

Path: Trait definition

trait-definition $\rightarrow$ γ *type path*

A *trait-definition* is a path to a [trait definition](#). It consists of the character γ followed by the *type* which is the `self` type of the referrer, followed by the *path* to the trait definition.

Recommended Demangling

A *trait-definition* can be displayed as a qualified path segment using the `< type as path >` syntax.

Example:

```
trait Trait {
    fn example() {}
}
struct Example;
impl Trait for Example {}
```

The symbol for `example` in the trait `Trait` implemented for `Example` is:

`_RNvYNtCs15kBYyAo9fc_7mycrate7ExampleNtB4_5Trait7exampleB4_`

"mycrate::Example" — path to the implementing type

Trait — path to the trait "Trait"

example — path to the trait definition

— trait-definition

Recommended demangling: `<mycrate::Example as mycrate::Trait>::example`

Path: Nested path

nested-path → N *namespace path identifier*

A *nested-path* is a path representing an optionally named entity. It consists of the character N followed by a *namespace* indicating the namespace of the entity, followed by a *path* which is a path representing the parent of the entity, followed by an *identifier* of the entity.

The identifier of the entity may have a length of 0 when the entity is not named. For example, entities like closures, tuple-like struct constructors, and anonymous constants may not have a name. The identifier may still have a disambiguator unless the disambiguator is 0.

Recommended Demangling

A *nested-path* can be displayed by first displaying the *path* followed by a `::` separator followed by the *identifier*. If the *identifier* is empty, then the separating `::` should not be displayed.

If a *namespace* is specified, then extra context may be added such as:

path :: { *namespace* (: *identifier*)_{opt} # *disambiguator*_{as base-10 number} }

Here the namespace `c` may be printed as `closure` and `s` as `shim`. Others may be printed by their character tag. The `: name` portion may be skipped if the name is empty.

The *disambiguator* in the *identifier* may be displayed if a *namespace* is specified. In other situations, it is usually not necessary to display the *disambiguator*. If it is displayed, it is recommended to place it in brackets, for example `[284a76a8b41a7fd3]`. If the *disambiguator* is not present, then its value is 0 and it can always be omitted from display.

Example:

```
fn main() {
    let x = || {};
    let y = || {};
    x();
    y();
}
```

The symbol for the closure `x` in crate `mycrate` is:

```
_RNCNvCsgStHSCytQ6I_7mycrate4main0B3_
```

Diagram illustrating the components of the symbol `_RNCNvCsgStHSCytQ6I_7mycrate4main0B3_`:

- `_RNCNvCsgStHSCytQ6I_`: nested-path
- `7mycrate4main`: closure namespace
- `0`: path to "mycrate::main"
- `B3_`: identifier with length 0

The symbol for the closure `y` is similar, with a disambiguator:

```
_RNCNvCsgStHSCytQ6I_7mycrate4mains_0B3_
```

Diagram illustrating the components of the symbol `_RNCNvCsgStHSCytQ6I_7mycrate4mains_0B3_`:

- `0`: base-62-number 0
- `1`: disambiguator 1 (base-62-number+1)

Recommended demangling:

- `x : mycrate::main::{closure#0}`
- `y : mycrate::main::{closure#1}`

Path: Generic arguments

generic-args → I *path* {*generic-arg*} E

generic-arg →
lifetime

| *type*
| K *const*

A *generic-args* is a path representing a list of generic arguments. It consists of the character `⊔` followed by a *path* to the defining entity, followed by zero or more *generic-args* terminated by the character `⊔`.

Each *generic-arg* is either a *lifetime* (starting with the character `L`), a *type*, or the character `K` followed by a *const* representing a const argument.

Recommended Demangling

A *generic-args* may be printed as: *path* `::` *opt* `<` comma-separated list of args `>` The `::` separator may be elided for type paths (similar to Rust's rules).

Example:

```
fn main() {
    example([123]);
}

fn example<T, const N: usize>(x: [T; N]) {}
```

The symbol for the function `example` is:

`_RINvCsgStHSCytQ6I_7mycrate7example1Kj1_EB2_`

- end of generic-args
- end of const-data
- const value `'1'`
- const type `'usize'`
- const generic
- generic type `i32`
- path to `"mycrate::example"`
- generic-args

Recommended demangling: `mycrate::example::<i32, 1>`

Namespace

namespace → *lower* | *upper*

A *namespace* is used to segregate names into separate logical groups, allowing identical names to otherwise avoid collisions. It consists of a single character of an upper or lowercase ASCII letter. Lowercase letters are reserved for implementation-internal disambiguation categories (and demanglers should never show them). Uppercase letters are used for special namespaces which demanglers may display in a special way.

Uppercase namespaces are:

- `c` — A closure.
 - `s` — A shim. Shims are added by the compiler in some situations where an intermediate is needed. For example, a `fn()` pointer to a function with the `#[track_caller]` attribute needs a shim to deal with the implicit caller location.
-

Recommended Demangling

See *nested-path* for recommended demangling.

Identifier

identifier → *disambiguator*_{opt} *undisambiguated-identifier*

undisambiguated-identifier → `u`_{opt} *decimal-number* – `_`_{opt} *bytes*

bytes → {UTF-8 bytes}

An *identifier* is a named label used in a *path* to refer to an entity. It consists of an optional *disambiguator* followed by an *undisambiguated-identifier*.

The disambiguator is used to disambiguate identical identifiers that should not otherwise be considered the same. For example, closures have no name, so the disambiguator is the only differentiating element between two different closures in the same parent path.

The undisambiguated-identifier starts with an optional `u` character, which indicates that the identifier is encoded in [Punycode](#). The next part is a *decimal-number* which indicates the length of the *bytes*.

Following the identifier size is an optional `_` character which is used to separate the length value from the identifier itself. The `_` is mandatory if the *bytes* starts with a decimal digit or `_` in order to keep it unambiguous where the *decimal-number* ends and the *bytes* starts.

bytes is the identifier itself encoded in UTF-8.

Recommended Demangling

The display of an *identifier* can depend on its context. If it is Punycode-encoded, then it may first be decoded before being displayed.

The *disambiguator* may or may not be displayed; see recommendations for rules that use *identifier*.

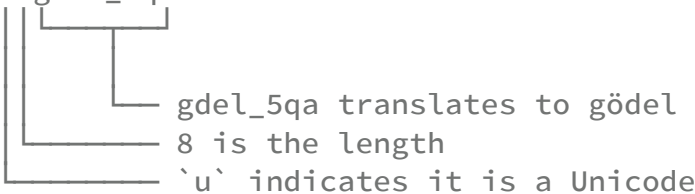
Punycode identifiers

Because some environments are restricted to ASCII alphanumerics and `_`, Rust's [Unicode identifiers](#) may be encoded using a modified version of [Punycode](#).

For example, the function:

```
mod gödel {  
    mod escher {  
        fn bach() {}  
    }  
}
```

_RNvNtNtCsg0H4LzxkuMq_7mycrateu8gdeI_5qa6escher4bach



identifier

Standard Punycode generates strings of the form `([[:ascii:]]+-)?[[:alnum:]]+`. This is problematic because the `-` character (which is used to separate the ASCII part from the base-36 encoding) is not in the supported character set for symbols. For this reason, `-` characters in the Punycode encoding are replaced with `_`.

Here are some examples:

Original	Punycode	Punycode + Encoding
føø	f-5gaa	f_5gaa
α_ω	_-ylb7e	__ylb7e
铁锈	n84amf	n84amf
👉	fq9h	fq9h
ρυστ	2xaedc	2xaedc

Note: It is up to the compiler to decide whether or not to encode identifiers using Punycode or not. Some platforms may have native support for UTF-8 symbols, and the compiler may decide to use the UTF-8 encoding directly. Demanglers should be prepared to support either form.

Disambiguator

disambiguator → s *base-62-number*

A *disambiguator* is used in various parts of a symbol *path* to uniquely identify path elements that would otherwise be identical but should not be considered the same. It starts with the character `s` and is followed by a *base-62-number*.

If the *disambiguator* is not specified, then its value can be assumed to be zero. Otherwise, when demangling, the value 1 should be added to the *base-62-number* (thus a *base-62-number* of zero encoded as `_` has a value of 1). This allows disambiguators that are encoded sequentially to use minimal bytes.

Recommended Demangling

The *disambiguator* may or may not be displayed; see recommendations for rules that use *disambiguator*. Generally, it is recommended that zero disambiguators are never displayed unless their accompanying identifier is empty (like is the case for unnamed items such as closures). When rendering a disambiguator, it can be shortened to a length reasonable for the context, similar to how git commit hashes are rarely displayed in full.

Lifetime

lifetime → `L` *base-62-number*

A *lifetime* is used to encode an anonymous (numbered) lifetime, either erased or *higher-ranked*. It starts with the character `L` and is followed by a *base-62-number*. Index 0 is always erased. Indices starting from 1 refer (as de Bruijn indices) to a higher-ranked lifetime bound by one of the enclosing *binders*.

Recommended Demangling

A *lifetime* may be displayed like a Rust lifetime using a single quote.

Index 0 should be displayed as `'_`. Index 0 should not be displayed for lifetimes in a

ref-type, *mut-ref-type*, or *dyn-trait-type*.

A lifetime can be displayed by converting the De Bruijn index to a De Bruijn level (level = number of bound lifetimes - index) and selecting a unique name for each level. For example, starting with single lowercase letters such as 'a' for level 0. Levels over 25 may consider printing the numeric lifetime as in '_123'. See [binder](#) for more on lifetime indexes and ordering.

Example:

```
fn main() {
    example::<fn(&u8, &u16)>();
}

pub fn example<T>() {}
```

The symbol for the function `example` is:

`_RINvCs7qp2U7fqm6G_7mycrate7exampleFG0_RL1_hRL0_tEuEB2_`

lifetimes

Recommended demangling: `mycrate::example::<for<'a, 'b> fn(&'a u8, &'b u16)>`

Const

```

const →
    type const-data
    | p
    | backref

const-data → nopt {hex-digit} _

hex-digit → digit | a | b | c | d | e | f

```

A *const* is used to encode a const value used in generics and types. It has the following forms:

- A constant value encoded as a *type* which represents the type of the constant and *const-data* which is the constant value, followed by `_` to terminate the *const*.
- The character `p` which represents a *placeholder*.
- A *backref* to a previously encoded *const* of the same value.

The encoding of the *const-data* depends on the type:

- `bool` — The value `false` is encoded as `0_`, the value `true` is encoded as `1_`.
 - `char` — The Unicode scalar value of the character is encoded in hexadecimal.
 - Unsigned integers — The value is encoded in hexadecimal.
 - Signed integers — The character `n` is a prefix to indicate that it is negative, followed by the absolute value encoded in hexadecimal.
-

Recommended Demangling

A *const* may be displayed by the const value depending on the type.

The `p` placeholder should be displayed as the `_` character.

For specific types:

- `b` (`bool`) — Display as `true` or `false`.
 - `c` (`char`) — Display the character in as a Rust character (such as `'A'` or `'\n'`).
 - integers — Display the integer (either in decimal or hex).
-
-

```
fn main() {  
    example::<0x12345678>();  
}  
  
pub fn example<const N: u64>() {}
```

```
_RINvCs7qp2U7fqm6G_7mycrate7exampleKy12345678_EB2_
```

The diagram illustrates the breakdown of the string `_RINvCs7qp2U7fqm6G_7mycrate7exampleKy12345678_EB2_`. The string is shown at the top, with brackets underneath indicating its components:

- `const-data 0x12345678`: Points to the hexadecimal value `12345678`.
- `const type u64`: Points to the character sequence `Ky12345678`, which represents the constant type.
- `const generic arg`: Points to the entire string, representing the generic argument.

A *placeholder* may occur in circumstances where a type or const value is not relevant.

```
pub struct Example<T, const N: usize>([T; N]);

impl<T, const N: usize> Example<T, N> {
    pub fn foo() -> &'static () {
        static EXAMPLE_STATIC: () = ();
        &EXAMPLE_STATIC
    }
}
```

Page 659 of 675

Recommended demangling: `<mycrate::Example<_, _>::foo::EXAMPLE_STATIC`

```
type →
  basic-type
| array-type
| slice-type
| tuple-type
| ref-type
| mut-ref-type
| const-ptr-type
| mut-ptr-type
| fn-type
| dyn-trait-type
| path
| backref
```

- A *basic-type* is encoded as a single character:
 - a — i8
 - b — bool
 - c — char
 - d — f64

- e — str
- f — f32
- h — u8
- i — isize
- j — usize
- l — i32
- m — u32
- n — i128
- o — u128
- s — i16
- t — u16
- u — unit ()
- v — variadic ...
- x — i64
- y — u64
- z — !
- p — [placeholder](#) _

Remaining primitives are encoded as a crate production, e.g. `C4f128` .

- A — An [array](#) [T; N] .

array-type → A [type](#) *const*

The tag A is followed by the [type](#) of the array followed by a [const](#) for the array size.

- s — A [slice](#) [T] .

slice-type → s [type](#)

The tag s is followed by the [type](#) of the slice.

- T — A [tuple](#) (T1, T2, T3, ...) .

$$\text{tuple-type} \rightarrow T \{ \text{type} \} E$$

The tag T is followed by one or more *types* indicating the type of each field, followed by a terminating E character.

Note that a zero-length tuple (unit) is encoded with the u *basic-type*.

- R — A *reference* $\&T$.
-

$$\text{ref-type} \rightarrow R \text{ lifetime}_{\text{opt}} \text{ type}$$

The tag R is followed by an optional *lifetime* followed by the *type* of the reference. The lifetime is not included if it has been erased.

- Q — A *mutable reference* $\&\text{mut } T$.
-

$$\text{mut-ref-type} \rightarrow Q \text{ lifetime}_{\text{opt}} \text{ type}$$

The tag Q is followed by an optional *lifetime* followed by the *type* of the mutable reference. The lifetime is not included if it has been erased.

- P — A *constant raw pointer* $*\text{const } T$.

The tag P is followed by the *type* of the pointer.

$$\text{const-ptr-type} \rightarrow P \text{ type}$$

- O — A *mutable raw pointer* $*\text{mut } T$.
-

$$\text{mut-ptr-type} \rightarrow O \text{ type}$$

The tag `o` is followed by the *type* of the pointer.

- `F` — A *function pointer* `fn(...) -> ...`.

`fn-type` $\rightarrow$ `F` *fn-sig*

`fn-sig` $\rightarrow$ *binder*_{opt} `U`_{opt} (`K` *abi*)_{opt} `{type}` `E` *type*

`abi` $\rightarrow$

`c`

| *undisambiguated-identifier*

The tag `F` is followed by a *fn-sig* of the function signature. A *fn-sig* is the signature for a function pointer.

It starts with an optional *binder* which represents the higher-ranked trait bounds (`for<...>`).

Following that is an optional `u` character which is present for an `unsafe` function.

Following that is an optional `k` character which indicates that an *abi* is specified. If the ABI is not specified, it is assumed to be the "Rust" ABI.

The *abi* can be the letter `c` to indicate it is the "C" ABI. Otherwise it is an *undisambiguated-identifier* of the ABI string with dashes converted to underscores.

Following that is zero or more *types* which indicate the input parameters of the function.

Following that is the character `E` and then the *type* of the return value.

- `D` — A *trait object* `dyn Trait<Assoc=X> + Send + 'a`.

`dyn-trait-type` $\rightarrow$ `D` *dyn-bounds lifetime*

`dyn-bounds` $\rightarrow$ *binder*_{opt} `{dyn-trait}` `E`

`dyn-trait` $\rightarrow$ *path* `{dyn-trait-assoc-binding}`

`dyn-trait-assoc-binding` → `p` *undisambiguated-identifier type*

The tag `D` is followed by a *dyn-bounds* which encodes the trait bounds, followed by a *lifetime* of the trait object lifetime bound.

A *dyn-bounds* starts with an optional *binder* which represents the higher-ranked trait bounds (`for<...>`). Following that is a sequence of *dyn-trait* terminated by the character `E` .

Each *dyn-trait* represents a trait bound, which consists of a *path* to the trait followed by zero or more *dyn-trait-assoc-binding* which list the associated types.

Each *dyn-trait-assoc-binding* consists of a character `p` followed a *undisambiguated-identifier* representing the associated binding name, and finally a *type*.

- A *path* to a named type.
 - A *backref* to refer to a previously encoded type.
-

Recommended Demangling

A *type* may be displayed as the type it represents, using typical Rust syntax to represent the type.

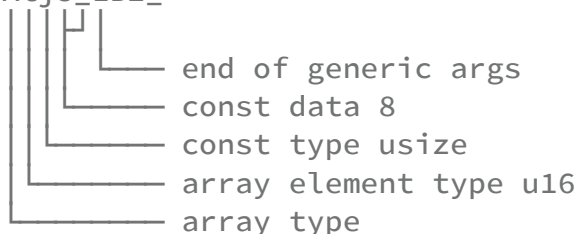
Example:

```
fn main() {
    example::<[u16; 8]>();
}

pub fn example<T>() {}
```

The symbol for function `example` is:


```
_RINvCs7qp2U7fqm6G_7mycrate7exampleAtj8_EB2_
```



Recommended demangling: `mycrate::example::<[u16; 8]>`

Binder

binder → `g` *base-62-number*

A *binder* represents the number of [higher-ranked trait bound](#) lifetimes to bind. It consists of the character `g` followed by a *base-62-number*. The value 1 should be added to the *base-62-number* when decoding (such that the *base-62-number* encoding of `_` is interpreted as having 1 binder).

A *lifetime* rule can then refer to these numbered lifetimes. The lowest indices represent the innermost lifetimes. The number of bound lifetimes is the value of *base-62-number* plus one.

For example, in `for<'a, 'b> fn(for<'c> fn (...))`, any *lifetimes* in `...` (but not inside more binders) will observe the indices 1, 2, and 3 to refer to `'c`, `'b`, and `'a`, respectively.

Recommended Demangling

A *binder* may be printed using `for<...>` syntax listing the lifetimes as recommended in [lifetime](#). See [lifetime](#) for an example.

Backref

backref → `B` *base-62-number*

A *backref* is used to refer to a previous part of the mangled symbol. This provides a simple form of compression to reduce the length of the mangled symbol. This can help reduce the amount of work and resources needed by the compiler, linker, and loader.

It consists of the character `B` followed by a *base-62-number*. The number indicates the 0-based offset in bytes starting from just after the `_R` prefix of the symbol. The *backref* represents the corresponding element starting at that position.

backrefs always refer to a position before the *backref* itself.

The *backref* compression relies on the fact that all substitutable symbol elements have a self-terminating mangled form. Given the start position of the encoded node, the grammar guarantees that it is always unambiguous where the node ends. This is ensured by not allowing optional or repeating elements at the end of substitutable productions.

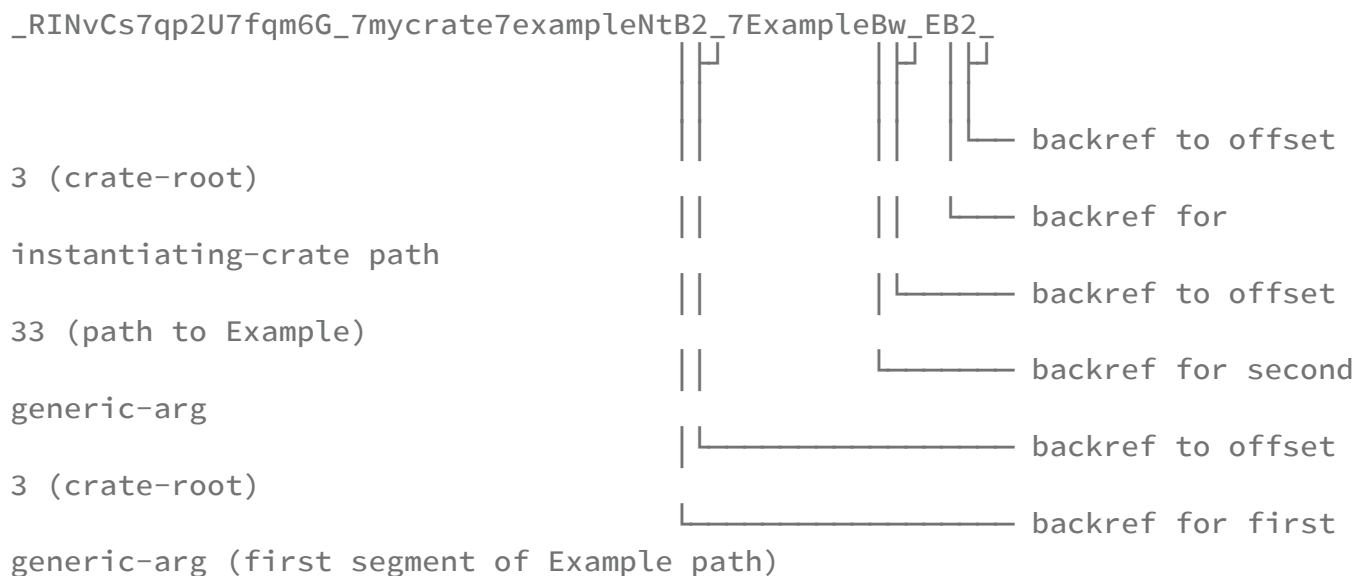
Recommended Demangling

A *backref* should be demangled by rendering the element that it points to. Care should be considered when handling deeply nested backrefs to avoid using too much stack.

Example:

```
fn main() {  
    example::<Example, Example>();  
}  
  
struct Example;  
  
pub fn example<T, U>() {}
```

The symbol for function `example` is:



Recommended demangling: `mycrate::example::<mycrate::Example, mycrate::Example>`

Instantiating crate

`instantiating-crate` → [path](#)

The *instantiating-crate* is an optional element of the [symbol-name](#) which can be used to indicate which crate is instantiating the symbol. It consists of a single [path](#).

This helps differentiate symbols that would otherwise be identical, for example the monomorphization of a function from an external crate may result in a duplicate if another crate is also instantiating the same generic function with the same types.

In practice, the instantiating crate is also often the crate where the symbol is defined, so it is usually encoded as a [backref](#) to the [crate-root](#) encoded elsewhere in the symbol.

Recommended Demangling

The *instantiating-crate* usually need not be displayed.

Example:

```
std::path::Path::new("example");
```

The symbol for `Path::new::<str>` instantiated from the `mycrate` crate is:

```
_RINvMsY_NtCseXNvpPnDBDp_3std4pathNtB6_4Path3neweECs7qp2U7fqm6G_7mycrate
```



instantiating crate identifier ``mycrate``

Recommended demangling: `<std::path::Path>::new::<str>`

Vendor-specific suffix

vendor-specific-suffix $\rightarrow$ (. | \$) *suffix*

suffix $\rightarrow$ {byte}

The *vendor-specific-suffix* is an optional element at the end of the *symbol-name*. It consists of either a `.` or `$` character followed by zero or more bytes. There are no restrictions on the characters following the period or dollar sign.

This suffix is added as needed by the implementation. One example where this can happen is when locally unique names need to become globally unique. LLVM can append a `.llvm.<numbers>` suffix during LTO to ensure a unique name, and `$` can be used for thread-local data on Mach-O. In these situations it's generally fine to ignore the suffix; the suffixed name has the same semantics as the original.

Recommended Demangling

The *vendor-specific-suffix* usually need not be displayed.

Example:

```
thread_local! {
    pub static EXAMPLE: RefCell<u32> = RefCell::new(1);
}
```

The symbol for `EXAMPLE` on macOS may have the following for thread-local data:

```
_RNVnVnVcs7qp2U7fqm6G_7mycrate7EXAMPLE7___getit5___KEY$tlv$init
                                     └── vendor-
specific-suffix
```

Recommended demangling: `mycrate::EXAMPLE::__getit::__KEY`

Common rules

decimal-number →

0

| *non-zero-digit* {*digit*}

non-zero-digit → 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

digit → 0 | *non-zero-digit*

lower → a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t
| u | v | w | x | y | z

upper → A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q |
R | S | T | U | V | W | X | Y | Z

A *decimal-number* is encoded as one or more *digits* indicating a numeric value in decimal.

The value zero is encoded as a single byte `0`. Beware that there are situations where `0` may be followed by another digit that should not be decoded as part of the decimal-number. For example, a zero-length *identifier* within a *nested-path* which is in turn inside another *nested-path* will result in two identifiers in a row, where the first one only has the encoding of `0`.

A *digit* is an ASCII number.

A *lower* and *upper* is an ASCII lower and uppercase letter respectively.

base-62-number

`base-62-number` → { *digit* | *lower* | *upper* } `_`

A *base-62-number* is an encoding of a numeric value. It uses ASCII numbers and lowercase and uppercase letters. The value is terminated with the `_` character. If the value is 0, then the encoding is the `_` character without any digits. Otherwise, one is subtracted from the value, and it is encoded with the mapping:

- `0 - 9` maps to 0-9
- `a - z` maps to 10 to 35
- `A - Z` maps to 36 to 61

The number is repeatedly divided by 62 (with integer division round towards zero) to choose the next character in the sequence. The remainder of each division is used in the mapping to choose the next character. This is repeated until the number is 0. The final sequence of characters is then reversed.

Decoding is a similar process in reverse.

Examples:

Value	Encoding
0	<code>_</code>

1	0_
11	a_
62	z_
63	10_
1000	g7_

Symbol grammar summary

The following is a summary of all of the productions of the symbol grammar.

symbol-name → *_R decimal-number_{opt} path instantiating-crate_{opt} vendor-specific-suffix_{opt}*

path →
 crate-root
 | *inherent-impl*
 | *trait-impl*
 | *trait-definition*
 | *nested-path*
 | *generic-args*
 | *backref*

crate-root → *c identifier*
inherent-impl → *M impl-path type*
trait-impl → *x impl-path type path*
trait-definition → *Y type path*
nested-path → *N namespace path identifier*
generic-args → *I path {generic-arg} E*

identifier → *disambiguator_{opt} undisambiguated-identifier*
undisambiguated-identifier → *u_{opt} decimal-number __{opt} bytes*
bytes → *{UTF-8 bytes}*

disambiguator $\rightarrow$ s *base-62-number*

impl-path $\rightarrow$ *disambiguator*_{opt} *path*

type $\rightarrow$

- basic-type*
- | *array-type*
- | *slice-type*
- | *tuple-type*
- | *ref-type*
- | *mut-ref-type*
- | *const-ptr-type*
- | *mut-ptr-type*
- | *fn-type*
- | *dyn-trait-type*
- | *path*
- | *backref*

basic-type $\rightarrow$ *lower*

array-type $\rightarrow$ A *type* *const*

slice-type $\rightarrow$ S *type*

tuple-type $\rightarrow$ T {*type*} E

ref-type $\rightarrow$ R *lifetime*_{opt} *type*

mut-ref-type $\rightarrow$ Q *lifetime*_{opt} *type*

const-ptr-type $\rightarrow$ P *type*

mut-ptr-type $\rightarrow$ O *type*

fn-type $\rightarrow$ F *fn-sig*

dyn-trait-type $\rightarrow$ D *dyn-bounds* *lifetime*

namespace $\rightarrow$ *lower* | *upper*

generic-arg $\rightarrow$

- lifetime*
- | *type*
- | K *const*

lifetime $\rightarrow$ L *base-62-number*

`const` →

type const-data

| `p`

| *backref*

`const-data` → `n` _{opt} {*hex-digit*} `_`

`hex-digit` → *digit* | `a` | `b` | `c` | `d` | `e` | `f`

`fn-sig` → *binder* _{opt} `U` _{opt} (`K` *abi*) _{opt} {*type*} `E` *type*

`abi` →

`C`

| *undisambiguated-identifier*

`dyn-bounds` → *binder* _{opt} {*dyn-trait*} `E`

`dyn-trait` → *path* {*dyn-trait-assoc-binding*}

`dyn-trait-assoc-binding` → `p` *undisambiguated-identifier* *type*

`binder` → `G` *base-62-number*

`backref` → `B` *base-62-number*

`instantiating-crate` → *path*

`vendor-specific-suffix` → (`.` | `$`) *suffix*

`suffix` → {*byte*}

`decimal-number` →

`0`

| *non-zero-digit* {*digit*}

`base-62-number` → { *digit* | *lower* | *upper* } `_`

`non-zero-digit` → `1` | `2` | `3` | `4` | `5` | `6` | `7` | `8` | `9`

`digit` → `0` | *non-zero-digit*

`lower` → `a` | `b` | `c` | `d` | `e` | `f` | `g` | `h` | `i` | `j` | `k` | `l` | `m` | `n` | `o` | `p` | `q` | `r` | `s` | `t`
| `u` | `v` | `w` | `x` | `y` | `z`

`upper` → `A` | `B` | `C` | `D` | `E` | `F` | `G` | `H` | `I` | `J` | `K` | `L` | `M` | `N` | `O` | `P` | `Q` |

R | S | T | U | V | W | X | Y | Z

Encoding of Rust entities

The following are guidelines for how Rust entities are encoded in a symbol. The compiler has some latitude in how an entity is encoded as long as the symbol is unambiguous.

- Named functions, methods, and statics shall be represented by a *path* production.
- Paths should be rooted at the inner-most entity that can act as a path root. Roots can be crate-ids, inherent impls, trait impls, and (for items within default methods) trait definitions.
- The compiler is free to choose disambiguation indices and namespace tags from the reserved ranges as long as it ascertains identifier unambiguity.
- Generic arguments that are equal to the default should not be encoded in order to save space.

Contributing to rustc

We'd love to have your help improving `rustc` ! To that end, we've written [a whole book](#) on its internals, how it works, and how to get started working on it. To learn more, you'll want to check that out.

If you would like to contribute to *this* book, you can find its source in the rustc source at [src/doc/rustc](#).